



Symbolic Value-Based Abstract Interpretation a Semantic Approach to Program Analyses and Transformations

(DRAFT) Habilitation à diriger les recherches de l'Université
Paris-Saclay
Présentée et soutenue par

Matthieu LEMERRE

Jury:

Reviewer	Laure GONNORD	<i>Professeure, Grenoble INP</i>
Reviewer	François POTTIER	<i>Directeur de Recherche, INRIA</i>
Reviewer	Helmut SEIDL	<i>Professor, TUM</i>
Examiner	Mads DAM	<i>Professor, KTH Royal Institute of Technology</i>
Examiner	Xavier RIVAL	<i>Directeur de recherche, INRIA</i>
Examiner	Mihaela SIGHIREANU	<i>Professeure, ENS Paris-Saclay</i>

Version: November 19, 2025

Contents

1	Introduction	4
1.1	Static analysis by abstract interpretation	4
1.2	Symbolic abstractions	7
1.3	Value-based static analysis	9
1.4	Overview of the manuscript	12
2	Syntax, translations, and concrete semantics	14
2.1	Syntax as a signature, semantics as an implementation	14
2.2	Functor as nanopasses	17
2.3	Types, program variables, and stores	18
2.4	Store	20
2.5	Monads	21
2.6	Interpreter transformation	22
2.7	Expressions with variables	23
2.8	Statements	24
2.9	A generic WHILE language and interpreter	25
3	Fixpoint computation	28
3.1	Standard domain signature	29
3.2	Fixpoint computation	32
3.3	Free algebra over the ABSTRACT_DOMAIN signature	34
3.4	Summary & conclusion	38
3.5	Related work and future directions	39
4	Symbolic expression analysis, SSA Abstract Domains, and SSA translation	45
4.1	Symbolic expression analysis	47
4.1.1	Term_With_Symvar: extending terms with symbolic variables	49
4.1.2	Symbolic_store	51
4.1.3	Heterogeneous_List_Of_Pair and Phi_Arguments	51
4.1.4	Symbolic_Expression_Domain	51
4.1.5	Complete instantiation	55
4.2	SSA_Translation to Value_Based_Domains	56
4.2.1	(INTEGER_AND_BOOLEAN_) VALUE_BASED_DOMAINs	58
4.2.2	The SSA_Translation functor	62
4.2.3	SSA_Domains are symbolic Value_Based_Domains	65
4.2.4	Complete instantiation	72
4.3	Related work and possible directions	73
4.3.1	Difference with published work	73

4.3.2	Related work	74
4.3.3	Future directions	75
5	Numerical SSA domains	78
5.1	Single-value Abstractions	79
5.2	Shortcomings of standard non-relational analysis	81
5.3	Cartesian SSA-based analysis	82
5.4	Related work and possible directions	89
6	Abstract Abstract Datatypes	92
6.1	A theory of stateless AADTs	93
6.1.1	Interface	93
6.1.2	Specification of AADTs and implementation of pairs	95
6.1.3	Regular operations and assume	96
6.1.4	Anti-unification	98
6.2	Some AADTs	100
6.2.1	Monadic constructs for disjunctive reasoning	101
6.2.2	Finite arrays	101
6.2.3	Local disjunctions	104
6.3	Discussion, related and future work	109
6.3.1	Summary of the method	109
6.3.2	Comparison with other techniques to build abstractions	110
6.3.3	More examples of AADTs	112
6.3.4	Flow-sensitive vs flow-insensitive information and limitations	113
6.3.5	Symbolic view of AADT, Exactness, and Compilation.	114
7	Memory analyses	116
7.1	Whole-program analysis for a finite, C-like memory model	116
7.1.1	Memory model	117
7.1.2	Syntax	118
7.1.3	Concrete semantics	119
7.1.4	Abstract semantics	119
7.2	Modular memory analysis of dynamically-allocating code	121
7.2.1	Shape analysis	122
7.2.2	Type-based analysis using a structural type system	124
8	Conclusion	129
8.1	From operating systems code to formal methods	130
8.2	Static analysis of low-level code	132
8.3	Future directions	138
9	Bibliography	140
9.1	Publications	140
9.2	Other references	144
10	Appendices	164

A	Module PatriciaTree	165
A.1	Homogeneous maps	165
A.2	Heterogeneous keys	166
A.3	Heterogeneous sets	167
A.4	Heterogeneous maps	167
B	Module Term	169
B.1	Interface	169
B.2	Implementation	170
C	Module Cfg2dot	175
D	Module Symbolic_Store_2_Dot	177
E	Module Term_Option_Pair_Map	181
F	Standard non-relational abstract interpretation	184
F.1	Module Single_Value_Abstraction	184
F.1.1	A Standard_Non_Relational_Abstract_Domain	192
G	Module Ite_Abstraction	196
G.1	The Ite Abstract Abstract Datatype	196
H	Module Finite_Heap_Abstraction (draft)	199
H.1	The Finite Heap Abstract Abstract Datatype	199
I	Module Concrete_C_Like_Memory_Model (draft)	203
J	Module Abstract_C_Like_Memory_Model (draft)	209
J.1	Preamble: AADT and Make_Domain_From_Stateless_AADT	209
J.2	Lift_Integer_and_Boolean_Value_Based_Domain	210
J.3		212
K	Module Abstract_Type_Based_Analysis (draft)	223
K.1	The type AST	223
K.2	The typedC AST	224
K.3	The example in OCaml	224
K.4		225

Chapter 1

Introduction

 HIS MANUSCRIPT describes the design of sound static analysers that are *symbolic* and *value-based*. This design is well-suited for implementing practical, precise, and efficient static analysers for imperative languages, especially low-level system languages like C or machine code. It evolved from my experience implementing the CODEX static analyser, applying it to challenging case studies, and examining its properties; all these activities have been central to my research objectives.

A sound static analyser comprises multiple interacting components. For instance, the machine code analysis in [RTAS'21](#) simultaneously extracts a control-flow graph from the binary program, which is necessary to perform an SSA translation. The resulting translation is used to conduct the numerical analysis, upon which the memory analysis relies. Both analyses contribute to reconstructing the control-flow graph. Each of these analyses is itself further decomposed into different components. The design is a crucial aspect of such an analyser : components must be separated for software engineering purposes, but must also cooperate closely to achieve the best precision. There are several such seemingly conflicting goals, such as analyser efficiency versus precision, or automation (minimal user effort) versus precision. The challenge lies in finding an analyser design that overcomes these conflicts, and makes the analyser practically useful.

I have published (with my co-authors) several papers discussing specific parts or applications of the CODEX analyser. However, no comprehensive document describes how these parts fit into a global design. This manuscript addresses that gap.

To begin, let us define what a sound static analyser is, and what I mean by symbolic and value-based. We will also discuss why these properties are useful for a static analyser.

1.1 Static analysis by abstract interpretation

Abstract interpretation ([Cousot and Cousot, 1977](#)) is a theory of approximation of the semantics of programs. One of the main application of abstract interpretation is reasoning about *sound static analyses*, which are automated methods for inferring invariants or verifying properties about a program.

Abstract element and concretization Abstract interpretation is often perceived as complicated, but I believe this reputation is unjustified. Here, I will be presenting the core elements that we need to understand this manuscript. Abstract interpretation connects abstract elements (or abstractions) $\mathbb{E}^\#$ (corresponding to data structures in the static analyser) to what they represent, called the concrete elements. In most cases, a concrete element is a set of elements $\mathcal{P}(\mathbb{E})$ that are used in the formal semantics of a programming language: a set of values, a set of stores, a set of heaps, a set of states, etc. There are several possible ways to connect an abstract element with a concrete element (Cousot and Cousot, 1992). The ideal one is a Galois connection, which is a pair of functions (α, γ) : α (the abstraction function) maps concrete elements to abstract ones, and γ (the *concretization function*) maps abstract elements to concrete one; each function deriving from the other.

γ : concretization

Example 1.1.1: Integer intervals

Integer intervals $\mathbb{Z}^\# \triangleq \mathbb{Z} \times \mathbb{Z}$ are composed of a pair (m, M) or integers (that we note $[m..M]$). There is a Galois connection $\mathbb{Z}^\# \xrightleftharpoons[\alpha]{\gamma} \mathcal{P}_{\text{fin}}(\mathbb{Z})$ from intervals to the finite set of integers that they represent:

$$\begin{aligned} \gamma[m..M] &\triangleq \{z \in \mathbb{Z} \mid m \leq z \wedge z \leq M\} \\ \alpha(Z) &\triangleq \begin{cases} (\min Z, \max Z) & (\text{when } Z \text{ is not empty}) \\ [1..0] & (\text{when } Z \text{ is empty}) \end{cases} \end{aligned}$$

Sometimes a Galois connection cannot be defined, and in particular, the abstraction function may not exist. On the other hand, our analyser always manipulates data structures, and we want to give a meaning of the operations it performs. Thus we can generally define the meaning of the analyser operations using only a γ function; this is what we will do in this manuscript. Note that a function $\gamma : \mathbb{E}^\# \rightarrow \mathcal{P}(\mathbb{E})$ is isomorphic to a function $\gamma : \mathbb{E}^\# \rightarrow \mathbb{E} \rightarrow \text{bool}$ (Pichardie, 2005), and the latter can be implemented (which is useful for testing purposes).

Sound abstract operations Once we have defined the data structures of our analyser (the abstract elements) and their meaning (the concretization), we must specify the operations performed by the analyser. These operations, called *transfer functions* or *abstract operations*, can be described mathematically (or in a functional language like OCaml) as functions inputting and outputting abstract elements. Furthermore, we want to specify that the functions are sound, i.e. that it captures all the behaviours of a corresponding concrete transfer function. These specifications are given as *soundness theorems*: proving that the implementation meets the specification corresponds to proving that the transfer functions are sound. For instance, an abstract addition operation over abstract integers $+^\# : \mathbb{Z}^\# \times \mathbb{Z}^\# \rightarrow \mathbb{Z}^\#$ is sound if

transfer functions

abstract operations

soundness theorems

$$\gamma(z_1^\# +^\# z_2^\#) \supseteq \{z_1 + z_2 \mid z_1 \in \gamma(z_1^\#) \wedge z_2 \in \gamma(z_2^\#)\}. \quad (\text{PLUSOUND})$$

When there is a Galois connection, soundness theorem can also be specified using α instead of γ . For interval addition, this gives:

$$\alpha(Z_1) +^\# \alpha(Z_2) \sqsupseteq \alpha(\{z_1 + z_2 \mid z_1 \in Z_1 \wedge z_2 \in Z_2\}) \quad (\text{PLUSOUNDALPHA})$$

where Z_1 and Z_2 are any set of integers, and $\sqsupseteq$ is interval inclusion.

Defining abstract operations The next question is how to define the abstract operations. When there is a Galois connection, we can define the best (meaning, most precise) abstract operation of a given concrete operation f as $\alpha \circ f \circ \gamma$ (Cousot and Cousot, 1979). This best abstract operation may not be computable, but we can derive an implementable version from it (Cousot, 1999). For this reason, the full definition of each transfer function is generally omitted in papers due to lack of space. However, in a practical analyser, the implementation is important. We may not always try to implement the most precise abstract operation, which can be prohibitively expensive; instead the implementation is a compromise between information worth remembering, the time and memory needed to obtain this information, and the effort required to implement and maintain an implementation. For this reason, I chose in this manuscript to detail the implementation of a complete analyser.

For interval arithmetics, we can define $[m_1..M_1] +^\# [m_2..M_2] \triangleq [m_1 + m_2..M_1 + M_2]$, and prove that it is sound (satisfies [PLUSOUND](#)). In addition, we can prove that it corresponds to the best implementation of integer addition on intervals.

Exactness and completeness The $\supseteq$ symbol in [PLUSOUND](#) indicates that the operation can be imprecise and only return an over-approximation of the real behaviours. If we replace it with an equal, then the operation is *exact*, which is an ideal situation. Exact is also called γ -complete or forward-complete (Giacobazzi and Quintarelli, 2001). If we replace the $\supseteq$ symbol in Equation ([PLUSOUNDALPHA](#)) above by $=$, then the operation is α -complete (or just *complete*). Note that while the two formulations of soundness are equivalent, α -completeness is a different notion than γ -completeness (Example 1.1.2). Furthermore, only the best transfer function can be exact or complete, which means that exactness and completeness describe the compatibility between an abstraction and an operation in the concrete semantics, and not merely properties of specific abstract operation. To summarize, exactness means that the best abstract operation does not lose any information and exactly captures the meaning of the concrete operation (for the inputs representable by the domain), while completeness means that the operation does not lose any information expressible using the abstraction.

exact

complete

Example 1.1.2: Completeness and exactness

Addition of an interval with a constant is both exact and complete. For instance:

$$\begin{aligned}\alpha(\{z + 3 \mid z \in \{2, 3, 4\}\}) &= \alpha(\{5, 6, 7\}) = [5..7] = [2..4] +^\# [3..3] \\ \gamma([2..4] +^\# [3..3]) &= \gamma([5..7]) = \{5, 6, 7\} = \{z + 3 \mid z \in \gamma([2..4])\}\end{aligned}$$

Multiplication by a constant is complete, but is not an exact operation. For instance:

$$\begin{aligned}\alpha(\{z * 3 \mid z \in \{2, 3, 4\}\}) &= \alpha(\{6, 9, 12\}) = [6..12] = [2..4] *^\# [3..3] \\ \gamma([2..4] *^\# [3..3]) &= \gamma([6..12]) = \{6, 7, 8, 9, 10, 11, 12\} \supset \{6, 9, 12\}\end{aligned}$$

Xoring an interval with a constant is neither exact nor complete :

$$\begin{aligned}\alpha(\{z \text{ xor } 1 \mid z \in \{2, 4\}\}) &= \alpha(\{3, 5\}) = [3..5] \sqsubset \alpha(\{2, 4\}) \text{ xor }^\# \alpha(\{1\}) = [2..5] \\ \{z_1 \text{ xor } z_2 \mid z_1 \in \gamma([2..4]), z_2 \in \gamma([1..1])\} &= \{2, 3, 5\} \subset \gamma([2..5]) = \{2, 3, 4, 5\}\end{aligned}$$

When abstracting $\mathcal{P}(\{0, 1, 2\})$ using the abstraction $\mathbb{Z}^\# = \{\{0, 1\}, \{1\}, \{1, 2\}, \{2\}, \{0, 1, 2\}\}$, the best abstraction $\text{inc}^\#$ of the function $\text{inc} : x \mapsto \min(2, x + 1)$ is exact, but not complete. For instance:

$$\alpha(\{\text{inc}(z) \mid z \in \{0\}\}) = \alpha(\{1\}) = \{1\} \subset \text{inc}^\#(\alpha(\{0\})) = \text{inc}^\#(\{0, 1\}) = \{1, 2\}$$

Exactness and completeness are thus incomparable notions in theory. In practice, exactness is a stronger notion: if an abstract element can exactly represent the set of inputs, then an exact operation exactly represents the concrete semantics, while a complete operation only states that the operation does not add additional imprecision. Exactness is often believed as a notion that is too strong to be interesting, or at least is much less interesting than completeness (e.g. (Møller and Schwartzbach, 2018, ch.12)). On the contrary, in this manuscript we will not be using completeness, while exactness plays an important role: for the symbolic abstractions of Chapter 3 and Chapter 4 (and POPL'23; PLDI'24), or in the study of the labelled union-find abstract domain (NSAD'24; PLDI'25); and deserves to be studied more. Even if an exact operation does not lose information, it is still doing a useful computation: sometimes the role of a static analysis is to uncover information that already exists in the program; exactness is just the perfect situation where we can uncover everything.

Post-fixpoint, semi-lattices and termination There are other important notions in abstract interpretation, such as lattices, post-fixpoint, or the use of widening to ensure convergence of the fixpoint when the lattice has an infinite height, that can be found in introductory textbooks (Miné, 2017; Rival and Yi, 2020), but are not fundamental to understand this manuscript. These notions are especially important when we want to prove that the analysis terminates. In this manuscript, I will be focusing on soundness of the operations, i.e. provide the properties of the abstract operations that are necessary to prove that the result of an analysis is a sound over-approximation of the program semantics. In particular, I will not be detailing why our algorithms compute a fixpoint, and rather provide means to check that the result, if provided, is a fixpoint. Note that some of the abstract elements presented in this manuscript, such as abstract abstract datatypes (Chapter 6), have similarities with lattices but are not lattices themselves.

1.2 Symbolic abstractions

One of the goals of a static analysis design is to avoid losing precision. An important source of imprecision is the decomposition of an analysis step into multiple steps. This is unfortunate, as the decomposition of the analysis into a fixed number of atomic operations is unavoidable: we cannot provide an abstract operation for every combination of program instructions. Given two concrete operation f and g , the best abstract operation for $f \circ g$ is $\alpha \circ f \circ g \circ \gamma$, while the composition of the best abstract operation of f and the best abstract operation for g is $\alpha \circ f \circ \gamma \circ \alpha \circ g \circ \gamma$: the intermediate call to $\gamma \circ \alpha$ introduces imprecision.

Example 1.2.1: Decomposition is a source of imprecision

For instance, let $f(x) = x \text{ xor } 1$. We want to compute the value of the expression $f(f(x))$ when initially we know that $x \in [2..4]$. The best approximation for the whole expression would compute $(\alpha \circ f \circ f \circ \gamma)([2..4]) = (\alpha \circ \text{id} \circ \gamma)([2..4]) = [2..4]$. However, a step-by-step analysis would compute $(\alpha \circ f \circ \gamma)((\alpha \circ f \circ \gamma)([2..4])) = (\alpha \circ f \circ \gamma)([2..5]) = [2..5]$.

However, in the case where the abstraction is exact ($\gamma \circ \alpha = \text{id}$), then the composition of transfer functions is also exact, and we add no imprecision. In other words, *with an exact abstraction, we can decompose the analysis into a small number of atomic operations without losing precision*. This is quite important: often static analysers try to preserve the structure of the original source code to avoid losing precision (Journault et al., 2019); analysers working on three-address code are less precise than those working on the source (Logozzo and Fähndrich, 2008). In CODEX, we decompose the analysis into the smallest set of constructs as soon as we can, and our numerical analysis is equally precise after a translation from three-address code (PLDI’24)¹.

Symbolic abstractions One way to obtain an exact abstraction is to use a *symbolic* abstraction. For instance, a symbolic abstract operation corresponding to the f function of Example 1.2.1 would be a function inputting a *symbolic expression* (or *term*) $\hat{e}$, and returning the term “ $\hat{e} \text{ xor } 1$ ”. The use of symbolic abstractions to improve the precision of a static analysis is known in the literature (Miné, 2006c; Reps et al., 2004; Gange et al., 2016; Gulwani and Necula, 2004; Graf and Saïdi, 1997). What I propose is to use symbolic abstraction to perform exact analyses on the entire program. There are technical difficulties to achieve this goal, described in this manuscript; such as the need to reuse widening points to break cycles when producing cyclic terms, or the need to perform anti-unifications when we join abstract states.

There are multiple benefits to performing symbolic analysis. One is the ability to change or rewrite the input program to improve the precision of a static analysis. For instance, a symbolic abstraction can rewrite $(x \text{ xor } y) \text{ xor } y$ into just x , which can be analysed more precisely using e.g. an interval abstraction. These kinds of rewrites are very useful, for instance, in static analysis of machine code (Djoudi et al., 2016), where we need to undo compiler transformations and decomposition of CISC instructions in microinstructions. In PLDI’24, we showed how the use of symbolic techniques allows reasoning on byte code as precisely as on the source code. In general, static analysis heavily depends on the intrinsic properties of the program (Bruni et al., 2020; Namjoshi and Pavlinovic, 2018), and adapting the program to the analysis is a good way to improve precision.

Another useful application of symbolic abstractions is the ability to postpone the decision of performing some analysis steps, or to refine the execution done in previous steps. For instance, in PLDI’24 and Chapter 5, we use the symbolic expressions computed by the analysis to perform constraint propagation : we refine the non-relational value associated with each symbolic expression upon reaching a conditional jump, propagating constraints starting from the fact that the condition is true. Another application is the ability to perform a more precise analysis if we cannot

symbolic

symbolic expression

term

¹Note that working from the source code is still more precise than analysis of compiled code, as some undefined behaviours in the source, like signed integer overflows in C, may be defined in the target language. This is why in CODEX our frontends translates the source to a small set of constructs while preserving all assertions related to undefined behaviours.

prove a specific property; for instance CODEX can output its symbolic intermediate form to an automatic solver like SPACER if it cannot prove a specific goal. Working with symbolic abstractions requires distinguishing between precise information that is currently unavailable and imprecise information that is available, as well as their interactions. This is why some of our abstractions have multiple concretization functions γ .

The interface of an abstract domain defines a language In CODEX, symbolic transformations and analysis goes hand-in-hand. Not only we use analysis results to transform the program (Cousot and Cousot, 2002b), but we use program transformations to improve the analysis (PLDI’24). Something that makes this close integration possible is that the interface to abstract domains is viewed as a language (in the tagless-final form of Carette et al. (2009); Kiselyov (2010)). Thus, when the abstract domain is symbolic, our analysis is performing a translation to this language. In Chapter 3, we will see that the classical abstract domain interface corresponds to the flowchart language used in Cousot and Cousot (1977). In Chapter 4, we will see that the interface to value-based abstract domains corresponds to the SSA language—more precisely, to the sea-of-nodes variant of SSA (Click and Paleczny, 1995).

There are interesting consequences of this. First, the soundness proof (and specification) of each abstract domain is clear: each domain is sound with respect to the language defined by its interface. Second, we can interpret many static analysis operations as performing translations. For instance, domain transformers (corresponding to functors in OCaml) can be viewed as compilation passes, whose soundness correspond to forward simulation, and exactness to bisimulation (PLDI’24). Memory objects (Chapters 6 and 7) can be interpreted as (possibly approximate) translations from a language with memory to a language without memory, with the interesting property that the translation can be exact when the source program only refers to a finite amount of memory cells. In general, this opens the way of using semantic methods, based on abstract interpretation, to perform and prove correct program transformations, an interesting (and unforeseen) topic to explore.

1.3 Value-based static analysis

We now saw that we could decompose the analysis of a whole program into a succession of atomic operations. But how small can these atomic operations be? The answer depends on the structure of the abstract interpreter. I came to realize that there are (at least) three different structures of (forward, flow-sensitive) abstract interpreters for imperative languages, that I call *non-relational* analysers, *variable-based* analysers, and *value-based* analysers (Example 1.3.1).

non-relational

variable-based

value-based

Example 1.3.1: Different analyser structures with the same concretization.

Non-relational	Variable-based (LValue-based)	Value-based
$p \mapsto \{.x \mapsto [1..3], .y \mapsto [2..4]\}$	$p.x \in [1..3] \wedge p.y \in [2..4]$	$p \mapsto \{.x \mapsto \alpha, .y \mapsto \beta\}$ $\wedge \alpha \in [1..3] \wedge \beta \in [2..4]$
$p.x := p.x + p.y$		
$p \mapsto \{.x \mapsto [3..7], .y \mapsto [2..4]\}$	$p.x \in [3..7] \wedge p.y \in [2..4]$	$p \mapsto \{.x \mapsto \alpha + \beta, .y \mapsto \beta\}$ $\wedge (\alpha + \beta) \in [3..7]$ $\wedge \beta \in [2..4] \wedge \alpha \in [1..3]$
No relations	Relations between memory locations	Relations between symbolic values
Composable by expressions $[1..3] + [2..4] = [3..7]$	Not composable by expressions $p.x + p.y = ?$	Composable by expressions $\alpha + \beta = (\alpha + \beta)$
Referentially transparent	Referentially opaque	Referentially transparent

Non-relational analysers In a non-relational analyser, the static analysis tracks the possible values of each variable or program expression independently. It is easy to construct abstractions by composition: integer values are represented by intervals; a record value with two integer fields can be represented by a pair of intervals; the store is represented by a mapping from program variables to abstract values. The analysis of a statement can be decomposed into steps that follows that of a concrete interpreter; for instance handling the statement $p.x := p.x + p.y$ consists in getting the intervals for $p.x$ and $p.y$, summing them, then updating the abstract record in the store. This is how interpreters like the **VALUE** plugin of **FRAMA-C**, or value-set analyses ([Balakrishnan and Reps, 2004](#)), are structured. Their main drawback is that they cannot represent relations between variables.

Variable-based analysers Most static analysers that can handle relations, like **Astrée** ([Blanchet et al., 2002](#)), **MOPSA** ([Journault et al., 2019](#)), **EVA** ([Blazy et al., 2017](#)), **Goblint** ([Vojdani et al., 2016](#)), **Verasco** ([Jourdan et al., 2015](#))... follow a structure which is quite different. The idea is to map program variables (or, in general, memory cells ([Miné, 2006a](#))) to dimensions in a numerical abstract domain that represents $\mathbb{Z}^n$ (where n is the total number of dimensions). The basic operations on the domains, assignments, conditionals, and joining two abstractions, are then translated to the corresponding operations on the numerical domain (libraries like **APRON** ([Jeannet and Miné, 2009](#)) support these operations). This static analyser structure has succeeded in verifying software with high precision, notably in embedded systems. Nevertheless, I believe there are two disadvantages in that structure, deriving from the fact that the most important operation in each abstract domain is assignment. The first issue is that you cannot decompose statements into smaller analysis operations, like addition, loading or storing a variable value : you have to handle assignments as a whole. The reason is that all information is attached to a memory location, but an expression like $p.x + p.y$ does not correspond to any memory location. The second issue is absence of referential transparency : the core analysis operation, (abstract) assignment, has side effects, which makes it more difficult to compose operations. For instance, [Monat \(2021, ch. 3.3.4\)](#) explains how assignments in **MOPSA** can make domains temporarily unsound, which requires a merge operator that recovers back soundness. Another

example is the CheckedC type system (Elliott et al., 2018), which allows linking a variable p that is a pointer to an array, to another variable l containing the length of the array. It is difficult to maintain the relation between these variables when either can be modified; for instance, after the statement $l := l/2$ (using integer division), p becomes a pointer to an array which is either of length $2 * l$ or of length $2 * l + 1$. Consequently, CheckedC disallows assignment to variables that are related, unlike our handling of TYPEDC specifications in CODEX (OOPSLA'24).

Value-based analysers There exists a third structure of relational analysers that is lesser known, that I call value-based analysers, that combines the compositional structure of non-relational analysers while handling relations. Examples of this kind of structure can be found in abstract domains working on terms (Chang and Leino, 2005; Gange et al., 2016) or in shape analysers (Chang and Rival, 2008; Gulwani and Tiwari, 2007). The idea is that the evaluation of each expression returns a fresh identifier for the value of the expression, called a symbolic value; and numerical abstract domains can track the relations between symbolic values. The analysis of program statements is decomposed into basic operations which perform (abstract) evaluation steps, inputting and returning symbolic values. For instance, the analysis of $p.x := p.x + p.y$ in Example 1.3.1 first retrieves the symbolic values α and β representing the values of $p.x$ and $p.y$; then create a new symbolic value $(\alpha + \beta)$ representing their sum while updating the numerical domain; finally, the abstract store is modified so that $p.x$ points to this new symbolic value. All of these operations preserve referential transparency, as they do not modify symbolic values, but rather extend the domain with additional information about the freshly-created symbolic value. In a way, variable-based analysers, with their core assignment operation, have a structure more akin imperative languages; while value-based analysers have a structure resembling functional languages (even when analysing imperative languages). A final benefit of the value-based structure is that it separates the numerical abstraction (constraining the symbolic values) from the memory abstraction (which is a data structure containing symbolic values).

While this analyser structure was promising, I felt that it was less understood than the variable-based one (although it was formalized in the above-mentioned papers, using a notion of renaming that I wanted to better understand). For instance, the fact that each operation returns fresh symbolic values naturally raises concerns about termination. The fact that diverging branches create different symbolic values raises issues about how to reconcile these branches; somehow if a program variable x has a symbolic value of α in one branch and a symbolic value of β in the other, then we should "join" these symbolic variables when we join the two states. This merging of symbolic values is related to the "phoney" function ϕ of SSA, used to reconcile differing variables, and the notion of *anti-unification* (Cerna and Kutsia, 2023) between terms. We detail this in this manuscript, as this was the motivation for the work in POPL'23; PLDI'24. The core idea is that while the interface of variable-based domains correspond to a flowchart language of Cousot and Cousot (1977) (or to the control-flow graph representation in a compiler), the interface of value-based domains correspond to the SSA language; it is possible to lift a value-based domain into a variable-based domain, using a functor that performs an SSA translation (Chapter 4).

anti-unification

1.4 Overview of the manuscript

The organization of the manuscript is as follows. We build increasingly complex symbolic and value-based static analysers. Except for Chapter 2 (which provides concrete semantics, i.e., an interpreter), each chapter presents the code of a full static analyser as a literate program (where the less interesting parts have been moved to the appendice). There are several reasons for this choice. The first is to remove ambiguity: with the actual code, the reader should not be left wondering how a particular feature works. The second is to make the manuscript accessible to engineers or researchers working on programming languages, that would be more interested in understanding how some feature can be implemented than by the theory (that is already described in the papers). For these reasons, I have kept a mathematical description of the abstractions of the analysers, their concretization and the soundness theorems of the transfer functions, but I have omitted all the proofs. The last reason is that it allows discussing software engineering issues that I find more difficult to discuss in papers.

- Chapter 2 presents the concrete semantics, introduce concepts of concrete interpretation (such as tagless-final interpretation, monads, or OCaml functors as compilation passes) that will also be useful in abstract interpretation.
- Chapter 3 presents a standard fixpoint computation algorithm, the standard (variable-based) abstract domain interface, and the symbolic domain corresponding to this interface. We show that the standard abstract domain corresponds to a control-flow graph language (as found in compilers or in [Cousot and Cousot \(1977\)](#)) and consequently a static analysis using this domain corresponds to the construction of the control-flow graph. This illustrates how we can use semantic methods for program translation.
- Chapter 4 first presents Symbolic Expression Analysis, which generalizes symbolic execution to handle all the program traces, and builds a symbolic representation of the program dataflow. Then, we introduce the value-based domain interface, whose symbolic version corresponds to the SSA language; and an SSA translation functor that translates from the control-flow language to the SSA language.
- Chapter 5 enriches the symbolic value-based domain with a non-relational numerical analysis, using the symbolic expressions to perform constraint propagation.
- Chapter 6 presents a theory of Abstract Abstract Datatypes (AADT), i.e. abstractions of compound data structures that are implemented as datastructures containing symbolic values. We provide some examples of AADTs to handle pairs, records, arrays or disjunctions, that we use to implement the memory model of a C-like language in a whole-program analyser.
- Chapter 7 shows how Abstract Abstract Datatypes can be used to implement memory abstractions. We present three different memory analyses: whole-program analysis (similar to that found in static analysers for embedding systems), shape analyses, and type-based analysis that perform dependent type-checking with abstract interpretation.

- Chapter 8 concludes, presenting how symbolic value-based static analysers fits within my general research objectives.
- The appendix contains modules used to implement static analysers, or examples of full analyses.

Chapter 2

Syntax, translations, and concrete semantics

 OR THE REST of this manuscript, we will concentrate on methods to write abstract interpreters and semantic methods for program translations. But some of these methods extend or derive from classic techniques for concrete interpretation and compilation. We present some of these methods here, that will be useful in the next chapters. All content in this chapter is background work, except for the observation that OCaml modules can be used to implement nanopasses in the tagless-final style.

2.1 Syntax as a signature, semantics as an implementation

We first describe the syntax and semantics of the program expressions that we will analyse. However, we do it in an unusual way because we are presenting the language in the tagless-final style of [Carette et al. \(2009\)](#); [Kiselyov \(2010\)](#), i.e. the syntax of a program corresponds to the signature of an algebra (or to an OCaml module signature). To this signature we can associate different semantics, or denotations, using different algebras (or OCaml modules). This presentation is an introduction to the next chapters or the work in [POPL'23](#); [PLDI'24](#), that can be viewed as tagless-final *abstract* interpreters.

```
module type BOOLEAN_EXPRESSIONS = sig
  type  $\tau$  expr
  val true: ['boolean] expr
  val  $\neg$ : ['boolean] expr  $\rightarrow$  ['boolean] expr
  val  $\wedge$ : ['boolean] expr  $\rightarrow$  ['boolean] expr  $\rightarrow$  ['boolean] expr
  val  $\vee$ : ['boolean] expr  $\rightarrow$  ['boolean] expr  $\rightarrow$  ['boolean] expr
  val unknown_boolean: unit  $\rightarrow$  ['boolean] expr
end

module type INTEGER_AND_BOOLEAN_EXPRESSIONS = sig
  include BOOLEAN_EXPRESSIONS
  val const: Z.t  $\rightarrow$  ['integer] expr
  val unknown_integer: unit  $\rightarrow$  ['integer] expr
  val add: ['integer] expr  $\rightarrow$  ['integer] expr  $\rightarrow$  ['integer] expr
end
```

```

val sub : ['integer] expr → ['integer] expr → ['integer] expr
val mul : ['integer] expr → ['integer] expr → ['integer] expr
val div : ['integer] expr → ['integer] expr → ['integer] expr
val eq  : ['integer] expr → ['integer] expr → ['boolean] expr
val leq : ['integer] expr → ['integer] expr → ['boolean] expr
end

```

The syntax of the expressions presented here should not be surprising: we have the classical operations of the boolean algebra and integer arithmetics. In addition, we have non-deterministic operations that create unknown boolean and integer values. These operations take a unit argument, as they create a fresh variable whenever they are called. Note that we use polymorphic variant types such as `['integer]` and `['boolean]` as phantom type parameters, used to statically represent the type of the expression. This ensures that only well-typed expressions can be constructed: types constrain the grammar of expressions, following the Church interpretation of types as sets of syntactically well-formed terms.

```

module Concrete_Value = struct
  type  $\tau$  t =
    | IntV: Z.t → ['integer] t
    | BoolV: bool → ['boolean] t end

```

The standard meaning of these expressions is also not surprising: our concrete values are either boolean values or integer values.

```

type undefined_behaviour = ..
type undefined_behaviour +=
  | Division_by_zero
  | Read_variable_not_initialized of string
exception Undefined_Behaviour of undefined_behaviour

```

In our language, we can make programming errors, i.e. write an erroneous program that reads a variable that was not previously initialized, or performs a division by 0, that we represent explicitly in our interpreter using OCaml exceptions. They correspond to the fact that some transitions are partial, i.e., that some precondition must be fulfilled before doing some operations. The usual compilation strategy for languages like C is to assume that the program does not have such programming errors, leading to *undefined behaviours* if the program contains such an error (actually in C, the term undefined behaviour is used to denote both programming errors and their consequence (Lee et al., 2017)). I think programming languages should have programming errors, and should not do things like throwing non-fatal exceptions or returning default values in such situations: this adds overhead, hides real bugs, and it makes static analyses (or even tests) unable to distinguish an error from an expected behaviour. By contrast, programming errors are a good source of *implicit properties* (RTAS'21), i.e., properties about the program that can be verified without having to specify them manually.

implicit properties

As it happens that programmers still make programming errors, how should we handle them? The main technique is to add dynamic tests, to properly report the error if it happens. However, this adds computation overhead, does not guarantee that the program will not crash, and finding the reason for an error can be hard (often the reason for an error is not in the module where the error happens, but in a caller

that did not fulfill some precondition, which is another programming error, even if the precondition is often not explicitly written). The other technique is sound static analysis, which guarantees absence of errors, let the compiler run the code as fast as possible, and promotes the use of annotations to better communicate the expectations and responsibilities of different modules. In high-level languages like OCaml or Rust, a syntactic type system can be used to guarantee the absence of most programming errors (the others are handled dynamically), and helps the programmer fix their error when there is one. I believe that similarly, it is possible to help fix and guarantee the absence of programming errors in C/C++ and binary code programs, using modular static analysis helped by annotations in a dependent type system like TypedC (VMCAI'22; OOPSLA'24).

```

module Concrete_Interp_Integer_And_Boolean_Expressions:
  INTEGER_AND_BOOLEAN_EXPRESSIONS
  with type  $\tau$  expr =  $\tau$  Concrete_Value.t =
  struct
    open Concrete_Value
    type  $\tau$  expr =  $\tau$  Concrete_Value.t
    let i (IntV x) = x      let to_i x = IntV x
    let b (BoolV x) = x    let to_b x = BoolV x
    let true = to_b true  let  $\neg$  x = to_b (not (b x))
    let ar2 fa fb fres op = fun a b  $\rightarrow$  fres (op (fa a) (fb b))
    let  $\wedge$  = ar2 b b to_b ( $\wedge$ )    let  $\vee$  = ar2 b b to_b ( $\vee$ )
    let add = ar2 i i to_i Z.add    let sub = ar2 i i to_i Z.sub
    let mul = ar2 i i to_i Z.mul    let eq = ar2 i i to_b Z.equal
    let leq = ar2 i i to_b Z.leq    let const k = to_i k
    let unknown_boolean() = to_b (Random.bool())
    let unknown_integer() = to_i (Z.of_int (Random.bits()))
    let div (IntV a) (IntV b) =
      if Z.equal b Z.zero then raise (Undefined_Behaviour Division_by_zero)
      else to_i (Z.div a b)
  end

```

The code above defines the standard, concrete interpretation for our program expressions. However, a benefit of the tagless-final style is that it readily allows defining multiple interpretations of a language. Kiselyov (2010) gives examples of non-standard interpretations that pretty-print or compile the program (while preserving their types). In this document, our non-standard interpretations will mostly be about abstract interpretation (that, as we will see, is very related to compilation). In particular, we use in this manuscript:

- A pretty-printing interpretation: **type** τ expr = formatter $\rightarrow \tau \rightarrow$ unit;
- A symbolic (term-building) interpretation: **type** τ expr = τ Term.t;
- A non-relational abstract interpretation: **type** τ expr = τ Interval_Single_Value_Abstraction.expr;
- Monadic interpretations: **type** τ expr = τ expr m;
- Different relational abstract interpretations.

Another benefit of the tagless-final style is that our WHILE language automatically benefits from the type system of the host language (OCaml in our case), saving us the need to implement a type-checker. Here we can already distinguish boolean from integer expressions, and following the Church interpretation of types (i.e., the types

are seen as a grammar that prevents us from writing ill-typed programs), it saves us from the need to define the meaning of addition of boolean values (in the concrete or the abstract)¹.

A last benefit is that interpretation is fast: we don't have any interpretative costs caused by pattern matching on a tag (hence—the tagless name), and the correct function is called directly. The implementations are also often succinct because we don't have to write this pattern matching code.

2.2 Functor as nanopasses

If an OCaml signature can correspond to the syntax of some language, then an OCaml functor, that transforms a module with a given signature to another module with another signature, can implement a translation from a program in one syntax to another. Indeed, I believe that functors are an effective way to decompose a transformation into a sequence of simple transformations (PLDI'24), an approach known as nanopasses (Keep and Dybvig, 2013). In particular, in nanopass compilers it is important to express intermediate languages as a "patch" (list of differences) applied to a previous language, and to express transformations as operating on that patch, and we can do that very simply using tagless-final interpreters and inclusion between signatures.

```
module type INTEGER_AND_BOOLEAN_EXPRESSIONS_FULL = sig
  include INTEGER_AND_BOOLEAN_EXPRESSIONS
  val false: ['boolean] expr
  val lt: ['integer] expr → ['integer] expr → ['boolean] expr
  val geq: ['integer] expr → ['integer] expr → ['boolean] expr
  val gt: ['integer] expr → ['integer] expr → ['boolean] expr
  val neq: ['integer] expr → ['integer] expr → ['boolean] expr
  val unary_minus: ['integer] expr → ['integer] expr
end
```

For instance, `INTEGER_AND_BOOLEAN_EXPRESSIONS_FULL` is expressed here as an extension of `INTEGER_AND_BOOLEAN_EXPRESSIONS` with some missing common operators. While these operators are convenient to use in the surface language, it is easy to replace them with constructs of the smaller language.

```
module Desugar(E: INTEGER_AND_BOOLEAN_EXPRESSIONS)
  : INTEGER_AND_BOOLEAN_EXPRESSIONS_FULL with type τ expr = τ E.expr =
struct
  include E
  let false = ¬ true
  let neq a b = ¬ (eq a b)
  let geq a b = leq b a
  let lt a b = ¬ (geq a b)
  let gt a b = ¬ (leq a b)
  let unary_minus x = sub (const Z.zero) x
end
```

¹Later in this manuscript we will analyze programs whose variables can contain values with different dynamic types, like addresses or integer values; these are encoded using disjunctions between values, that we see in Chapter 6.

The implementation can be seen as extending the base language with more constructs; or, alternatively, as a way to desugar constructs from the full language into constructs of the base language. Note that its implementation depends only on its argument E 's syntax, and not on its interpretation : the desugaring functor works for any interpretation satisfying the `INTEGER_AND_BOOLEAN_EXPRESSIONS` syntax. Let us see how this can be used with the standard interpretation:

```
let () =
  let open Desugar(Concrete_Interp_Integer_And_Boolean_Expressions) in
  assert ((lt (unary_minus (const @@ Z.of_int 3))
              (sub (const @@ Z.of_int 2) (const @@ Z.of_int 8)))
          = Concrete_Value.BoolV false)
```

Note that the implementation of this desugaring pass corresponds to the nanopass view (Keep and Dybvig, 2013) that passes should be handling only the new constructs of the language. The implementation is particularly efficient, as the implementation of base constructs (like `add`) directly goes through the code in E , i.e., does not touch any code from the `Desugar` functor.

The benefits of a smaller base language are that there are fewer constructs that need to be handled by the analyser. However, in static analysis, decomposing a transfer function into a combination of smaller ones can introduce imprecision, so we should remove only the constructs that do not lead to precision losses. For instance, we could express subtraction as a combination of addition and multiplication by -1 , but some abstract domains, like Zones Miné (2001), Octagons Miné (2006b), or the constant offset domain (PLDI'25), do not handle multiplication well. Thus, the size of the base language is a compromise between development efficiency and the precision of the analysis. In our analyses, the above constructs can be removed without introducing any loss in precision.

2.3 Types, program variables, and stores

Our goal is now to extend the `INTEGER_AND_BOOLEAN_EXPRESSIONS` with program variables, i.e., we want to interpret expressions such as $a \wedge x \leq 3$.

```
module Cmp = struct
  type ( $\tau, \tau_2$ ) cmp = ( $\tau, \tau_2$ ) PatriciaTree.cmp =
    | Eq : ( $\kappa, \kappa$ ) cmp
    | Diff : ( $\kappa_1, \kappa_2$ ) cmp
end

module Typ = struct
  type ! $\tau$  tag = ..
  module type TAG = sig
    type t
    type  $\tau$  tag += Tag: t tag
  end
  type  $\tau$  t = (module TAG with type t =  $\tau$ )
  let new_tag (type a) () : a t =
    let module T = struct
      type t = a
      type _ tag += Tag : a tag
    end
```

```

end in
(module T)

let polyeq (type a b) (a: a t) (b: b t): (a, b) Cmp.cmp =
  let (module A) = a and (module B) = b in
  match A.Tag with B.Tag → Eq | _ → Diff

let equal (type a b) (a:a t) (b:b t) =
  match polyeq a b with Eq → true | Diff → false

let boolean:[‘boolean] t = new_tag()
let integer:[‘integer] t = new_tag()
module Boolean = (val boolean)
module Integer = (val integer)
end

```

Some parts of our interpreters and analysers manipulate values independently of their types, and some parts depend on the type. Thus, runtime information representing the type (a tag) is often useful; and we want to make it extensible so that it is easy to add new types. The difficult function to implement is to `polyeq`, that compares two heterogeneous tags, and either concludes that they are different, or, if they are equal, also returns a proof that they correspond to the same type. This function is used by `PatriciaTree`, our heterogeneous map implementation that maps different types of keys to different types of values. The difficulty is that `polyeq` must conclude that equal tags have the same type on tags that have not yet been defined. This is solved by passing as an argument to `polyeq` the modules in which these tags will be defined; for this we use the modules themselves to represent the dynamic type information².

```

module ProgVar = struct
  type !τ t = { typ:τ Typ.t; id:int; name:string}

```

Our first use of these types is the definition of identifiers for (mutable) *program variables* $x \in \mathbb{X}$ (or variables for short), that are distinguished by the type of the value that they contain. We also remember the name of the variable (for debugging purposes), and an integer used to uniquely identify the variable.

$x \in \mathbb{X} : \text{program variables}$

```

let to_int x = x.id
let count = ref 0;;
let create typ name = incr count; {typ;id = !count; name}
type any = Any: τ t → any [@@unboxed]
let equal a b = (Any a) == (Any b)
let polyeq (type a) (type b) (a:a t) (b:b t) : (a,b) PatriciaTree.cmp =
  if equal a b then Typ.polyeq a.typ b.typ else Cmp.Diff
end

```

We need some small helper functions, to create new variables, compare them retrieve their id. The last function, `polyeq`, both compares the variable, and provides a proof that equal variables have the same type, reusing the `polyeq` function on tags that we saw earlier.

²Thanks to François Pottier for telling me about this technique, who was communicated to him by Frédéric Bour.

2.4 Store

The main semantic object that we need is the *store*, a partial mapping $\sigma \in \Sigma \triangleq \mathbb{X} \rightarrow \mathbb{V}$ that associates program variables to values. The mapping is partial as it only contains the value for variables that have been previously assigned; attempting to read the value of a variable that has never been assigned is an undefined behaviour.

 $\sigma \in \Sigma : \text{concrete store}$

```

module type STORE = sig
  type t          type  $\tau$  value
  val find:  $\tau$  ProgVar.t  $\rightarrow$  t  $\rightarrow$   $\tau$  value
  val add:  $\tau$  ProgVar.t  $\rightarrow$   $\tau$  value  $\rightarrow$  t  $\rightarrow$  t
  val empty: t
  type  $\tau_2$  polyfold = { f:  $\tau$ .  $\tau$  ProgVar.t  $\rightarrow$   $\tau$  value  $\rightarrow$   $\tau_2 \rightarrow \tau_2$  }
  val fold :  $\tau_2$  polyfold  $\rightarrow$  t  $\rightarrow$   $\tau_2 \rightarrow \tau_2$ 
  type polyunion = { f:  $\tau$ .  $\tau$  ProgVar.t  $\rightarrow$   $\tau$  value  $\rightarrow$   $\tau$  value  $\rightarrow$   $\tau$  value }
  val idempotent_union: polyunion  $\rightarrow$  t  $\rightarrow$  t  $\rightarrow$  t
  type  $\tau_2$  polyfold2_union =
    { f:  $\tau$ .  $\tau$  ProgVar.t  $\rightarrow$   $\tau$  value option  $\rightarrow$   $\tau$  value option  $\rightarrow$   $\tau_2 \rightarrow \tau_2$  }
  val fold_on_nonequal_union :  $\tau_2$  polyfold2_union  $\rightarrow$  t  $\rightarrow$  t  $\rightarrow$   $\tau_2 \rightarrow \tau_2$ 
end

```

We implement this store using [PatriciaTree](#), an implementation of Okasaki's fast mergable integer maps ([Okasaki and Gill, 1998](#)) written by me and my Phd student Dorian Lesbre. We will need usual map functions like `find` (retrieves the value from a key), `add` (adds a binding) and `empty` (the empty map).

We use a single store to associate variables of different types (integer and boolean variables) to their respective value. For this we use a heterogeneous map, that can map keys of different types to different values (I got this idea from the `dmap` implementation by Benoît Montagu). To support heterogeneous operations, the argument of some functions (like `fold`) must be polymorphic, which is supported by OCaml only if you wrap them in records.

The [PatriciaTree](#) module is implemented to take maximum advantage of physical sharing. In flow-sensitive abstract interpretation, we often duplicate maps when the control-flow splits, which leads to merging trees with common subtrees, and it is important to take advantage of that [Blanchet et al. \(2002\)](#). Thus, `idempotent_union` computes the union of the bindings of the map, merging the values that appear twice, but it assumes idempotence (so that the union of identical trees returns the same tree). `idempotent_union` iterates on all the bindings that differ on the value possibly attached to the key, which also takes advantage of physical equality to skip common subtrees.

```

module Make_Store(Value:sig type  $\tau$  t end)
  : STORE with type  $\tau$  value =  $\tau$  Value.t =
struct
  module M = PatriciaTree.MakeHeterogeneousMap(ProgVar)(struct
    type ( $\kappa$ , 'm) t =  $\kappa$  Value.t
  end )
  include M
end

module Concrete_Store = Make_Store(Concrete_Value)

```

The implementation is generic in the type of the value in the store, as we will reuse `Make_Store` to build other values. For the concrete interpretation, we are interested in the `Concrete_Store`, that maps program variables to `Concrete_Values`.

2.5 Monads

Now that we have a store, we need to pass it to the evaluation of expressions, so that when we evaluate x in $3 + x$, the evaluation has access to the store. We will do this by systematically transforming the denotation (Scott and Strachey, 1971) of each expression from a $\tau \text{Concrete_Value.t}$ to a $\text{Concrete_Store.t} \rightarrow \tau \text{Concrete_Value.t}$: the meaning of an expression like $3 + x/y$ is a partial function that possibly returns a value when given a `Store`. To do this transformation of expressions, we will be using a monad, and in particular the `Reader_Monad`.

```
module type MONAD = sig
  type  $\tau$  m
  val return :  $\tau \rightarrow \tau$  m
  val (let*) :  $\tau$  m  $\rightarrow$  ( $\tau \rightarrow \tau_2$  m)  $\rightarrow \tau_2$  m
end
```

```
module Reader_Monad(State:sig type t end):MONAD with type  $\tau$  m = State.t  $\rightarrow \tau$  =
struct
  type  $\tau$  m = State.t  $\rightarrow \tau$ 
  let return x = fun _s  $\rightarrow$  x
  let (let*) a k = fun s  $\rightarrow$ 
    let b = a s in
    k b s
end
```

This manuscript should be understandable without a good knowledge of what is a monad. You can mostly ignore these `let*` and `return` when reading the code, only knowing that they are used to transport a state (like the store) between the evaluation of expressions, or to short-circuit the evaluation when an expression does not return a result (which is represented by `None`). Here, the `Reader_Monad` will transform every expression of type τ to an expression of type $\text{State.t} \rightarrow \tau$, to make an immutable state available during the evaluation of every expression.

```
module Identity_Monad:MONAD with type  $\tau$  m =  $\tau$  = struct
  type  $\tau$  m =  $\tau$ 
  let return x = x
  let (let*) a k = k a
end
```

A simple monad is the `Identity_Monad`, which does nothing: `let*` is just a function call (i.e., a `let`).

```
module Polymorphic_State_Option_Monad = struct
  type ( $\tau$ , 'state) m = 'state  $\rightarrow$  ( $\tau * 'state$ ) option
  let return x = fun s  $\rightarrow$  Some(x,s)
  let (let*) a k = fun s  $\rightarrow$ 
```

```

    match a s with
    | None → None
    | Some(b,s) → k b s
  let get_state = fun s → Some(s,s)
  let set_state s = fun _ → Some((),s)
  let return_bottom = fun s → None
end

module State_Option_Monad(State:sig type t end):
  MONAD with type  $\tau$  m = State.t → ( $\tau$  * State.t) option =
  struct
    include Polymorphic_State_Option_Monad
    type  $\tau$  m = ( $\tau$ ,State.t) Polymorphic_State_Option_Monad.m
  end

```

The monad that we will be using the most (but not in this chapter) is the [State_Option_Monad](#), that exists in two versions (one where the state is a polymorphic type, and one where it is part of the monadic type). The [State_Option_Monad](#) is a combination of two monads:

- A state monad, that represents a computation that changes a state. It is similar to the [Reader_Monad](#) in that it allows to access a state, but it also allows this state to be modified;
- An option monad, that represents a computation that returns no result (and no state). When this happens, the rest of the computation is short-circuited.

We use this monad to represent the (concrete or abstract) interpretation of an expression in an operational semantics. In the concrete, the state changes correspond to transitions between states, and the option to a computation that fails: for instance, a division by zero returns no result, and no "real" state (only the error state).

This is similar in the abstract: we use the state monad to represent updates to the abstract state, and the option monad to explicitly represent the case when the analysis has inferred that it represents an empty set of states (i.e., that it has reached the most precise abstract state, called bottom ($\perp$), and represented by `None` in OCaml). When the (abstract) state is $\perp$, there is no need to continue the (abstract) interpretation of the program, which is what the "short-circuiting behaviour" of the option monad provides. In CODEX, we often use assignment and exceptions in the host language instead of monads.

2.6 Interpreter transformation

We have seen that following [Carette et al. \(2009\)](#), OCaml signatures define the syntax of a language, and OCaml modules define different (concrete or abstract) interpretation. In many situations, we want to apply a generic transformation of an interpretation, such as encapsulating the interpretation inside of a [Monad](#), or changing the interpretation and representation of base types like boolean and integer. In this section, we define such a transformation for the [INTEGER_AND_BOOLEAN_EXPRESSIONS](#) language.

```

module Lift_Integer_And_Boolean_Expressions
  (Sub:INTEGER_AND_BOOLEAN_EXPRESSIONS)

```

```

(Monad:MONAD)
(Rewrite:sig
  type  $\tau$  expr
  val to_integer: ['integer] Sub.expr Monad.m  $\rightarrow$  ['integer] expr
  val from_integer: ['integer] expr  $\rightarrow$  ['integer] Sub.expr Monad.m
  val to_boolean: ['boolean] Sub.expr Monad.m  $\rightarrow$  ['boolean] expr
  val from_boolean: ['boolean] expr  $\rightarrow$  ['boolean] Sub.expr Monad.m
end):INTEGER_AND_BOOLEAN_EXPRESSIONS
with type  $\tau$  expr =  $\tau$  Rewrite.expr = struct
open Monad
include Rewrite
let true = to_boolean (return Sub.true)
let unknown_boolean() = to_boolean (return @@ Sub.unknown_boolean())
let const k = to_integer (return (Sub.const k))
let unknown_integer() = to_integer (return @@ Sub.unknown_integer())
let  $\neg$  x =
  to_boolean
  (let* x = (from_boolean x) in
   return @@ Sub. $\neg$  x)

let bin op a b =
  let* a = a in
  let* b = b in
  return @@ op a b

let boolop op a b = to_boolean (bin op (from_boolean a) (from_boolean b))
let binop op a b = to_integer (bin op (from_integer a) (from_integer b))
let binpred op a b = to_boolean (bin op (from_integer a) (from_integer b))

let  $\wedge$  = boolop Sub. $\wedge$  let  $\vee$  = boolop Sub. $\vee$ 
let add = binop Sub.add let sub = binop Sub.sub let mul = binop Sub.mul
let div = binop Sub.div let eq = binpred Sub.eq let leq = binpred Sub.leq
end

```

Often we do not need to perform any rewrite, and just want to encapsulate expressions in a monad:

```

module Apply_Monad_Over_Integer_And_Boolean_Expressions
(Sub:INTEGER_AND_BOOLEAN_EXPRESSIONS)(Monad:MONAD):
INTEGER_AND_BOOLEAN_EXPRESSIONS with type  $\tau$  expr =  $\tau$  Sub.expr Monad.m =
Lift_Integer_And_Boolean_Expressions(Sub)(Monad)(struct
  type  $\tau$  expr =  $\tau$  Sub.expr Monad.m
  let to_integer x = x let from_integer x = x
  let to_boolean x = x let from_boolean x = x
end)

```

2.7 Expressions with variables

We now use monadic encapsulation to lift our evaluation of arithmetic expressions to an evaluation of expressions that can contain variables. To do this, we need to thread the store, so that when we evaluate $3 + x$, the evaluation of x can access the

value of x in the store. Consequently, the denotation of expressions is no longer a value, but a partial function from store to value.

```

module Store_To_Integer_And_Boolean_Concrete_Values =
  Apply_Monad_Over_Integer_And_Boolean_Expressions
    (Concrete_Interp_Integer_And_Boolean_Expressions)
    (Reader_Monad(Concrete_Store))
let () =
  let var x store = Concrete_Store.find x store in
  let x = ProgVar.create Typ.integer "x" in
  let open Store_To_Integer_And_Boolean_Concrete_Values in
  let expr = add (const (Z.of_int 3)) (var x) in
  let initial_store = Concrete_Store.(add x (IntV (Z.of_int 4)) empty) in
  assert (expr initial_store = IntV (Z.of_int 7))

```

The `expr` variable above is a function of type `Concrete_Store.t → ['integer] Concrete_Value.t`, corresponding to the denotation of the $3 + x$ expression; it returns 7 when given the store $[x \mapsto 4]$.

2.8 Statements

We now add statements in our language, that can change the store or change the control-flow. The simplest language that we can provide, **WHILE**, only consists in assignment, sequences, and **if** and **while** statements of structured programs. The syntax for **WHILE** statements is as follows:

```

module type WHILE_LANG_STATEMENTS = sig
  type τ expr
  type statement
  val assign: τ ProgVar.t → τ expr → statement
  val seq: statement → statement → statement
  val if_: ['boolean] expr → statement → statement → statement
  val while_: ['boolean] expr → statement → statement
end

```

We can provide a generic interpretation of these statements, that will be useful to build other variations of the **WHILE** language with other expressions than `INTEGER_AND_BOOLEAN_EXPRESSIONS`. The evaluation of statements only needs to understand the meaning of boolean values for the conditional jumps done in `if_` and `while_`. Note that normal denotation for a **WHILE** statement would be a function from store to store. Instead here, our statements are functions from list of stores to list of stores, representing the sequence of all the successive states produced by an assignment from the start of the program. This corresponds to a finite trace semantics (Cousot, 2002), which is mainly of mathematical interest for the next chapter.

```

module Concrete_Interp_While_Statements(Store:STORE)(Interp:sig
  val boolean: ['boolean] Store.value → bool
end):sig
  include WHILE_LANG_STATEMENTS
  with type τ expr = Store.t → τ Store.value
  and type statement = Store.t list → Store.t list
  val var: τ ProgVar.t → τ expr

```

```

end
= struct
  type state = Store.t
  type  $\tau$  expr = Store.t  $\rightarrow$   $\tau$  Store.value
  let var v s = match Store.find v s with
    | exception Not_found  $\rightarrow$ 
      raise (Undefined_Behaviour (Read_variable_not_initialized v.name))
    | value  $\rightarrow$  value

  type statement = state list  $\rightarrow$  state list

  let last states = List.hd states

  let assign x e states =
    let state = last states in
    (Store.add x (e state) state)::states
  let seq stmt1 stmt2 states = stmt2 (stmt1 states)
  let if_ p stmt_then stmt_else states =
    let v = p (last states) in
    if Interp.boolean v then stmt_then states
    else stmt_else states
  let rec while_ p stmt_body states =
    let v = p (last states) in
    if Interp.boolean v then while_ p stmt_body (stmt_body states)
    else states
end

```

The main semantic object that we use is the *state*. Often this state consists in a memory part, with the store and the heap (mapping from addresses to values); and a control part (such as the current program location). Here, the state will contain only the store: the memory part is reduced to the store because we will integrate the heap inside the store, as a special variable of type *heap* (Section 7.1), and we use a denotational semantics that abstracts away from information about the control flow. Specifically, standard denotational semantics interpret statements as partial functions from states to states (Scott and Strachey, 1971). Here, our denotational semantics interpret statements as partial functions from finite *traces* to finite traces, where a trace is a sequence of states (the initial state, plus one per assignment). The function is partial because we do not model failures of the WHILE language explicitly; instead they are represented by failures of the OCaml interpreter (i.e., formally, by the fact that the denotation of a WHILE program is represented by a *partial* function). One of these failures is to read a variable that was not previously assigned.

$s \in \mathbb{S} : \text{concrete state}$

trace

While this finite trace semantics (Cousot, 2002) is not the most precise one, it suffices for the proof of all safety properties (Alpern and Schneider, 1987) (as safety properties are refuted with a finite counter-example, reasoning about the set of all finite traces suffices to prove safety properties). It will also suffice for our compilation proofs.

2.9 A generic WHILE language and interpreter

We can now define the complete syntax and semantics for a WHILE language. We want to actually define a *family* of languages, independent of the type of expressions,

so that we can easily add new types and constructs to the language. Many parts of this manuscript, including the fixpoint computation of Chapter 3 and SSA translation of Chapter 4, are independent of the program expressions.

```

module type GENERIC_EXPRESSIONS = sig
  module type EXPR_TYPE
  module Expr:EXPR_TYPE
  type  $\tau$  expr
end

module type WHILE_LANG_GENERIC = sig
  include WHILE_LANG_STATEMENTS
  include GENERIC_EXPRESSIONS with type  $\tau$  expr :=  $\tau$  expr
  val var:  $\tau$  ProgVar.t  $\rightarrow$   $\tau$  expr
end

```

The `Generic_Expressions` groups a module `Expr` with its module type `EXPR_TYPE`, which is supposed to contain a τ `expr` type (but we cannot enforce this yet). It is used in `While_Lang_Generic`, that defines a `WHILE` language parameterized by its expressions.

```

module type WHILE_LANG_INTEGER_AND_BOOLEAN = sig
  type  $\tau$  expr
  include WHILE_LANG_GENERIC
  with type  $\tau$  expr :=  $\tau$  expr
  and module type EXPR_TYPE :=
    (INTEGER_AND_BOOLEAN_EXPRESSIONS with type  $\tau$  expr =  $\tau$  expr)
end

```

We can specialize `While_Lang_Generic` to the `WHILE` language whose expressions are the arithmetic and boolean expressions.

```

module Example(W:WHILE_LANG_INTEGER_AND_BOOLEAN) = struct
  let x = ProgVar.create Typ.integer "x"
  let y = ProgVar.create Typ.integer "y"

  let factorial5 =
    let open Desugar(W.Expr) in
    W.(seq
      (seq
        (assign x (const (Z.of_int 5)))
        (assign y (const (Z.of_int 1))))
      (seq
        (while_ (gt (var x) (const Z.zero))
          (seq
            (assign y (mul (var y) (var x)))
            (assign x (sub (var x) (const Z.one))))))
      (seq
        (assign x (mul (var y) (var x)))
        (assign y (add (var x) (var y))))))
end

```

The above code is our first program in this language. It will be our motivating example in Chapters 3 and 4. It is parameterized by module `W`, representing the interpreter for this language. When given the pretty-printing interpretation `PP_While_Lang` (not shown for space reasons), it is :

```
module Example_PP = Example(PP_While_Lang);;
assert ((Format.asprintf "%t" Example_PP.factorial5) =
  "x := 5; y := 1; " ^
  "while not x <= 0 do y := y * x; x := x - 1 done; " ^
  "x := y * x; y := x + y")
```

Finally, we can build the concrete interpreter for `While_Lang_Integer_and_Boolean`:

```
module While_Lang_Integer_and_Boolean_Interpreter = struct
  module Expr = Store_To_Integer_And_Boolean_Concrete_Values
  include Concrete_Interp_While_Statements(Concrete_Store)(struct let boolean
    (BoolV x) = x
  end)
end
```

and use it to interpret our example program:

```
module Inst = Example(While_Lang_Integer_and_Boolean_Interpreter);;
let final_store = List.hd (Inst.factorial5 [Concrete_Store.empty]) in
assert (Format.asprintf "%a" Concrete_Store.pp final_store =
  "[x -> 0 y -> 120 ]")
```

Chapter 3

Fixpoint computation

 HE GOAL of this chapter is twofold. Firstly, it provides a fixpoint computation algorithm that will be needed in the rest of this manuscript; this will serve as an opportunity to discuss my work on the area. Secondly, we use it to provide an implementation of the compiling-with-abstract-interpretation approach (PLDI'24) which is as simple as possible. The essence of the approach is to implement abstract domains symbolically as free/term algebras over the signature of abstract domains, where signature denotes in theory an algebraic signature, and practically, an OCaml module type.

The main thesis of this chapter is that *standard fixpoint computation* can be viewed as a *translation pass from the source language* (here, the WHILE language) to a *Control-flow-graph (CFG) language*. This is the basis for a *semantic method for program translation*: i.e., the reuse of abstract interpretation theory to build and prove correct compilers. One of the benefits of viewing program translation as an abstract domain is that it allows the translation and analysis to happen simultaneously as a reduced product (Cousot and Cousot, 1979); such that the translation can be simplified using the analysis (e.g., using deadcode elimination), and the program modification done by the translation can improve the precision of the analysis (Giacobazzi and Mastroeni, 2016).

One important point of the chapter is a *semantic proof method for compiler correctness*, entirely based on abstract interpretation. In my previous papers on the subject (POPL'23; PLDI'24), the proof resembled usual compilation proofs (Leroy, 2006): provide an operational semantics for the source and target languages, and finding a simulation relation between the source and target programs. By contrast, here, the proof (sketch) is purely semantic: as we have a sound and exact abstraction of the program traces, the result of the fixpoint computation is an object that preserves all the program traces. It just so happens that this object can also be equipped with an operational semantics, and is thus executable.

Implementation-wise, the constructs described in this chapter are standard. An interesting question is *why* we want a translation (by fixpoint computation) to standard domain signatures, rather than having an abstract domain that directly interprets WHILE constructs (as in MOPSA (Journault et al., 2019)). The answer is that this helps maintain a modular structure of the analyser:

- Like many functors in this manuscript, *Fixpoint* computation decomposes complex constructs like `while(e) stmt` into a smaller set of core constructs that are easier to handle. These core constructs (`join`, `assign`, `assume`, ...) closely cor-

respond to the basic operations over sets of states or traces, that a static analysis abstracts.

- There are different ways (and a whole literature) on how to perform a fixpoint computation over the domain interface, as we want to improve computing efficiency (Kennedy, 1981), memory usage (Kim et al., 2020), or perform incremental fixpoint computation (Stein et al., 2024)... Thus, decoupling the fixpoint computation from the abstract domain implementation allows easily changing and reusing of the fixpoint computation algorithm. I have been particularly interested in trace partitioning (Rival and Mauborgne, 2007; Sankaranarayanan et al., 2006) techniques that improve precision, which can also be considered as techniques for performing control-flow transformations when fixpoint computation is seen as a program translation.
- The decomposition of compound, "big-step" constructs, like `while`, into a sequence of more atomic "small-step" constructs like assignments, is necessary for some domain compositions, like reduced product (Cousot and Cousot, 1979), an important method to structure abstract interpreters. The intuition behind this is that `while` constructs are similar to the use of an internal iterator (like `List.iter f in OCaml`), in that this nicely describes the overall behaviour of a program, but does not allow intertwining operations on individual program steps. Decomposing into a sequence of constructs is like using external iterators (like `OCaml Seq.t`), which can create a sequence of pairs from two sequences, and allows better coordination between the simultaneous analyses.

The outline of the chapter is as follows: I first present the standard `ABSTRACT_DOMAIN` interface, then a standard `Fixpoint` computation algorithm; then we instantiate it with the `Term` algebra over the standard `ABSTRACT_DOMAIN` signature, resulting in a CFG construction by abstract interpretation. I conclude with a discussion of work I did that relates to this area, and ideas for future directions.

3.1 Standard domain signature

In this section we provide the canonical signature for a forward, *flow-sensitive*¹ abstract interpreter of the `While` language. This signature is a variation of the one provided by Cousot and Cousot (1977), itself an extension of the signature of the dataflow-analysis algorithm of Kildall (1973) to incorporate widening, with some variations.

flow-sensitive

flow-insensitive

```
module type GENERIC_EXPRESSIONS_WITH_BOOLEAN = sig
  include GENERIC_EXPRESSIONS
  include BOOLEAN_EXPRESSIONS with type  $\tau$  expr :=  $\tau$  expr
end
```

The abstract domain must understand the expressions that are part of the `While` language, but the fixpoint engine does not have to, which is why the type of expressions in the domain can be abstract. Only a minimal understanding of expressions by the fixpoint engine is needed to create *guards*, the conditional expressions that the

guard

¹An abstraction is flow-sensitive if it is split according to the control-flow locations in the program, i.e., if the main abstraction conceptually contains a mapping from control-flow locations to this abstraction. An abstraction is flow-insensitive if only a single instance of this abstraction is present at every step of the analysis.

program must fulfill to continue. Here we only need to manipulate the `not_` boolean expression to negate the conditions that appear in `while` and `if`; if `WHILE` had more complicated control structures (C-like `switch`, pattern matching, ...), the fixpoint engine would need to know more about the language of expressions.

```
type loop_id = int
```

A difference with the standard abstract domains signature is that our widening operation (`fixpoint_step`) requires a *loop identifier* (of type `loop_id`). This identifier is used to detect cycles and is necessary for the static analysis to terminate. A lot of domains that I discuss in this manuscript are based on the symbolic manipulations of terms and symbolic variables. However, we cannot always create fresh variables and terms as this would prevent the analysis from terminating. My solution to this problem is to work on cyclic term graphs (Ariola and Klop, 1996). In cyclic term graphs, some nodes are given a name to break the recursion; the purpose of `loop_id` is to create these names deterministically, so that we can easily detect when the analysis has reached a fixpoint. In [POPL'23](#) I proposed to use the names of control-flow location as names. In [PLDI'24](#) we used only the names of widening points for the same purpose, discovering an interesting analogy in the role of widening points and names in cyclic terms to break cycles. But we can actually just generate fresh identifiers on the fly, which simplifies the interaction with control-flow transformations that might duplicate control-flow locations (e.g. loop unrolling or inlining).

loop identifier

```
module type ABSTRACT_DOMAIN = sig

  module Expr: GENERIC_EXPRESSIONS_WITH_BOOLEAN
  val var:  $\tau$  ProgVar.t  $\rightarrow$   $\tau$  Expr.expr

  (** Abstraction of states. *)
  type state

  (** Abstraction of the possible entry states. *)
  val entry_state: state

  (** Standard join operation, over-approximating union. *)
  val join: state  $\rightarrow$  state  $\rightarrow$  state

  (** [fixpoint_step loop_id previous next] detects whether fixpoint
      computation is over (then it returns true and the final abstract
      state), or it tries to extrapolate (widen) from the evolution
      from [previous] to [next] (returning false and a widened
      state). *)
  val fixpoint_step:
    loop_id  $\rightarrow$  state  $\rightarrow$  state  $\rightarrow$  (state * bool)

  (** [assume cond state] filters [state] to only contain
      those for which [cond] is true. *)
  val assume: ['boolean] Expr.expr  $\rightarrow$  state  $\rightarrow$  state option

  (** [assign x e state] returns an over-approximation of the states
      in [state] after they have been applied the statement x := e. *)
  val assign:  $\tau$  ProgVar.t  $\rightarrow$   $\tau$  Expr.expr  $\rightarrow$  state  $\rightarrow$  state option
end
```

The signature for our abstract domain is standard, except that we perform widening and inclusion checking in the same function that we call `fixpoint_step`. This simplifies implementation and makes the analysis more efficient. Furthermore, the `assume` and `assign` functions return an option type, where `None` represents *bottom* ($\perp$) (denoting the empty set of states).

$\perp$: *bottom*

Meaning and formalization

I will denote by $\mathbb{S}^\#$ the set of *abstract states* (remember that $\mathbb{S}$ denotes concrete states). In the code, abstract states correspond to the `state` type. The `state` option type is represented by $\mathbb{S}_\perp^\#$. The meaning of the abstract domain could be defined by a concretization function $\gamma : \mathbb{S}^\# \rightarrow \mathcal{P}(\mathbb{S})$, that makes each abstract state correspond to a set of concrete states. However here we are interested in proving compilation correctness, i.e. preservation of traces. Thus, we will concretize into the set of finite *traces*, where, for the sake of simplicity, a trace is the sequence of states obtained after each assignment. Note that by the hierarchy of program semantics (Cousot, 2002), an abstraction that is sound for the finite traces abstraction is also sound for a concretization into *transformations* (FMSD'21; SAS'20) (i.e., input/output relations, represented as the first and last elements of the trace), or into sets of states (just the last element). Note that our notion of trace preservation is very strong, which is interesting from an analysis point of view, but not when performing translations, as this precludes some optimizations, such as removing useless assignments. We discuss this link between concretization and optimizations at the end of the chapter. Note that we do not consider infinite trace in the concretization above (we discuss them too at the end of the chapter), and this enables some optimizations that we will soon see.

$\mathbb{S}^\#$: *abstract state*

trace

What the transfer functions have to do is given by soundness conditions, the invariants that `ABSTRACT_DOMAINS` must fulfill (in an analyser written in Coq/Rocq like Verasco (Jourdan, 2016), these requirements would appear in the module signature). Here we state them as follows: $\forall s^\#, s_1^\#, s_2^\# \in \mathbb{S}^\#, \forall x \in \mathbb{X}$:

Definition 3.1.1: Soundness theorems for `ABSTRACT_DOMAIN`.

$$\begin{aligned} \gamma(\text{entry_state}) &\supseteq \{\langle s \rangle \mid s \in (\tau \text{ progvar} \rightarrow \mathbb{V}_\tau)\} \\ \gamma(\text{join } s_1^\# s_2^\#) &\supseteq \gamma(s_1^\#) \cup \gamma(s_2^\#) \\ \gamma_\perp(\text{assume } e s^\#) &\supseteq \{t \in \gamma(s^\#) \mid \text{eval } e (\text{last } t) = \text{true}\} \\ \gamma_\perp(\text{assign } x e s^\#) &\supseteq \{s[x \leftarrow \text{eval } e s] :: t \mid t \in \gamma(s^\#), s = \text{last } t\} \\ (\text{fixpoint_step } s_1^\# s_2^\#) = (s_3^\#, \text{true}) &\Rightarrow \gamma(s_1^\#) \supseteq \gamma(s_2^\#) \wedge \gamma(s_3^\#) \supseteq \gamma(s_2^\#) \end{aligned}$$

where $\gamma_\perp$ is defined as $\gamma_\perp(\text{None}) \triangleq \{\}$, $\gamma_\perp(\text{Some}(s^\#)) \triangleq \gamma(s^\#)$, *last* designates the element of a sequence that was appended last, and $\text{eval}(e, s)$ represents the evaluation of an expression using a state s^2 . In short, `entry_state` is a singleton trace containing an arbitrary state; `join` overapproximates the union of traces; `assume` overapproximates the filtering of traces according to a predicate; `assign` represents the traces where a new store has been appended on the existing traces. We do not require anything on `fixpoint_step`, except that if it returns true, then a fixpoint must have been

²There is a simplification here in that in our implementation the expressions are abstract, while we assume that they are terms in this formalization.

reached (another requirement is that repeated calls to `fixpoint_step` should eventually converge, which is required to guarantee the termination of the analysis, but this is not necessary to prove soundness). The requirements for exactness are similar, except that the $\supseteq$ are replaced by $=$.

3.2 Fixpoint computation

In this section we provide a syntax-based dataflow analysis algorithm (originally called high-level dataflow analysis (Rosen, 1977)) that performs a fixpoint computation directly from the program syntax, i.e. without requiring the computation of a control flow graph. The algorithm that we provide here is standard (see, e.g. Fig 3.10 in Miné, 2017). It is implemented as an OCaml functor, parameterized by the implementation of an abstract domain.

```
module Fixpoint(D:ABSTRACT_DOMAIN):WHILE_LANG_GENERIC
  with type statement = D.state → D.state option
  and module type EXPR_TYPE = D.Expr(EXPR_TYPE)
  and module Expr = D.Expr(Expr)
  and type  $\tau$  expr =  $\tau$  D.Expr(expr) =
  struct
    type statement = D.state → D.state option
```

In abstract interpretation articles, syntax-directed fixpoint computation is often called the *denotational style of abstract interpretation* (Miné, 2017) (and sometimes also called big-step abstract interpreters, or definitional abstract interpreters (Keidel et al., 2023)). We can see above why: the method provides an interpretation of each statement as a function from an abstract state to an abstract state, which is an abstraction of the denotational semantics of each statement as a partial function from states to states (Scott and Strachey, 1971). Our interpretation makes explicit (using None) the empty abstract state $\perp$, so that the analysis skips unreachable code, making it both faster and more precise.

```
include D.Expr
let var = D.var          let assign = D.assign          let assume = D.assume
```

The functor’s implementation is independent from the expressions used in the source language (the EXPR_TYPE signature exported by the functor corresponds to the one in the argument). We reuse the transfer functions handling basic statements and expressions that is provided by the abstract domain.

```
let (let*) = Option.bind
let seq c1 c2 s# = let* s1# = c1 s# in c2 s1#
```

None represents the bottom case, and we use the option monad to avoid analysing statements that the analyser can prove unreachable: i.e., we skip the analysis of c2 if c1 returns None. In CODEX, we also use exceptions in the interpreter to do the same thing.

```
let join_state_option s1# s2# = match s1#, s2# with
  | None, s# | s#, None → s#
```

```

| Some  $s_1^\#$ , Some  $s_2^\# \rightarrow \text{Some}(\text{D.join } s_1^\# \ s_2^\#)$ 

let if_ (cond:[‘boolean] expr) cthen celse  $s^\#$  =
  let then_ $s^\#$  =
    let* then_ $s^\#$  = assume cond  $s^\#$  in cthen then_ $s^\#$  in
  let else_ $s^\#$  =
    let* else_ $s^\#$  = assume (D.Expr. $\neg$  cond)  $s^\#$  in celse else_ $s^\#$  in
  join_state_option then_ $s^\#$  else_ $s^\#$ 

```

We extend `join` to handle the bottom case. The handling of `if_` statements decompose conditionals using two core constructs of the concrete and abstract semantics: filtering (using `assume`) and union (using `join`). But, as we will see, this code can also be seen as a translation of `if_` statements into non-deterministic choice and guards, like in Dijkstra’s Guarded Command Language (Dijkstra, 1975).

```

let fresh_loop_id =
  let count = ref (-1) in
  fun ()  $\rightarrow$  incr count; !count

let while_ cond (cbody:statement) entry_ $s^\#$  =
  let loop_id = fresh_loop_id () in
  let rec loop  $s^\#$  =
    let body_ $s^\#$  = let*  $s^\#$  = assume cond  $s^\#$  in cbody  $s^\#$  in
    match body_ $s^\#$  with
    | None  $\rightarrow$  entry_ $s^\#$  (* The loop body stops, i.e., there is no loop. *)
    | Some body_ $s^\#$   $\rightarrow$ 
      let  $s^\#$  = D.join entry_ $s^\#$  body_ $s^\#$  in
      let  $s^\#$ , fp_reached = D.fixpoint_step loop_id  $s^\#$   $s^\#$  in
      if fp_reached then  $s^\#$  else loop  $s^\#$  in
  let final_ $s^\#$  = loop entry_ $s^\#$  in
  assume (D.Expr. $\neg$  cond) final_ $s^\#$ 
end

```

The analysis of `while_` statement performs a local fixpoint iteration (thus following a recursive fixpoint iteration strategy (Bourdoncle, 1993)), assigning a fresh identifier to each fixpoint computation. The most interesting part here is the case where `body_state` is `None`, i.e., when the analysis of the body returns no state (because of division by zero, infinite inner loop, etc. in the loop body; that is, the end of the loop is not reachable). In this case, we return the abstract state at the loop entry. The concretization into finite traces provide a good justification for this: as the end of the loop body is unreachable, any trace that enters the loop does not exit the loop. Thus, the traces that exit the loop are those that never enter it, i.e., are the traces for which the loop condition is initially false. Because the relational and state abstractions are abstractions of the set of finite traces (Cousot, 2002), this optimization in the fixpoint trace computation is also sound for the relational and state concretizations.

It is interesting to understand how this corresponds to a program transformation. Informally, we are transforming a statement $w = \text{while}(\text{cond}) \text{ body}$ by a blocking assertion like `assert(!cond);` (corresponding to the `assume` operation in the abstract). This assertion could be removed to make the code more efficient, but the transformation would then be over-approximating: there would be traces in the initial

program that would reach the end of the transformed program, while this was not the case in the initial program. It is interesting to see that the interpretation of fixpoint computation as a program transformation automatically performs and justifies such transformations.

Note that this transformation/optimization is only valid if we are interested in preservation of the denotation (Section 3.5); it is not valid if we are also interested in preserving abruptly-terminating or infinite traces. This optimization thus probably requires knowing the intent of the programmer (Bowman, 2025).

Formalization

The formalization is standard. Let us denote by $\llbracket w \rrbracket^\# : \mathbb{S}^\# \rightarrow \mathbb{S}_\perp^\#$ the abstract interpretation of a WHILE statement w , and by $\llbracket w \rrbracket : \mathbb{S}^* \rightarrow \mathbb{S}^*$ its concrete interpretation (Section 2.8). We can prove that:

$$\forall w \in \mathbb{W}, \forall s^\# \in \mathbb{S}^\# : \gamma(\llbracket w \rrbracket^\#(s^\#)) \supseteq \{\llbracket w \rrbracket(t) \mid t \in \gamma(s^\#)\}$$

Furthermore, if all the operations in $\text{Fixpoint}(D)$ are exact, then we have

$$\forall w \in \mathbb{W}, \forall s^\# \in \mathbb{S}^\# : \gamma(\llbracket w \rrbracket^\#(s^\#)) = \{\llbracket w \rrbracket(t) \mid t \in \gamma(s^\#)\}$$

These statements are proved by induction on w , using the soundness/exactness of D (Definition 3.1.1) and the Tarskian fixpoint approximation theorem (Miné, 2017, Theorem 2.8).

3.3 Free algebra over the ABSTRACT_DOMAIN signature

The above exposition is a quite standard presentation of a forward abstract interpreter. An original idea that we have investigated with my PhD student Dorian Lesbre (PLDI'24), is to view abstract interpretation as a way to translate programs, either exactly or approximately. We saw in Section 2.1 that tagless-final interpreters can represent the syntax of programs as module signatures, and that functors describe translation between languages with different interfaces. The idea here is to see the ABSTRACT_DOMAIN interface as the description of some "ABSTRACT_DOMAIN language", which means that the `Fixpoint` functor is a translator from the `While_Lang_Generic` language to this ABSTRACT_DOMAIN language.

What is the language corresponding to ABSTRACT_DOMAIN? It is a flowchart language similar to the ones used to present abstract interpretation in the seminal papers of Cousot and Cousot (1977, 1979) (a choice that can be explained by the close correspondence with the abstract interpretation framework that they were describing). Another way to see this is that our fixpoint computation is building a CFG (control flow graph), one of the first passes performed in a compiler.

In tagless-final interpreters, the language can be recovered by mapping the signature to an implementation that builds terms (i.e., using a free algebra over the domain signature (PLDI'24)), and this is what we are going to do here. These terms can then be seen as the (initial) description of the control flow graph.

Terms representing arithmetic expressions

We need some code to define terms over arithmetic expressions and build them; we will be using this also in the rest of the manuscript. In particular, we will need the

terms to be hashconsed there, which requires a bit of boilerplate code (which is not very long and is put in `Appendix_term`); here I provide only the signature for the term definition and term-building code.

```

module Term: sig
  type  $\tau$  function_symbol =  $\tau$  Term.function_symbol = ..
  type  $\tau$  function_symbol +=
    | True: ['boolean] function_symbol
    | Const: Z.t  $\rightarrow$  ['integer] function_symbol
    | Not: (['boolean]  $\rightarrow$  ['boolean]) function_symbol
    | Add: (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['integer]) function_symbol
    | Sub: (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['integer]) function_symbol
    | Mul: (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['integer]) function_symbol
    | Div: (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['integer]) function_symbol
    | Eq: (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['boolean]) function_symbol
    | Leq: (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['boolean]) function_symbol
    | Unknown:  $\tau$  Typ.t  $\rightarrow$   $\tau$  function_symbol (* Fresh value, e.g. read from
      input. *)

  type  $\tau$  t =  $\tau$  Term.t =
    | Ar0: {id:int; symb: $\tau$  function_symbol}  $\rightarrow$   $\tau$  t
    | Ar1: {id:int; symb:( $\tau \rightarrow \tau_2$ ) function_symbol; arg: $\tau$  t}  $\rightarrow \tau_2$  t
    | Ar2: {id:int; symb:( $\tau \rightarrow \tau_2 \rightarrow \tau_3$ ) function_symbol;
      arg1: $\tau$  t; arg2: $\tau_2$  t}  $\rightarrow \tau_3$  t

  val phys_equal:  $\tau$  t  $\rightarrow \tau_2$  t  $\rightarrow$  bool
  val typeof:  $\tau$  t  $\rightarrow$   $\tau$  Typ.t
  val to_int:  $\tau$  t  $\rightarrow$  int
end = Term

module Build_Integer_And_Boolean_Expressions:
  INTEGER_AND_BOOLEAN_EXPRESSIONS with type  $\tau$  expr =  $\tau$  Term.t
  = Build_Integer_And_Boolean_Expressions

```

We have defined function symbols as an extensible variant type so that it is easy to add new function symbols. These function symbols correspond to those appearing in the `INTEGER_AND_BOOLEAN_EXPRESSIONS` interface. We use a GADT to provide the specific type of each function symbol.

The terms τ t consist in a unique identifier id; a function_symbol; and a number of arguments that depend on the arity. They are implemented as a variant type, where each variant represents a different arity of the function symbol. We provide different utility functions for terms. `phys_equal` is a polymorphic test of physical equality. Because terms are hash-consed, this allows for a fast test of term equality. `typeof` returns the type of a term. `to_int` returns the id field in each term; we use these ids as indices in maps.

Finally, `Build_Integer_And_Boolean_Expressions` is a module where each function is a term builder (e.g., `add e1 e2` creates the term `Add(e1,e2)`), and obeys the `INTEGER_AND_BOOLEAN_EXPRESSIONS` signature that we saw in Chapter 2.

ABSTRACT_DOMAIN terms

We begin by extending our terms so that they can refer to program variables.

```

module Term_With_ProgVar = struct
  open Term
  type  $\tau$  Term.function_symbol +=
    | ProgVar:  $\tau$  ProgVar.t  $\rightarrow$   $\tau$  function_symbol
end

```

We then complement these terms with terms representing the CFG of statements:

```

module Cfg = struct
  type node =
    | Entry: node
    | Join: node * node  $\rightarrow$  node
    | Loop_head: {loop_id:loop_id;mutable def:node}  $\rightarrow$  node
    | Assume: ['boolean] Term.t * node  $\rightarrow$  node
    | Assign:  $\tau$  ProgVar.t *  $\tau$  Term.t * node  $\rightarrow$  node
end

```

All the constructors here match the functions in the `ABSTRACT_DOMAIN` signature, with the exception of `fixpoint_step`, which has been replaced by `Loop_head`. Intuitively, a `Loop_head{loop_id;def}` constructor can be seen as labeling the `def` instruction with label `loop_id`, that can then be used as a "goto" target. More formally, the `node` type should be seen as a cyclic term graph (Ariola and Klop, 1996). `Loop_head` corresponds to recursion variables in the cyclic term graph, used to break cycles by introducing names. The `loop_id` field is used as the variable name (uniquely identifying the variable/loop head), while `def` holds the node definition (to which it unfolds). This field is mutable so that we can represent cycle in the CFG directly by a cyclic node structure (built in two passes), instead of breaking the cycle using the `loop_id` name and a map from `loop_id` to its node. The `loop_id` field is thus no longer useful once the graph has been constructed, as the addresses of the `nodes` are used to identify nodes.

Abstract domain building terms over the `ABSTRACT_DOMAIN` signature

With these definitions, we can provide now the complete implementation for our CFG-building abstract domain:

```

module Generic_Build_Integer_And_Boolean_Expressions = struct
  module Expr = Build_Integer_And_Boolean_Expressions
  module type EXPR_TYPE = module type of Expr
  include Expr
end

module Build_CFG:ABSTRACT_DOMAIN
  with type state = Cfg.node
  and module Expr = Generic_Build_Integer_And_Boolean_Expressions =
  struct
    module Expr = Generic_Build_Integer_And_Boolean_Expressions

    open Term;; open Term_With_ProgVar;; open Cfg;;

    let var vi =
      let id = !term_id_count in
      incr term_id_count; Ar0{id;symb=ProgVar vi}
  end

```

```

type state = node
let entry_state = Entry
let join a b = Join(a,b)
let assume cond node = Some(Assume(cond,node))
let assign var expr node = Some(Assign(var,expr,node))
let fixpoint_step loop_id prev next = match prev with
| Loop_head lh when lh.loop_id = loop_id →
  lh.def ← next; (prev,true)
| _ → Loop_head{loop_id;def=next},false
end

```

Most of this code is straightforward. The module `Generic_Build_Integer_And_Boolean_Expressions` complies with the `Generic_Expressions_With_Boolean` signature (which is independent of the list of function symbols and types). The `var` function is the missing term builder, as `ProgVar` is not part of `Term` (`term_id_count` is the reference used to provide fresh variable ids). The code interprets all `ABSTRACT_DOMAIN` operations symbolically, creating the corresponding terms. The only significant part here is `fixpoint_step`, the combined widening/inclusion testing operation. The idea of joining terms is to create a fresh variable to represent the generalized term. However, fresh variables would not allow the analysis to converge; so instead we use a "deterministic" name, provided here by `loop_id` (in previous papers, we reused the name of control locations of the control flow graph used as inputs). This is what is done in `fixpoint_step`: the operation returns a term of the form `Loop_head{loop_id}`; if this term already exists, then we remember its definition which is provided by `next`.

In [POPL'23](#), I joined terms by detecting if the two terms are different; if so we replace each term by a variable, whose name derives from an additional identifier (like `loop_id`) passed as an argument. This meant that variable creation was done during join, but also that we had to create variables for each join, unlike the join operation here. We changed that in [PLDI'24](#) where the join creates a term like here, and only the widening operation renames "join terms" into variables to ensure convergence.

From terms to CFG

Values of type `Cfg.node`, which describe cyclic term graphs, can easily be translated as graphs: each term represents a node, and the subterm relation corresponds to the `predecessors` relations in the graph (here a function). Indeed, as we build our CFG in a forward analysis, the CFG is built backward, with newer nodes pointing to previous ones (for instance, `Assign(x, e, p)` represents a program `p`, followed by the statement `x := e`).

```

let predecessors = let open Cfg in function
| Entry → []
| Loop_head{def;_} → [def]
| Join(a,b) → [a;b]
| Assume(_,node) → [node]
| Assign(_,_,node) → [node]

```

There are other ways to translate the `node` terms in a graph : for instance, in [PLDI'24](#) we carried operations on the edges, and we considered some terms as equivalent, which was necessary to prove graph isomorphism between the source flowchart

language and its target compilation. The above translation has the benefit of simplicity, but has more nodes (and some transitions do nothing).

Soundness and exactness

We can show that the `Build_CFG` symbolic abstract domains is a sound and exact abstraction of finite traces. For this, we need to define a concretization $\gamma : \text{Cfg.node} \rightarrow \mathcal{P}(\mathbb{S}^*)$ as a set of finite traces. We can define γ recursively, as the smallest solution to the following equations:

Definition 3.3.1: Concretization of `node`

$$\begin{aligned} \gamma(\text{Entry}) &= \{\langle s \rangle \mid s \in (\tau \text{ progv} \rightarrow \mathbb{V}_\tau)\} \\ \gamma(\text{Join}(n_1, n_2)) &= \gamma(n_1) \cup \gamma(n_2) \\ \gamma(\text{Assume}(e, n)) &= \{t \in \gamma(s^\#) \mid \text{eval}(e, \text{head}(t)) = \text{true}\} \\ \gamma(\text{Assign}(x, e, n)) &= \{s[x \leftarrow \text{eval}(e, s)] \mid t \in \gamma(s^\#), s = \text{head}(t)\} \\ \gamma(\text{Loop_head}\{\text{loop_id}, \text{def} = n\}) &= \gamma(n) \end{aligned}$$

where $\text{eval} : \mathbb{E} \times \mathbb{S} \rightarrow \mathbb{V}$ interprets expressions in a given state.

In other words, this concretization γ that we have just defined, corresponds to the definition of a collecting semantics of traces using a set of recursive equations starting from the program control flow graph (Miné, 2017, §3.4). The set of traces for a given `node` correspond to the least fixpoint of these equations. Proving soundness and exactness of the abstract domain is easy, given the correspondence between the above definition of γ , the soundness/exactness requirements for the transfer functions in the abstract domain, and the correspondence between transfer functions and term constructors.

The only non-trivial part is the soundness of `fixpoint_step`. The main idea is to ignore feedback edges (that go back to the loop) while the fixpoint is not reached (POPL'23). Note that our implementation of `fixpoint_step` is not exact during the fixpoint computation (the field `def` contains an incorrect value), and is only sound and exact once the fixpoint is reached.

3.4 Summary & conclusion

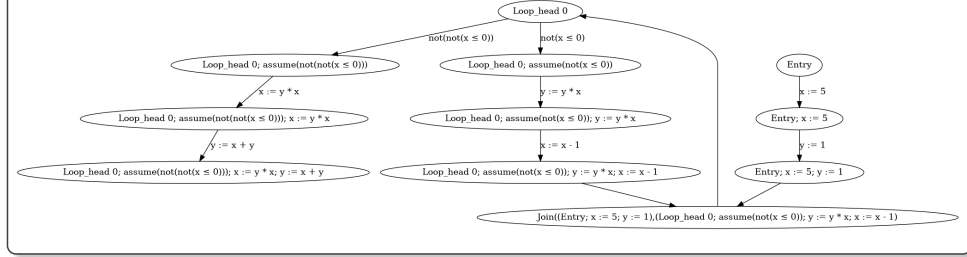
We can now put everything together, translating our `WHILE` family of languages to a control flow graph by forward abstract interpretation.

```
module Example_CFG = Example(Fixpoint(Build_CFG));;
let final_state:Cfg.node option = Example_CFG.factorial5 Build_CFG.entry_state;;
```

With some boilerplate code (Chapter C) generating a Graphviz graph description from the term, we obtain the following control flow graph for the `factorial5` program. To be precise, this control flow graph only consists of the traces that reach the exit of `factorial5`; if there were infinite loops in that program, we would not see them in this graph.

```
#include "cfg2dot.ml"
CFG2Dot.dump_graph "generated/factorial5cfg.dot" final_state
```

Figure 3.4.1: CFG representing program traces reaching the end of `factorial5`.



To summarize, we have interpreted the standard `ABSTRACT_DOMAIN` interface as the syntax of a flowchart language. By using an abstract domain of terms of the `ABSTRACT_DOMAIN` language, we have transformed `Fixpoint` as a translation from the `While_Lang_Generic` language to the `ABSTRACT_DOMAIN` language. This operation corresponds to the generation of a control-flow graph in traditional compilers. By contrast with usual compilers where passes are done in sequence, here the translations are simultaneous: the translation done by `Fixpoint` is done simultaneously with other translations, such as the one done by `Desugar`. In the other chapter, we describe other translations that are also done simultaneously, without having to "build" the intermediate representations.

One interest of this program translation technique is the proof, which is purely semantic. In short, if we have an abstract domain $\mathbb{S}^\#$ that concretizes into a set of finite traces, and if the operations on this domain are sound and exact, then the result $s^\#$ of the analysis is sound and exact, and is thus an object that preserves the traces. If it also happens that $s^\#$ can be made executable, then $s^\#$ is the result of a semantics-preserving compilation.

The construction is not only theoretical, but also practical. It is probably difficult to perform control-flow generation in a more concise way than what was done here. It is quite efficient, as each fixpoint computation terminates in two iterations (a pessimistic fixpoint algorithm would allow computing the fixpoint in a single pass). It is easy to change the definition of the transfer functions to perform more simplifications on the fly (e.g., rewrite `(not (not x))` as `x` is our example), and these simplifications can benefit from a simultaneous (Cousot and Cousot, 1979) numerical analysis for constant propagation, deadcode eliminations, etc. Simultaneously, numerical analyses can benefit from simultaneous program transformation (Lerner et al., 2002).

3.5 Related work and future directions

Graph-based methods to retrieve a CFG in machine code

The method presented in this chapter uses a flow-sensitive term graph as a control flow abstraction. This abstraction represents, for a given program point, the set of traces leading to this program point.

We can also use abstract interpretation to directly construct a single graph (Kinder et al., 2009; Bardin et al., 2011), a flow-insensitive abstraction representing the set of all the traces. When we can access the names for all the control locations (e.g. program addresses, with their callstack, in machine code; name of labels in C): the constructed CFG is the subset of reachable control locations and edges inside the

complete graph of all the possible control locations. If it contains sufficient information, such a retrieved graph can also contain a sound and exact abstraction of the program (as in [POPL'23](#)).

We use such an abstraction in the Codex binary code analyser ([RTAS'21](#); [Nicole, 2022](#), §7.2.2). In binary code analysis, the control flow graph cannot be approximated using syntactic methods, and a semantic analysis is necessary. In turn, the CFG is required for the fixpoint computation, e.g. for the placement of widening points, so we need to perform both dataflow analysis and control-flow analysis simultaneously. We came up with several interesting techniques, such as using trace abstraction for finding delimitation of function boundaries (needed to define what is a callstack), or alternating regular analysis with "exploration" phases (analysing instructions using any abstract state while we do not return to a known address) to speed up fixpoint computation. A method that could be more efficient in this case would be to use a dynamic fixpoint solver, such as the top-down solver ([Stade et al., 2024](#)).

Preservation of all the traces

The concrete semantics that we have used only accounts for the finite traces that reach the exit, and our fixpoint algorithm only attempts to preserve those. Thus, even our sound and exact (i.e., maximally precise) abstraction cannot account for the other traces in the system. In particular, traces that abruptly stops execution (e.g., because of a division by zero) or enter an infinite loop are not represented by the resulting term.

This has some benefits: our while-loop-suppression optimization would not be legitimate if we wanted to preserve all the traces, while it is correct if we only want to preserve finite traces or the denotation (input-output relation) of the program. It is often the case that we are interested only by the result of a computation, and the optimization is interesting and useful in this case. One can view the result of our analysis as performing program slicing ([Weiser, 1984](#)), removing the parts of the execution that do not contribute to the traces that reach the exit.

However, there are also many cases where we are interested in preserving all the traces, including the infinite traces, for instance in embedded systems or operating systems. This is why ([Dockins, 2012](#)) the COMPCERT ([Leroy, 2006, 2009](#)) compiler has shifted from preservation of finite traces to preservation of infinite ones.

The compiling-with-abstract-interpretation approach should also be usable when preservation of all the traces is required. First, preservation of all the finite traces (including those that do not reach the exit) can be obtained by replacing the term abstraction of type `node` by a graph or infinite tree ([Xia et al., 2020](#)), or by a mapping from all the reachable states of the program to a term of type `node`, as in [PLDI'24](#). To account for all infinite traces, we can use the property that under some conditions (each state only has a finite number of successors), finite trace equivalence implies infinite trace equivalence ([Rot, 2018](#); [Bonsangue et al., 2014](#)). Thus, a sound and exact translation that preserves all the finite traces also preserve all the infinite ones.

Other methods for preserving all the traces are probably possible, such as concretizing abstract states to finite and infinite traces; these would be interesting directions to explore in future work.

Proving trace preservation

In this chapter, I have insisted on a purely semantic proof method for compiler correctness based on abstract interpretation. Its benefits are that the proof automatically derives from the soundness and exactness of the transfer functions. This is different from the method we used in other works: in [POPL'23](#); [PLDI'24](#) we reasoned directly on the abstraction to show its bisimulation with the source, after having defined an operational semantics for the source and target abstractions. The drawback is that the proof of trace-preservation was tied to the specific term and source program that we use, the benefits were that it also accounted for infinite traces.

Observe that the concrete semantics that I used in this paper abstracts away the control flow (the control locations do not appear in the trace, as what is abstracted is only the sequence of stores). Using this abstraction, it would be possible to integrate in the fixpoint computation transformations that modify the control flow, such as function inlining and loop unrolling. However, this is not abstract enough to enable other transformations, like code hoisting or removal of dead assignments. To integrate such transformations, we should preserve only further abstractions of the traces. For instance, hoisting or removing dead assignments to local variables can be done if we do not need to preserve assignment to these local variables. Even more transformations can be done if concretize to a more abstract representation (in the sense of the hierarchy of program semantics ([Cousot, 2002](#))), such as transformations (i.e., input/output relation, or denotational semantics); for instance in this case, it is possible to change the complexity of the original algorithm.

In compiler proof methods based on simulation, such considerations corresponds to changing the notion of trace equivalence ([Abate et al., 2021](#)); here the equivalence relation is preservation of an abstraction of the trace. A specific difficulty is that if the abstraction allows steps to stutter, then preservation of infinite traces may be lost ([Leroy, 2009](#)).

In general, there seems to be strong relations between the soundness and exactness of the `assign` and `assume` function and the proof by simulation diagrams, that would be interesting to explore. Some connections were done in [PLDI'24](#), where we proved that the soundness (resp. γ -completeness) of transformation functors (than only modify the definition of `assign` and `assume` functions) can be transformed into forward (resp. backward) simulation proofs, and moreover, that the soundness of these functors can be established by reasoning only over the state collecting semantics. In other words, this lifts a semantic proof over sets of states to a simulation proof.

Control-flow transformations

I distinguish two kinds of control flow transformations. *Local* transformations inspects and modifies `assign` or `assume` statements, or the expressions they contain, which can affects the control flow. Instances of such local transformations are transformation of an `if` expression to an `if` statement, or adding runtime assertions (and alarms) to detect undefined behaviours on overflow, division by zero etc. The natural place for such transformations is as functors over the `ABSTRACT_DOMAIN` interface ([PLDI'24](#)). Interestingly, we can also prove the soundness and exactness of these transformations using purely semantic methods: we proposed to lift the proof of functor domains when operating on sets of states to a simulation proof when the functor is applied to `Build_CFG`. Local transformations allow lowering a complex

language like C to a simpler one with fewer constructs, emitting assertions corresponding to undefined behaviours along the way; they are very useful and can be also seen, for instance, in MOPSA (Journault et al., 2019). More generally, local transformations have been proposed as a natural way to combine information in compiler passes (Lerner et al., 2002).

By contrast, the `Fixpoint` functor is a natural place to perform more *global* control flow transformations, such as function inlining, loop unrolling etc. Indeed, these control-flow transformations can be naturally expressed (and proved correct) using trace partitioning (Rival and Mauborgne, 2007). Trace partitioning is a practical method to enhance the precision of a static analyser. One of the most useful techniques is loop unrolling, which consists in discovering if there is a constant bound n for a loop which is not too large, and if so analyse the loop body up to n times instead of doing a fixpoint computation. Symbolic methods like those described in this manuscript are useful to find the existence of such loop bounds: we can sometimes replace a symbolic inductive variable by a closed-form term depending on the number of loop iterations, and lift constraints on the terms to retrieve constraints on the number of iterations.

My PhD student Charles Babu has conducted an investigation (SAS’24) on systematic methods for improving the precision of an analysis using trace partitioning domain. Separating program traces in a program graph is a compromise between the size of the graph and the precision. We devised a method that alternates searches for precision improvement and phases that reduce the graph size while maintaining the same precision and studied its properties. This method is automatic, works for different kinds of control-flow modifications (loop unrolling, delaying joins, context-sensitivity...) and is a practical and generic way to improve the precision of an analysis.

Control-flow analysis

It is easy to integrate some control-flow analysis (Muchnick, 1997; Allen, 1970) in the symbolic translation of `Build_CFG`, which allows for simple algorithms. For instance, if we extend `Cfg` with a field containing the immediate dominator for the `Join` and `Loop_head` nodes, then the output is not only the program CFG, but also its dominator tree (their combination corresponds to a DJ-graph Sreedhar (1995)). Computing dominator trees using dataflow analysis has already been proposed by Cooper et al. (2001), noting in particular the simplicity of the resulting framework. One source of inefficiency in their algorithm is the way they compute the dominator between two nodes – essentially a nearest-common ancestor algorithm, using indices to know which branch we should climb. The same can be performed by maintaining the depth of the nodes in the node data structure. I found a simple and efficient way to improve the nearest-common ancestor query, by having each node of depth d pointing to its 2^k predecessor if d is a multiple of 2^d (i.e. we have a kind of skiplist). The complexity then matches that of Tsakalidis (1988), allowing to answer nearest-common ancestor queries in time logarithmic in the height of the tree, which is sufficiently fast in practice. More efficient algorithms now exist for this nearest-common-ancestor with interleaved addition of leaves problem (Gabow, 1990; Alstrup and Thorup, 2000; Cole and Hariharan, 2005), but they are significantly more complex; moreover our algorithm does not require changing the parent nodes or an external table, making it a better fit for OCaml. Note that the use of classical algorithms like Lengauer and Tarjan (1979) seems not easy, because they re-

quire several linear passes over all the nodes in different orders, and cannot thus be used when the nodes are incrementally built like in our settings.

One application of this is debugging the SSA translation of Chapter 4, to assert that the dominance property of SSA (Rastello and Bouchez Tichadou, 2022) is preserved, or to compute the scope (POPL’23) of symbolic expressions. To do this, the node data structure just has to include pre- and post-order ordering for answering whether a node dominates another in constant time (Dietz, 1982).

Another application is to perform interval and structural analyses (Allen, 1970; Sharir, 1980; Muchnick, 1997), which partition arbitrary control-flow graphs into higher-level control-flow structures. Such analyses can be used to optimize dataflow computations or to make low-level code more readable (Brumley et al., 2013); we have preliminary ideas and experiments on this subject that indicates that implementing such algorithms as an abstract interpretation leads to simple yet efficient algorithms.

Applications of program transformation by abstract interpretation

I am convinced that program transformation by abstract interpretation can be useful in a variety of contexts. I currently have three main applications in mind.

The first is program simplifications before further analyses. For instance, the Frama-C platform has an RTE plugin whose role is to generate proof obligation for each operation where a run-time error (RTE, i.e., undefined behaviour in C) may occur. To avoid generating a large quantity of proof obligations which are easily discharged, such a plugin could be written using a basic abstract interpretation paired with a module like `Build_CFG`, that would only emit the proof obligations that it cannot directly prove. In analysis of machine code, a language whose constructs perform many useless operations, like flag updates, doing such a preprocessing pass using abstract interpretation could be used to simplify the control flow graph, unroll finite loops etc., which would speed up a following more expensive analysis like symbolic execution.

Going further, the technique could be applied to situations where retrieving the control-flow graph is not trivial. I think we could use generation of CFG by abstract interpretation to undo some obfuscation techniques, like virtualization (Salwan et al., 2018), control flow flattening (Blazy and Trieu, 2016), or insertion of opaque predicates (Bardin et al., 2017). Abstract interpretation can help against weaknesses of symbolic execution in this area, like path explosion (Ollivier et al., 2019); it can also find the program invariants on which an obfuscation relies, to prove that some code is unreachable (PLDI’24, Fig. 2). Finally, we could use the technique for sound decompilation (Cifuentes and Gough, 1995) e.g. automated transformation from machine code to C, or sound binary lifting (Park et al., 2025).

A final interesting direction is model extraction. Instead of producing a using a sound and exact abstraction of the program traces, we can produce abstractions of some aspects of the trace. For instance, it would be easy to use our method on a program that calls `print` on constant strings, to transform this into a nondeterministic finite automaton, that characterizes the strings that a program can output. Another application would be the extraction of time-constrained automaton (ICE’10), an abstraction of real-time tasks that I have proposed when I was researching real-time systems. In these systems, schedulability analysis (the assurance that every task reaches its deadline) is important, and I am convinced that abstract interpretation is a very useful tool in that area. I think we can find many other applications to this idea,

like extracting a network protocol from a code implementing it, extracting some UML diagrams from code, etc.

Chapter 4

Symbolic expression analysis, SSA Abstract Domains, and SSA translation

S WE SAW in Chapter 3, *Fixpoint* computation corresponds to a translation that decomposes compound WHILE statements into simpler elementary statements. This allows for a modular design of the analyser, where abstract domains need only implement the simpler transfer functions, and the decomposing translation (the *Fixpoint* functor) is shared among different possible implementations of abstract domains. This decomposing translation does not lose any precision in the concrete semantics, as the source and target programs correspond to the same traces.

Using SSA translation (Cytron et al., 1991) in an abstract interpreter (POPL’23; PLDI’24) can be seen as pushing this idea further, allowing further decomposition of statements into elementary expressions while still preserving the semantics. SSA translation deals with the notion of reading and assigning program variables, the decomposition of nested expressions into atomic arguments, the choice of an ordering for the evaluation of expressions, etc., so that SSA-based abstract domains can focus on the precise analysis of elementary operations (like addition or multiplication). In particular, SSA-based abstract domains do not have to handle side effects such as killing or assignment, which change the program relations and can be hard to analyse (as seen in Section 1.3).

Note that the fact that SSA translation is a decomposition that preserves the concrete semantics does not mean that it also preserves the abstract semantics. Indeed, decomposition may introduce a loss in precision: for a given abstract domain, the best abstraction of $f \circ g$ is in general more precise than the composition of the best abstraction of f and the best abstraction of g . This is what makes, for instance, the analysis of bytecode harder than the analysis of source code (Logozzo and Fähndrich, 2008). However, the symbolic (or free-algebra (PLDI’24)) SSA Abstract Domain is sound and exact, allowing the precise representation of the code to be available to the other domains (it also allows using the result of this analysis as the output of a semantics-preserving compiler (POPL’23)). This allows an SSA-based abstract domain to recover from this loss of precision, to the point that its precision can exceed that of a standard numerical analysis; for instance, in PLDI’24, we show that SSA-

based non-relational domains are always more precise than standard non-relational domains, and that SSA-based abstract interpretation can recover from the precision loss induced by byte-compilation. We will cover these aspects in Chapter 5.

In [POPL'23](#), we showed that many compiler analyses and transformations can be seen as a continuum of symbolic value-based analyses: the most precise is SSA translation, and techniques such as global value numbering ([Alpern et al., 1988](#); [Rosen et al., 1988](#)), or construction of the global value graph ([Reif and Lewis, 1986](#)) can be viewed as computing only a part of the information of this SSA translation. Following this idea, this chapter is organised in two parts. The first (Section 4.1) is a minimal symbolic expression analysis ([POPL'23](#)), which maps program variables to elementary symbolic expressions (implemented with the `Term` module) which represent their values. This analysis is similar to symbolic execution ([King, 1976](#)) (but over all program traces), or to term-based value numbering implementations¹. A core operation of this analysis is a form of *anti-unification/generalization* ([Cerna and Kutsia, 2023](#)), which generalizes two terms by introducing a new variable, and corresponds to the ϕ operation in SSA. This operation introduces a loss of precision in the symbolic expression analysis.

anti-unification

generalization

This loss of precision is recovered in the second part (Section 4.2) by adding control-flow information to the symbolic expressions, resulting in an SSA translation. While the symbolic expression analysis of the first part only serves as an introduction, the second one is more complete and proposes several further improvements. First, we extract a generic `SSA_Translation` functor from the symbolic expression analysis, allowing a variety of `VALUE_BASED_DOMAINS` which are not all building symbolic terms. Second, the new symbolic expression analysis is parameterized by an `ABSTRACT_ENVIRONMENT`, which serves as the basis for our numerical domains in Chapter 5. Finally, this symbolic expression analysis integrates an optimistic form of Global Value Numbering ([Alpern et al., 1988](#)) (giving the same name to variables that are equal) as part of the analysis.

Even though adding control state recovers the precision lost in the symbolic expression analysis, the latter remains valuable. The analysis that we develop in this chapter has several concretizations, ranging from imprecise but readily-available information (such as "the value of x is positive"), to precise but latent information (such as "here is a semantically-equivalent version of the program"). SSA-based abstract interpretation allows interesting combinations of these simultaneous forms of information about the program.

Comparison with other known SSA construction algorithms To summarize, our method for SSA construction is based on a dataflow analysis. In particular, we detect where we need to insert ϕ nodes in the join operation, by comparing the stores to find program variables that are mapped to different symbolic expressions. The algorithm is quite different from classical ones ([Cytron et al., 1991](#); ?) as in particular we do not need to compute the dominator tree or dominance frontier (see [POPL'23](#) for a full comparison). However, it is close to the "parse-time optimization" technique for fast SSA generation of [Click \(1995\)](#) that I have discovered recently. Interestingly, recasting this algorithm as an abstract interpretation allows to better understand why this algorithm works.

¹as found, for instance, in CompCert: <https://compcert.org/doc/html/compcert.backend.CSE.html>.

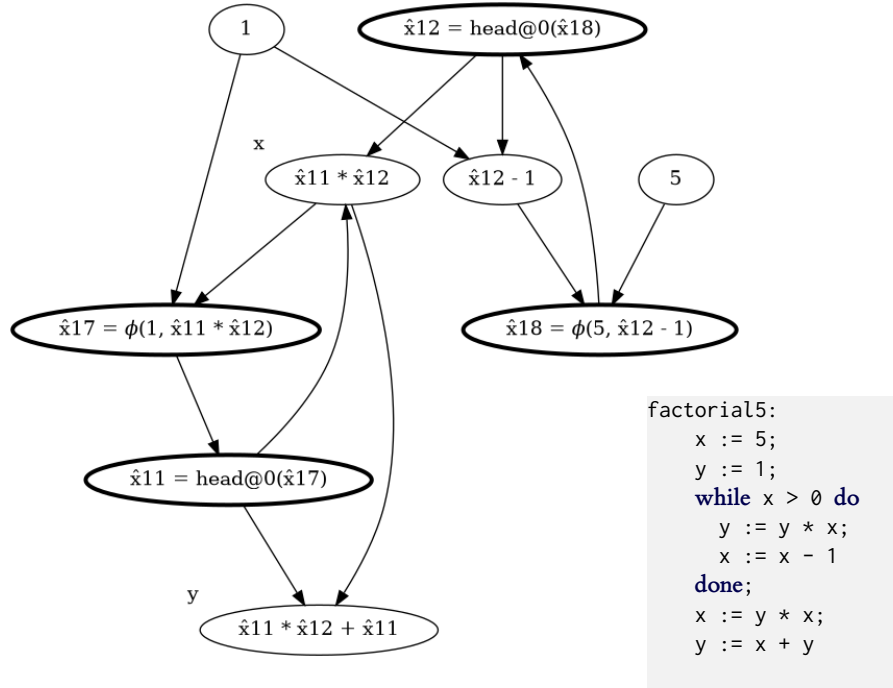


Figure 4.1: Result of symbolic expression analysis on factorial5. Nodes represent symbolic expressions, and bold nodes are symbolic variables. The arrows represents the flow of data, either because a symbolic expression is a subterm of another, or the definition of a symbolic variable depends on an expression. The x and y labels represent the symbolic store at the end of the program, mapping program variables to their symbolic expression.

4.1 Symbolic expression analysis

The goal of this section is to describe the [Symbolic_Expression_Domain](#), an `ABSTRACT_DOMAIN` whose result on factorial5 yields Figure 4.1.

This figure represents a view of the dataflow of the program called the *global value graph* (Reif and Lewis, 1986). The figure can be read as follows: at the end of the analysis, the program variable y is mapped to the `Term` (or *symbolic expression* $\hat{e} \in \hat{\mathbb{E}}$) $\hat{x}_{11} * \hat{x}_{12} + \hat{x}_{11}$, while x is mapped to $\hat{x}_{11} * \hat{x}_{12}$. The main structure for this analysis is indeed a *symbolic store* $\hat{\sigma} \in \hat{\Sigma} \triangleq \mathbb{X} \rightarrow \hat{\mathbb{E}}$, a partial mapping from the program variables that have been assigned to symbolic expressions, and the resulting symbolic store at the end of the analysis is $\hat{\sigma}_{\text{end}} = [x \mapsto \hat{x}_{11} * \hat{x}_{12}, y \mapsto (\hat{x}_{11} * \hat{x}_{12}) + \hat{x}_{11}]$. Inside these symbolic expressions, $\hat{x}_{11}$ and $\hat{x}_{12}$ are *symbolic variables* $\hat{x} \in \hat{\mathbb{X}}$, i.e., specially-named terms. Indeed, the global value graph is an example of a *cyclic term graph* (Ariola and Klop, 1996), i.e., a maximally-shared term where we name some terms to break the cycles (there must be at least one variable in each cycle of the graph). The way we create symbolic variables ensure this property, as we create a symbolic variable at each widening point (and widening points are used to break cycles in the control flow graph). Here, both $\hat{x}_{11}$ and $\hat{x}_{12}$ are variables created at the head of a loop identified with `id 0`, which is what the `@0` means. In the formalization, I will use the symbol $\hat{x} \in \hat{\mathbb{X}}$ to represent symbolic variables.

global value graph

$\hat{e} \in \hat{\mathbb{E}} : \text{symbolic expression}$

$\hat{\sigma} \in \hat{\Sigma} : \text{symbolic store}$

$\hat{x} \in \hat{\mathbb{X}} : \text{symbolic variable}$

cyclic term graph

The terms $\hat{x}_{17}$ and $\hat{x}_{18}$ also are symbolic variables. Intuitively, when joining two symbolic stores, one where x is bound to $\hat{e}$ and one where it is bound to $\hat{e}'$, the resulting symbolic store maps x to a non-deterministic choice between $\hat{e}$ and $\hat{e}'$ written $\phi(\hat{e}, \hat{e}')$ (where ϕ corresponds to the "phoney" (Rastello and Bouchez Tichadou, 2022) function of SSA). The symbolic variable allows representing the result of this non-deterministic choice. A more precise way to see this operation is that we are performing an anti-unification of two terms that are generalized by creating a new symbolic variable.

Example 4.1.1

When the fixpoint is reached, at the end of the loop, the symbolic expression analysis joins the symbolic store $\hat{\sigma} = [x \mapsto 5; y \mapsto 1]$, corresponding to the state at the beginning of the loop, with the symbolic store $\hat{\sigma}' = [x \mapsto \hat{x}_{12} - 1; y \mapsto \hat{x}_{11} * \hat{x}_{12}]$ which is the state at the end of the loop body.

The algorithm needs to:

1. Find all the program variables that are mapped to different symbolic expressions, in our example, x and y . Indeed we have $\hat{\sigma}[x] \neq \hat{\sigma}'[x]$ and $\hat{\sigma}[y] \neq \hat{\sigma}'[y]$. If we had in our example, a program variable z that would not be modified inside the loop, then, on the contrary, we would have $\hat{\sigma}[z] = \hat{\sigma}'[z]$.
2. Get the set of pairs of symbolic expressions that have to be anti-unified. In our example, it is $\{(5, \hat{x}_{12} - 1), (1, \hat{x}_{11} * \hat{x}_{12})\}$.
3. Use this set as the domain $\text{dom } \phi$ of a partial function ϕ that maps each different pair of symbolic expression to a fresh variable:

$$\phi = \{(5, \hat{x}_{12} - 1) \mapsto \hat{x}_{18}, (1, \hat{x}_{11} * \hat{x}_{12}) \mapsto \hat{x}_{17}\}$$

. We can also record the identities $\hat{x}_{18} = \phi((5, \hat{x}_{12} - 1))$ and $\hat{x}_{17} = \phi((1, \hat{x}_{11} * \hat{x}_{12}))$ in the global value graph.

4. Use ϕ to rebuild a new symbolic store $\hat{\sigma}''$. If we see both $\hat{\sigma}$ and $\hat{\sigma}'$ as (OCaml) maps, then ϕ is the "value merge" functional argument of `Map.merge`.

The algorithm is somewhat complicated because it operates on two phases: one to scan the stores to obtain the list of differing pairs, and one to create the resulting store with the phi function. However, after Chapter 6, we will be able to express this computation algebraically as:

$$\begin{aligned} \Phi(\hat{\sigma}, \hat{\sigma}') &= [x \mapsto \Phi(\hat{\sigma}[x], \hat{\sigma}'[x]); y \mapsto \Phi(\hat{\sigma}[y], \hat{\sigma}'[y])] \\ &= [x \mapsto \Phi(5, \hat{x}_{12} - 1); y \mapsto \Phi(1, \hat{x}_{11} * \hat{x}_{12})] \\ &= [x \mapsto \hat{x}_{18}; y \mapsto \hat{x}_{17}] \end{aligned}$$

where Φ represents the anti-unification operation extended to stores.

In the figure, the variables $\hat{x}_{12}$ and $\hat{x}_{18}$ are equal (they correspond to the value of x in the loop, which is the current factor in the factorial computation), and so are $\hat{x}_{11}$ and $\hat{x}_{17}$ (which corresponds to the value of y , the product of the numbers seen so far). In other presentations (POPL'23; PLDI'24), we would have defined only one variable instead of two. Here, the distinction highlights the fact that $\hat{x}_{17}$ and $\hat{x}_{18}$

are created by the `join` operation (and represent a non-deterministic choice between terms), while $\hat{x}_{11}$ and $\hat{x}_{12}$ are created by `fixpoint_step` (and are used to allow cycles in the term graph). In the graph, $\hat{x}_{12}/\hat{x}_{18}$ is initially 5 and then recursively defined as $\hat{x}_{12} - 1$, while $\hat{x}_{11}/\hat{x}_{17}$ is initially 1 and then recursively defined as $\hat{x}_{11} * \hat{x}_{12}$.

In this section, the concretization will not even interpret ϕ as a non-deterministic choice between two terms, and head as the identity. We will give an even coarser interpretation, where ϕ and head are uninterpreted and thus name a new symbolic variable, which is bound to an arbitrary value. This interpretation more accurately represents the use of symbolic expression analysis in a state analysis (trying to prove state properties), where variables are only used to give a name to values, and we have the numerical domains to constrain the possible values of the variables. Formally, the meaning of a symbolic term $\hat{e} \in \hat{\mathbb{E}}$ is a partial function that returns the value $v \in \mathbb{V}$ of a term when given an *environment* $\rho \in \wp \triangleq \hat{\mathbb{X}} \rightarrow \mathbb{V}$, i.e., a mapping from the symbolic variables to values.

 $\rho \in \wp : \text{environment}$

$$\begin{aligned} \gamma_{\hat{\mathbb{E}}} : \hat{\mathbb{E}} &\rightarrow (\wp \rightarrow \mathbb{V}) \\ \gamma_{\hat{\mathbb{E}}}(\hat{e}) &= [\rho \in \wp \mapsto \text{eval}(\hat{e}, \rho)] \end{aligned}$$

The meaning of a symbolic store is the value of the symbolic terms it contains, when provided with an environment:

$$\begin{aligned} \gamma_{\hat{\mathbb{S}}} : \hat{\mathbb{S}} &\rightarrow (\wp \rightarrow \mathbb{S}) \\ \gamma_{\hat{\mathbb{S}}}(\hat{\sigma}) &= [\rho \in \wp \mapsto [x \in \mathbb{X} \mapsto \gamma_{\hat{\mathbb{E}}}(\hat{e})(\rho)]] \end{aligned}$$

It is the fact that all symbolic expressions in the symbolic store share their environment in the concretizations that makes this domain relational.

Example 4.1.2

The concretization for the symbolic store at the end of the analysis is:

$$\begin{aligned} \gamma([x \mapsto \hat{x}_{11} * \hat{x}_{12}, y \mapsto (\hat{x}_{11} * \hat{x}_{12}) + \hat{x}_{11}]) = \\ \{[x \mapsto \hat{x}_{11} * \hat{x}_{12}, y \mapsto (\hat{x}_{11} * \hat{x}_{12}) + \hat{x}_{11}] \mid (\hat{x}_{11}, \hat{x}_{12}) \in \mathbb{Z} \times \mathbb{Z}\} \end{aligned}$$

This example shows that the symbolic expression analysis loses precision when variables are defined: there are no constraints on $\hat{x}_{11}$ and $\hat{x}_{12}$.

4.1.1 Term_With_Symvar: extending terms with symbolic variables

We start our implementation by extending the `Term` module representing ground terms with a notion of symbolic variable, representing the result of anti-unification of two ground terms, resulting in `Term_With_Symvar`.

```
| type from_loop = loop_id option
```

During fixpoint computation, we need to find out whether a variable has been created in the loop that we are processing, in another loop, or not in a loop (e.g., by a `join` operation at the end of an `if` statement). None is used in this latter case.

```
| type control_state = ..
| type control_state += No_control_state
```

In the extension from symbolic expression analysis to SSA translation, each symbolic variable is linked to the `control_state` where the variable is created. This type can be ignored for now, it will be used in the second part of this chapter.

```
type  $\tau$  definition =
| Phi of  $\tau$  Term.t option *  $\tau$  Term.t option
| Head of  $\tau$  Term.t option
```

The subterm relationship represents how the data flows in the SSA program: the term $e_1 + e_2$ is created from the data present in the term e_1 and the data present in e_2 . This does not work for symbolic variables, that have no subterms. To fix this, we attach to symbolic variables their definition, which is either `phi(e1,e2)` (for variables created during a `join`), or `head(e)` (for variables created by a `fixpoint_step`). This information is needed to completely represent the global value graph. However, it is not needed to compute the fixpoint computation; we could retrieve it after the fixpoint is attained. Moreover, the information that it carries, about the dataflow for symbolic variables can be extracted out of the `control_state` that we define when we revisit this analysis in Section 4.2.

```
module Term_With_Symvar = struct
  open Term
  type  $\tau$  t =  $\tau$  Term.t
  let to_int = Term.to_int

  type  $\tau$  Term.function_symbol +=
  | Var: { typ: $\tau$  Typ.t; from_loop:from_loop;
          control_state:control_state;
          mutable definition: $\tau$  definition }  $\rightarrow$   $\tau$  Term.function_symbol

  let fresh_var
    typ from_loop ?(control_state=No_control_state) definition =
    let symb = Var {typ;from_loop; control_state; definition} in
    let res = Term.Ar0{id = !term_id_count; symb } in
    incr term_id_count; res
```

We extend the terms with a new `Var` constructor for symbolic variables, using the types that we have defined above. `fresh_var` is a simple function to build the term.

```
let typeof = function
| Term.Ar0{symb=Var{typ;_};_}  $\rightarrow$  typ
| t  $\rightarrow$  Term.typeof t

let polyeq (type a) (a:a t) (type b) (b:b t) : (a,b) Cmp.cmp =
  if phys_equal a b then Typ.polyeq (typeof a) (typeof b) else Diff
end
```

We extend the `Term` functions to deal with the `Var` case. The definition for `polyeq` has not changed, but we redefine it because of the new definition of `typeof` (and because OCaml modules do not support late binding).

4.1.2 Symbolic_store

```
module Symbolic_Store = Make_Store(struct type  $\tau$  t =  $\tau$  Term.t end)
```

The symbolic store is just a mapping from program variables to terms.

4.1.3 Heterogeneous_List_Of_Pair and Phi_Arguments

```
module Heterogeneous_List_Of_Pair(Elt:sig type  $\tau$  t end) = struct

  type 'list t =
    | Empty: unit t
    | Cons:  $\tau$  Elt.t *  $\tau$  Elt.t * 'rest t  $\rightarrow$  ( $\tau$  * 'rest) t

  type  $\delta$  f = {f: $\tau$ .  $\delta \rightarrow \tau$  Elt.t  $\rightarrow \tau$  Elt.t  $\rightarrow \delta$ }

  let fold_left (type acc) {f} (acc:acc) list =
    let rec loop:  $\tau$  . acc  $\rightarrow \tau$  t  $\rightarrow$  acc =
      fun (type a) acc ( $\phi$ _args:a t)  $\rightarrow$  match  $\phi$ _args with
        | Empty  $\rightarrow$  acc
        | Cons(a,b,rest)  $\rightarrow$  loop (f acc a b) rest in
    loop acc list

end

module Phi_Arguments =
  Heterogeneous_List_Of_Pair(struct type  $\tau$  t =  $\tau$  Term.t option end)
type  $\tau$   $\phi$ _arguments =  $\tau$  Phi_Arguments.t
```

The heart of the symbolic expression analysis is its anti-unification algorithm, which requires maintaining a list of pairs of differing variables to anti-unify; we use `Heterogeneous_List_Of_Pair` to fulfill this role. The result of the anti-unification, i.e. the ϕ function of SSA, is a function $\hat{\mathbb{E}}_{\perp} \times \hat{\mathbb{E}}_{\perp} \rightarrow \hat{\mathbb{E}}$ mapping each pair of differing term to a symbolic variable that anti-unifies them. In other words, the `Heterogeneous_List_Of_Pair` corresponds to the list of arguments (or domain) of the ϕ function. Note the $\perp$ subscript (where $\perp$ corresponds to the None case in option types): we will sometimes want to anti-unify a term which has no counterpart in the other state. This corresponds to undefined values in SSA-based compilers (e.g., Lee et al., 2017, Fig.2)).

A note on the implementation: the heterogeneous list of pairs uses a GADT to build a list that contains values of different types. The `fold_left` function thus simultaneously requires polymorphic recursion and use of GADT; as the type of this function cannot be inferred automatically, we need to help it with type annotations. Also note that we need a higher-rank type for the functional argument to `fold_left`, which is possible in OCaml if we enclose it inside a record.

4.1.4 Symbolic_Expression_Domain

We can now describe the `ABSTRACT_DOMAIN` performing the *symbolic expression analysis*.

symbolic expression analysis

```

module Symbolic_Expression_Domain: ABSTRACT_DOMAIN
  with type state = Symbolic_Store.t
  and type  $\tau$  Expr.expr = Symbolic_Store.t  $\rightarrow$   $\tau$  Term.t
  and module type Expr.EXPR_TYPE =
    (INTEGER_AND_BOOLEAN_EXPRESSIONS
     with type  $\tau$  expr = Symbolic_Store.t  $\rightarrow$   $\tau$  Term.t) =
  struct
    type state = Symbolic_Store.t
    module State = struct type t = state end
    module M = Reader_Monad(State)
    module Expr = struct
      module Expr =
        Apply_Monad_Over_Integer_And_Boolean_Expressions
          (Build_Integer_And_Boolean_Expressions)
            (M)
      module type EXPR_TYPE = module type of Expr
      include Expr
    end
  end
  let entry_state = Symbolic_Store.empty
  let var x state = Symbolic_Store.find x state
  let assign x e state = Some(Symbolic_Store.add x (e state) state)
  let assume _cond state = Some state

```

Most of the domain implementation amounts to performing a symbolic execution (King, 1976). In the concrete semantics, we interpret program expressions as functions from concrete stores to values (i.e. we lift program expressions using a `Reader_Monad`); this allows moving around the concrete store so that we can interpret expressions "x" as accessing the contents of x in the store. Here, we interpret program expressions as functions from symbolic stores to symbolic expressions, and we interpret "x" as the access of the symbolic store. This interpretation is performed by calling `Apply_Monad` on the term-building `Build_Integer_And_Boolean_Expressions` module using the `Reader_Monad`, which could be formalized as:

$$\begin{aligned}
\langle \cdot \rangle^\#(\cdot) : \mathbb{E} \times \hat{\Sigma} &\rightarrow \hat{\mathbb{E}} & \langle e_1 \diamond e_2 \rangle^\#(\hat{\sigma}) &= \langle e_1 \rangle^\#(\hat{\sigma}) \diamond \langle e_2 \rangle^\#(\hat{\sigma}) \\
\langle c \rangle^\#(\hat{\sigma}) &= c & \langle x \rangle^\#(\hat{\sigma}) &= \hat{\sigma}[x]
\end{aligned}$$

where $\langle \cdot \rangle^\#(\cdot)$ represents the abstract evaluation of a program expression. It is easy to show that this operation is sound and exact, i.e.,

$$\forall \rho, e, \hat{\sigma} : \gamma_{\hat{\Sigma}}(\langle e \rangle^\#(\hat{\sigma}))(\rho) = \text{eval}(e, \gamma_{\hat{\Sigma}}(\hat{\sigma})(\rho))$$

The rest of the code deals with the merging (with anti-unification) of symbolic expressions. This is an operation not performed by symbolic execution or evaluation, that analyses execution traces one at a time.

```

type  $\tau$   $\phi$ _result =
  | Empty_ $\phi$ _result: unit  $\phi$ _result
  | Cons_ $\phi$ _result:  $\tau$  Term.t *  $\tau_2$   $\phi$ _result  $\rightarrow$  ( $\tau$  *  $\tau_2$ )  $\phi$ _result

type serialization =
  Serialization:  $\tau$   $\phi$ _arguments * ( $\tau$   $\phi$ _result  $\rightarrow$  state)  $\rightarrow$  serialization

let serialize_state: state  $\rightarrow$  state  $\rightarrow$  serialization = fun s1 s2  $\rightarrow$ 

```

```

let f key v1 v2 (Serialization( $\phi$ _args,deserialize)) =
  let  $\phi$ _args = Phi_Arguments.Cons(v1,v2, $\phi$ _args) in
  let deserialize (Cons_ $\phi$ _result(v,rest)) =
    let state = deserialize rest in
    let state = Symbolic_Store.add key v state in
    state in
  Serialization( $\phi$ _args,deserialize) in
let init =
  (Serialization(Phi_Arguments.Empty,fun Empty_ $\phi$ _result  $\rightarrow$  s1)) in
Symbolic_Store.fold_on_nonequal_union {f} s1 s2 init

```

The core algorithm of the symbolic expression analysis is `serialize_state`, a one-pass algorithm that anti-unifies two symbolic stores (whose type is `state`). A call to `serialize_state` returns two things:

- A τ ϕ _arguments, i.e., a list of differing pairs of terms that must be anti-unified². For instance, if in state s_1 the program variable x is mapped to $\hat{x}_1$, and in s_2 it is mapped to $\hat{x}_2$, then `serialize_state` s_1 s_2 will add the pair $(\hat{x}_1, \hat{x}_2)$ to the phi arguments.
- A function of type τ ϕ _result $\rightarrow$ state. Conceptually, it is a function that rebuilds the new state when we have finished computing ϕ . Following our previous example, `serialize_state` will map the program variable x to $\phi(\hat{x}_1, \hat{x}_2)$ in the state that it returns. Here, the ϕ function is not represented by a map, but as a sequence of resulting symbolic values $\phi(\hat{e}_1, \hat{e}_2)$ whose index correspond to the $(\hat{e}_1, \hat{e}_2)$ pairs in the τ ϕ _arguments.

The type τ shared by both the argument and the result, representing the types of the terms in the lists, is hidden using an existential type provided by the `Serialization` constructor. This existential quantification allows `serialize_state` (and other serialization functions) to typecheck when we add a varying number of elements to the τ ϕ _arguments.

To build the list of phi arguments, we fold on the elements that are different in both symbolic stores. This is what `fold_on_nonequal_union` does: this function directly skips the subtrees that are physically equal. This is why we implement stores using Patricia Trees (which allows fast merges (Okasaki and Gill, 1998)), and why the `PatriciaTree` library maintains physical equality whenever possible. Skipping the parts of the state that are equal when merging states was noted as a very important optimization in Astrée (Blanchet et al., 2002), and I have regularly observed that this is indeed the case.

To compactly formalize the `serialize_state` operation, we extend the notion of anti-unification of terms to the notion of anti-unification of symbolic stores, defined as the anti-unification of the terms that it contains:

$$\phi_{\hat{x}}(\hat{\sigma}_1, \hat{\sigma}_2) \triangleq [x \in \mathbb{X} \rightarrow \phi(\hat{\sigma}_1[x], \hat{\sigma}_2[x])]$$

where ϕ of an equal pair of terms return that term, and ϕ over different terms returns the variable that anti-unifies them. This formalization expresses both how to build the domain of ϕ from the definition of $\phi_{\hat{x}}$, and how to define $\phi_{\hat{x}}$ from ϕ , and

²The name `serialize` comes from the fact that the function is producing a list.

represents thus the two phases of our algorithm. We will see more examples of such formalizations in Chapter 6³.

```

type f = {f:τ. τ Term.t option → τ Term.t option → τ Term.t};;
let rec map_args_to_result: τ. τ φ_arguments → f → τ φ_result =
  fun (type a) (args:a φ_arguments) {f}: a φ_result → match args with
  | Phi_Arguments.Empty → Empty_φ_result
  | Phi_Arguments.Cons(a,b,rest) →
    let res = f a b in
    Cons_φ_result(res,map_args_to_result rest {f})

let join sa sb =
  let Serialization(φ_args,deserialize) = serialize_state sa sb in
  let φ_res = map_args_to_result φ_args {f=fun a b → match a,b with
    | None, None → assert false
    | Some a, Some b when Term.phys_equal a b → a
    | Some x, _ | _, Some x →
      let typ = Term_With_Symvar.typeof x in
      Term_With_Symvar.fresh_var typ None (Phi(a,b)) } in
  deserialize φ_res

```

Joining abstract states (which only contain symbolic stores) amounts to:

1. Finding the differing pairs of terms in each symbolic store (serialize_state);
2. Mapping each differing pair of terms to a fresh variable of the same type (map_args_to_result);
3. Rebuilding a symbolic store with the new variables (deserialize).

This scheme is ubiquitous in CODEX and in this manuscript, and can be applied to many different datastructures instead of just stores; we return to this in Chapter 6.

We did not yet detail step 2., which computes the $\phi \in \hat{\mathbb{E}}_{\perp} \times \hat{\mathbb{E}}_{\perp} \rightarrow \hat{\mathbb{E}}$ function⁴. In the analysis that we build in this chapter, a term always represents a value; thus the case where both arguments to ϕ are $\perp$ (None) should not happen, as then the fresh variable would represent no value. In analysers that can represent option types (e.g., Liu and Rival (2015b)); we will also see an example of this in Chapter 6), this operation would be allowed.

The case where we test equality between terms is not needed here because PatriciaTree.fold_on_nonequal_union already skips them, but would be useful if we used another data structures (such as Map.merge from the OCaml standard library).

Finally, the last case (differing terms) just create a fresh variable. Note that this does not compute the most precise anti-unification: for instance we would return a fresh variable for $\phi(2 * e_1, 2 * e_2)$ instead of transforming this to $2 * \phi(e_1, e_2)$. Some imprecision in the symbolic expression analysis is not very important as we will complement it with numerical abstraction providing numerical information about the symbolic variables in Chapter 5. Adapting the algorithm to compute more precise anti-unification should be possible, but is not easy (Gulwani and Necula, 2004); for instance anti-unifying $\phi(2 + 3, 4 + 1)$ as $\phi(2, 4) + \phi(3, 1)$ would be a bad idea. This is an interesting topic to explore in the future.

³One issue is the fact that ϕ is not a function in this initial implementation, which is solved in Section 4.2, so let us assume that it is the case here.

⁴It is actually not a function because the same pair of terms can be mapped to different symbolic variables; we fix this later in this chapter.

```

module TermSet = PatriciaTree.MakeHeterogeneousSet(Term_With_Symvar)

let fixpoint_step loop_id prev next =
  let Serialization( $\phi$ _args,deserialize) = serialize_state prev next in
  let seen = ref TermSet.empty in
  let fp_reached = ref true in
  let open Term_With_Symvar in
  let  $\phi$ _res = map_args_to_result  $\phi$ _args {f=fun a b  $\rightarrow$  match a,b with
    | None, None  $\rightarrow$  assert false
    | Some a, Some b when Term.phys_equal a b  $\rightarrow$  a
    | Some(Term.Ar0{id;symb=Var({from_loop} as record)) as prev,next
      when from_loop = Some loop_id  $\wedge$  not (TermSet.mem prev !seen)  $\rightarrow$ 
        record.definition  $\leftarrow$  Head(next);
        seen := TermSet.add prev !seen;
        prev
    | Some x, _ | _, Some x  $\rightarrow$  begin
      fp_reached := false;
      let typ = Term_With_Symvar.typeof x in
      Term_With_Symvar.fresh_var typ (Some loop_id) (Head None)
    end
  } in
  (deserialize  $\phi$ _res),!fp_reached
end

```

The algorithm for `fixpoint_step` is similar to the one for `join`. One difference is that the algorithm also tests for inclusion between `prev` and `next` (the boolean `fp_reached`). There is no inclusion if we have to create another variable.

The other main difference is the third case in the pattern matching, corresponding to cases where the variable has already been created in a previous fixpoint iteration. The key point here is that we can reuse the variable in this case, as creating a fresh variable every time would lead to a non-terminating analysis. There are some subtleties though; first, we have to make sure we do not mix variables created for the convergence of the "current" loop (given by `loop_id`) with other symbolic variables. Second, in the case where a single symbolic variable is associated with two different definitions, we need to split this symbolic variable into two distinct variables. We maintain the `TermSet` `seen` to check this condition, which should not happen in simple symbolic expression analyses like this one, but could when we serialize more complicated objects, like we begin to do in Chapter 6.

Eventually when the fixpoint is reached, we have computed a ϕ that either map two equal terms to the same term, or a new symbolic variable to its unique definition (that we record to display the global value graph, but is not needed to perform the fixpoint computation).

4.1.5 Complete instantiation

To produce the global value graph of Figure 4.1, the analysis can be run as follows (the code for transformation of the cyclic term graph to GraphViz is in Chapter D):

```

module Example_Symbolic_Domain =
  Example(Fixpoint(Symbolic_Expression_Domain));;
let initial_state_symbolic = Symbolic_Expression_Domain.entry_state;;
let final_state_symbolic = Option.get @@ Example_Symbolic_Domain.factorial5
  initial_state_symbolic;;

```

I will not detail here the soundness proof of the symbolic expression analysis, which is interesting but not easy; in particular, because the soundness of the `join` operation depends on some condition on the abstract elements that is met during the analysis. The details can be found in [POPL'23](#).

4.2 SSA_Translation to Value_Based_Domains

The `Symbolic_Expression_Domain` presented previously allowed us to present the main concepts behind a static analysis manipulating symbolic expressions, such as the accumulation of phi arguments that are pairs of terms that must be anti-unified and the one-pass algorithm for "serialization". We can now extend this to what will constitute the basis for SSA-based abstract interpretation. There will be three main independent extensions.

First, we will lift our symbolic expressions analysis to an analysis generating an SSA translation. Remember that there are two outputs from a symbolic expression analysis (Figure 4.1): symbolic stores, mapping program variables to symbolic expressions at the different program locations; and a global value graph, a cyclic term graph where each term points to the terms on which it depends (e.g., its subterms). This global value graph accurately represents the data flow of the program, but it lacks control flow information: The $\hat{x}_{11}$ and $\hat{x}_{12}$ variables seem to be updated separately (in particular, from the global value graph one cannot prove the main program invariant $\hat{x}_{11} = 5!/\hat{x}_{12}!$), and the global value graph does not represent the conditions of the while loop.

Contrast this with Figure 4.2, where we add the control-flow information to the global value graph. There, $(\hat{x}_{11}, \hat{x}_{12})$, and $(\hat{x}_{17}, \hat{x}_{18})$, have been grouped in tuples (in gray), which represent control points. Furthermore, there are thick arrows between these control points, one of which is carrying the condition corresponding to staying in the while loop. The rest of the Figure corresponds to the global value graph (with a different placement of nodes). This Figure, obtained by the static analysis that we are going to present, corresponds to a translation to a variant of Sea-of-node SSA ([Click and Paleczny, 1995](#)), which can be given an operational semantics ([Demange et al., 2018](#)): i.e., the analysis that we are going to present performs a translation from the source `factorial5` program to this SSA variant.

Second, the SSA language can be given a corresponding tagless-final ([Carette et al., 2009](#); [Kiselyov, 2010](#)) signature, like we have done for the `While_Lang` and `ABSTRACT_DOMAIN` signatures. This signature corresponds to what I call a `VALUE_BASED_DOMAIN`, which differs from `ABSTRACT_DOMAIN` by not using the notions of store or assignment; instead it is based on the evaluation of expressions, and will enrich the `join` and `fixpoint_step` operations with the merge of expressions corresponding to the phi function of SSA. Furthermore, like the `Fixpoint` module was a translation from the `While_Lang` language to the `ABSTRACT_DOMAIN` language, we will provide an `SSA_Translation` module that will translate `ABSTRACT_DOMAIN` programs to `VALUE_BASED_DOMAIN` ones. All the domains that we present in the rest of this manuscript will be `VALUE_BASED_DOMAIN`. To that end, the symbolic expressions of the `Symbolic_Expression_Domain` is replaced by an opaque symbolic value ([Chang and Rival, 2008](#)), which corresponds to symbolic expression in this chapter (or to variable in standard SSA), but not in further ones.

Last, we will also change the implementation of the symbolic expression analysis, notably to improve the precision by performing global value numbering ([Rosen et al.,](#)

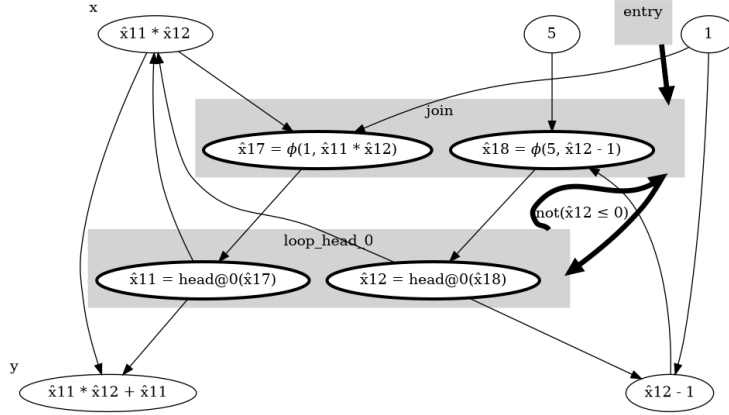


Figure 4.2: SSA translation of `factorial5`, which adds control-flow information to the global value graph of Figure 4.1. Each gray square is a control location; execution proceeds by moving from one control location to the next (if there is a transition between these control locations, symbolized using a thick arrow, and the condition on this transition evaluates to true). When we enter a control location, we interpret the definition of the symbolic variables and add them to the environment, similarly to how `let`-bindings are interpreted in functional languages. Thus, the first steps of the execution consists in

$$(\text{entry}, []); (\text{join}, \begin{bmatrix} \hat{x}_{17} \mapsto 1 \\ \hat{x}_{18} \mapsto 5 \end{bmatrix}); (\text{loop_head_0}, \begin{bmatrix} \hat{x}_{17} \mapsto 1 \\ \hat{x}_{18} \mapsto 5 \\ \hat{x}_{11} \mapsto 1 \\ \hat{x}_{12} \mapsto 5 \end{bmatrix})$$

1988), i.e., fusing symbolic variables that are equal. In more detail, we will turn ϕ into a function, by ensuring that $\phi(\hat{e}_1, \hat{e}_2) = \phi(\hat{e}_3, \hat{e}_4)$ when $\hat{e}_1 = \hat{e}_3 \wedge \hat{e}_2 = \hat{e}_4$, instead of creating a fresh variable at every invocation.

It is worth noting that performing an SSA translation does not make the symbolic expression go away: the symbolic expression analysis is still part of the SSA translation; this should be clear when comparing Figure 4.2 with Figure 4.1. Furthermore, while the SSA translation is sound and exact, the symbolic expression analysis that it embeds is not (it is only sound, but loses precision on join). Formally, this means that we need several concretizations for the SSA-translation analysis, one corresponding to the symbolic expression analysis, expressing the precision of the state properties that we describe, and one corresponding to the SSA translation, expressing the preservation of the traces. Actually, our analysis is parameterized by a numerical domain, which leads to further distinction of the concretizations. All these concretizations are useful and express different aspects of the analysis: in CODEX, the symbolic expression analysis is useful to prove state properties of the program, while the SSA translation is used to translate the source C program to simplified (but equivalent) Horn clauses and send it to a goal-oriented analyser (like SPACER (Gurfinkel, 2022a)) that can solve the assertions that we could not prove.

4.2.1 (INTEGER_AND_BOOLEAN_) VALUE_BASED_DOMAINs

The core interface for the SSA-based abstract interpreter defined in this manuscript is the following one:

```

module type VALUE_BASED_DOMAIN = sig
  (* Abstract state. *)
  type  $\hat{S}^\#$ 
  val entry:  $\hat{S}^\#$ 

  (* Symbolic value of type 'a. *)
  type  $\tau \ \mathcal{Q}$ 
  module Expr: GENERIC_EXPRESSIONS_WITH_BOOLEAN
  with type  $\tau \ \text{expr} = (\tau \ \mathcal{Q}, \hat{S}^\#)$  Polymorphic_State_Option_Monad.m
  val assume: [ $\text{boolean}$ ]  $\mathcal{Q} \rightarrow \hat{S}^\# \rightarrow \hat{S}^\# \ \text{option}$ 

  (* Join and fixpoint_step with phi. *)
  type  $\tau \ \phi\_arguments$ 
  type  $\tau \ \phi\_result$ 
  val join:  $\tau \ \phi\_arguments \rightarrow \hat{S}^\# \rightarrow \hat{S}^\# \rightarrow \tau \ \phi\_result * \hat{S}^\#$ 
  val fixpoint_step: loop_id  $\rightarrow \tau \ \phi\_arguments \rightarrow \text{previous}:\hat{S}^\# \rightarrow \text{next}:\hat{S}^\# \rightarrow$ 
    ( $\tau \ \phi\_result * \hat{S}^\# * \text{bool}$ )

  (* Phi-building data structures. *)
  val empty_ $\phi\_arguments$ : unit  $\phi\_arguments$ 
  type ( $\tau, \tau_2$ ) serialization =
    Serialization:
      'some  $\phi\_arguments *$ 
      ('some  $\phi\_result \rightarrow \hat{S}^\# \rightarrow \tau * (\tau_2 \ \phi\_result) * \hat{S}^\#$ )  $\rightarrow$ 
      ( $\tau, \tau_2$ ) serialization
  val serialize:  $\tau \ \text{Typ.t} \rightarrow$ 
     $\tau \ \mathcal{Q} \ \text{option} \rightarrow \hat{S}^\# \rightarrow \tau \ \mathcal{Q} \ \text{option} \rightarrow \hat{S}^\# \rightarrow$ 
     $\tau_2 \ \phi\_arguments \rightarrow$ 
    ( $\tau \ \mathcal{Q}, \tau_2$ ) serialization  $* \hat{S}^\# * \hat{S}^\#$ 
end

```

Like the `ABSTRACT_DOMAIN` signature described a language, the flowchart language, the `VALUE_BASED_DOMAIN` signature also describes a language, the SSA language. An example of program in the SSA language was given in Figure 4.2. There are similarities between these languages and signatures: they both describe a control-flow graph starting at `entry`, allow splitting control-flow using non-deterministic choice between several successor control locations, blocking control-flow using `assume` and merging control-flow back using `join`; and they both manipulate expressions (using `Expr`). The main difference is that there is no `assign` in the `VALUE_BASED_DOMAIN`, which describes a functional language based on the evaluation of expressions. There is no side effects, except `assume`, which does not change the state but blocks some executions (this property implies a fundamental property of our SSA language, the monotonic scope property (Theorem 1)). The main type of *SSA states*, represents a node in the SSA CFG and the set of properties inferred on this node.

Like `ABSTRACT_DOMAIN`, the `VALUE_BASED_DOMAIN` manipulates expressions $\tau \ \text{expr}$. However, an expression is not just an unspecified type used to interpret statements, but a *symbolic value* (Chang and Leino, 2005; Chang and Rival, 2008) $\alpha \in \mathcal{Q}$ encapsulated inside a state monad, where the state is $\hat{S}^\#$ (in the code, symbolic values $\mathcal{Q}$ use the

$\hat{S}^\# \in \hat{S}^\# : \text{SSA state}$

$\alpha \in \mathcal{Q} : \text{symbolic value}$

polymorphic type $\tau \ \mathbb{Q}$). In this chapter and Chapter 5, symbolic values will correspond to the symbolic expressions $\hat{e} \in \hat{\mathbb{E}}$ that we previously saw (i.e., $\tau \text{ Term.t}$ in the code), but this will no longer be the case in Chapter 6 (in Chang and Leino (2005); Chang and Rival (2008), they are also just unique identifiers). The state monad allows the evaluation of expressions in `VALUE_BASED_DOMAIN` to explicitly thread a $\hat{S}^\#$, that each evaluation step may update. This is why `VALUE_BASED_DOMAIN` decomposes the analysis in more atomic operations than what a regular `ABSTRACT_DOMAIN` does, which is already more decomposed than a direct analysis over the `While_Lang` signature.

A final difference between `ABSTRACT_DOMAIN` and `VALUE_BASED_DOMAIN` is that `join` and `fixpoint_step` takes a new argument (of type $\phi_arguments$) and returns a new result (of type ϕ_result). This corresponds to the need for ϕ in control-flow join nodes in SSA, that reconcile different expressions⁵. Note that such a "join with renaming" also appear in existing value-based abstract interpreters (Gulwani and Tiwari, 2006; Chang and Rival, 2008, 2013; Gange et al., 2016), and I think this operation is central in these kinds of analysers. There is also a `serialize` operation and a (τ, τ_2) serialization type, used to collect $\phi_arguments$ and generate the ϕ function that anti-unifies the symbolic values. To prepare Chapter 6, the (τ, τ_2) serialization type is more complex than the one we saw previously, because 1. we will serialize other objects than the symbolic store (previously, τ could only be the symbolic store); 2. we may serialize different objects one after the other (previously, τ_2 was fixed to `unit` representing the empty sequence), and 3. serialization functions now take and return abstract states $\hat{S}^\#$, because serialization may need to evaluate expressions, and evaluation may modify the state $\hat{S}^\#$. The result of the serialization can be formalized as a renaming function ϕ : note that compared to the simple analysis of Section 4.1, ϕ now operates on opaque symbolic values (which may or may not be symbolic expressions), and is now an injective function $\in \mathbb{Q} \times \mathbb{Q} \rightarrow \mathbb{Q}$ (implemented using a heterogeneous map).

$$\phi \in \mathbb{Q} \times \mathbb{Q} \rightarrow \mathbb{Q} : \phi$$

Soundness rules

Finally, let us provide the soundness conditions for these operations, i.e., the specification of what they do. Our abstract states concretize into sets of *valuations* $v \in \mathcal{V} \triangleq \mathbb{Q} \rightarrow \mathbb{V}$, which are partial mappings from symbolic values to concrete values. These valuations are what make the analysis relational, as they allow constraining several symbolic values simultaneously. Note that concretization into sequences of valuations would be possible (as we did for `ABSTRACT_DOMAIN` in Section 3.3), but we concentrate on state properties the sake of simplicity.

$$v \in \mathcal{V} : \text{valuation}$$

$$\begin{aligned} \gamma(\hat{s}^\#) : \hat{S}^\# &\rightarrow \mathcal{P}(\mathcal{V}) \\ \gamma(\text{entry} = \{\}) & \\ (\alpha, \hat{s}^{\#'}) = \text{op}_n^\#(\alpha_1, \dots, \alpha_n)(\hat{s}^\#) &\Rightarrow \gamma(\hat{s}^{\#'}) \supseteq \{v[\alpha \mapsto \text{op}_n(v[\alpha_1], \dots, v[\alpha_n]) \mid v \in \gamma(\hat{s}^\#)\} \\ &\quad \wedge \{v|_{\text{dom } \hat{s}^\#} \mid v \in \gamma(\hat{s}^{\#'})\} = \gamma(\hat{s}^\#) \\ \hat{s}^{\#'} = \text{assume } \alpha \hat{s}^\# &\Rightarrow \gamma(\hat{s}^{\#'}) \supseteq \{v \in \gamma(\hat{s}^\#) \mid v[\alpha] = \text{true}\} \end{aligned}$$

⁵When formalizing SSA as Continuation-passing style (Kelsey, 1995; Appel, 1998b), ϕ corresponds to the link between `goto-with-argument` and the arguments of a basic block.

An important property of our abstract states is that all the valuations that they represent refer to the same set of values. Formally:

$$\forall \hat{s}^\# \in \hat{\mathbb{S}}^\# : \exists S \in \mathcal{P}(\mathcal{Q}) : \forall v \in \gamma(\hat{s}^\#) : \text{dom } v = S.$$

We denote this set of symbolic values $\text{dom } \hat{s}^\#$.

Thus, the `entry` function is the abstract state referring to no symbolic value. Abstract operations extend the existing valuations by adding information about the resulting symbolic value α . Conceptually α would be a fresh symbol, but the analysis may actually be reusing an existing symbolic value (especially when symbolic values are symbolic expressions and we perform global value numbering). The relations between the existing symbolic values (i.e., other than α if α is fresh) is not modified, which makes the analysis referentially transparent (like functional programming languages, and unlike variable-based static analyses whose base transfer function is `assign`).

The `assume` function shrinks the number of possible valuations to those where the boolean condition α is true, without changing the set of symbolic variables that are being referred to. `assume` also allows to implement queries of the domain, i.e., to determine if a boolean expression is valid: if $\text{assume}(\alpha, \hat{s}^\#) = \perp$, where α is some boolean expression, then there is no valuation in $\gamma(\hat{s}^\#)$ in which α is true (α is unsatisfiable). Hence, α is false in the set of states represented by $\hat{s}^\#$, which we formally write as $\hat{s}^\# \vdash \neg \alpha$. These queries are used to check if some operation might fail (and we need to emit an alarm), or to optimize transfer functions (as in Chapter 6).

Formalizing the behaviour of the `join`, `fixpoint_step`, and `serialize` operations can be difficult. To simplify the formalization, we will differentiate the above functions, used to build the result of an analysis, and *soundness rules*, which are conditions that establish the soundness of the result, without detailing its construction. Already in `ABSTRACT_DOMAIN` we do not require anything from `fixpoint_step` when the fixpoint is not reached. We will go further in that direction: while in the implementation the `serialize` and `join` functions are used to build the ϕ function in intricate ways (first `serialize` computes the domain of ϕ , then ϕ is built in `join`, and finally ϕ is used in the "deserialization" part), we only need to establish the soundness of `join` with a given renaming function $\phi \in \mathcal{Q} \times \mathcal{Q} \rightarrow \mathcal{Q}$. In the formalization we will thus suppose that ϕ is an argument to the `join` and `fixpoint_step` functions. The algorithms for constructing ϕ can be derived⁶ from the specification provided by the soundness rules: as the soundness rules describe which terms must be anti-unified, this says what the domain of ϕ should be. For soundness, the image of ϕ should just associate different values to different pairs (i.e. ϕ must be injective); the reuse of variables is a problem only for the termination of the algorithm. This way of separating proof generation from proof checking, that I learned by working with Xavier Rival during the PhD of Hugo Illous (Illous, 2019), is similar to the distinction between type inference and generation of type constraints (Pottier and Rémy, 2005); it is also related the translation validation (Pnueli et al., 1998; Leroy, 2006) technique (in particular in the context of SSA transformation validation (Barthe et al., 2014)) in certified compilers.

$$\text{join}_\phi \hat{s}_1^\# \hat{s}_2^\# = \hat{s}^\# \quad \Rightarrow \quad \gamma(\hat{s}^\#) \supseteq \text{unbind}(\{ \text{bind}(\text{fst} \circ \phi^{-1}, v) \mid v \in \gamma(\hat{s}_1^\#) \} \cup \{ \text{bind}(\text{snd} \circ \phi^{-1}, v) \mid v \in \gamma(\hat{s}_2^\#) \})$$

⁶At least in this manuscript; the algorithms for computing a suitable ϕ in the separation-logic based analyses like Chang and Rival (2008); FMSD'21 are quite involved.

$$\text{fixpoint_step}_\phi \hat{s}_1^\# \hat{s}_2^\# = (\hat{s}_3^\#, \text{true}) \Rightarrow \begin{aligned} &\gamma(\hat{s}_1^\#) \supseteq \{\text{bind}(\text{snd} \circ \phi^{-1}, \nu) \mid \nu \in \gamma(\hat{s}_2^\#)\} \\ &\wedge \gamma(\hat{s}_3^\#) \supseteq \{\text{bind}(\text{snd} \circ \phi^{-1}, \nu) \mid \nu \in \gamma(\hat{s}_2^\#)\} \end{aligned}$$

where

$$\begin{aligned} \text{bind } B \nu &\triangleq \nu \cup [\alpha \in \text{dom } B \mapsto \nu[B[\alpha]]] \\ \text{unbind } S &\triangleq \{\nu|_I \mid \nu \in S, I = \bigcap_{\nu' \in S} \text{dom } \nu'\} \end{aligned}$$

The above formulas deserve an explanation. The bind operator takes a binding parameter $B \in \mathcal{Q} \rightarrow \mathcal{Q}$ and copies the values of the valuation according to the binding. This binding is extracted from the injective ϕ function, by inversion and split, (hence the binding is either $\text{fst} \circ \phi^{-1}$ or $\text{snd} \circ \phi^{-1}$)⁷. When $\mathcal{Q}$ is a symbolic expression $\hat{\mathbb{E}}$, then B is more precisely a partial function from symbolic variables to terms $\hat{\mathbb{X}} \rightarrow \hat{\mathbb{E}}$.

Example 1. In our factorial5 example of Figure 4.2, the definition of ϕ at the join point is $\phi = [(1, \hat{x}_{11} * \hat{x}_{12}) \mapsto \hat{x}_{17}; (5, \hat{x}_{12} - 1) \mapsto \hat{x}_{18}]$. This ϕ can be inverted to create two bindings $\text{fst} \circ \phi^{-1} = [\hat{x}_{17} \mapsto 1; \hat{x}_{18} \mapsto 5]$ and $\text{snd} \circ \phi^{-1} = [\hat{x}_{17} \mapsto \hat{x}_{11} * \hat{x}_{12}; \hat{x}_{18} \mapsto \hat{x}_{12} - 1]$.

The definition of unbind is uncommon. In SSA, one can view "single assignment" either as a permanent assignment, or as a binding (Schneider, 2013; POPL'23). In the latter, variables are unbound we control leaves the scope, delimited with the domination tree. This is better as unbinding allows getting rid of useless variables, allowing a gain in efficiency that we will see in Chapter 5. Here I have chosen to give a purely-semantic characterization of unbinding: we unbind from *each* valuation all the symbolic values that do not appear in *all* the valuations. The reason to do that is to maintain our invariant that each abstract state refers to the same set of symbolic value: here, we have the property $\text{dom } \text{join}_\phi \hat{s}_1^\# \hat{s}_2^\# = (\text{dom } \hat{s}_1^\# \cap \text{dom } \hat{s}_2^\#) \cup \text{img } \phi$.

Example 2. In our factorial5 example of Figure 4.2, we join an abstract state $\hat{s}_{\text{init}}^\#$ representing valuations coming from the program entry point ($\gamma(\hat{s}_{\text{init}}^\#) = \{[5 \mapsto 5; 1 \mapsto 1]\}$) with an abstract state $\hat{s}_{\text{loop}}^\#$ representing valuations coming from the loop-back edge, which contains the following symbolic values (by order of evaluation): $\text{dom } \hat{s}_{\text{loop}}^\# = \{5; 1; \hat{x}_{17}; \hat{x}_{18}; \hat{x}_{11}; \hat{x}_{12}; \hat{x}_{11} * \hat{x}_{12}; \hat{x}_{12} - 1; \hat{x}_{12} \leq 0; \text{not}(\hat{x}_{12} \leq 0)\}$. Thus, after binding with ϕ and unbinding, the $\text{dom}(\text{join } \phi \hat{s}_{\text{init}}^\# \hat{s}_{\text{loop}}^\#)$ is $\{5; 1; \hat{x}_{17}; \hat{x}_{18}\}$.

There are several advantages of this definition of unbinding: first, we do not need to refer to control location or domination tree, which simplifies the presentation; second, this semantic characterization is more accurate than the scope-based (hence syntactic) one, accounting for exemple for unreachable code. It also provides a better intuition than the use of domination: we just do not want to, and thus cannot, refer to symbolic values that do not exist in all the traces. Finally, having the semantics of single traces depend on the semantics of all the other traces is quite interesting.

Once bind and unbind are defined, the soundness conditions for `join` and `fixpoint_step` are unsurprising. They are the same as the ones in standard `ABSTRACT_DOMAIN`, with binding of the ϕ argument, and unbinding in the case of `join` (not that unbind appears in the definition of `join` and not for `fixpoint_step`, because the corresponding control-flow location has two predecessors in the case of `join` and only one for `fixpoint_step`, so unbinding would do nothing in the later case).

⁷This inversion and split can be seen as transforming the SSA form into the dual view of SSA, CPS (Appel, 1998b), where the ϕ function of SSA is replaced in CPS by calls with continuations that binds the formal argument to the value of the real argument.

INTEGER_AND_BOOLEAN_VALUE_BASED_DOMAIN

For our analyses, we need a specialised version of `VALUE_BASED_DOMAIN` when `EXPR_TYPE` is an `INTEGER_AND_BOOLEAN_EXPRESSIONS`.

```
module type INTEGER_AND_BOOLEAN_VALUE_BASED_DOMAIN = sig
  type  $\mathbb{S}^\#$ 
  type  $\tau \ \mathbb{Q}$ 
  module Expr: GENERIC_EXPRESSIONS_WITH_BOOLEAN
    with type  $\tau \ \text{expr} = (\tau \ \mathbb{Q}, \mathbb{S}^\#)$  Polymorphic_State_Option_Monad.m
    with module type EXPR_TYPE = (INTEGER_AND_BOOLEAN_EXPRESSIONS with type
       $\tau \ \text{expr} = (\tau \ \mathbb{Q}, \mathbb{S}^\#)$  Polymorphic_State_Option_Monad.m)
    and type  $\tau \ \text{Expr.expr} = (\tau \ \mathbb{Q}, \mathbb{S}^\#)$  Polymorphic_State_Option_Monad.m

  include VALUE_BASED_DOMAIN
  with type  $\mathbb{S}^\# := \mathbb{S}^\#$ 
  and type  $\tau \ \mathbb{Q} := \tau \ \mathbb{Q}$ 
  and module Expr := Expr
end
```

The goal here is just to define what is in `VALUE_BASED_DOMAIN` with a more specialised `EXPR_TYPE`. It is verbose just because we need to repeat type definitions so that they are in scope for the `with` constraints.

```
(** [may_be_true  $\alpha \ s$ ] returns false if there is no concrete valuation
     $v \in \gamma(s)$  where  $v[\alpha] = \text{true}$ . *)
val may_be_true: ['boolean]  $\mathbb{Q} \rightarrow \mathbb{S}^\# \rightarrow \text{bool}$ 

(** [may_be_false  $\alpha \ s$ ] returns false if there is no concrete
    valuation in  $v \in \gamma(s)$  where  $v[\alpha] = \text{false}$ . *)
val may_be_false: ['boolean]  $\mathbb{Q} \rightarrow \mathbb{S}^\# \rightarrow \text{bool}$ 

(** [fold_on_values  $\alpha \ s \ \sim_{\min} \ \sim_{\max} \dots$ ] performs a fold on the
    sequence of integer values  $i$  such that there is a concrete
    valuation  $v \in \gamma(s)$  where  $v[\alpha] = i$ . If min and/or max are set, then
    select only the values that are between min and/or max. *)
val fold_on_values: ['integer]  $\mathbb{Q} \rightarrow \mathbb{S}^\# \rightarrow \text{min:Z.t} \rightarrow \text{max:Z.t} \rightarrow \delta \rightarrow (\text{Z.t} \rightarrow \delta \rightarrow \delta) \rightarrow \delta$ 

end
```

We also specialize `VALUE_BASED_DOMAIN` by adding explicit *queries*, i.e., use of the analyser invariants to eliminate impossible cases and simplify other parts of the analysis. For instance, `may_be_true` and `may_be_false` can be used by `Fixpoint` to skip dead code in conditionals. `fold_on_values` will be used for proof by exhaustion (case analysis) in later chapters. In papers, we use the notation $d^\# \vdash p$ where $d^\#$ is an abstract element and p a predicate, to represent the fact that we can prove p from the knowledge we have in $d^\#$ (and the logic rules corresponding to the abstract domain operations).

query

4.2.2 The SSA_Translation functor

Now that we have an `VALUE_BASED_DOMAIN` signature, how can this fit into the abstract interpretation framework that we have defined up to here, i.e., how can we transform an `VALUE_BASED_DOMAIN` module into an `ABSTRACT_DOMAIN` module? This is the role

of the `SSA_Translation` functor, named like this because when the `ABSTRACT_DOMAIN` and `VALUE_BASED_DOMAIN` signatures are seen as the tagless-final representation of a language, then this module really correspond to an SSA translation.

```
module SSA_Translation(VBD:VALUE_BASED_DOMAIN):sig
  module Abstract_Store:STORE with type  $\tau$  value =  $\tau$  VBD. $\alpha$ 
  type state = { store:Abstract_Store.t;
                 ssa_state:VBD. $\hat{S}^\#$  }
  include ABSTRACT_DOMAIN with type state := state
                           and module Expr = VBD.Expr
end = struct

  module Abstract_Store = Make_Store(struct type  $\tau$  t =  $\tau$  VBD. $\alpha$  end)
  type state = {store:Abstract_Store.t; ssa_state:VBD. $\hat{S}^\#$ }
  let entry_state = {store=Abstract_Store.empty;
                     ssa_state=VBD.entry}
```

`SSA_Translation` is very similar to the `Symbolic_Expression_Domain` above. The `Abstract_Store` is like a `Symbolic_Store`, except that it points to symbolic values instead of terms⁸. The state also carries the `ssa_state` in addition to the abstract store.

```
module Expr = VBD.Expr
open Effect
open Effect.Deep

type  $\tau$  Effect.t += GetVar:  $\tau$  ProgVar.t  $\rightarrow$  ( $\tau$  Typ.t *  $\tau$  VBD. $\alpha$ ) t
let var x ssa_state = let _,res = perform (GetVar x) in Some(res,ssa_state)

let run ssa_expr {store;ssa_state} =
  try ssa_expr ssa_state with
  | effect (GetVar x),k  $\rightarrow$ 
    continue k (x.typ, Abstract_Store.find x store)
```

As we saw, the evaluation of expression in an `VALUE_BASED_DOMAIN` requires threading the `ssa_state` using a state monad. We also need, here, to thread the abstract store to implement `var`. We could have used a monad transformer to add a reader monad on top of the expressions. However, this would be quite complex, especially because we want our `EXPR_TYPE` to remain generic. The solution used here is to use `Effect.t` to implement this reader monad transformer, which is what `run` does.

Note that the `$\tau$  typ` parameter in `GetVar` is not used in the code; its purpose is just to make the OCaml type checker happy.

```
let assume cond state =
  match run cond state with
  | None  $\rightarrow$  None
  | Some (boolean,ssa_state)  $\rightarrow$  begin
    match VBD.assume boolean ssa_state with
    | None  $\rightarrow$  None
    | Some ssa_state  $\rightarrow$  Some{state with ssa_state = ssa_state}
```

⁸This store is often called symbolic store in the symbolic execution literature, and abstract store in abstract interpretation one. Because they are an exact abstraction (Chapter 6), “abstract store” and “symbolic store” are the same concept, and both names are synonyms.

```

end

let assign x e state =
  match run e state with
  | None → None
  | Some (value,ssa_state) → begin
      let store = Abstract_Store.add x value state.store in
      Some{store;ssa_state}
    end
end

```

Like for the `Symbolic_Expression_Domain`, the implementation of `assume` and `assign` is quite simple. `assume` evaluates the expression, then makes a query to stop the analysis when the condition is false. `assign` evaluates the expression and updates the store. Both also return the updated `ssa_state`.

```

open VBD
let serialize_abstract_store store1 s1 store2 s2  $\phi$ _args =
  let f (type a) (key:a ProgVar.t) (v1:a  $\mathbb{Q}$  option) (v2:a  $\mathbb{Q}$  option)
    (Serialization( $\phi$ _args,d_store),s1,s2) =
    let Serialization( $\phi$ _args,d_expr),s1,s2 =
      VBD.serialize key.typ v1 s1 v2 s2  $\phi$ _args in
    Serialization( $\phi$ _args,fun res s →
      let v,res,s = d_expr res s in
      let store,res,s = d_store res s in
      let store = Abstract_Store.add key v store in
      store,res,s),s1,s2 in
  let init = Serialization( $\phi$ _args,fun res s → store1,res,s),s1,s2 in
  Abstract_Store.fold_on_nonequal_union {f} store1 store2 init

let join state1 state2 =
  let Serialization( $\phi$ _args,d_store),ssa_state1,ssa_state2 =
    serialize_abstract_store state1.store state1.ssa_state state2.store
    state2.ssa_state VBD.empty_ $\phi$ _arguments in
  let  $\phi$ _res,ssa_state = VBD.join  $\phi$ _args ssa_state1 ssa_state2 in
  let store,(_:unit  $\phi$ _result),ssa_state = d_store  $\phi$ _res ssa_state in
  {store;ssa_state}

let fixpoint_step loop_id previous next =
  let Serialization( $\phi$ _args,d_store),prev_ssa_state,next_ssa_state =
    serialize_abstract_store previous.store previous.ssa_state next.store
    next.ssa_state VBD.empty_ $\phi$ _arguments in
  let  $\phi$ _res,ssa_state,includes =
    VBD.fixpoint_step loop_id  $\phi$ _args
    ~previous:prev_ssa_state ~next:next_ssa_state in
  let store,(_:unit  $\phi$ _result),ssa_state = d_store  $\phi$ _res ssa_state in
  {store;ssa_state},includes
end

```

The code for `join` and `fixpoint_step` follows the same structure as the ones in `Symbolic_Expression_Domain`: serialize the abstract store (using `serialize_abstract_store`) into pairs of differing terms, then create a ϕ function merging anti-unifying these terms, then create a new abstract store using ϕ and the closure returned by `serialize_abstract_store`. The difference is that we need to also `join` or apply

`fixpoint_step` on the `ssa_state` to get the ϕ function, and for this we call the corresponding operations in `VBD`.

Furthermore, the "serialization" of the abstract states now involve anti-unifying pairs of symbolic values instead of pairs of terms, which requires also calling the `serialize` function on the symbolic values. In general, `serialize_abstract_store` is an example of how we implement anti-unification of compound objects (here, a store) from the anti-unification of scalar objects by this "serialization" into a sequence of pairs of symbolic values; we will see more examples of this in Chapter 6.

Concretization and soundness

Given a concretization $\gamma(\hat{s}^\#) : \hat{\mathbb{S}}^\# \rightarrow \mathcal{P}(\mathcal{V})$ for a `VALUE_BASED_DOMAIN` argument `VBD`, we can concretize `SSA_Translation(VBD)` (whose state is a pair of an abstract store and `ssa` state) simply as:

$$\begin{aligned} \gamma : \hat{\Sigma} \times \hat{\mathbb{S}}^\# &\rightarrow \mathcal{P}(\mathbb{S}) \\ \gamma(\hat{\sigma}, \hat{s}^\#) &= \{[x \in \mathbb{X} \mapsto v[\hat{\sigma}[x]]] \mid v \in \gamma(\hat{s}^\#)\} \end{aligned}$$

The proof of soundness shows that if the `VBD` parameter of the functor is sound (wrt. the conditions on the `VALUE_BASED_DOMAIN` signature), then `SSA_Translation(VBD)` is sound (wrt. the conditions on the `ABSTRACT_DOMAIN` signature). Moreover, if `VALUE_BASED_DOMAIN` is exact, then `SSA_Translation(VBD)` is also exact. These semantic proofs also correspond to simulation proofs of the soundness of the SSA translation, in which the symbolic store corresponds to the simulation relation between the states in the `VALUE_BASED_DOMAIN` (SSA) language, and the states in the `ABSTRACT_DOMAIN` language. These details can be found in [POPL'23](#); [PLDI'24](#).

4.2.3 SSA_Domains are symbolic Value_Based_Domains

We now present the first implementation of a `VALUE_BASED_DOMAIN` signature. Like we did to build a control-flow graph in Section 3.3, this implementation is symbolic and mostly consists in a free algebra over the `VALUE_BASED_DOMAIN` signature: $\hat{\mathbb{S}}^\#$ will be represented symbolically, and $\tau \text{ @ } \alpha$ will consist in a τ `Term.t`. Together with the `SSA_Translation` functor, it will implement an analysis that supersedes the symbolic expression analysis of Section 4.1, implementing a full SSA translation.

Global value numbering: ϕ as a function

One of the improvements of this analysis over `Symbolic_Expression_Domain` is a better *Global Value Numbering* ([Reif and Tarjan, 1982](#); [Reif and Lewis, 1986](#); [Rosen et al., 1988](#); [Alpern et al., 1988](#)). Due to the fact that our `Term` implementation is based on a term graph (i.e., we hashcons every term), the `Symbolic_Expression_Domain` already detected the equality of some symbolic expressions. However, we missed some variable equalities at variable creation time: if, in a conditional branch `x` and `y` where both assigned the constant 2, and in the other branch they are both assigned the constant 3, the previous domain would still have assigned two different symbolic variables to the result of the join operation. This means that ϕ was not a function, as there were multiple variables as the result of $\phi(2, 3)$. In this section, this ϕ is a function, and $\phi(2, 3)$ corresponds to a single variable. To implement this, we will use the `Term_Option_Pair_Map` module provided in Chapter E, renamed as `Phi` here:

Global Value Numbering

```

#include "term_option_pair_map.ml"
module Phi:sig
  type t
  val empty: t
  val mem1:  $\tau$  Term.t option  $\rightarrow$  t  $\rightarrow$  bool
  val mem:  $\tau$  Term.t option  $\rightarrow$   $\tau$  Term.t option  $\rightarrow$  t  $\rightarrow$  bool
  val find:  $\tau$  Term.t option  $\rightarrow$   $\tau$  Term.t option  $\rightarrow$  t  $\rightarrow$   $\tau$  Term.t
  val add:  $\tau$  Term.t option  $\rightarrow$   $\tau$  Term.t option  $\rightarrow$   $\tau$  Term.t  $\rightarrow$  t  $\rightarrow$  t
  type ( $\delta$ ) polyfold = { f:  $\tau$ .  $\tau$  Term.t option  $\rightarrow$   $\tau$  Term.t option  $\rightarrow$ 
                         $\tau$  Term.t  $\rightarrow$   $\delta \rightarrow \delta$  } [@@unboxed]
  val fold:  $\tau$  polyfold  $\rightarrow$  t  $\rightarrow$   $\tau \rightarrow \tau$ 
end = Term_Option_Pair_Map

```

The goal of this module is to have a bijective map uniquely associating a pair of τ Term.t option to a variable; this represents a ϕ function (where None corresponds to an absent argument). One complexity is that this map uses heterogeneous keys (as terms are polymorphic), which is why `fold` uses a higher-rank type (which requires encapsulation of the function argument in the polyfold record type). We also implement the map of pairs as a map of map (τ Term.t option $\rightarrow$ τ Term.t option $\rightarrow$ τ Term), because the first map will replace the `TermSet` module (detecting if each variable is associated with only one definition). The `mem1` function allows testing membership in the first map.

Abstract environments

```

module type ABSTRACT_ENVIRONMENT = sig
  type t
  val entry: t
  val eval:  $\tau$  Term.t  $\rightarrow$  t  $\rightarrow$  t
  val assume: [ $\tau$  boolean] Term.t  $\rightarrow$  t  $\rightarrow$  t option
  val join: Phi.t  $\rightarrow$  t  $\rightarrow$  t  $\rightarrow$  t
  val fixpoint_step: includes:bool  $\rightarrow$ 
    Phi.t  $\rightarrow$  previous:t  $\rightarrow$  next:t  $\rightarrow$  t * bool
  val fold_on_values: [ $\tau$  integer] Term.t  $\rightarrow$  t  $\rightarrow$  min:Z.t  $\rightarrow$  max:Z.t  $\rightarrow$   $\delta \rightarrow$ 
    (Z.t  $\rightarrow$   $\delta \rightarrow \delta$ )  $\rightarrow$   $\delta$ 
end

```

The second main addition is that the implementation is parameterized by an `ABSTRACT_ENVIRONMENT` module, used to provide a numerical domain computing numerical information about the terms. The signature is close to `VALUE_BASED_DOMAIN`, with the difference that symbolic values are replaced by terms, and that the terms and ϕ functions are already built and given to the domain, instead of being built by the domain. In other terms, in `VALUE_BASED_DOMAIN` the symbolic values are opaque names, while in `ABSTRACT_ENVIRONMENT` they are terms. In `VALUE_BASED_DOMAIN` the transfer functions return a fresh symbolic value, while in `ABSTRACT_ENVIRONMENT` the term is already built, and we use `eval` to update the information known about it (for instance, in a non-relational domain which maps each term to an interval, `eval  $\hat{e}$` would update the domain by computing the interval corresponding to $\hat{e}$ from the interval of its subterms).

We also require a `fold_on_values` function, used to implement the homonymous function in `INTEGER_AND_BOOLEAN_VALUE_BASED_DOMAIN`. The concretization for this

domain would be similar to the ones for `VALUE_BASED_DOMAIN` in Section 4.2.1. A difference is that `fixpoint_step` takes a first includes argument: if inclusion does not hold (includes is false), the fixpoint is not reached, and we don't even need to perform further inclusion testing, which is useful if this operation is costly.

SSA control states

```
(** Conjunction of boolean constraints represented as a list. *)
type condition = ['boolean] Term.t list

type control_state +=
| Entry
| Loop_head of {id:loop_id;
                mutable  $\phi$ :Phi.t;
                mutable definition: control_state * condition }
| Join of { mutable  $\phi$ :Phi.t;
            predecessors: (control_state * condition)
                        * (control_state * condition) }
```

The last main addition is the addition of *control states*, which are what was the symbolic expression analysis missed to be an exact abstraction of the source program. The type `control_state` is quite similar to the type `node` of module `Cfg`, and indeed, they both describe control states in a control-flow graph. However, `control_state` is missing nodes for assignment (as in our SSA language, there are no assignments and only bindings). We also do not create nodes corresponding to `assume`; instead, we accumulate *conditions* on the edges from the predecessors of a `control_state` node to that `control_state`. This way, we create fewer control states, only in the case when new symbolic variables are created (i.e., on `join` and `fixpoint_step`).

control state

The fields are mutable to build circular references (from symbolic variables to their control states, and from control states to their variables through the ϕ field). In the case of `Loop_head`, the ϕ only contains bindings of the form $(\hat{x}, \hat{e}) \mapsto \hat{x}$ which we display as $\hat{x} = \text{head}(\hat{e})$ in Figure .

Note that it would be easy to extend `control_state` to carry more information, notably computing domination tree on the fly (Cooper et al., 2001); this is useful to help debugging scoping issues (e.g. addition of two terms that have an incompatible scope, i.e., do not follow the SSA dominance property (Rastello and Bouchez Tichadou, 2022)).

Abstract state : definition and three concretizations

```
module Make_SSA_Domain(Abs_Environment:ABSTRACT_ENVIRONMENT):
  INTEGER_AND_BOOLEAN_VALUE_BASED_DOMAIN =
  struct
    type  $\mathbb{S}^\#$  = {  $\rho^\#$ :Abs_Environment.t;          (* Numerical state. *)
                  control_state:control_state;    (* Last control state *)
                  condition: condition }          (* Condition since last control
                                                    state. *)
    type  $\tau \mathbb{Q} = \tau$  Term.t
```

The state of our symbolic abstract domain consists in the last `control_state` encountered (i.e. last control-flow join or loop head), plus the accumulated conjunction of conditions encountered since that last state. In addition, we also maintain the state of the `Abs_Environment` parameter.

As we saw in Section 4.2.1, the meaning of this triple is given by a concretization function $\gamma(\hat{s}^\#) : \hat{\mathbb{S}}^\# \rightarrow \mathcal{P}(\mathcal{V})$ that represent a set of valuations. But we will be actually be providing three such concretization functions. These concretizations do not interact (e.g., using a reduced product (Cousot and Cousot, 1979)) like we often do in abstract interpretation; instead, they represent different views of $\hat{\mathbb{S}}^\#$, from imprecise information which is immediately accessible, to more precise information which is possibly accessible.

The first two concretizations are based on $\gamma_\rho : (\rho^\# = \text{Abs_Environment.t}) \rightarrow \mathcal{P}(\mathcal{V})$, the concretization of the `Abs_Environment` parameter. Our first concretization is simply $\gamma_1(\hat{s}^\#) \triangleq \gamma_\rho(\hat{s}^\#. \rho^\#)$, which represents the knowledge that module `Abs_Environment` contains on each individual `Term`, seen as an opaque entity (a symbolic value $\mathbb{O}$). This concretization is used to reason about immediately available information, for instance, what are the possible results of queries.

Our second concretization is defined as:

$$\gamma_2(\hat{s}^\#) = \{v \in \gamma_1(\hat{s}^\#) \mid \exists \rho \in \rho : \forall \hat{e} \in \text{dom}(v) : v[\hat{e}] = \text{eval}(\hat{e}, \rho)\}.$$

Now the symbolic expressions are no longer seen as opaque identifiers, but are interpreted, and we remove the valuations are inconsistent. For instance, let $v = [\hat{x} \mapsto 2, \hat{x} + 1 \mapsto 9]$. It is possible that $v \in \gamma_1(\hat{s}^\#)$ for some $\hat{s}^\# \in \hat{\mathbb{S}}^\#$, but not that $v \in \gamma_2(\hat{s}^\#)$. This concretization represents the information contained in the reduced product between the symbolic expression analysis (Section 4.1) and the contents of the `Abs_Environment.t`; or, equivalently, what we could maximally learn from the abstract environment (for instance, if we dumped an abstract environment as an SMT formula and asked the solver whether a possible valuation belongs to that environment).

The last concretization, which does not depend on `Abs_Environment`, is the most precise (as it exactly describes the semantics of the SSA program) but corresponds to the most inaccessible information. Indeed, it just recomputes a collecting semantics from the symbolic expression in `control_state`, as we did in Chapter 3. This concretization is also defined inductively, and this inductive definition could also be transformed into an operational semantics for the `VALUE_BASED_DOMAIN` language.

$$\begin{aligned} \gamma_3(\hat{s}^\#) &= \gamma_{cs}(\hat{s}^\#. \text{control_state}, \hat{s}^\#. \text{condition}) \\ \gamma_{cs}(cs, \text{cond} :: \text{conds}) &= \{v \in \gamma_{cs}(cs, \text{conds}) \mid \text{eval}(\text{cond}, v)\} \\ \gamma_{cs}(\text{Entry}, []) &= \{[]\} \\ \gamma_{cs}(\text{Join}(\phi, s_1, s_2), []) &= \text{unbind}(\{ \text{bind}(\text{fst} \circ \phi^{-1}, v) \mid v \in \gamma_{cs}(s_1) \} \\ &\quad \cup \{ \text{bind}(\text{snd} \circ \phi^{-1}, v) \mid v \in \gamma_{cs}(s_2) \}) \\ \gamma_{cs}(\text{Loop_head}(\phi, s), []) &= \{ \text{bind}(\text{snd} \circ \phi^{-1}, v) \mid v \in \gamma_{cs}(s) \} \end{aligned}$$

Note that because this definition closely corresponds to the soundness rules (Section 4.2.1) of the `VALUE_BASED_DOMAIN`, it is easy to prove that with the γ_3 concretization, all transfer functions in the SSA domain that we will see, are sound and exact. This is why this concretization is useful to reason about the loss of precision of some program transformations, as done in Chapter 6 or PLDI'24; or about the results that

could be obtained if we redo the fixpoint computation with a more precise abstract domain, or when handling the SSA program to a software model-checking tool (as we do in CODEX when translating the SSA programs to Horn clauses that we analyse with SPACER).

Implementation: SSA translation as an abstract interpretation

```
let entry = {  $\rho^\#$  = Abs_Environment.entry;
              control_state = Entry;
              condition = [] }
let assume cond state =
  match Abs_Environment.assume cond state. $\rho^\#$  with
  | None → None
  | Some  $\rho^\#$  → Some {  $\rho^\#$ ;
                      condition=cond::state.condition;
                      control_state = state.control_state }
```

The functions `entry` and `assume` are simple transfer functions that manipulate the `ssa_state`. Like all transfer functions in this module, it performs two interpretations: one symbolic that updates the `control_state`, `condition`, and symbolic expressions; and one semantic, that updates the abstract environment by calling the corresponding functions in `Abs_Environment`.

```
module M = State_Option_Monad(struct type t =  $\hat{S}^\#$  end)
module Expr = struct
  module Expr = Lift_Integer_And_Boolean_Expressions(Build
    _Integer_And_Boolean_Expressions)(M)(struct
    type  $\tau$  expr =  $\tau$  Term.t M.m

    let replace:  $\tau$  Term.t M.m →  $\tau$  Term.t M.m = fun  $\hat{e}$  state →
      match  $\hat{e}$  state with
      | None → None
      | Some (symb_val, state) →
        let  $\rho^\#$  = Abs_Environment.eval symb_val state. $\rho^\#$  in
        Some(symb_val, {state with  $\rho^\#$ })

    let to_integer = replace    let from_integer  $\hat{e}$  =  $\hat{e}$ 
    let to_boolean = replace    let from_boolean  $\hat{e}$  =  $\hat{e}$ 

  end)
  module type EXPR_TYPE = module type of Expr
  include Expr
end
```

Here, we just encapsulate the evaluation of expressions in a state monad, so that the evaluation of each expression can access and modify the state using the `eval` function. In the next chapter, we use this to associate a single-value abstraction (like an interval) to each newly-created term.

```
type  $\tau$   $\phi$ _arguments =  $\tau$  Phi_Arguments.t
let empty_ $\phi$ _arguments = Phi_Arguments.Empty
```

```

type  $\tau$   $\phi\_result$  = Phi.t

type ( $\tau, \tau_2$ ) serialization =
  Serialization:
    'some  $\phi\_arguments$  *
    ('some  $\phi\_result \rightarrow \hat{S}^\# \rightarrow \tau * (\tau_2 \phi\_result) * \hat{S}^\#$ )  $\rightarrow$ 
    ( $\tau, \tau_2$ ) serialization

let serialize_typ  $\hat{e}_1$  state1  $\hat{e}_2$  state2  $\phi\_args$  = match  $\hat{e}_1, \hat{e}_2$  with
| None, None  $\rightarrow$  assert false (* Should not happen. *)
| Some  $\hat{e}_1$ , Some  $\hat{e}_2$  when Term.phys_equal  $\hat{e}_1$   $\hat{e}_2$   $\rightarrow$ 
  Serialization( $\phi\_args$ , fun  $\phi\_res$  state  $\rightarrow$   $\hat{e}_1$ ,  $\phi\_res$ , state),
  state1, state2
| _  $\rightarrow$ 
  let  $\phi\_args$  = Phi.Arguments.Cons( $\hat{e}_1, \hat{e}_2, \phi\_args$ ) in
  Serialization( $\phi\_args$ , fun  $\phi\_res$  state  $\rightarrow$ 
    let  $\hat{e}$  = Phi.find  $\hat{e}_1$   $\hat{e}_2$   $\phi\_res$  in
     $\hat{e}, \phi\_res, state$ ), state1, state2

```

Compared to the `Symbolic_Expression_Domain`, `$\phi\_result$` now returns a whole phi function (instead of just a sequence of results matching each pair of terms). The deserialization consists in finding the corresponding symbolic variable in the returned phi function. You can also note that the serialization no longer pushes equal terms on the list of phi arguments, and directly returns the result. Note that serialization here does not change the SSA states: we pass and return states in the function to be compatible with the same interface in Chapter 6.

```

let join  $\phi\_args$  state1 state2 =
  let (Join cs as control_state) =
    Join {  $\phi$ =Phi.empty;
          predecessors=(state1.control_state, state1.condition),
                    (state2.control_state, state2.condition) } in
  let upd_ $\phi$   $\phi$   $\hat{e}_1$   $\hat{e}_2$  =
    if Phi.mem  $\hat{e}_1$   $\hat{e}_2$   $\phi$  then  $\phi$ 
    else
      let typ = match  $\hat{e}_1, \hat{e}_2$  with
      | Some  $\hat{e}$ , _ | _, Some  $\hat{e}$   $\rightarrow$  Term_With_Symvar.typeof  $\hat{e}$ 
      | None, None  $\rightarrow$  assert false in
      let  $\hat{x}$  =
        Term_With_Symvar.fresh_var typ None ~control_state (Phi( $\hat{e}_1, \hat{e}_2$ )) in
      Phi.add  $\hat{e}_1$   $\hat{e}_2$   $\hat{x}$   $\phi$ 
  in
  let  $\phi$  = Phi.Arguments.fold_left {f=upd_ $\phi$ } Phi.empty  $\phi\_args$  in
  cs. $\phi$   $\leftarrow$   $\phi$ ;
  let  $\rho^\#$  = Abs_Environment.join  $\phi$  state1. $\rho^\#$  state2. $\rho^\#$  in
  let stateres = { $\rho^\#$ ; condition=[]; control_state} in
   $\phi$ , stateres

```

This `join` function only creates the `$\phi$` function from the list of pairs of terms that is `dom  $\phi$` , while doing global value numbering (returning the same variable for the same equal pair of arguments). In addition, we create a `control_state` (similarly to what we did in the `Build_CFG` module), link to it from the variables, and update the `ABSTRACT_ENVIRONMENT`. The `$\phi$` field is imperatively modified because the data

structure is cyclic: states contain a ϕ that contains fresh variables that point to the state where they were created.

```

let fixpoint_step loop_id  $\phi$ _args ~(previous: $\S^\#$ ) ~next =
  (* Create or reuse control state. *)
  let includes,cs = match previous.control_state with
  | (Loop_head {id}) when id == loop_id → (true,previous.control_state)
  | _ →
    (false,Loop_head { definition = (next.control_state,next.condition);
                        id=loop_id;  $\phi$ =Phi.empty }) in

  (* Iterate on pairs of terms and create phi. Try to reuse symbolic
     variables. *)
  let upd_ $\phi$  (includes, $\phi$ ) previous next =
    if Phi.mem previous next  $\phi$  then (includes, $\phi$ )
    else
      let open Term_With_Symvar in
      let includes, $\hat{x}$  = match previous,next with
      | None, None → assert false (* Impossible. *)
      | Some(Term.Ar0{id;symb=Var({control_state} as record)}) as  $\hat{x}$ ,_
        when includes (* No further test if fixpoint is not
reached. *)
           $\wedge$  control_state == cs
           $\wedge$  not (Phi.mem1 (Some  $\hat{x}$ )  $\phi$ ) →
            record.definition ← (Head next);
            (true,  $\hat{x}$ )
      | Some  $\hat{e}$ , _ | _, Some  $\hat{e}$  →
        let  $\hat{x}$  =
          Term\_With\_Symvar.fresh_var ~control_state:cs
          (Term\_With\_Symvar.typeof  $\hat{e}$ ) (Some loop_id) (Head None) in
        (false,  $\hat{x}$ ) in
      (includes, Phi.add previous next  \$\hat{x}\$   \$\phi\$ \)
  in
  let includes, \$\phi\$  =
    Phi\_Arguments.fold\_left {f=upd\_ \$\phi\$ } \(includes,Phi.empty\)  \$\phi\$ \_args in

  \(\* Update circular definitions and update the abstract environment.
     \*\)
  \(let Loop\_head cs = cs in
   cs. \$\phi\$  ←  \$\phi\$ ; cs.definition ← \(next.control\_state,next.condition\)\);
  let  \$\rho^\#\$ ,includes =
    Abs\\_Environment.fixpoint\_step ~includes  \$\phi\$ 
    ~previous:previous. \$\rho^\#\$  ~next:next. \$\rho^\#\$  in
  let state = { \$\rho^\#\$ ;condition=\[\];control\_state=cs} in
   \$\phi\$ ,state,includes

```

What happens here is a mix between the code for `fixpoint_step` in `Build_CFG` and the `Symbolic_Expression_Domain`. We create a `control_state` with an `loop_id` used to detect if it corresponds to the current loop; reuse it when the `loop_ids` correspond, and create a new one otherwise.

Then, we iterate on the list of pairs of terms, reusing symbolic variables when possible, or creating fresh ones when needed. Compared to `Symbolic_Expression_Domain`, we no longer need the `from_loop` field, and instead check whether the control

_state corresponds to the one we are creating for the current loop. We also reuse the existing ϕ map to ensure that each variable is associated with a single definition.

Finally, we update the circular references and update the `Abs_Environment` (passing an `includes` argument to `fixpoint_step`, to save it from computing inclusion if we already know that fixpoint is not reached).

```

let may_be_true  $\hat{e}$  state = Option.is_some (assume  $\hat{e}$  state)

let may_be_false  $\hat{e}$  state =
  match Expr. $\neg$  (M.return  $\hat{e}$ ) state with
  | None  $\rightarrow$  false
  | Some ( $\neg\hat{e}$ , state)  $\rightarrow$  may_be_true  $\neg\hat{e}$  state

let fold_on_values  $\hat{e}$  state = Abs_Environment.fold_on_values  $\hat{e}$  state. $\rho^\#$ 
end

```

The functions `may_be_true` and `may_be_false` implement queries to know if a condition is constant, by doing unsatisfiability tests: we assume the condition, and if there is a contradiction, then there are no valuations where the boolean is true. Implementing boolean queries that way is often more precise than doing boolean queries on the domain, because the new assumption may lead to propagations that allow discovering the contradiction. For instance using the constraint propagation domain of Chapter 5, we may initially have $\hat{x} \in \{-1, 1\}$, $\hat{x} * \hat{x} \in \{-1, 1\}$; the propagations after the query $\hat{x} * \hat{x} = -1$ can discover that this condition is false.

Finally, `fold_on_values` is a simple wrapper around the function provided by the module `Abs_Environment`.

4.2.4 Complete instantiation

We can now put everything together to produce the result of Figure 4.2. Finally, we instantiate the `SSA_Domain` with a trivial `ABSTRACT_ENVIRONMENT` (note: `fold_on_values` is currently not used).

```

module Unit_SSA_Domain = Make_SSA_Domain(struct
  type t = unit
  let entry = ()
  let join _ () () = ()
  let eval _ () = ()
  let fixpoint_step ~includes _ ~previous:() ~next:() = (), includes
  let assume _ () = Some ()
  let pp_fmt () = ()
  let fold_on_values _ _ ~min:_ ~max:_ = assert false
end)

```

Then we instantiate all the components to analyse the `factorial5` function:

```

module SSA_Translate_to_Term = SSA_Translation(Unit_SSA_Domain)
module Example_SSA_Translation_To_Term =
  Example(Fixpoint(SSA_Translate_to_Term))
let initial_state = SSA_Translate_to_Term.entry_state;;
let final_state = Option.get @@ Example_SSA_Translation_To_Term.factorial5
  initial_state;;

```

The code to convert `final_state` to a figure is described in Appendix D.

4.3 Related work and possible directions

4.3.1 Difference with published work

This chapter builds on my previous work (POPL’23; PLDI’24), with several subtle differences, which highlights the versatility of the symbolic abstract domain technique. I will explain and justify these differences here.

The `VALUE_BASED_DOMAIN` signature is language-independent

An important improvement is that the SSA language corresponding to the `VALUE_BASED_DOMAIN` signature is independent of the source language used, unlike in POPL’23; PLDI’24, where the names of program variable and of control locations or widening points was used to create the names of symbolic variables. This makes it easier to reuse `VALUE_BASED_DOMAIN` in different contexts where program variables are not easily available, like in Chapters 6 or 7 where we track the contents of the heap; or even in this chapter, where global value numbering means that a symbolic variable may represent the value of several program variables. This also makes the analyser easier to port to new languages (Codex can analyse multiple languages, and porting the analysis to new languages is easy). Note that using program variables in symbolic variable names could still be useful to help debugging.

Different number of control states

In this chapter and previous publications, we have experimented with different variants of the SSA syntax, which shows that the approach is easily adaptable. In POPL’23, there was one SSA control state per control location in the source program. In PLDI’24 we removed from the SSA state the control locations corresponding to assignments, but kept those corresponding to assume transitions. In this manuscript, I no longer need control states corresponding to assume, but I have separated the control states corresponding to join and widening (which could be merged). Fewer nodes allow, for instance, for a domination tree with a smaller height.

Further reducing the number of control states would require replacing join nodes by "if then else" expressions, in the style of Gated-SSA (Herklotz et al., 2023; Ballance et al., 1990). Note that we are already able to distinguish, in join nodes, loop heads from other nodes; it would be interesting to see if the symbolic-domain approach could be used to generate a program strictly conforming to the Gated-SSA form.

In earlier, unpublished versions of the formalization (Lemerre and Bardin, 2017) I used yet another different syntax, where only μ -recursion (Ariola and Klop, 1996) was possible. This was due to the fact that the corresponding `VALUE_BASED_DOMAIN` used a closure operator as found in dataflow-elimination frameworks (Ryder and Paull, 1986), rather than a standard widening operator. I found out how to support widening later, by introducing the idea of passing a loop identifier (a control location in POPL’23, the name of a widening point in PLDI’24, and a `loop_id` here) to the widening (or `fixpoint_step`) function, which allow symbolic domains with widening to terminate.

Distinction between `VALUE_BASED_DOMAIN` and `ABSTRACT_ENVIRONMENT`

The distinction that we operate in this chapter between the `VALUE_BASED_DOMAIN` signature, with opaque symbolic values built and `serialized` by the domain, and

`ABSTRACT_ENVIRONMENT`, which receives terms that have already built, is new. In [POPL'23](#) we provided a monolithic analysis like the `Symbolic_Expression_Domain`, and the SSA translation in [PLDI'24](#) grouped `SSA_Translation` and `Make_Symbolic_Value_Based_Domain` functors in a single "Lift" functor. This distinction is important, first to explain the abstract abstract datatypes in Chapter 6, but they also better describe how evaluation works in `VALUE_BASED_DOMAIN` (e.g., the mapping from terms to single value abstraction in [PLDI'24](#) is created lazily, which can negatively affect the precision of the join operation; while this mapping is created simultaneously with the creation of terms with the calls to `eval` here). In previous papers, a module like `VALUE_BASED_DOMAIN` was generally used in the formalization ([SAS'20](#); [FMSD'21](#); [VMCAI'22](#)), but [OOPSLA'24](#) used `ABSTRACT_ENVIRONMENT` instead.

Proof

The proofs presented here are considerably simplified compared to that in [POPL'23](#); [PLDI'24](#). Most importantly, our fixpoint computation here is "denotational-style" ([Miné, 2017](#)), i.e., based on the syntax rather than the control-flow graph. Thus, we do not keep a global map from program locations to abstract stores, which prevents us from establishing a correspondance between the source and SSA programs along a program trace. In other words, we can only prove state properties, or denotational / abstract transformations ([SAS'20](#)) properties from the entry point to some program location, or trace properties of the finite traces from the entry to a particular program location, but not global properties over all the traces of the program. The proofs of [POPL'23](#); [PLDI'24](#) is done in the settings of a such global trace properties (as in Chapter 3).

These papers also contain other technical details that I have omitted in this chapter, such as the soundness proof for the join operation of symbolic expressions, or the difficulty to maintain exactness during the construction of the SSA translation. Finally, they also start from the operational semantics of a SSA language, that we have omitted in this chapter.

4.3.2 Related work

The style of abstract interpretation that I describe as "value-based", based on the computation of symbolic values by composition of expressions, and implying a need to perform a renaming by a ϕ function on joins, has different occurrences in the literature; what I have proposed is an explanation of value-based domains by its interaction with the SSA language and translation, and the related concepts of symbolic expression analysis and anti-unification. Value-based domains particularly appear in memory and shape analyses, which I think is due to the fact that handling assignment in variable-based analyses is quite complex (but possible (e.g., [Sagiv et al., 1998](#))). For instance, in early papers on pointer analysis like [Chase et al. \(1990\)](#), SSA helped to reason about pointer analysis.

[Chang and Leino \(2005\)](#) describe an analysis with symbolic values, a join and renaming operations. In shape analysis, [Gulwani and Tiwari \(2007\)](#) describe the use of "dummy variables" (corresponding to our symbolic values) and [Gulwani and Tiwari \(2006\)](#) further describe a normalization (corresponding to our ϕ function computation). [Calcagno et al. \(2006\)](#); [Distefano et al. \(2006\)](#) describe a notion of primed variable (corresponding to symbolic values) and renaming, canonicalization, and variable removal operations. [Chang and Leino \(2005\)](#); [Chang and Rival \(2008, 2013\)](#);

Rival (2011) use a value-based domain to integrate numerical analysis in memory and shape analyses, and describe the notions of symbolic value and renaming (corresponding to our ϕ computation); this line of work (that I discovered working with Xavier, Hugo and Olivier in NFM’17; Illous (2019); SAS’20; FMSD’21; RTAS’21; VMCAI’22; Nicole (2022)) was very influential in the formalization presented in this chapter. The computation of ϕ function in shape analysis is particularly complex, and it would be interesting to see how well this would fit the design presented in this chapter.

Finally, note that analyses based on SSA may be qualified as value-based; an explicit reason for the use of SSA is the simplification brought by value-based analysis:

Being able to perform optimizations on a mostly functional representation is much easier than performing them on a representation that had all of the warts of a real programming language.

— Kenneth Zadeck and Mark N. Wegman, in the preface of Rastello and Bouchez Tichadou (2022).

This chapter is also related to existing symbolic expression analyses that have been proposed, some variable-based (Kildall, 1973; Gulwani and Necula, 2004; Miné, 2006c) and some value-based (Gange et al., 2016; Cousot, 2021, ch.48).

We can also relate it to program verification techniques which are not usually presented as abstract interpretations. For instance, symbolic execution (King, 1976) and bounded model checking (Biere et al., 1999) can be seen as applications of SSA applications on a single trace or on a bounded set of traces. Cousot (1978, , ch. 3.3) already describe symbolic execution as an abstract interpretation (that does not terminate). Deductive verification tools, often use a form of strongest-postcondition calculus (de Bakker and Meertens, 1975) to produce their proof obligation (Flanagan and Saxe, 2001; Leino, 2005), which is close to our SSA translation.

Finally, this chapter is related to analysis and compilation techniques that can be found in compilers. Symbolic expression analysis is a form of I see SSA translation as

4.3.3 Future directions

There are several directions that I would be interested in exploring.

Symbolic abstract transformations and abstract programs

One approach to interprocedural analysis is the relational approach (Sharir and Pnueli, 1981), consisting in computing *abstract transformations* (SAS’20) (i.e., a relation between the input and output of the program) that can be reused on the different call sites. The symbolic expression analysis, possibly completed with symbolic control states, could be used as a precise basis for such abstract transformation; this would correspond to the symbolic relational separate analysis of Cousot and Cousot (2002a). The benefit of such a domain is its precision, as it can exactly represent the behavior of a subroutine. But the summary can also be abstracted, i.e. be made more compact by forgetting information. For instance, to perform a pointer analysis, it is unlikely that floating point computations, functions that perform some output and the computation needed for these output, or string manipulations would provide useful information; these can thus be removed from the summary.

In general, abstracting information during SSA translation allows the generation of *abstract programs*, i.e. abstract models of the program that the program refines,

abstract transformation

abstract program

which I believe have many useful applications. For instance, my PhD student Amira Methni translated C code to TLA+ (FTSCS’14) to perform explicit-state model-checking of the concurrent PharOS OS kernel, and used abstraction (ISCEA’15) and abstract interpretation (Methni, 2016, §5.4.2) to reduce the number of states. In future applications, I would be interested, for instance, in abstracting the writes of a program to a file using pushdown or regular automata, to characterize the contents of a file using a context-free grammar or a regular expression. Another interesting application could be to generate an abstract model of a program’s behaviour regarding its network interactions, to be further analysed by protocol analysis tools.

Cheap-expensive simultaneous analyses

Depending on the level of abstractions, abstract interpretation allows choosing between inexpensive but imprecise analyses (e.g., using intervals (Cousot and Cousot, 1977)), or costly more precise analyses (e.g., using polyhedra (Cousot and Halbwachs, 1978)). Sound and exact symbolic domains like the symbolic value based domain of Section 4.2.3, enables the simultaneous interaction of a cheap analysis (done eagerly), and a slow more precise analysis, done on-demand when the cheap analysis is not sufficient (for instance, to prove an absence of alarm, to refine the result of a `may_be_true` or `may_be_false` query, or more generally when a precision requirement (Bardin et al., 2011) is not met). Indeed, it suffices to redo the analysis using the terms produced by the symbolic expressions. Many analyses use pre-analyses to improve their efficiency (e.g. Oh et al., 2012), and I think better interaction between the first and second analyses could improve the overall efficiency.

In CODEX, when there are unproved assertions about the program, we take the SSA translation of the program leading to the point of the assertion⁹, translate this to Horn clauses, and send it to the SPACER (Gurfinkel, 2022b) solver, a strategy that works quite well on the SVComp benchmark (Lemerre and Bardin, 2017). It would be possible to directly work on Horn clauses (Gange et al., 2015), however I believe that the fact that SSA has more structure than Horn clauses is beneficial (for instance, Horn clauses cannot be nested, and thus inner clauses in require more arguments: the translation from SSA to Horn has a closure conversion pass that identifies captured variables to pass them as argument in Horn clauses).

Rewriting

A final interesting direction is the interaction of rewriting techniques in the symbolic expression analysis. This is often important when analysing machine code (Djoudi et al., 2016), where we have to undo some compiler transformations to analyse the simpler source program, or because simple source expressions (like comparison) are translated into complex sequence of machine instructions. As an example, Julien Simonnet has implemented an induction variable analysis that reverses the strength reduction optimization, replacing counters that are incremented synchronously (e.g., `x += 3`) with references to their induction variable (`x + 3*i`). This automatically creates a relation between these counters.

While destructive rewrites are possible, the use of egraphs (Willsey et al., 2021; Nelson and Oppen, 1980) would be beneficial, as it allows non-destructive rewrites,

⁹Notice that our term-based analyses only take into account the states that lead to some statement, i.e. they automatically perform some program slicing.

which can only improve precision without degrading it. Of course, e-graphs can be expensive, so careful engineering is needed for such an interaction to be possible.

Note that the interaction of classical global value numbering and algebraic simplifications¹⁰ is not easy (Rastello and Bouchez Tichadou, 2022, §11.5.1), and the combinaison of our symbolic expression analysis with rewriting could provide an interesting solution to this problem.

¹⁰Also called instruction combining, notably in LLVM.

Chapter 5

Numerical SSA domains



NUMERICAL ANALYSIS is an important component of static analyses used for formal verification of low-level programs. In addition to analyses where they are the most important component (e.g. analysis of the precision of control-command software or numeric simulations), numerical analysis is useful to verify the absence of many basic but common errors, such as out-of-bound access errors, null pointer dereference, integer overflow, or division by zero.

One can perfectly reuse standard variable-based numerical abstract domains, that concretize into $\mathbb{Z}^n$ or $\mathbb{R}^n$ (such as boxes, octagons (Miné, 2006b) or polyhedra (Cousot and Halbwachs, 1978)), as a value-based or SSA-based domain. It suffices to follow the assignment semantics of SSA, where the "single assignment" is seen as a real assignment, that standard implementations (like Apron (Jeannet and Miné, 2009)) provide; and to map symbolic values or symbolic expressions to a numerical dimension. To avoid having too many variables, dimensions corresponding to symbolic values that no longer appear in the symbolic store can be "garbage collected" (Chang and Leino, 2005; Chang and Rival, 2013).

This is undoubtedly useful, but doing this does not make use of the term information contained in our symbolic abstract domains (it suffices for symbolic values to be implemented as just an integer using a counter to create fresh symbolic values). In this chapter we investigate another direction, which is the combination of symbolic abstract domains and non-relational abstractions (PLDI'24). There are several reasons for this. First, non-relational domains are still useful even in presence of relational domains, because there are many elementary operations that cannot be handled precisely by relational domains (e.g. non-linear arithmetics, bitwise operations, etc.) Second, we want to take advantage of the symbolic information computed in our symbolic abstract domain, and the combination between the symbolic abstract domain and a non-relational domain is a simple and effective way to do that. Indeed, the resulting combination is surprisingly precise when compared to a basic non-relational domain, yet still inexpensive, such that it can be enabled by default as a replacement for a standard non-relational domain (it is, in particular, always more precise than a standard non-relational analysis (PLDI'24)). One of its main characteristics is that it can analyse bytecode (or code decomposed into small parts) as precisely as the source code (PLDI'24), which is useful not only when analysing low-level code like bytecode or machine code, but also simplifies the program transformation pipeline (e.g., our analyser does not attempt to preserve the structure of the original source code, and instead performs an `SSA_Translation` to the simpler language described by

VALUE_BASED_DOMAIN).

While this cheap numerical analysis (described thereafter) is sufficiently accurate for the majority of queries, it can also be easily complemented with a very precise on-demand analysis used only for the remaining queries, as it suffices to convert the SSA form produced in Chapter 4 into Horn clauses and send it to a specialized goal-oriented solver (we chose not to describe this further in the manuscript).

At the end of the chapter, I will describe ongoing and future extensions to further improve the precision of the domain, investigate other numeric abstract domains inhering generally useful properties while still being sufficiently cheap to be enabled by default (and avoiding cost mitigation heuristics like packing (Blanchet et al., 2003)), or take advantage of the immutability of symbolic values and SSA variables.

5.1 Single-value Abstractions

At the heart of a non-relational abstract interpreter is what I call a *single-value abstraction*, i.e. an abstraction which concretize into a set of (single) values. The most well-known examples of single-value abstractions are intervals (Cousot and Cousot, 1977), congruences (Granger, 1989), their combination as strided intervals (Reps et al., 2006), and the bitwise/bitfield/known bits/tristate abstraction (Regehr and Duongsaa, 2006; Monniaux, 2007; Miné, 2012; Vishwanathan et al., 2022). Here, we will use intervals as an abstraction of integers.

single-value abstraction

```
module Integer_Or_Inf:sig
  type t = Integer_Or_Inf.t =
    | Ninf          (* Negative infinity *)
    | Pinf          (* Positive infinity. *)
    | Finite of Z.t (* Finite value. *)
end = Integer_Or_Inf

module Interval:sig
  type t = Integer_Or_Inf.t * Integer_Or_Inf.t
end = Interval
```

An *interval* is just a pair of values in $(\mathbb{Z} \cup \{-\infty\} \times \mathbb{Z} \cup \{+\infty\})$ (written $[m, M]$). Its concretization is the set of all the values in the interval (included):

interval

Definition 5.1.1: Concretization of the interval abstraction

$$\begin{aligned} \gamma_V [m, M] &= \{z \mid m \leq z \leq M\} & \gamma_V [-\infty, +\infty] &= \mathbb{Z} \\ \gamma_V [-\infty, M] &= \{z \mid z \leq M\} & \gamma_V [m, +\infty] &= \{z \mid m \leq z\} \end{aligned}$$

We will not just use intervals: as our expression may have several types (integer and boolean), our single-value abstractions also are multi-sorted. Thus, we will use a combination of the interval and quadrivalent abstraction.

```
module Quadrivalent:sig
  type t = Quadrivalent.t = Bottom | True | False | Top
end = Quadrivalent
```

The *quadrivalent* abstraction is an exact representation of $\mathcal{P}(\text{true}, \text{false})$, i.e.

quadrivalent

Definition 5.1.2: Concretization of the quadrivalent abstraction

$$\begin{aligned} \gamma_v(\text{Bottom}) &\triangleq \{\} & \gamma_v(\text{Top}) &\triangleq \{\text{true}, \text{false}\} \\ \gamma_v(\text{True}) &\triangleq \{\text{true}\} & \gamma_v(\text{False}) &\triangleq \{\text{false}\} \end{aligned}$$

I name this quadrivalent because it corresponds to a four-valued propositional logic.

```

module Interval_Single_Value_Abstraction: sig
  include INTEGER_AND_BOOLEAN_EXPRESSIONS
  val to_quadrivalent: ['boolean] expr → Quadrivalent.t
  val to_interval: ['integer] expr → Interval.t
  val join: τ Typ.t → τ expr → τ expr → τ expr
  val fixpoint_step: τ Typ.t → τ expr → τ expr → τ expr * bool
  module Backward: sig
    val ¬: ['boolean] expr → ['boolean] expr →
      ['boolean] expr option
    val eq: ['integer] expr → ['integer] expr → ['boolean] expr →
      ['integer] expr option * ['integer] expr option
    val leq: ['integer] expr → ['integer] expr → ['boolean] expr →
      ['integer] expr option * ['integer] expr option
    val add: ['integer] expr → ['integer] expr → ['integer] expr →
      ['integer] expr option * ['integer] expr option
    val sub: ['integer] expr → ['integer] expr → ['integer] expr →
      ['integer] expr option * ['integer] expr option
    val mul: ['integer] expr → ['integer] expr → ['integer] expr →
      ['integer] expr option * ['integer] expr option
    val div: ['integer] expr → ['integer] expr → ['integer] expr →
      ['integer] expr option * ['integer] expr option
    val ∧: ['boolean] expr → ['boolean] expr → ['boolean] expr →
      ['boolean] expr option * ['boolean] expr option
    val ∨: ['boolean] expr → ['boolean] expr → ['boolean] expr →
      ['boolean] expr option * ['boolean] expr option
  end
end
end = Interval_Single_Value_Abstraction

```

Finally, the `Interval_Single_Value_Abstraction` is just a `Single_Value_Abstraction` (described in Chapter F) using intervals for the integer abstraction. There, a `['boolean] expr` corresponds to a `Quadrivalent.t`, and a `['integer] expr` corresponds to an `Interval.t` (unfortunately OCaml does not allow type-level computations which would define `type ['boolean] expr = Quadrivalent.t`, hence the need for the conversion functions `to_quadrivalent` and `to_interval`).

The functions provided by this interface, and their soundness requirement, is standard¹. We first have functions for `join` (over-approximation of union), the combined widening/inclusion operator `fixpoint_step`, and forward propagation of abstract values on unary (f_1) and binary (f_2) operators and predicate:

¹Similar ones can be found in (Miné, 2017, §4.6).

Definition 5.1.3: Soundness theorems for forward operations

$$\begin{aligned}
\gamma_v(\text{join } v_1^\# v_2^\#) &\supseteq \gamma_v(v_1^\#) \cup \gamma_v(v_2^\#) \\
(\text{fixpoint_step } v_1^\# v_2^\#) = (v_3^\#, \text{true}) &\Rightarrow \gamma_v(v_1^\#) \supseteq \gamma_v(v_2^\#) \wedge \gamma_v(v_3^\#) \supseteq \gamma_v(v_2^\#) \\
\gamma_v(f_1(v_1^\#)) &\supseteq \{f_1(v_1) \mid v_1 \in \gamma_v(v_1^\#)\} \\
\gamma_v(f_2(v_1^\#, v_2^\#)) &\supseteq \{f_2(v_1, v_2) \mid v_1 \in \gamma_v(v_1^\#), v_2 \in \gamma_v(v_2^\#)\}
\end{aligned}$$

We also have backward operations, that allows refining the arguments of an operation knowing the previous version of the argument, and the value of the result:

Definition 5.1.4: Soundness theorems for backward operations

$$\begin{aligned}
\tilde{f}_1(v_1^\#, v^\#) = v_1'^\# &\Rightarrow \\
&\{(v_1, f_1(v_1)) \mid v_1 \in \gamma_v(v_1'^\#)\} \supseteq \{(v_1, f_1(v_1)) \mid v_1 \in \gamma_v(v_1^\#) \wedge f_1(v_1) \in \gamma_v(v^\#)\} \\
\tilde{f}_2(v_1^\#, v_2^\#, v^\#) = (v_1'^\#, v_2'^\#) &\Rightarrow \\
&\{(v_1, v_2, f_2(v_1, v_2)) \mid v_1 \in \gamma_v(v_1'^\#) \wedge v_2 \in \gamma_v(v_2'^\#)\} \\
&\supseteq \{(v_1, v_2, f_2(v_1, v_2)) \mid v_1 \in \gamma_v(v_1^\#) \wedge v_2 \in \gamma_v(v_2^\#) \wedge f_2(v_1, v_2) \in \gamma_v(v^\#)\}
\end{aligned}$$

In addition, backward propagation ensures that the returned value are more precise than the previous one, or return None if the returned value has the same precision (this allows stopping constraint propagation).

5.2 Shortcomings of standard non-relational analysis

The module `Standard_Non_Relational_Abstract_Domain` in Appendix F presents a small variable-based non-relational abstract domain for the `While_Lang` language (that we do not represent it here for the sake of concision). Such a domain is formalized simply as:

$$\begin{aligned}
\mathbb{S}^\# &\triangleq \mathbb{X} \rightarrow \mathbb{V}^\# \\
\gamma(s^\#) &= \{\sigma \in \Sigma \mid \forall x \in X : \sigma[x] \in \gamma_v(s^\#[x])\}
\end{aligned}$$

i.e., an abstract state is just an abstract store mapping from variable to abstract values, representing stores σ such that each program variable x obeys the corresponding constraint in the abstract store. The implementation in module `Standard_Non_Relational_Abstract_Domain` is small but quite feature-complete, with in particular constraint-programming (Miné, 2017, §4.6) so that we can learn from tests (e.g., `assert(x == 3)`; updates an abstract state $s^\#$ with $s^\#[x \mapsto (3, 3)]$).

I have been involved in the development of the `VALUE` analyser (development community, [n. d.]), predecessor of `Eva`, and all the abstract domains there were built as compositions of non-relational abstractions. There are several advantages to this technique:

1. The analyser structure is simple and compositional: it is easy to build compound abstract states as a composition of smaller abstract elements; to evaluate compound expressions by composition of simpler expressions, etc.
2. Because the abstract state is a composition of independent states and transfer functions are built in a divide-and-conquer style on these smaller states, memoisation (Michie, 1968) is particularly effective. The `VALUE` analyser combines

this with heavy use of trace partitionning (Mauborgne and Rival, 2005; Rival and Mauborgne, 2007), almost transforming the abstract interpreter into an explicit-state model checker, which works very well (in terms of precision and efficiency) on embedded systems (where most memory allocation is finite and recursion and there is no recursion).

Despite these advantages, the non-relational analysis also has a number of issues that cause imprecisions on common code patterns:

1. The *absence of any relation*, not only those that can be inferred by the domain (e.g. by transitive closure in Miné (2006b)), but also those that already exist in the program, is problematic. For instance in the program `y := x + 1; assume (y == 3)`, the domain does not learn that `x = 2` after the test.
2. A related issue is the *code window* (Logozzo and Fähndrich, 2008) problem: the only relation that the analysis can exploit is present in the expressions of the source program statements. Thus, a small refactoring like splitting a statement in two can have a significant impact on precision. For instance, replacing `if (x < 3)` by `tmp = x < 3; if (tmp)` changes what we can learn about `x`. This is problematic when analysing bytecode (Logozzo and Fähndrich, 2008) or machine code, as this is the kind of transformations that compiler do; but also when analysing automatically-generated C code, or when using the analyser as a continuous integration (the fact that some refactoring changes the analyser precision is inevitable, but it is better to avoid it when possible)
3. A last issue is that we cannot remember information that is not on a variable. For instance, the domain cannot prove the assertion in the program `while (x < n) assert (x < n)`; (a very common situation when checking array bound accesses), as there is nothing to learn on `x` or `n` from the test `x < n`. Similarly, the domain cannot retain any information after the test `if (x * y == 0)`.

Those are very common patterns. In fact, the need to gain precision on these patterns that we often encountered in case studies with VALUE is what prompted me to begin working on symbolic value-based static analysis. And indeed, we'll see that this solves all of these problems almost for free (the non-relational standard domain and the cartesian value-based domain have the same complexity in terms of memory and CPU time), thanks to a simple combination of the cheap relational symbolic expression analysis, with a single-value abstraction.

5.3 Cartesian SSA-based analysis

We can now describe our cartesian SSA-based abstract domain. Following Cousot (2021) I call this domain cartesian instead of non-relational, because it is relational, thanks to the relations provided by the symbolic expressions. In fact, the main feature of this domain is the reduction (or constraint propagation) between the cartesian information and the information contained in the symbolic expression. I also call this domain SSA-based (instead of value-based) because it is an `ABSTRACT_ENVIRONMENT` making use of the information available in symbolic expressions (the symbolic values are opaque in a `VALUE_BASED_DOMAIN`).

This domain is simple, but the combination between a symbolic expression analysis and single-value abstraction is surprisingly powerful yet almost as efficient as a

standard non-relational domain. Thus, it is an interesting "base" abstraction that can be enabled by default in any abstract interpreter, as a replacement for the standard non-relational domain (as this domain is always more precise than the standard non-relational domain, as we show theoretically and empirically in [PLDI'24](#)).

Definition A *Cartesian_Abstract_Environment* $\rho^\# \in \wp^\#$ is just a partial mapping $\hat{\mathbb{E}} \rightarrow \mathbb{V}^\#$ from symbolic expressions to abstract values in a single value abstraction (SVA). We use the custom syntax $\rho^\#.\text{@}[\text{term}]$ to denote access to the abstract environment.

```

module SVA = Interval_Single_Value_Abstraction
module Value = struct type ( $\kappa, \mu$ ) t =  $\kappa$  SVA.expr end
module TermMap = PatriciaTree.MakeHeterogeneousMap(Term_With_Symvar)(Value)
let ( .@[] )  $\rho^\#$  term = TermMap.find term  $\rho^\#$ 
module Cartesian_Abstract_Environment = struct
  type t = unit TermMap.t
  let entry = TermMap.empty

```

Concretizations Similarly to Section 4.2.3, we can provide several meanings for an abstract environment.

Definition 5.3.1: Concretizations for abstract environments

$$\begin{aligned}
\gamma_v &: \wp^\# \rightarrow \mathcal{P}(\mathcal{V}) \\
\gamma_v(\rho^\#) &\triangleq \{v \in \mathcal{V} \mid \text{dom}(v) = \text{dom}(\rho^\#) \wedge \forall \hat{e} \in \text{dom}(v) : v[\hat{e}] \in \gamma_v(\rho^\#[\hat{e}])\} \\
\gamma_\rho &: \wp^\# \rightarrow \mathcal{P}(\wp) \\
\gamma_\rho(\rho^\#) &\triangleq \{\rho \in \wp \mid \exists v \in \gamma_v(\rho^\#) : \forall \hat{e} \in \text{dom}(v) : v[\hat{e}] = \text{eval}(\hat{e}, \rho)\} \\
\gamma'_v &: \wp^\# \rightarrow \mathcal{P}(\mathcal{V}) \\
\gamma'_v(\rho^\#) &\triangleq \{[\hat{e} \in \text{dom}(\rho^\#) \mapsto \text{eval}(\hat{e}, \rho) \mid \rho \in \gamma_\rho(\rho^\#)]\}
\end{aligned}$$

Remember that a concrete valuation $v \in \mathcal{V} \triangleq \hat{\mathbb{E}} \rightarrow \mathbb{V}$ is a mapping from symbolic expressions to values, while a concrete environment $\rho \in \wp \triangleq \hat{\mathbb{X}} \rightarrow \mathbb{V}$ is a mapping from symbolic variables to values. Symbolic expressions are expressions that can contain symbolic variables.

In the first concretization γ_v , the abstract environment is non-relational: every term, seen as an opaque symbolic value, is constrained independently. In the second concretization γ_ρ , abstract environments are a relational abstraction: the terms are interpreted as constraints over the symbolic variables that they contain, not as independent dimensions on their own. The constraint over each symbolic variable $\hat{x}$ is the intersection of all the constraints over all the terms in which $\hat{x}$ occurs. The fact that terms are related (as they can share symbolic variables) makes this abstraction relational. Finally, the last concretization γ'_v directly derives from γ_ρ , and is also relational. As it concretizes to the same object as γ_v , this allows comparison.

All of these concretizations are useful. γ_v expresses the information that is directly available about each term, and γ'_v represents what can be inferred (it is an upper limit of what constraint propagation can learn). The second concretization is always more precise than the first: $\forall \rho^\# \in \wp^\# : \gamma'_v(\rho^\#) \subseteq \gamma_v(\rho^\#)$. γ_ρ represents what can be inferred about symbolic variables. Because the evaluation of expressions is deterministic, γ_ρ and γ'_v carry the same information.

Forward propagation

```

let eval (type a) (ê:a Term.t) ρ# =
  let ar2 f arg1 arg2 =
    let v = f ρ#.[arg1] ρ#.[arg2] in
    TermMap.add ê v ρ#
  in
  let open Term in match ê with
  | Ar0{symb=Const k} → TermMap.add ê (SVA.const k) ρ#
  | Ar0{symb=True} → TermMap.add ê SVA.true ρ#
  | Ar0{symb=Unknown (module T)} → begin match T.Tag with
    | Typ.Boolean.Tag → TermMap.add ê (SVA.unknown_boolean()) ρ#
    | Typ.Integer.Tag → TermMap.add ê (SVA.unknown_integer()) ρ#
  end
  | Ar1{symb=Not;arg} → TermMap.add ê (SVA.¬ (ρ#.[arg])) ρ#
  | Ar2{symb=Add;arg1;arg2} → ar2 SVA.add arg1 arg2
  | Ar2{symb=Sub;arg1;arg2} → ar2 SVA.sub arg1 arg2
  | Ar2{symb=Mul;arg1;arg2} → ar2 SVA.mul arg1 arg2
  | Ar2{symb=Div;arg1;arg2} → ar2 SVA.div arg1 arg2
  | Ar2{symb=Eq;arg1;arg2} → ar2 SVA.eq arg1 arg2
  | Ar2{symb=Leq;arg1;arg2} → ar2 SVA.leq arg1 arg2
  | Ar2{symb=And;arg1;arg2} → ar2 SVA.∧ arg1 arg2
  | Ar2{symb=Or;arg1;arg2} → ar2 SVA.∨ arg1 arg2
  | _ → assert false

```

The `eval` function just interprets the term to update the `TermMap`. Here the evaluation is eager, i.e., the information about a term exists as soon as it is created (in [PLDI'24](#), the evaluation was performed lazily). Laziness can be a virtue (in particular because it simplifies formalisation, but it can also be interesting in very expensive domains like k -sets [Bardin et al. \(2011\)](#)). However, laziness has a cost in precision when the domain is not distributive: in the program `if(x == 1) y = x * x; else if (y == -1) y = x * x; else while(1); assert(y == 1)`; the lazy evaluation of $\hat{x} \times \hat{x}$ returns the interval $[-1; 1]$ (and the assertion cannot be proved), while the eager analysis can prove the assertion. This is an instance of the fact that in abstract domains we lose precision if we do not apply reduction/closure/constraint propagation before a join operation. On the contrary, our domain could perform some trace partitioning [Mauborgne and Rival \(2005\)](#); [Rival and Mauborgne \(2007\)](#) by just "proactive" evaluation of a term before doing a `join`.

Backward propagation

```

let backward_eval (type a) (ê:a Term.t) ρ# : t * Term.any list =
  let ar1 f arg ê =
    match f (ρ#.[arg]) (ρ#.[ê]) with
    | None → ρ#, []
    | Some v → TermMap.add arg v ρ#, [Term.Any arg] in
  let ar2 f arg1 arg2 ê =
    let v1,v2 = f (ρ#.[arg1]) (ρ#.[arg2]) (ρ#.[ê]) in
    let ρ#, changed = match v1 with
    | None → ρ#, []
    | Some v1 → TermMap.add arg1 v1 ρ#, [Term.Any arg1] in
    let ρ#, changed = match v2 with

```

```

| None →  $\rho^\#$ , changed
| Some v2 → TermMap.add arg2 v2  $\rho^\#$ , (Term.Any arg2)::changed in
 $\rho^\#$ , changed in
let open Term in match  $\hat{e}$  with
| Ar0{symb=Const k} →  $\rho^\#$ , []
| Ar0{symb=True} →  $\rho^\#$ , []
| Ar0{symb=Unknown _} →  $\rho^\#$ , []
| Ar0{symb=Term_With_Symvar.Var _} →  $\rho^\#$ , []
| Ar1{symb=Not;arg} → ar1 SVA.Backward. $\neg$  arg  $\hat{e}$ 
| Ar2{symb=Add;arg1;arg2} → ar2 SVA.Backward.add arg1 arg2  $\hat{e}$ 
| Ar2{symb=Sub;arg1;arg2} → ar2 SVA.Backward.sub arg1 arg2  $\hat{e}$ 
| Ar2{symb=Mul;arg1;arg2} → ar2 SVA.Backward.mul arg1 arg2  $\hat{e}$ 
| Ar2{symb=Div;arg1;arg2} → ar2 SVA.Backward.div arg1 arg2  $\hat{e}$ 
| Ar2{symb=Eq;arg1;arg2} → ar2 SVA.Backward.eq arg1 arg2  $\hat{e}$ 
| Ar2{symb=Leq;arg1;arg2} → ar2 SVA.Backward.leq arg1 arg2  $\hat{e}$ 
| Ar2{symb=And;arg1;arg2} → ar2 SVA.Backward. $\wedge$  arg1 arg2  $\hat{e}$ 
| Ar2{symb=Or;arg1;arg2} → ar2 SVA.Backward. $\vee$  arg1 arg2  $\hat{e}$ 
| _ → assert false

```

Backward evaluation takes a term $\hat{e}$ and an abstract environment $\rho^\#$, applies the SVA.Backward transfer functions, to possibly update the subterms of $\hat{e}$, and return the list of subterms that have been modified (to continue the constraint propagation with these terms).

Constraint propagation

```

let assume  $\hat{e}$   $\rho^\#$  =
  match SVA.to_quadrivalent  $\rho^\#$ .@[ $\hat{e}$ ] with
  | Quadrivalent.(False | Bottom) → None
  | Quadrivalent.True → Some  $\rho^\#$ 
  | Quadrivalent.Top → begin
    let  $\rho^\#$  = TermMap.add  $\hat{e}$  SVA.true  $\rho^\#$  in
    let module Worklist = Set.Make(struct
      type t = Term.any
      let to_int (Term.Any  $\hat{e}$ ) = Term.to_int  $\hat{e}$ 
      let compare a b = (to_int b) - (to_int a)
    end) in
    let worklist = Worklist.singleton (Term.Any  $\hat{e}$ ) in
    let rec loop worklist  $\rho^\#$  =
      match Worklist.max_elt worklist with
      | exception Not_found →  $\rho^\#$ 
      |  $\hat{e}$  →
        let worklist = Worklist.remove  $\hat{e}$  worklist in
        let Term.Any  $\hat{e}$  =  $\hat{e}$  in
        let  $\rho^\#$ , l = backward_eval  $\hat{e}$   $\rho^\#$  in
        let worklist = List.fold_right Worklist.add l worklist in
        loop worklist  $\rho^\#$  in
    Some(loop worklist  $\rho^\#$ )
  end
end

```

The `assume` function first queries the value of a condition; if Top, it sets the condition to `true` and then performs a constraint propagation using a variant of the AC-3

algorithm (Mackworth, 1977) known as HC-4 (Benhamou et al., 1999). This algorithm maintains a worklist of terms that have been modified, and handles each until a fixpoint is reached. The HC-4 variant consists in doing that based on the term order (note that this order is a benefit of SSA form compared to, e.g., Horn clauses). Here we only do a backward propagation pass, but alternating between backward and forward passes is also possible. In CODEX, we also set a constant upper bound on the number of propagations that can be performed, in particular as a work-around to the slow convergence phenomena (Bordeaux et al., 2007).

join

```
let join  $\phi$   $\rho_1^\#$   $\rho_2^\#$  =
  (* First: intersection of the common terms. *)
  let  $\rho^\#$  =
    let f  $\hat{e}$  v1 v2 = SVA.join (Term_With_Symvar.typeof  $\hat{e}$ ) v1 v2 in
    TermMap.idempotent_inter {f}  $\rho_1^\#$   $\rho_2^\#$  in
  (* Second: add the terms whose name change in phi. *)
  let f  $\hat{e}_1$   $\hat{e}_2$   $\hat{e}$  acc =
    let v = match  $\hat{e}_1, \hat{e}_2$  with
      | None, None  $\rightarrow$  assert false
      | Some  $\hat{e}_1$ , None  $\rightarrow$   $\rho_1^\#$ .@[ $\hat{e}_1$ ]
      | None, Some  $\hat{e}_2$   $\rightarrow$   $\rho_2^\#$ .@[ $\hat{e}_2$ ]
      | Some  $\hat{e}_1$ , Some  $\hat{e}_2$   $\rightarrow$ 
        SVA.join (Term_With_Symvar.typeof  $\hat{e}_1$ )  $\rho_1^\#$ .@[ $\hat{e}_1$ ]  $\rho_2^\#$ .@[ $\hat{e}_2$ ]
    in TermMap.add  $\hat{e}$  v acc
  in
  Phi.fold {f}  $\phi$   $\rho^\#$ 
```

Remember the definition of unbind in Section 4.2.1: the symbolic values (here, terms) that have not been evaluated in all the execution paths are inaccessible after a join operation (this corresponds to the dominance property of SSA (Rastello and Bouchez Tichadou, 2022)). Consequently, we get rid of the terms that are not in both abstract environments by performing an intersection of the abstract environments. After that, we add the bindings that come from the ϕ function. Note that, following the semantics, it would be more logical to first use the ϕ function to duplicate the bindings in both abstract environments, and then to perform an intersection. However, in the cartesian abstract environment, doing the intersection first is equivalent and slightly more efficient. Also note that if we used the assignment semantics of SSA instead of the binding semantics (POPL'23; Schneider, 2013), this operation would have to perform a union of abstract environments, instead of an assignment, and would thus be less efficient.

fixpoint_step

```
let fixpoint_step ~includes  $\phi$  ~previous ~next =
  let r_inc = ref includes in
  (* First compare the common terms. *)
  let  $\rho^\#$  =
    let f key v1 v2 =
      let v, inc = SVA.fixpoint_step (Term_With_Symvar.typeof key) v1 v2 in
      r_inc := !r_inc  $\wedge$  inc; v
    in
```

```

TermMap.idempotent_inter {f} previous next in
(* Second: compare the terms that change. *)
let f  $\hat{e}_1$   $\hat{e}_2$   $\phi\_res$  acc =
  let v = match  $\hat{e}_1, \hat{e}_2$  with
  | None, None → assert false
  | Some  $\hat{e}_1$ , None → assert false (* Keys should not disappear *)
  | None, Some  $\hat{e}_2$  → r_inc := false; next.@[ $\hat{e}_2$ ]
  | Some  $\hat{e}_1$ , Some  $\hat{e}_2$  →
    let v, bool =
      SVA.fixpoint_step (Term_With_Symvar.typeof  $\hat{e}_1$ )
        previous.@[ $\hat{e}_1$ ] next.@[ $\hat{e}_2$ ] in
    r_inc := !r_inc  $\wedge$  bool; v
  in TermMap.add  $\phi\_res$  v acc
in
let  $\rho^\#$  = Phi.fold {f}  $\phi$   $\rho^\#$  in
 $\rho^\#, !r\_inc$ 

```

The code for `fixpoint_step` is very similar to the code of `join`; the only difference is that it also checks for inclusion, and relies on the `fixpoint_step` function of module `Interval_Single_Value_Abstraction`, instead of `join`.

More precise constraint propagation

```

let assume  $\hat{e}$   $\rho^\#$  = let open Term in
  let ext_join a b = match a, b with
  | None, x | x, None → x
  | Some a, Some b → Some(join Phi.empty a b) in
  let rec assume_pos: ['boolean] Term.t → _ → _ =
    fun  $\hat{e}$   $\rho^\#$  → match  $\hat{e}$  with
    | Ar0{symb=True} → Some  $\rho^\#$ 
    | Ar2{symb=And;arg1;arg2} →
      Option.bind (assume_pos arg1  $\rho^\#$ ) (assume_pos arg2)
    | Ar2{symb=Or;arg1;arg2} →
      let  $\rho_1^\#$  = assume_pos arg1  $\rho^\#$  in
      let  $\rho_2^\#$  = assume_pos arg2  $\rho^\#$  in
      ext_join  $\rho_1^\#$   $\rho_2^\#$ 
    | _ → assume  $\hat{e}$   $\rho^\#$ 
  in assume_pos  $\hat{e}$   $\rho^\#$ 

```

Here we redefine `assume` for a more precise handling of disjunctions: assuming that $c_1 \vee c_2$ holds now perform two constraint propagations yielding two different states, that are then `joined`. This could be pushed further using De Morgan's law to systematically push down negations to atomic predicates.

Queries

```

let fold_on_values term  $\rho^\#$  ~min ~max acc f =
  let itv_min, itv_max = SVA.to_interval  $\rho^\#$ .@[term] in
  let open Integer_Or_Inf in (* Intersection of itv with (min,max) *)
  let min = match itv_min with Ninf → min | Finite m → Z.max min m in
  let max = match itv_max with Pinf → max | Finite m → Z.min max m in
  let rec loop i acc =

```

```

    if Z.gt i max then acc
    else loop (Z.succ i) (f i acc)
  in loop min acc
end

```

The implementation of the query function `fold_on_values` is straightforward, but note that the bounds guarantee termination.

Using the numerical domain

```

module Numerical_SSA_Domain: INTEGER_AND_BOOLEAN_VALUE_BASED_DOMAIN =
  Make_SSA_Domain(Cartesian_Abstract_Environment)

```

We can use the `Cartesian_Abstract_Environment` to build an `Integer_and_Boolean_Value_Based_Domain`, using the symbolic domain `Make_SSA_Domain` presented in Chapter 4. Compared to the previous instantiation (the module `Unit_SSA_Domain`), this new domain computes semantics numerical properties about the symbolic expressions being created by the SSA domain. The analysis of the SSA program is performed while the SSA translation is being performed; the results of this analysis can help the SSA translation (for instance, dead conditional branches are removed, which automatically means that there will be fewer variables created by a ϕ in the SSA translation).

```

module SSA_Translate_Numerical = SSA_Translation(Numerical_SSA_Domain)
module Example_SSA_Translate_Numerical =
  Example(Fixpoint(SSA_Translate_Numerical));;
let initial_state = SSA_Translate_Numerical.entry_state in
let final_state = Option.get @@ Example_SSA_Translate_Numerical.factorial5
  initial_state in
let output = Format.asprintf "Result: @[<hv>%a@ %a@"
  SSA_Translate_Numerical.Abstract_Store.pp final_state.store
  Numerical_SSA_Domain.pp_ssa_state final_state.ssa_state in
assert (output = "Result: [x -> x11 * x12 y -> x11 * x12 + x11 ]
  {abs_environment:[1 -> [1,1]
    0 -> [0,0]
    5 -> [5,5]
    x11 -> [1,+oo]
    x12 -> [0,0]
    x12 ≤ 0 -> True
    not(x12 ≤ 0) -> False
    x11 * x12 -> [0,0]
    not(not(x12 ≤ 0)) -> True
    x11 * x12 + x11 -> [1,+oo]
  ];condition:not(not(x12 ≤ 0))}");;

```

Finally, we apply the analysis on `factorial5`, which returns again the SSA translation of Figure 4.2, but this time also with an abstract environment mapping all the symbolic terms to their abstract value.

5.4 Related work and possible directions

Additional properties of the domain In [PLDI'24](#) we prove additional properties about the SSA-based cartesian domain. The first is that the domain enjoys strong relative completeness, defined by [Logozzo and Fähndrich \(2008\)](#) as requiring that the analysis of bytecode does not lose information (compared to the analysis of source code) when using the same abstract domain. This paper discusses the loss of precision induced by bytecompilation (i.e., by decomposing the statements in smaller atomic operations, similar to tree-address code), which is a problem that symbolic domains can help solve; the fact that our domain enjoys strong relative completeness is a nice demonstration of this fact.

Another interesting property is that the SSA-based cartesian domain is always more precise than the standard non-relational domain. We prove this using a simple Galois connection between [SSA_Translation\(Numerical_SSA_Domain\)](#) and [Standard_Non_Relational_Abstract_Domain](#), whose abstraction function consists in forgetting about the symbolic expressions:

$$\alpha : \hat{\Sigma} \times \rho^\# \rightarrow \mathbb{S}^\# \quad \alpha(\hat{\sigma}, \rho^\#) = [x \in \mathbb{X} \mapsto \rho^\#[\hat{\sigma}[x]]]$$

Finally, a close look at the complexity of the domain operations in [SSA_Translation\(Numerical_SSA_Domain\)](#) shows that the domain is quite efficient: [assign](#) takes $\log(|\hat{\Sigma}|)$ time, [join](#) and [fixpoint_step](#) takes a time proportional to the difference in the store and abstract environments (generally small), and the number of propagations in [assume](#) can be given an upper bound. This makes the domain compelling as a drop-in replacement to the standard non-relational numerical domain.

Dimensions of value-based abstract domains In general, value-based analysis can be a good replacement for variable-based analysis. A possible issue is the dimension of the numerical domain: as can be seen in our example, the number of "live" symbolic expressions can be larger than the number of program variables $|\mathbb{X}|$, which is problematic as the efficiency of relational abstract domains is supra-linear in terms of this dimension.

However, if we are only interested in state properties, there is no need to keep track of all the reachable symbolic expressions; it suffices to use those that appear in the symbolic store, and eliminate the other (which amounts to an existential projection or "garbage collection" ([Chang and Leino, 2005](#); [Chang and Rival, 2013](#))). With this operation, value-based relational abstract domains always have fewer dimensions than a variable-based analysis, as some values may be common between some variables (as is the case after an assignment $y := x$), without any loss of precision.

In the cartesian abstract domain, having fewer dimensions is also beneficial, not only for efficiency, but also for precision. Indeed, if x and y share the same symbolic value, then anything learned about x will also be learned about y (which is not the case in the standard non-relational domain).

Because of these considerations, Global Value Numbering is always beneficial in a value-based analysis, as it improves both efficiency and precision by reducing the number of symbolic values.

Rewriting One way to further reduce the number of symbolic values is by performing some rewrites. In the example, we could rewrite $\text{not}(\text{not}(\hat{x}_{12} \leq 0))$ to the equivalent $\hat{x}_{12} \leq 0$, that already appears in the abstract environment. We can also

take advantage of the abstract environment to replace symbolic terms when they are constants; e.g. replace $\hat{x}_{12}$ with 0, $\hat{x}_{12} \leq 0$ with True, and then $\hat{x}_{11} * \hat{x}_{12} + \hat{x}_{11}$ with $\hat{x}_{11}$, etc. Such rewrite rules could be automatically generated, or obtained from peephole optimizers (Mukherjee and Regehr, 2024).

However, one must be careful when doing such rewrites, as this may introduce imprecision. For instance, an abstract environment may contain that $0 * (3/\hat{x}_{27}) \in [0; 0]$, but replacing the term $0 * (3/\hat{x}_{27})$ by 0 would lose precision, as the term is bound to the empty set if we later find out that $\hat{x}_{27}$. To avoid this, the condition $\hat{x}_{27} = 0$ must be remembered somewhere, either "locally" if optional values Liu and Rival (2015b); Siegel et al. (2013) are allowed, or globally if this condition means that the whole state is bottom. For instance, in PLDI'24 we propose a transformation that adds conditions on undefined behaviours such as division by zero, and would thus allow remembering the condition (and we should improve our cartesian SSA-based analysis to return a bottom state if one of the current conditions becomes false).

Different programs that are functionally equivalent can yield different results when analysed with the same abstract interpreter, and therefore it is possible to modify a program to improve the precision of its analysis (Giacobazzi and Mastroeni, 2016; Bruni et al., 2020). The use of symbolic abstract domains where symbolic expressions can be rewritten is a practical method to do this. An example of precision-improving rewrite is transforming $\hat{e} + \hat{e}$ into $2 * \hat{e}$, which allows the congruence domain (Granger, 1989) to infer that the expression is even. But note that the reverse transformation would decrease precision; thus developing a framework for precision-enhancing rewrites would be an interesting task. A useful tool for this is the equivalence graph (e-graph) (Nelson, 1980; Willsey et al., 2021), which allows performing rewrites without losing the previous expression, and thus guarantees an increase of precision (at the cost of having to maintain several versions of the expression).

A particular form of rewriting is the computation of closed forms for our symbolic expressions (which are recursive if we account for the definition of symbolic variables). A *closed form* is an expression whose only variables are the loop counters (and the program inputs). For instance, my PhD student Julien Simonnet has implemented an induction variable analysis (Rastello and Bouchez Tichadou, 2022), where in Figure the definition $\hat{x}_{12} = \text{head}(\phi(5, \hat{x}_{12} - 1))$ would be replaced by the expression $5 + (-1) * \hat{x}_i$ where $\hat{x}_i$ is a new symbolic variable used as a loop counter. This rewrites allows two symbolic expressions to become related, as they would both become expressions over the same loop counter. This rewrite is also often a simple way to find an upper bound of the number of loop iterations (which allows loop unrolling). The technique is related to algebraic program analysis (Tarjan, 1981a; Kincaid et al., 2021) (which features a closure operator for computing closed form, but does not take context into account) and abstract acceleration (Gonnord and Schrammel, 2014; Jeannet et al., 2014).

closed form

Labeled Union-find In addition to rewriting, we can sometime reduce the number of symbolic values if we can express some values as a function over another value. Consider the symbolic expression $\hat{e}_2 \triangleq \hat{e}_1 + 3$: it is linked to the symbolic expression $\hat{e}_1$ by the property that in any valuation, $v[\hat{e}_2] = v[\hat{e}_1] + 3$. Consequently, we may not have to represent $\hat{e}_2$ in an abstract environment; it may suffice to recover its value from $\hat{e}_1$.

This line of reasoning led Dorian Lesbre and me (NSAD'24; PLDI'25), with the

later help from Hichem Rami Ait-El-Hara and François Bobot, to start investigating a new family of abstract domain, based on a structure that we name the labeled-union-find data structure. The structure works like the classical union-find structure (Galler and Fischer, 1964; Tarjan and van Leeuwen, 1984), except that the link to the parent contains a label. Labels are equipped with a composition relation so that you can retrieve the label between any two related members by transitivity; moreover the composition of relation on every path must be the same, which implies that the natural structure for labels is a group. We studied the group of abstract relations and found that they must describe injective functions between equivalence classes of values. Interesting examples of abstract relations include $y = q * x + b$ on integers and $y = x \text{ rot } a \text{ xor } b$ with a, b and q constant, among many others.

I believe that this family of abstract domain is very promising. By itself, it can infer many useful relations, and its low-cost should make it activable by default. It is also useful in conjunction with other domains: by considering "equivalence classes" of symbolic values, we can improve the efficiency of constant propagation in `Cartesian_Abstract_Environment`, or of other relational abstract domains whose complexity depends on the dimension.

Sparse and on-demand analysis A final direction that I would like to investigate are sparse and on-demand analyses. The analyses that I present in this manuscript are dense: at each program location ℓ , we have a mapping from each program variable $\hat{x}$ to a symbolic expression $\hat{e}$; and we have a mapping from many symbolic expressions $\hat{e}$ to an abstract value $v^\#$. Most of these mappings are the same between different program locations. Although (notably with the `PatriciaTree` library that we have developed in Codex) we try to maximize and take advantage of the sharing between subtrees in each map, this is a source of inefficiency; it would make more sense to not repeat these bindings, which is the idea behind *sparse analyses*. One idea that I have explored is to replace, in the `Cartesian_Abstract_Environment`, the mapping "location $\rightarrow$ symbolic expression $\rightarrow$ abstract value" by a mapping "symbolic expression $\rightarrow$ location $\rightarrow$ abstract value", where only "meaningful" locations would be kept. A fundamental property of our SSA form is that scopes are monotonic:

sparse analysis

Theorem 1 (Monotonic Scopes). *If control location ℓ dominates ℓ' , then the possible values for a symbolic expression at ℓ includes the possible values for symbolic expressions at ℓ' .*

This makes it possible to make use of interesting data structures like context trees Wegbreit (1976); Dietz (1982). This would resemble dynamic generation of an e-SSA Bodík et al. (2000) or SSI Ananian (2001) form, which allow sparse analyses with constraint propagation of tests in SSA Tavares et al. (2014). There has also been interesting work in the direction of sparse relational analyses (e.g. Miriaz and Pichardie, 2022; Paisante et al., 2016; Maalej et al., 2018) that I would like to investigate.

Another interesting aspect is on-demand analysis, where costly analyses is done only if cheaper analyses cannot handle an important query. The simplified program built by our symbolic abstract domain could be used as the basis of such on-demand analyses. We already explained that we can convert our generated SSA program to Horn clause, but cheaper alternatives could be also be used, for instance Bodík et al. (2000) is an effective method for verifying bound checks.

Chapter 6

Abstract Abstract Datatypes

 IN CHAPTER 4, the `VALUE_BASED_DOMAIN` signature defines two main opaque types: the abstract state $\mathbb{S}^\#$, and the symbolic value (of type τ) $\tau \mathbb{V}$. In Chapter 5 we saw that different abstract states are possible (for different numerical domains), but we only mentioned two kind of symbolic values: symbolic expressions, and unique identifiers (using integers). What we will see in this chapter is that many data structures can be used as symbolic values; we call these data structures *abstract abstract datatypes* (AADT for short). This name reflects both that AADT are abstract datatypes used to define abstractions, but also that they can define abstractions of abstract datatypes. We actually already saw one example of AADT: the `Symbolic_Store`, which holds symbolic values to represent a concrete store.

AADT allows to build abstractions of compound objects, like memory heaps, stores, environments, records, arrays, variants, etc., modularly, by composition of abstractions of simpler objects. But despite this modularity, it still allows preservation of the relations between the values contained in these compound datastructures. I think this dilemma between modularity (which requires decomposition in independent parts) and relations is the reason why implementing a precise library of memory abstractions is known to be difficult. AADT is a way to solve this problem, and we will present a variety of precise abstractions of compound objects in this chapter. Actually, all the abstractions in this chapter are exact (when the concretization of the base domain is exact Chapter 4), and quite precise when the base domain is doing approximations.

A benefit of AADTs is that transfer functions are simple (abstract) evaluation of referentially transparent expressions, making their implementation similar to writing a functional program; in particular, we do not require implementing complex operations like constraint propagation or assignment, as found in variable-based analysers. The absence of side-effects in the abstract helps combining multiple abstraction and proving the soundness of their combination. Apart from transfer functions, the most important operation defined on abstract abstract datatypes is anti-unification, defined using our `serialize` operation.

In this chapter, we will begin by defining what is an AADTs (Section 6.1), then we will give some examples (Section 6.2), covering in particular the use of AADTs to represent disjunctions. In Chapter 7, we use AADTs to build a different memory abstractions, such as a whole-program analyser for a C-like memory model (Section 7.1), capable of exact translation from a C-like language to a SSA program that

abstract abstract datatype

AADT

only has numerical variables.

6.1 A theory of stateless AADTs

To sum up, an AADT is a data structure containing symbolic values, that becomes itself a symbolic value. AADTs must implement some operations and follow a specification provided by soundness theorems. In this section, we define the interface of AADT and we illustrate some of the concepts.

6.1.1 Interface

A formal interface to AADT, containing all the definitions necessary to define and prove sound an AADT, is provided in Definition 6.1.1. An OCaml signature, defining only the implementation parts for an abstraction of a pair, is defined in the module type `PAIR_ABSTRACTION`.

Definition 6.1.1: Formal interface of AADT

Imported definitions (signature of a `VALUE_BASED_DOMAIN`):

$\text{type } \hat{S}^\#$	Underlying set of abstract states
$\alpha \in \mathcal{Q}$	Base symbolic values
$\underline{v} \in \mathbb{V}$	Base concrete values
$\underline{v} \in \underline{\mathcal{V}} \triangleq \mathcal{Q} \rightarrow \mathbb{V}$	Base valuations
$\phi \in \Phi \triangleq \mathcal{Q} \times \mathcal{Q} \rightarrow \mathcal{Q}$	Base anti-unification
$\gamma_{\hat{S}^\#} : \hat{S}^\# \rightarrow \mathcal{P}(\underline{\mathcal{V}})$	Concretisation for abstract states

Imported (recursively) and exported definitions:

$\alpha \in \mathcal{Q}$	Definition of AADT (extended symbolic value)
$v \in \mathbb{V}$	Concrete values (concrete ADT)
$\gamma_{\mathcal{Q}} : \mathcal{Q} \times \hat{S}^\# \rightarrow \mathcal{P}(\mathbb{V} \times \underline{\mathcal{V}})$	Concretization for AADT
$\text{op}_n^\# : \mathcal{Q}^n \times \hat{S}^\# \rightarrow \mathcal{Q} \times \hat{S}^\#$	Basic operations
$\text{assume}^\# : \mathcal{Q} \times \hat{S}^\# \rightarrow \hat{S}^\#$	Extended assume
$\Phi_\phi : \mathcal{Q} \times \mathcal{Q} \rightarrow \mathcal{Q}$	Anti-unification (generalization) of AADT
$\text{serialize} : (\mathcal{Q} \times \hat{S}^\#) \times (\mathcal{Q} \times \hat{S}^\#) \times \mathcal{P}(\mathcal{Q} \times \mathcal{Q}) \rightarrow \mathcal{P}(\mathcal{Q} \times \mathcal{Q}) \times (\Phi \times \hat{S}^\# \rightarrow \mathcal{Q} \times \hat{S}^\#) \times \hat{S}^\# \times \hat{S}^\#$	

```

module type PAIR_ABSTRACTION = sig
  module Base:VALUE_BASED_DOMAIN;;   open Base

  type Q_1           type Q_2           type Q = Q_1 * Q_2

  (* Unused functions, that closely correspond to the formal definition
   . *)
  val _cons: Q_1 → Q_2 → S# → Q * S#
  val _first: Q → S# → Q_1 * S#
  val _second: Q → S# → Q_2 * S#

```

```

(* The same functions using the state option monad. *)
open Polymorphic_State_Option_Monad
val cons: ( $\mathcal{Q}_1, \hat{S}^\#$ ) m → ( $\mathcal{Q}_2, \hat{S}^\#$ ) m → ( $\mathcal{Q}, \hat{S}^\#$ ) m
val first: ( $\mathcal{Q}, \hat{S}^\#$ ) m → ( $\mathcal{Q}_1, \hat{S}^\#$ ) m
val second: ( $\mathcal{Q}, \hat{S}^\#$ ) m → ( $\mathcal{Q}_2, \hat{S}^\#$ ) m

val serialize:  $\mathcal{Q} \rightarrow \hat{S}^\# \rightarrow \mathcal{Q} \rightarrow \hat{S}^\# \rightarrow \tau$   $\phi\_arguments \rightarrow$ 
  ( $\mathcal{Q}, \tau$ ) serialization *  $\hat{S}^\#$  *  $\hat{S}^\#$ 
end

```

An AADT defines new symbolic values in a `VALUE_BASED_DOMAIN` without changing the abstract state. Thus, AADTs depend on the definition of a base domain. In the code, the signature requires a Base module of signature `VALUE_BASED_DOMAIN`, while in the formal definition, we reference the components of a value-based domain (symbolic values, concrete values, concretization, etc.) that we distinguish by underlining them.

Regarding the concretization $\gamma_{\hat{S}^\#}$, we only require the abstract state to concretize into a set of valuations (i.e., we do not need the possible control or trace information that may be contained in the abstract state). All the different concretizations defined in Section 4.2.3 can be used; each correspond to a different meaning for the AADT. We also require the concretization for an abstract state $\hat{S}^\#$ to be such that all the valuations have the same domain, i.e. define a value for the same set of symbolic values, that we note $\text{dom}(\hat{S}^\#)$. Formally:

$\text{dom}(\hat{S}^\#)$

$$\forall v \in \gamma_{\hat{S}^\#}(\hat{S}^\#) : \text{dom}(v) = \text{dom}(\hat{S}^\#).$$

An AADT is simply a symbolic value implemented as an abstract data type whose fields may be other symbolic values (base or AADT). Thus in both the code and formal interface, we define a type/set of symbolic values $\mathcal{Q}$ (possibly divided into different sorts τ , in which case we note them $\mathcal{Q}_\tau$; or in different subsets that we denote as $\mathcal{Q}_1, \mathcal{Q}_2$ etc.).

An AADT is given a meaning using a set of concrete values $v \in \mathbb{V}$. Like the AADT is an abstract datatype containing symbolic values, the concrete values v generally is an ADT containing other concrete values. For instance in a `PAIR_ABSTRACTION`, if $\mathcal{Q}_1$ represent values in a set $\mathbb{V}_1$ and if $\mathcal{Q}_2$ represent values in a set $\mathbb{V}_2$, then $\mathcal{Q} = \mathcal{Q}_1 * \mathcal{Q}_2$ represent values in a set $\mathbb{V} = \mathbb{V}_1 \times \mathbb{V}_2$.

The relation between a symbolic value α and the set of values it represent is formalized using the concretization function $\gamma_\alpha : \mathcal{Q} \times \hat{S}^\# \rightarrow \mathcal{P}(\mathbb{V} \times \mathcal{V})^1$. This function represent a reduced product of a symbolic value α in a given abstract state $\hat{S}^\#$, and attach a valuation $\underline{v}$ to each v in the set. It is through these valuations that an AADT maintains a relation between its parts. For instance, given γ_1 and γ_2 the concretizations for $\mathcal{Q}_1$ and $\mathcal{Q}_2$, the concretization for pairs $\mathcal{Q}_1 * \mathcal{Q}_2$ is defined as

$\gamma_\alpha : \mathcal{Q} \times \hat{S}^\# \rightarrow \mathcal{P}(\mathbb{V} \times \mathcal{V})$

$$\gamma_\alpha((\alpha_1, \alpha_2), \hat{S}^\#) \triangleq \{((v_1, v_2), \underline{v}) \mid (v_1, \underline{v}) \in \gamma_1(\alpha_1, \hat{S}^\#) \wedge (v_2, \underline{v}) \in \gamma_2(\alpha_2, \hat{S}^\#)\}.$$

Given an abstract state $\hat{S}^\#$ that constrain two base symbolic values α_1 and α_2 to be such that $7 \leq \alpha_1 \leq 8$ and $\alpha_2 = \alpha_1 + 1$ (e.g., $\gamma_{\hat{S}^\#} = \{\underline{v}_7, \underline{v}_8\}$ where $\underline{v}_7 = [\alpha_1 \mapsto 7; \alpha_2 \mapsto 8]$ and $\underline{v}_8 = [\alpha_1 \mapsto 8; \alpha_2 \mapsto 9]$), we have $\gamma_\alpha((\alpha_1, \alpha_2), \hat{S}^\#) = \{(7, 8, \underline{v}_7), (8, 9, \underline{v}_8)\}$, which

¹ $\gamma_\alpha(\alpha, \hat{S}^\#)$ is defined only if $\text{support}(\alpha) \subseteq \text{dom}(\hat{S}^\#)$, but we mostly omit this condition to simplify the formalization.

shows that our definition of pair preserves the relation between the fields (despite the modularity of the implementation, which does not even require that both elements in a pair are of the same type).

6.1.2 Specification of AADTs and implementation of pairs

As usual, we specify the behavior of operations on abstract element using soundness theorems (Definition 6.1.2). We will gradually explain these operations and soundness theorems in the following sections. The module `Pair_Abstraction` provides a concrete implementation of this interface for the abstraction of pairs of elements.

Note that the implementation in `Pair_Abstraction` provides a monadic interface for the `cons`, `first` and `second` functions. They simply derive from simpler operations `_cons`, `_first` and `_second` that more closely match the interface in Definition 6.1.1. The reason for this difference is that the simpler operations are easier to define and prove, but the monadic operation better fit the monadic interface that we use in this manuscript.

Definition 6.1.2: Soundness theorems for AADT

$$\begin{aligned}
& \hat{s}^{\#'} \text{ extends } \hat{s}^{\#} \triangleq \text{dom}(\hat{s}^{\#'}) \supseteq \text{dom}(\hat{s}^{\#}) \wedge \gamma_{\hat{s}^{\#}}(\hat{s}^{\#}) = \{v_{|\text{dom}(\hat{s}^{\#})} \mid v \in \gamma_{\hat{s}^{\#}}(\hat{s}^{\#'})\} & (\text{EXTENDS}) \\
& (\alpha', \hat{s}^{\#'}) \hat{\supseteq} (\alpha, \hat{s}^{\#}) \triangleq \hat{s}^{\#'} \text{ extends } \hat{s}^{\#} \wedge \gamma_{\alpha}(\alpha', \hat{s}^{\#'}) \supseteq \gamma_{\alpha}(\alpha, \hat{s}^{\#}) & (\text{WEAKENS}) \\
\\
& (\alpha, \hat{s}^{\#'}) = \text{op}_n^{\#}(\alpha_1, \dots, \alpha_n) \hat{s}^{\#} \Rightarrow & (\text{OP}_n \text{SOUND}) \\
& \{(v, v_{|\text{dom} \hat{s}^{\#}}) \mid (v, v) \in \gamma_{\alpha}(\alpha, \hat{s}^{\#'})\} \supseteq \{(\text{op}_n(v_1, \dots, v_n), v) \mid \forall i \in 1..n : (v_i, v) \in \gamma_{\alpha}(\alpha_i, \hat{s}^{\#})\} \\
\\
& \hat{s}^{\#'} = \text{assume}^{\#}(\alpha) \hat{s}^{\#'} \Rightarrow & (\text{ASSUME SOUND}) \\
& \text{dom}(\hat{s}^{\#'}) \supseteq \text{dom}(\hat{s}^{\#}) \wedge \gamma_{\hat{s}^{\#}}(\hat{s}^{\#'}) \supseteq \{v \in \gamma_{\hat{s}^{\#}}(\hat{s}^{\#}) \mid (\text{true}, v) \in \gamma_{\alpha}(\alpha, \hat{s}^{\#})\} \\
\\
& \hat{s}^{\#'} = \text{join}_{\phi} \hat{s}_1^{\#} \hat{s}_2^{\#} \wedge \alpha' = \Phi_{\phi}(\alpha_1, \alpha_2) \Rightarrow & (\text{PHI SOUND}) \\
& \gamma_{\alpha}(\alpha', \hat{s}^{\#'}) \supseteq \gamma_{\alpha}(\alpha_1, \hat{s}_1^{\#}) \cup \gamma_{\alpha}(\alpha_2, \hat{s}_2^{\#}) \\
\\
& \text{serialize } (\alpha_1, \hat{s}_1^{\#}) (\alpha_2, \hat{s}_2^{\#}) S = S', d, \hat{s}_1^{\#'}, \hat{s}_2^{\#'} \Rightarrow & (\text{SERIALIZE SOUND}) \\
& S' \supseteq S \wedge \forall \phi : \text{dom } \phi = S' \Rightarrow (\alpha_1, \alpha_2) \in \text{dom } \Phi_{\phi} \\
& \wedge \exists (\alpha'_1, \alpha'_2) \in \mathbb{Q} \times \mathbb{Q} : \\
& \quad (\alpha'_1, \hat{s}_1^{\#'}) \hat{\supseteq} (\alpha_1, \hat{s}_1^{\#}) \wedge (\alpha'_2, \hat{s}_2^{\#'}) \hat{\supseteq} (\alpha_2, \hat{s}_2^{\#}) \\
& \quad \wedge \forall \phi : \text{dom } \Phi_{\phi} \supseteq S' \\
& \quad \forall \hat{s} : \text{dom}(\hat{s}) \supseteq \text{img}(\phi) \\
& \quad \forall \hat{s}^{\#'} : \hat{s}^{\#'} \text{ extends } \text{join}_{\phi} \hat{s}_1^{\#'} \hat{s}_2^{\#'} \\
& \quad d \phi \hat{s}^{\#} = \alpha, \hat{s}^{\#'} \Rightarrow \\
& \quad \hat{s}^{\#'} \text{ extends } \hat{s}^{\#} \wedge \alpha = \Phi_{\phi}(\alpha'_1, \alpha'_2)
\end{aligned}$$

`module` `Pair_Abstraction`(Arg: sig

```

module Base:VALUE_BASED_DOMAIN;; open Base
type  $\mathcal{Q}_1$ ;; type  $\mathcal{Q}_2$ 
val serialize_1:  $\mathcal{Q}_1 \rightarrow \hat{S}^\# \rightarrow \mathcal{Q}_1 \rightarrow \hat{S}^\# \rightarrow \tau \phi\_arguments$ 
   $\rightarrow (\mathcal{Q}_1, \tau)$  serialization *  $\hat{S}^\# * \hat{S}^\#$ 
val serialize_2:  $\mathcal{Q}_2 \rightarrow \hat{S}^\# \rightarrow \mathcal{Q}_2 \rightarrow \hat{S}^\# \rightarrow \tau \phi\_arguments$ 
   $\rightarrow (\mathcal{Q}_2, \tau)$  serialization *  $\hat{S}^\# * \hat{S}^\#$ 
end):PAIR_ABSTRACTION = struct
include Arg
open Base
type  $\mathcal{Q} = \mathcal{Q}_1 * \mathcal{Q}_2$ 

open Polymorphic_State_Option_Monad
let _cons v1 v2 s = (v1,v2),s
let _first v s = let (v1,_) = v in v1,s
let _second v s = let (_,v2) = v in v2,s

let cons v1 v2 = let* v1 = v1 in let* v2 = v2 in return (v1,v2)
let first v = let* (v1,_) = v in return v1
let second v = let* (_,v2) = v in return v2

let serialize (symval11,symval12) s1 (symval21,symval22) s2  $\phi\_args$  =
  let Serialization( $\phi\_args$ ,d1),s1,s2 =
    serialize_1 symval11 s1 symval21 s2  $\phi\_args$  in
  let Serialization( $\phi\_args$ ,d2),s1,s2 =
    serialize_2 symval12 s1 symval22 s2  $\phi\_args$  in
  Serialization( $\phi\_args$ ,fun  $\phi\_res$  s  $\rightarrow$ 
    let res2, $\phi\_res$ ,s = d2  $\phi\_res$  s in
    let res1, $\phi\_res$ ,s = d1  $\phi\_res$  s in
    (res1,res2), $\phi\_res$ ,s),s1,s2
end

```

6.1.3 Regular operations and assume

There are three main ways to modify an abstract state: by extension using normal operations, by `assume`, and by `join`. We examine the two first here. Note that there is no `assume` operation for pairs as pairs are not booleans (even if pairs can be transparently involved in boolean expressions, like `first(cons(true,1))`). Thus, we will instead use the `Abstract_Store` of Chapter 4, which is also an AADT, to illustrate how `assume` works.

Regular operations $op_n^\#$ attach properties to new symbolic values, but do not alter the properties of previously-known symbolic values. This implies that symbolic values are *referentially transparent*, which simplifies the design and proof of complex abstractions. Referential transparency is formalized by the notion of *extension* (EXTENDS): when $\hat{s}^\#$ extends $\hat{s}^\#$, the concretization of $\hat{s}^\#$ are valuations involving more base symbolic values, but the values for the existing base symbolic values remains unchanged (they are neither more nor less precise). This implies that the valuations of AADTs, that depend only on the valuation of the base symbolic values, also remain unchanged. The concept of extension derives from Chapter 4, where symbolic values were all basic symbolic values, and all the operations extended the abstract state with the newly-defined symbolic value. With AADT, the evaluation of compound symbolic values involves an unpredictable number of operations, making it difficult

referentially transparent

extension

to determine how many base variables are added to the abstract state; the role of `extends` is to abstract away this information. Note that because extension does not alter precision, reasoning about precision stays simple.²

The fact that operations $\text{op}_n^\#$ can only change the abstract state by extension means that these operations need only to perform a forward evaluation of its arguments to return the most precise symbolic value. All of our simple `_cons`, `_first` and `_second` operations are as precise as possible, as they are exact.

On the contrary, the `assume` operation does not return any new symbolic value: its goal is only to shrink the concretization when we learn that a particular condition is true. In other words, while $\text{op}_n^\#$ only does forward evaluation, `assume` does only a kind of backward propagation.

This design is interesting because the backward constraint propagation that `assume` does is generally more complicated than what the implementation of a forward evaluation: e.g., updating the store when a symbolic value α changes would require maintaining a list of memory locations that are related to α to update them too. While this is possible for simple memory abstractions like an abstract store (as done in our `Standard_Non_Relational_Abstract_Domain` in Appendix), it becomes cumbersome in more complex settings, e.g. if the value α appears as part of an array index, where the array is one of the possible targets of a pointer contained in some record.

Example 6.1.3: Combination of regular operations and `assume`

Consider the following program:

```
y := x + 1; if(! (y ≤ 3)) exit(0);
```

In a non-relational store abstraction $\Sigma^\# = \mathbb{X} \rightarrow \mathbb{V}^\#$ mapping program variables to intervals, starting from an initial store $\sigma_0^\# \triangleq [x \mapsto \top, y \mapsto \top]$, a standard analysis (as in Chapter F or Blazy et al. (2017)) would return as a final store $\sigma^{\#'} = [x \mapsto \top, y \mapsto [-\infty..3]]$. Note that to get this result, the backward propagation of the condition $y \leq 3$ had to modify the value of y in the store $\sigma^{\#'}$. If we also wanted to learn that $\sigma^{\#'}[x] \subseteq [-\infty..2]$, we would have to make non-trivial changes to the analysis.

By contrast, the operations on the abstract store on Chapter 4, which is an AADT, are much simpler. Starting from an initial state $(\hat{\sigma}_0, \hat{s}_0^\#) \triangleq ([x \mapsto \alpha_x, y \mapsto \alpha_y], [\alpha_x \mapsto \top, \alpha_y \mapsto \top])$, the final state is now $(\hat{\sigma}', \hat{s}^{\#'}) \triangleq ([x \mapsto \alpha_x, y \mapsto \alpha_x + 1], [\alpha_x \mapsto [-\infty..2], \alpha_x + 1 \mapsto [-\infty..3], \alpha_y \mapsto \top])$. The abstract store is only modified by the assignment operation $y := x + 1$ (where $\hat{s}_0^\#$ is only extended to also contain $\alpha_x + 1$), and is not changed by the condition $!(y \leq 3)$, where only the abstract state is refined (as described in Chapter 5).

²The notion of extension is an interesting property that holds in the abstractions defined in this manuscript, but may be limiting in some situations. There are at least three cases where extension does not hold. First, one may want to forget about some information in the abstract state; for instance Chang and Rival (2013) perform garbage collection on unused symbolic values. This limitation could be sidestepped by separating the information-losing operation in a separate operation. Second, some operations are partial (like division by zero or loading addresses), and we may want the resulting state to contain only the valuations for which the evaluation succeeds. This can also be sidestepped by adding transformation passes that adds and `assume` preconditions to these partial operations (and emit alarms), as we proposed in PLDI'24. Finally, there are cases where some constraints cannot be remembered in the abstract state, but is contained in the AADT, that may re-issue the information to the abstract state using `assume`. In this case, the operations are referentially transparent only if the base concretization is exact. For these reasons, extension may not hold in the AADTs of CODEX; instead, if the result of an operation is $(\alpha, \hat{s}^{\#'})$, then $\hat{s}^{\#'}$ is constrained by $\gamma_{\hat{s}^{\#'}}(\hat{s}^{\#'}) \supseteq \{v \mid \exists v : (v, v) \in \gamma_Q(\alpha', \hat{s}^{\#'})\}$.

Thus, we use symbolic values as an *indirection*: rather than changing the symbolic value everywhere that it appears, we change the information attached to the symbolic value, which is stored in the abstract SSA state $\hat{s}^\#$. This works particularly well in Example 6.1.3 because the conditions that appear in `assume` are numeric and thus handled by Chapter 5. However, we will see in Section 6.2.3 that it also extends to situations that do not appear as numeric.

6.1.4 Anti-unification

The last operation that we did not describe is `serialize`. In Chapter 4, we had a `serialize` operation on base symbolic values which was used on join points, when a program variable was mapped to two different symbolic values, to create the ϕ function that reconciles these values by anti-unification. The `serialize` function here extends this to the case where symbolic values are AADT, i.e. it can be formalized as creating a ϕ function that anti-unifies two AADT values. We can thus characterize a call to `serialize` $(\alpha_1, \hat{s}_1^\#) (\alpha_2, \hat{s}_2^\#)$ as returning a α that anti-unifies α_1 (coming from state $\hat{s}_1^\#$) and α_2 (coming from state $\hat{s}_2^\#$), i.e. $\alpha = \phi(\alpha_1, \alpha_2)$.

Specifying anti-unification using Φ

The implementation of `serialize` for pairs, and its soundness theorem `SERIALIZE-SOUND`, seems very complex. As a comparison, the similar operation (`join`) on the non-relational abstraction of pairs $\mathbb{V}^\# \times \mathbb{V}^\#$ is defined simply as $(v_1^\#, v_2^\#) \sqcup (v_3^\#, v_4^\#) \triangleq (v_1^\# \sqcup_{\mathbb{V}^\#} v_3^\#, v_2^\# \sqcup_{\mathbb{V}^\#} v_4^\#)$. But actually, what `serialize` does is fully specified by the definition

$$\Phi((v_1^\#, v_2^\#), (v_3^\#, v_4^\#)) = (\Phi(v_1^\#, v_3^\#), \Phi(v_2^\#, v_4^\#)).$$

Similarly, `serialize_abstract_store` in Chapter 4 is specified simply by

$$\Phi(\hat{\sigma}_1, \hat{\sigma}_2) = [x \in \mathbb{X} \mapsto \Phi(\hat{\sigma}_1[x], \hat{\sigma}_2[x])].$$

In the above constraints, $\Phi : \mathbb{A} \times \mathbb{A} \rightarrow \mathbb{A}$ is an unknown function representing possible anti-unification functions ϕ , and the above equations are constraints that solutions ϕ must fulfill.

Example 6.1.4: ϕ function example

If we want to anti-unify pairs of a abstract store and a base variable, like $(\hat{\sigma}_0, \underline{\alpha}_1)$ with $\hat{\sigma}_0 = [x \mapsto \underline{\alpha}_0, y \mapsto 2]$, and $(\hat{\sigma}_1, \underline{\alpha}_4)$ with $\hat{\sigma}_1 = [x \mapsto \underline{\alpha}_0, y \mapsto \underline{\alpha}_3]$, we can use the function ϕ defined as:

$$\begin{aligned} \underline{\phi} &= [(\underline{\alpha}_0, \underline{\alpha}_0) \mapsto \underline{\alpha}_0; (2, \underline{\alpha}_3) \mapsto \underline{\alpha}_5; (\underline{\alpha}_1, \underline{\alpha}_4) \mapsto \underline{\alpha}_6] \text{ where } \underline{\alpha}_5 \text{ and } \underline{\alpha}_6 \text{ are fresh} \\ \phi &= \underline{\phi} \cup [(\hat{\sigma}_0, \hat{\sigma}_1) \mapsto \hat{\sigma}_2; ((\hat{\sigma}_0, \underline{\alpha}_1), (\hat{\sigma}_1, \underline{\alpha}_4)) \mapsto (\hat{\sigma}_2, \underline{\alpha}_6)] \text{ where } \hat{\sigma}_2 = [x \mapsto \underline{\alpha}_0, y \mapsto \underline{\alpha}_5] \end{aligned}$$

This ϕ function meets the constraints on Φ defined above. Note that we can easily compose AADT as we did above, e.g. we could also have pairs inside of abstract stores. Note also that $\text{dom } \phi$ is minimal, and that what ϕ computes derives from the definition of Φ and $\underline{\phi}$.

In addition to constraints specific to the AADT, we have two additional constraints on Φ :

1. When the arguments of Φ are base variables, then Φ should correspond to some anti-unification $\underline{\phi}$ of the base domain, as described in Chapter 4. In practice this means that the `serialize` function of base symbolic values $\underline{\alpha} \in \underline{\mathcal{Q}}$ correspond to the `serialize` function of the base `VALUE_BASED_DOMAIN` module). Actually, the constraints on Φ can also be seen as the definition of a recursive function $\Phi_{\underline{\phi}}$, whose base cases (i.e., the base symbolic values) correspond to $\underline{\phi}$. Note in particular that $\Phi_{\underline{\phi}}$ is determined by $\underline{\phi}$.
2. We require that $\forall \alpha : \Phi(\alpha, \alpha) = \alpha$ (which corresponds to Global Value Numbering, see Section 4.2.3). This allows important (Blanchet et al., 2002) optimizations: when merging large maps that have some physically equal common parts, we don't need to iterate inside this common part. Our implementation of `serialize_abstract_store` calls the `fold_on_nonequal_union` function of the PatriciaTree library (Chapter A), which I designed to maximally take advantage and preserve physical identity, for this purpose.

Because anti-unification is fully specified using the constraints on Φ , the most important soundness theorem regarding anti-unification is **PHISOUND**. Note that it is very similar to the soundness condition for a join $\sqcup$ operation.

Weakening

Often we cannot anti-unify two AADTs values α_1 and α_2 as is, because they are too different. This explains why Φ is partial: it cannot perform anti-unification on every pair of values. When this happens, we need to *weaken* α_1 and α_2 into two AADTS α'_1 and α'_2 that will be more similar shape, before anti-unification. This weakening operation is formalized using the $\hat{=}$ operator, defined in **WEAKENS** as:

weaken

$$(\alpha', \hat{s}^{\#'}) \hat{=} (\alpha, \hat{s}^{\#}) \triangleq \hat{s}^{\#'} \text{ extends } \hat{s}^{\#} \wedge \gamma_{\mathcal{Q}}(\alpha', \hat{s}^{\#'}) \supseteq \gamma_{\mathcal{Q}}(\alpha, \hat{s}^{\#}).$$

Example 6.1.5: Weakening example

There is no need for weakening for pairs or abstract stores. But consider the case where we anti-unify two memory blocks composed by concatenation of bitvectors, in the case where the concatenations do not align. For instance, let us compute

$\phi(\alpha_3, \alpha_4)$ where $\alpha_3 = \begin{array}{|c|c|c|} \hline \alpha_2 & \alpha_1 & \alpha_0 \\ \hline \end{array}$ and $\alpha_4 = \begin{array}{|c|c|c|} \hline \alpha_5 & \alpha_4 & \alpha_0 \\ \hline \end{array}$. It

makes no sense to anti-unify bitvector symbolic values that have different lengths, so we have to first weaken α_3 and α_4 to align them by splitting or concatenating cells, resulting (if weakening concatenates cells) in $\alpha'_3 = \begin{array}{|c|c|} \hline \alpha_2 :: \alpha_1 & \alpha_0 \\ \hline \end{array}$

and $\alpha'_4 = \begin{array}{|c|c|} \hline \alpha_5 :: \alpha_4 & \alpha_0 \\ \hline \end{array}$. These can be unified, to result in $\phi(\alpha'_3, \alpha'_4) =$

$$\begin{array}{|c|c|} \hline \phi(\alpha_2 :: \alpha_1, \alpha_5 :: \alpha_4) & \alpha_0 \\ \hline \end{array}.$$

Note that we did not require $\hat{=}$ to define an ordering between symbolic values in the abstract. Indeed, $\hat{=}$ is a minimal definition to state the soundness of a `serialize` function, but does not explain how weakening is performed in practice in `serialize`,

where the goal is to simultaneously weaken two arguments so that they can be anti-unified. Defining an order in the abstract can be useful to define how to implement such an algorithm. Indeed, this algorithm is similar to what a binary join ($\sqcup$) operator does (weakening its two arguments to find an upper bound of both), except that we need to return two "compatible" (i.e., anti-unifiable) symbolic values instead of one upper bound. We found out that, in practice, it is easy to derive `serialize` from a non-relational join algorithm (where symbolic values $\alpha \in \mathbb{Q}$ are replaced by abstract values $v^\# \in \mathbb{V}$, like intervals). For instance in the case of finite memory blocks, we need to define an ordering on blocks that states either that weakening should merge or split fields; depending on this order, the shape of the weakened symbolic values is found by considering either the intersection or the union of all the possible boundaries between fields. In the example above, only the $\{8\}$ boundary was kept.

Implementation of the anti-unification: `serialize`

Ideally, we would want `serialize` to just take a pair of AADT values α_1 and α_2 , and return $\Phi(\alpha_1, \alpha_2)$. But this is not possible for three reasons:

1. As we said, sometimes we need to weaken symbolic values before they can be anti-unified. For this reason, `serialize` also takes two abstract states $\hat{s}_1^\#$ and $\hat{s}_2^\#$, and returns two states $\hat{s}_1^{\#'}$ and $\hat{s}_2^{\#'}$, corresponding to the possible updates of the states due to the computation of weakening operations. Note that `SERIALIZE-SOUND` refers to two symbolic variables α_1' and α_2' for this reason, that are, in the simplest cases, (respectively) equal to α_1 and α_2 .
2. The fact that there are relations between symbolic values (including AADTs) comes from the fact that we compute a ϕ function, which is derived from a ϕ function, which is created by the `join` function of the base `VALUE_BASED_DOMAIN` when provided with $\text{dom } \phi$. Thus, `serialize` operations take and return another argument S which accumulates all the pairs of base variable to be anti-unified, corresponding to $\text{dom } \phi$.
3. Because `join` creates the ϕ function that we need to do the anti-unification, and because creating this ϕ requires to collect all the pair of base symbolic values that must be anti-unified to be computed, anti-unification must proceed in three passes: first, we compute $\text{dom } \phi$ and possibly weaken arguments so that they can be anti-unified, then `join` creates ϕ from $\text{dom } \phi$, and finally we can use ϕ to do the anti-unification following the definition of Φ_ϕ . This succession of phases may be observed, for instance, in the implementation of `join` and `fixpoint_step` in the `SSA_Translation`. This also explains why `serialize` returns a function in `SERIALIZE-SOUND`: the first phase is building a ϕ anti-unification function that will be used in the last phase, when ϕ and the final abstract state is known.

In the end, we have a simple specification of anti-unification thanks to the constraints on Φ , and implementing `serialize`, once Φ and the weakening rules are known, is relatively simple, even if the function signatures appear complicated.

6.2 Some AADTs

In this section we present some important abstract abstract datatypes, that we will put into use in the next section.

6.2.1 Monadic constructs for disjunctive reasoning

The implementation of many transfer functions rely on case-based reasoning. For instance, loading a value from an array depends on conditions on the index of this array. To help implement these case-based constructs, we provide monadic constructs that helps building such cases.

```

module Polymorphic_State_Option_Monad = struct
  include Polymorphic_State_Option_Monad
  let assumem assume = fun cond state →
    match assume cond state with
    | None → None
    | Some state → Some((),state)

  let fork_join merge (l:(τ,'state) m list) = fun state →
    (* Apply the state on each branch, then filter dead branches. *)
    match List.filter_map (fun x → x state) l with
    | [] → None (* No remaining undead branch = bottom. *)
    | hd::tl → Some(List.fold_left merge hd tl) (* reduce using merge. *)
end
open Polymorphic_State_Option_Monad

```

The first construct, `assumem`, just makes `assume` comply to the monadic interface (by returning an encapsulated unit). The second construct, `fork_join` allows building a transfer function as a union of transfer functions. When applied to a union of two transfer functions f_1 and f_2 that do not return the $\perp$ state, `fork_join` obeys to this specification:

$$\frac{
\begin{array}{l}
(\alpha_1, \hat{s}_1^\#) = f_1 \hat{s}^\# \quad (\alpha_2, \hat{s}_2^\#) = f_2 \hat{s}^\# \\
\alpha = \phi(\alpha_1, \alpha_2) \quad \hat{s}^{\#'} = \text{join}_\phi \hat{s}_1^\# \hat{s}_2^\# \quad (\alpha, \hat{s}^{\#'}) = m(\alpha_1, \hat{s}_1^\#) (\alpha_2, \hat{s}_2^\#)
\end{array}
}{
(\alpha, \hat{s}^{\#'}) = \text{fork_join } m [f_1; f_2] \hat{s}^\#
}$$

The transfer functions f_1 and f_2 are typically closures, that make use of `assumem` to refine the the state on their branch (e.g., we encode `if...then...else` construct using `fork_join` and `assumem`).

Note that when doing transfer functions, we want $\hat{s}^{\#'}$ to extend $\hat{s}^\#$. This is true only if the set of conditions that we use inside each case is exhaustive, e.g., if we `assume` c_1 inside f_1 and `assume` c_2 inside f_2 , then $c_1 \vee c_2$ is true in $\hat{s}^\#$.

Finally, note that when we view our base domain as producing an SSA program (as in Chapter 4), then `fork_join` corresponds to a kind of `switch-case` or Dijkstra's guarded command [Dijkstra \(1975\)](#) construct (or just an `if then else` when there are only two branches). Viewing things this way sometimes help writing the transfer function.

6.2.2 Finite arrays

In this section we will see how we can implement *array expansion* ([Blanchet et al., 2002](#)), i.e., abstracting arrays by representing the content of every cell individually, as an AADT. The abstraction is quite similar to pairs and records that we already saw. The main difference is that the array index used as argument of the `read` and `write` functions is variable, while it is constant for tuples and pairs. Because of

array expansion

this, all elements of the arrays are of the same type, and the operations will be using the abstract state $\hat{s}^\#$ argument to reason about the possible values of the array index.

```

module IntMap = struct
  include PatriciaTree.MakeMap(struct include Int let to_int x = x end)
  let max m = fst (unsigned_max_binding m)
end

module Finite_Array_Abstraction(Arg:sig
  module Sub:VALUE_BASED_DOMAIN;; open Sub
  type  $\mathbb{Q}$ _idx
  type  $\mathbb{Q}$ _elt
  (** Fold on all the possible values of an abstract index between 0
    and max. *)
  val fold_on_idx:  $\mathbb{Q}$ _idx  $\rightarrow$  max:Z.t  $\rightarrow$   $\hat{S}^\# \rightarrow \tau \rightarrow (Z.t \rightarrow \tau \rightarrow \tau) \rightarrow \tau$ 
  (** Create the condition idx = c. *)
  val idx_equal_to:  $\mathbb{Q}$ _idx  $\rightarrow$  Z.t  $\rightarrow$   $\hat{S}^\# \rightarrow ([\text{boolean}] \text{ Sub.}\mathbb{Q} * \hat{S}^\#)$  option

  val serialize_elt:  $\mathbb{Q}$ _elt  $\rightarrow$   $\hat{S}^\# \rightarrow \mathbb{Q}$ _elt  $\rightarrow$   $\hat{S}^\# \rightarrow \tau$   $\phi$ _arguments
     $\rightarrow (\mathbb{Q}$ _elt, $\tau$ ) Sub.serialization *  $\hat{S}^\# * \hat{S}^\#$ 
end):sig
  include Monomorphic_AADT with module Base = Arg.Sub
  type  $\tau$  m = ( $\tau$ ,Base. $\hat{S}^\#$ ) Polymorphic_State_Option_Monad.m
  val read:  $\mathbb{Q} \rightarrow$  Arg. $\mathbb{Q}$ _idx  $\rightarrow$  Arg. $\mathbb{Q}$ _elt m
  val write:  $\mathbb{Q} \rightarrow$  Arg. $\mathbb{Q}$ _idx  $\rightarrow$  Arg. $\mathbb{Q}$ _elt  $\rightarrow$   $\mathbb{Q}$  m
  val make: int  $\rightarrow$  (int  $\rightarrow$  Arg. $\mathbb{Q}$ _elt m)  $\rightarrow$   $\mathbb{Q}$  m
end
=

```

The array abstraction makes use of symbolic values representing indexes, and of symbolic values representing elements (i.e., the array contents). On indices, we need to be able to fold on all the possible values (cropped by the interval $0..max$, so the enumeration terminates), and also to create the condition $idx = c$ for some constant index c . Regarding elements that appear in the array, we need to serialize them. Note that do not need to serialize symbolic indices, as they do not appear in the array.

```

struct
  module Base = Arg.Sub
  type  $\tau$  m = ( $\tau$ ,Base. $\hat{S}^\#$ ) Polymorphic_State_Option_Monad.m
  type  $\mathbb{Q}$  = Arg. $\mathbb{Q}$ _elt IntMap.t

  let make len f =
    let open Polymorphic_State_Option_Monad in
    let l = Seq.(take len (ints 0)) in
    let do_i mapm i =
      let* map = mapm in let* v = f i in return (IntMap.add i v map) in
    Seq.fold_left do_i (return IntMap.empty) l

```

We represent an array of length l using functional maps with bindings in $[0..l-1]$. The meaning of such a finite symbolic array α of length l is simply

$$\gamma_{\mathbb{Q}}(\alpha, \hat{s}^\#) \triangleq \{(a, \nu) \mid \text{dom}(a) = [0..l-1] \wedge \forall i \in [0..l-1] : (a[i], \nu) \in \gamma_{\mathbb{Q}}(\alpha[i], \hat{s}^\#)\}.$$

This abstraction is relational because of the shared valuation v . The `make` function creates a new array where all the cells are initialized with the same element (i.e., they are all equal, and this property is naturally preserved in a value-based abstract interpretation). XXX: No longer. Instead, I have an array whose initialization depends on the index, like `Array.make` in OCaml.

```
let join_elt (elt1,s1) (elt2,s2) =
  let Serialization( $\phi$ _args,d_elt),s1,s2 =
    Arg.serialize_elt elt1 s1 elt2 s2 Arg.Sub.empty_ $\phi$ _arguments in
  let  $\phi$ _res,s = Arg.Sub.join  $\phi$ _args s1 s2 in
  let elt,(_:unit Arg.Sub. $\phi$ _result),s = d_elt  $\phi$ _res s in
  elt,s
```

`join_elt` is our first use of a serialize function outside of a full join of the state; instead it creates a new symbolic value that "joins" (by anti-unification) two existing symbolic values. This mimicks what a non-relational abstraction does (joining abstract elements), but in a relational way. For instance, replacing the non-relational abstraction of array elements in Cousot et al. (2011) by symbolic elements transform it into a relational abstraction; this requires replacing their non-relational join operation with our relational `join_elt`.

```
let switch_on_idx max @_idx f = fun state →
  let l = Arg.fold_on_idx @_idx ~max state [] (fun key acc →
    let res =
      let* cond = Arg.idx_equal_to @_idx key in
      let* () = assumem Arg.Sub.assume cond in
      f key in
    res::acc) in
  fork_join join_elt l state

let read map @_idx =
  let open Polymorphic_State_Option_Monad in
  let max = Z.of_int (IntMap.max map) in
  switch_on_idx max @_idx (fun key →
    return (IntMap.find (Z.to_int key) map))
```

In a non-relational abstraction, `read ar idx` would compute $\bigsqcup_{i \in \gamma_{V\#}(idx)} ar.content[i]$.

In our relational setting, we just replace $\bigsqcup$ by `join_elt`. Another way to view this operation, when our analysis is seen as performing a SSA translation, is that it creates a kind of C `switch(idx)` statement, that splits execution according to the value of the `idx` variable; and each of the different branches return the value `ar.content[i]`.

```
let write map symb_key symb_elt =
  let* state = get_state in
  let max = Z.of_int (IntMap.max map) in
  Arg.fold_on_idx symb_key ~max state (return map) (fun key acc →
    let* map = acc in
    let key' = Z.to_int key in
    let* cond = Arg.idx_equal_to symb_key key in
    let* notcond = Arg.Sub.Expr.¬ (return cond) in
    let* element = fork_join join_elt
```

```

[ (* if idx = key *) begin
  let* () = assumem Arg.Sub.assume cond in
  return symb_elt
end;
(* else *) begin
  let* () = assumem Arg.Sub.assume notcond in
  return @@ IntMap.find key' map
end ] in
let map = IntMap.add key' element map in
return map)

```

We could implement `write` like `read`, i.e. by joining (using a `join_array` function) all the possible arrays for the different possible values of the index `idx`. However, this is less efficient than the code above, as joining arrays is more expensive than joining single elements. Instead, what `write arr idx element` essentially does is to replace `arr [i]` with `if idx = i then element else arr[i]`, when `idx` may have value `i`. Note that this code will transparently do a "weak" or "strong" update, depending on whether `idx` is a singleton. If we replace `join_elt` with `Ite_Abstraction`, a disjunctive abstraction that we will see in Section 6.2.3, it can even do a "fluid" update (Dillig et al., 2010)).

```

open Arg.Sub
let serialize map1 s1 map2 s2 ϕ_args =
  assert (IntMap.max map1 == IntMap.max map2); (* Same bindings. *)
  let f idx v1 v2 (Serialization(ϕ_args,d_rest),s1,s2) =
    let Serialization(ϕ_args,d_elt),s1,s2 =
      Arg.serialize_elt v1 s1 v2 s2 ϕ_args in
    let d_map = fun ϕ_res s →
      let elt,ϕ_res,s = d_elt ϕ_res s in
      let rest,ϕ_res,s = d_rest ϕ_res s in
      let map = IntMap.add idx elt rest in
      (map,ϕ_res,s)
    in Serialization(ϕ_args,d_map),s1,s2 in
  let init = Serialization(ϕ_args, fun ϕ_res s → map1,ϕ_res,s),s1,s2 in
  IntMap.fold_on_nonequal_inter f map1 map2 init
end

```

Nothing surprising about `serialize`: it is similar to `serialize_abstract_store`, i.e. it implements the specification $\phi(m_1, m_2) = [i \mapsto \phi(m_1[i], m_2[i])]$.

6.2.3 Local disjunctions

In an abstract domain $\mathbb{V}^\#$, `join` ($\sqcup_{\mathbb{V}^\#}$) is a natural way to compute an over-approximation of a disjunction, but sometimes we want to use more precise disjunctions. There are different constructs that can be used for this purpose (Miné, 2017), obtained by transformation of $\mathbb{V}^\#$: powerset completion $\mathcal{P}(\mathbb{V}^\#)$, partitionning $\mathcal{P} \rightarrow \mathbb{V}^\#$ (where elements in $\mathcal{P}$ partition the concrete space), the use of decision trees, etc.

Using these constructs, one can easily build complex disjunctive abstractions. For instance, in the `VALUE` and `EVA` abstract interpreters for `FRAMA-C`, a pointer value can be represented by the map $[\&x \mapsto [1..3], \&y \mapsto [5..7]]$, representing the union of addresses pointing either to the global variable `x` with an offset between 1 and 3, or to the addresses pointing to `y` with an offset between 5 and 7.

However, proceeding like this can only provide non-relational abstractions. In the above examples, there are no relations between the offset of the pointer and other values (such as the fact that the offset is smaller than some other variable). For this reason, many precise analysers (e.g., [Chang and Rival \(2013\)](#) or [Bautista et al. \(2022\)](#)) only do "top-level" disjunctions (on the set of states, where we do not need to maintain relations). However, doing so this is akin to a transformation in disjunctive normal form and can suffer from combinatory explosion (e.g., the transformation of 5 independent disjunctions leads to a set of $2^5 = 32$ different states) which must be mitigated ([Li et al., 2017](#)) to better scale. By contrast, *local disjunctions*, like the partitioning of offsets according to the target, does not suffer from combinatory explosion: we can have several pointer values, each pointing to different targets, without this being a problem as we do not enumerate all the possible combinations of possible target for each pointer value. This is more easily achieved in a non-relational settings, where there is less need to reason about the different cases of several disjunctive values simultaneously.

local disjunctions

Using AADT, we can have abstractions that can both have local disjunction *and* maintain relations. Moreover, the construct for these AADT derives from the familiar non-relational lattice transformations that we mentioned above. Consider for instance the AADT value $[\&x \mapsto \alpha_1, \&y \mapsto \alpha_2]$: it is a symbolic address representing a local disjunction (either the address points to $\&x$ with offset α_1 , or to $\&y$ with offset α_2); where α_1 and α_2 may be related to other symbolic values (for instance, it may be that both are smaller than some symbolic value α_3).

In this section, we will see how disjunctive abstractions can be implemented as an AADT. We will see in particular the partitioning abstraction that decomposes a value according to a bounded number of partitions. We name this construct the *Switch_Abstraction*, because when viewed as a program translation, it has similarities with the `switch(...){ case ... }` programming construct in C.

```
module Switch_Abstraction(Arg:sig
  module Sub:INTEGER_AND_BOOLEAN_VALUE_BASED_DOMAIN;; open Sub;; open
    Polymorphic_State_Option_Monad
  type  $\mathcal{Q}$ 
  val eq: ( $\mathcal{Q}, \hat{S}^\#$ ) m  $\rightarrow$  ( $\mathcal{Q}, \hat{S}^\#$ ) m  $\rightarrow$  ([‘boolean] Sub. $\mathcal{Q}, \hat{S}^\#$ ) m
  val serialize:  $\mathcal{Q}$  option  $\rightarrow \hat{S}^\# \rightarrow \mathcal{Q}$  option  $\rightarrow \hat{S}^\# \rightarrow \tau$  Sub. $\phi$ _arguments
     $\rightarrow (\mathcal{Q}, \tau)$  Sub.serialization *  $\hat{S}^\#$  *  $\hat{S}^\#$ 
end) = struct
  module Sub = Arg.Sub

  type  $\mathcal{Q}$  = { choice: [‘integer] Sub. $\mathcal{Q}$ ; map: Arg. $\mathcal{Q}$  IntMap.t }
```

Switch_Abstraction can be applied to any symbolic value (we only require an implementation of the `serialize` function). The resulting AADT is essentially a map from partition identifiers (represented by ints) to this symbolic value. However, to improve precision, we add a *choice variable*, a symbolic value that represents the possible active partitions. This gives the resulting formal definition:

choice variable

Definition 6.2.1: The Switch AADT

$$\begin{aligned} \text{Switch}(\mathcal{Q}) &\triangleq \{\text{choice} = \mathcal{Q}_{\mathbb{Z}}; \text{map} = (\mathbb{Z} \rightarrow \mathcal{Q})\} \\ \gamma_{\mathcal{Q}}(\alpha, \hat{S}^\#) &\triangleq \{(v, v) \mid (v_c, v) \in \gamma_{\mathcal{Q}}(\alpha.\text{choice}, \hat{S}^\#) \wedge (v, v) \in \gamma_{\mathcal{Q}}(\alpha.\text{map}[v_c], \hat{S}^\#)\} \end{aligned}$$

The reason for this extra choice variable is that AADTs cannot be modified by a call to `assume`, that only changes the abstract state (see Section 6.1.3). So instead, we embed this choice variable whose values in the abstract state indicates the possible choices. If we later discover that some choices are impossible, we can record this by changing the value of this variable in the abstract state. For instance after an equality test between two pointers, we know that the pointers must point to the same block, which we can encode by saying that the choice variables will be equal.

Note that by simplicity we have used an integer variable to represent this choice but abstracting the possible choices using an interval would create imprecision; instead, in CODEX we have a dedicated enum type that represent a finite set of cases, and is precisely enumerated.

```
open Polymorphic_State_Option_Monad

let reduce symval state =
  let map = IntMap.filter (fun variant value →
    let const = Sub.Expr.Expr.const (Z.of_int variant) in
    let Some(cond, state) =
      Sub.Expr.Expr.eq (return symval.choice) const state in
    match Sub.assume cond state with
    | None → false | Some _ → true) symval.map in
  { choice = symval.choice; map }
```

Because of this choice variable there may be cases that are in the map (because they were possible when the AADT was created) but later became impossible (due to an `assume` call). The `reduce` function is used to remove from the map these impossible cases. It is an example of reduction, i.e. an operation that does not change the precision in the concrete, but improves knowledge in the abstract.

```
let ar1_lift op symval =
  let* state = get_state in
  let symval = reduce symval state in
  let f i value mapm =
    let* map = mapm in
    let* value' = op i value in
    return (IntMap.add i value' map) in
  let* map = IntMap.fold f symval.map (return IntMap.empty) in
  return { choice = symval.choice; map }
```

`ar1_lift op symval` just calls `op` on each element of `symval.map`, using the monad to incrementally extend the state. An interesting observation is that the same choice is shared between the input and output symbolic values. Because pointer addition is implemented using this function, this allows to prove that, for instance, if `p + 4` cannot point to `&x`, then neither does `p`.

In this implementation of `ar1_lift`, we did not perform a case split by calling `assume` on each case to express that the choice variable has a specific value. It would be possible to do this³; but note that it is not required for the operation to be sound and exact (provided that `op` is).

³There are some drawbacks in doing so; in particular the base symbolic values would always be symbolic variables, because we would have to join the newly-created variable with `None` as done in `serialize` below, which hinders precision. Some work is needed to solve this problem, for instance by expressing a term as a separate variable and scope, like the SSI representation (Boissinot et al., 2012; Ananian, 2001) does.

```

let equal eqf symval1 symval2 =
  let module E = Arg.Sub.Expr.Expr in
  let* state = get_state in
  let symval1 = reduce symval1 state in let symval2 = reduce symval2 state in
  let eq_choice = E.eq (return symval1.choice) (return symval2.choice) in
  let common= IntMap.nonidempotent_inter_no_share (fun _ a b → (a,b))
    symval1.map symval2.map in
  let f choice (offset1,offset2) acc =
    let cond =
      let choice = E.const (Z.of_int choice) in
      let c1 = E.eq (return symval1.choice) choice in
      let c2 = E.eq (return symval2.choice) choice in
      E.∧ (E.∧ c1 c2) (Arg.eq (return offset1) (return offset2)) in
    E.V acc cond in
  let false = E.(¬ true) in
  let eq_offset = IntMap.fold f common false in
  E.∧ eq_choice eq_offset

```

The above code defines "equality between two [Switch_Abstraction](#) symbolic values α_1 and α_2 " as " α_1 and α_2 are in the same case, and in each case the value correspond". Formally:

$$\text{equal eqf } \alpha_1 \alpha_2 \triangleq \alpha_1.\text{choice} = \alpha_2.\text{choice} \wedge \left(\bigvee_{\substack{(c,\alpha'_1) \in \alpha_1.\text{map} \\ (c,\alpha'_2) \in \alpha_2.\text{map}}} \alpha_1.\text{choice} = c \wedge \text{eqf } \alpha'_1 \alpha'_2 \right)$$

(SWITCHEQ)

The code implements the above formula (also adding the condition $\alpha_2.\text{choice} = c$ because our simple analyser does not infer that from $\alpha_1.\text{choice} = \alpha_2.\text{choice} \wedge \alpha_1.\text{choice} = c$). While this is a simple formula, there are many things happening here.

Consider this example (also in ex2 in Chapter I), where we compute the equality of addresses between $\{\text{choice} = \alpha_1, \text{map} = [\&x \mapsto \alpha_2, \&y \mapsto \alpha_3, \&z \mapsto \alpha_4]\}$ and $\{\text{choice} = \alpha_5, \text{map} = [\&x \mapsto \alpha_6, \&y \mapsto \alpha_7]\}$ with numerical constraints $\alpha_2 = \alpha_4 = 0, 0 \leq \alpha_3 \leq 1, 1 \leq \alpha_7 \leq 3, \alpha_6 = 1$. The result of pointer comparison is the formula $\alpha_1 = \alpha_5 \wedge ((\alpha_1 = \&x \wedge \alpha_2 = \alpha_6) \vee (\alpha_1 = \&y \wedge \alpha_3 = \alpha_7))$. Observe first how some simplifications have been made to this formula (that excludes the case where the first pointer points to $\&z$).

This formula evaluates to [Quadrivalent](#).Top (the pointers may or may not be equal). However, when we [assume](#) that they are equal, the propagation will automatically perform a case split that eliminates the case where both pointers point to $\&x$ (because the offsets are incompatible), and refines α_3 and α_7 to the singleton 1. Thus when we write to one of these pointers, we perform a strong update at the location $\&y + 1$. This example illustrates that the cooperation of (flow-insensitive) AADTs with (flow-sensitive) constraint propagation can perform sophisticated reasoning, including the use of local disjunctions, simply and efficiently.

There are interesting variations around this function. For instance, the C standard defines inequality ($\leq$) comparison between addresses pointing to different memory blocks to be an undefined behaviour. This allows to gain precision and to drop the first condition ($\alpha_1.\text{choice} = \alpha_2.\text{choice}$); this exemplifies how undefined behaviour

allows improving precision in a static analysis. Another variation is presented in Section 7.2.2. There, a choice no longer indicates that the pointer belongs to a specific block, but that it has some type (i.e., is inside a set of blocks). In this setting, the `SwitchAbstraction` can no longer be used to prove the equality between pointers, but we can still prove disequality (i.e., absence of aliasing). We can still reuse `SWITCHEQ` if we "join/anti-unify" it with `false`, the intuitive idea being "either the type and corresponding offsets are incompatible and we know that the addresses are different, or they are compatible, in which case the result is unknown, i.e. is a non-deterministic choice between true and false". Note that by doing so, the transfer function is no longer exact (while it is here).

```

open Sub
let serialize_map map1 s1 map2 s2 ϕ_args =
  let f variant v1 v2 (Serialization(ϕ_args,d_map),s1,s2) =
    let Serialization(ϕ_args,d_value),s1,s2 =
      Arg.serialize v1 s1 v2 s2 ϕ_args in
    let d_map ϕ_res s =
      let value,ϕ_res,s = d_value ϕ_res s in
      let map,ϕ_res,s = d_map ϕ_res s in
      (IntMap.add variant value map, ϕ_res, s) in
    Serialization(ϕ_args,d_map),s1,s2 in
  let init = (Serialization(ϕ_args,fun ϕ_res s → map1,ϕ_res,s)),s1,s2 in
  IntMap.fold_on_nonequal_union f map1 map2 init

let serialize Ⓚ1 s1 Ⓚ2 s2 ϕ_args =
  let reduce_option Ⓚ s = match Ⓚ with
  | None → None, IntMap.empty
  | Some Ⓚ →
    let Ⓚ = reduce Ⓚ s in
    Some Ⓚ.choice,Ⓚ.map in
  let choice1,map1 = reduce_option Ⓚ1 s1 in
  let choice2,map2 = reduce_option Ⓚ2 s2 in
  let Serialization(ϕ_args,d_choice),s1,s2 =
    Sub.serialize Typ.integer choice1 s1 choice2 s2 ϕ_args in
  let Serialization(ϕ_args,d_map),s1,s2 =
    serialize_map map1 s1 map2 s2 ϕ_args in
  Serialization(ϕ_args,fun ϕ_res s →
    let map,ϕ_res,s = d_map ϕ_res s in
    let choice,ϕ_res,s = d_choice ϕ_res s in
    {choice;map},ϕ_res,s),s1,s2
end

```

The `serialize` operation consists in merging the map and choice variables. It can be specified as follows:

$$\begin{aligned}
\Phi(m_1, m_2) = & [k \mapsto \Phi(m_1[k], m_2[k]) \mid k \in \text{dom } m_1 \cap \text{dom } m_2] \\
& \cup [k \mapsto \Phi(m_1[k], \text{none}) \mid k \in \text{dom } m_1 \setminus \text{dom } m_2] \\
& \cup [k \mapsto \Phi(\text{none}, m_2[k]) \mid k \in \text{dom } m_2 \setminus \text{dom } m_1]
\end{aligned}$$

$$\Phi(\alpha_1, \alpha_2) = \{\text{choice} = \Phi(\alpha_1.\text{choice}, \alpha_2.\text{choice}); \text{map} = \Phi(\alpha_1.\text{map}, \alpha_2.\text{map})\}$$

In the formal definition above, `none` corresponds to the absence of value. When implementing `serialize` on AADTs with local disjunctions, it is often the case that

we have to anti-unify a symbolic value with something that has no counterpart in the other branch, in which case we use the `none` special value. This is the reason why all our `serialize` operations in the code take an option type, including the `serialize` function above, which uses `reduce_option` to reason about the `none` case. A similar construct exists in compiler SSA, where this `none` corresponds to an `undef` value (Lee et al., 2017). Note that $\Phi(\alpha, \text{none})$ cannot be replaced by α in general, as this could break the scoping constraints of SSA that definition dominates their uses.

The specification above does not include the call to `reduce` in the implementation, which can be formalized as a weakening step (that actually increases precision in the abstract, but maintain the same precision in the concrete, so weakening is not a fit name here).

Variation: the `Ite_Abstraction`

Section G.1 contains a variant of the `Switch_Abstraction`, the `Ite_Abstraction`. In this abstraction the choice variable is a boolean, and the map only has at most two elements (one if the condition is true, called the `left` value, and one if it is false, the `right` value). The main difference with the `Switch_Abstraction` is that the `left` and `right` symbolic values may have different types.

```
| module Ite_Abstraction = Appendix_ite_abstraction.Ite_Abstraction
```

Definition 6.2.2: The `Ite` AADT

$$\begin{aligned} \text{Ite}(\mathbb{Q}_{\mathbb{B}}, \mathbb{Q}_1, \mathbb{Q}_2) &\triangleq \mathbb{Q}_{\mathbb{B}} \times (\mathbb{Q}_1 \cup \{\text{none}\}) \times (\mathbb{Q}_2 \cup \{\text{none}\}) \\ \gamma_{\mathbb{Q}}(\text{none}, \hat{s}^{\#}) &\triangleq \{\} \\ \gamma_{\mathbb{Q}}((\alpha_b, \alpha_1, \alpha_2), \hat{s}^{\#}) &\triangleq \{(v_1, v) \in \gamma_{\mathbb{Q}}(\alpha_1, \hat{s}^{\#}) \mid v[\alpha_b] = \text{true}\} \\ &\quad \cup \{(v_2, v) \in \gamma_{\mathbb{Q}}(\alpha_2, \hat{s}^{\#}) \mid v[\alpha_b] = \text{false}\} \end{aligned}$$

6.3 Discussion, related and future work

6.3.1 Summary of the method

Abstract abstract datatypes are an effective and simple solution to build modular and precise abstraction of complex objects. It solves the dilemma between modularity and relationality, where having modular abstractions requires considering objects independently from the (implementation of) other objects, and relational abstractions require treating related objects together. This dilemma is solved by relating different AADT values using a shared abstract SSA state and shared set of names : the base symbolic values of Chapter 4. Therefore, the meaning of an AADT values depends on the possible valuations of the base values that it contains, and these valuations are constrained by the abstract SSA state.

This sharing of names and abstract state prevents joining two AADT values independently. Instead of joining AADT values, we perform an anti-unification operation that builds upon the renaming function ϕ that anti-unifies the base values. The implementation of this operation appears complex due to the separation of a join operation in three different phases : identify the base symbolic values that need to be anti-unified, join the abstract SSA state and create the renaming ϕ function,

then use ϕ to anti-unify the AADT values. Furthermore, some AADT values require transformation before they can be anti-unified. Fortunately, the specification of the anti-unification operation for an AADT is generally simple.

Outside of this serialization operation, operating on AADT values consist only of referentially transparent forward operations, using queries on the abstract state to simplify the result. This makes the implementation of such operations quite similar to the implementation of functional programs, that create new values by composition of values. The preservation of relations is achieved naturally, by reuse of symbolic values or by relying on the fact that symbolic values are related by the abstract state. Moreover, AADT do not require implementing complex constraint propagation operations, like transitive closures, when the program *assumes* a new condition. Instead, such operations will modify the abstract state, without changing the AADT value (cf. Example 6.1.3).

6.3.2 Comparison with other techniques to build abstractions

The contents of this chapter may appear simple; it is precisely the point. I believe the fact that AADTs are simple to specify, implement and prove to be one of its main advantages, in addition to its composability. In particular, building a composable library of memory abstraction was found to be a hard task ⁴, and AADT can be used to solve this problem.

As mentioned in Section 1.3, we can divide the structure of abstract interpreters in three groups. Non-relational interpreters (like VALUE or value-set analyses (Balakrishnan and Reps, 2004)) can be built modularly but are imprecise. However, their implementation can serve as the basis for an AADT. For instance, the array abstraction of Cousot et al. (2011) uses a non-relational abstraction of array values (paired with indices); by replacing these non-relational array values by symbolic values, we automatically transform it into a relational abstraction. Variable-based interpreters compute relation between program variables (or, more generally, between memory locations in the program (Miné, 2006a)). This type of interpreters is found, for instance, in Frama-C/Eva (Blazy et al., 2017), Astrée (Cousot et al., 2006; Chevalier and Feret, 2020; Chevalier, 2020) or MOPSA (Journault et al., 2019; Monat, 2021). They allow inferring precise numerical relations between the program variables, but generally focus on whole-program analysis of embedded systems, which does not require too much reasoning on infinite numbers of memory cells (except when doing array smashing Blanchet et al. (2002) or dealing with calls to `malloc` in a loop (Balakrishnan and Reps, 2006)). Finally, value-based interpreters compute relations between the programs values that is contained in the programs. They are more commonly seen in shape analysers (Chang et al., 2020) (e.g. Chang and Rival, 2008; Gulwani and Tiwari, 2007), and I think this is because doing complex memory reasoning like shape analysis is much more complicated in a variable-based than in a value-based framework. Here, I would like to explain the reasons why this is the case.

Composition by expression The main operation in variable-based analyses is assignment: nothing happens if no memory location is modified. As a consequence, the structure of the analyser is organised around assignments, making its operation similar to imperative programming. By contrast, operations on AADTs

⁴In the Dagstuhl seminar 23281, Patrick Cousot reported that they originally wanted to develop both a library of numerical abstraction, which was quickly developped as APRON (Jeannet and Miné, 2009), and then a library of memory abstractions, but the latter one was never implemented because developing a modular library of memory abstractions was difficult.

combine several symbolic values and return a fresh symbolic value (and the abstract state is not changed, only extended). Such operation can be performed on each evaluation step of the program expression. This makes the structure of the analyser more akin to pure functional programming, but also allows finer-grained decomposition. One illustration of this problem is seen in EVA [Blazy et al. \(2017\)](#), which only propagates non-relational values when operating on expression. In MOPSA (p.63 [Monat, 2021](#)), there is a need for a merge function that reconciliates diverging states that are temporarily unsound due to partial assignments, i.e., cannot be proved modularly. AADT and value-based analyses solve these issues.

Referential transparency In variable-based analysis, knowledge about a variable needs to be updated when the variable may be modified. For instance, we may have to implement kill operations (that completely remove knowledge about variables whose content is erased). By contrast in value-based domains, the knowledge about a variable is permanent, thanks to referential transparency, and these kinds of updates is not required: we only need to drop unused information when variables go out of scope, and possibly propagate improvements on calls to [assume](#). An illustration of this problem can be found in the limitations of the Checked C type system (§3.7 [Elliott et al., 2018](#)) : because their type system uses variable-based relations between a pointer variable and an integer variable, it is difficult to maintain these relations when either change, so they disallow these changes (or aliases that may lead to them to change) completely. By contrast the relation between the array size and content of integer in TypedC ([OOPSLA’24](#); [VMCAI’22](#); [Nicole, 2022](#)) is value-based, and thus does not have such restrictions.

Copy In memory analyses, a common operation which requires precision is copying data between different memory cells. In variable-based analysis, this requires adding an equality relation between the memory cells (e.g., §4.4.4 [Miné, 2006a](#); [Bautista et al., 2024](#)). By contrast in a value-based analysis, it suffices to copy the symbolic value from one AADT to the other.

Creation of new variables and handling of disjunctions One strength of the combination of value-based domains with AADT is that it is easy to create new symbolic values. This can in particular be used to implement local disjunctions. This situation seems to be more difficult in variable-based analysers; for instance, in [Chevalier and Feret \(2020\)](#)’s work on ghost variables, it is mentioned ([Chevalier, 2020](#), p.279) that maintaining a different offset for each possible target (block name) of a pointer would be difficult, because a target-specific ghost variable for a given target would need to be maintained for each pointer variable, even if the pointer variable does not point to that target. By opposition, our [Switch_Abstraction](#) maintain an offset only for the possible targets of a pointer value, as seen in Section 7.1. Another examples is the handling of disjunctions in the variable-based analyser of ADTs in [Bautista et al. \(2022\)](#); [Bautista \(2023\)](#), which cannot make use of local disjunctions, and require a "top-level disjunctive completion" domain subject to combinatory explosions.

Composition of abstractions AADTs allows complex compositions of abstractions, and in particular recursive dependencies. For instance in our TYPEDC checker ([OOPSLA’24](#)), we have values typed by abstract types that are AADT containing values. We also have block abstractions that may be arrays of blocks, which

is a recursive dependency between the array and block abstractions. While a lot of work focus only of arrays of integers, the ability to abstract arrays of more complex data structures is in practice very important (Bautista et al., 2024). These examples illustrate the need for complex interdependencies between abstractions, beyond trees (Cousot et al. (2006); Jourdan et al. (2015)) or DAGs (Monat, 2021; Journault et al., 2019). Note that if we can compose AADT recursively, this is due to the fact that they are stateless (they do not change the definition of abstract state); our abstract domains are composed using a tree structure corresponding to successive compilation passes (PLDI’24).

6.3.3 More examples of AADTs

In Section 6.2 we provided some basic examples of AADTs. In CODEX there are a lot of interesting examples of AADTs that we can quickly present here. Note that shape graphs can be considered as AADTs, but we discuss them in Chapter 7.

Disjunctive AADTs

Optional values (Liu and Rival (2015b); Siegel et al. (2013)) are simple to implement using AADTs : it suffices to reuse the `Ite_Abstraction`, replacing the right value by `unit`. This AADT is useful, for instance, to represent possibly-uninitialized data.

A more complex variation of the abstract addresses examples of Section 7.1.4 (where we maintain a separate offset for each possible block targeting the pointer) is the notion of *abstract offset*. Instead of representing an offset within a block as a simple integer symbolic value, an abstract offset represent a disjunctive set of possible *access paths* inside a block, where an access path (like `.f.a[3].b[α][2].u`) is a succession of field accesses (like `.f`, `.a`, `.b`, or `.u`) or array index accesses (like `[3]`, `[α]`, or `[2]`), where the index may be a symbolic value (like `α`). A prefix tree allows maintaining this set of paths efficiently. This representation of offsets improves the precision of addresses: for instance a pointer to different fields in a structure no longer point to the fields in between, as would be the case using a integer symbolic value mapped to an interval. Furthermore, maintaining symbolic indices allows comparing array indices instead of byte offsets, which is often more precise (in particular, the zone numerical domain (Miné, 2001) generally suffices in places where more complex ones like TVPI (Simon and King, 2010) or linear inequalities (Cousot and Halbwachs, 1978) would otherwise be needed). Maintaining set of paths separately is not very costly because most offsets only point to a single access path, and maintaining a strong precision on addresses is important in general because these imprecisions can cascade.

abstract offset

access paths

A final interesting example of disjunctive AADT is the disjunction of blocks corresponding to the cases in a `union` type as specified in TYPEDC, which has been designed and implemented by my PhD student (co-supervised by Mihaela Sighireanu) Julien Simonnet. One of the interesting features of this AADT is that each case contains a boolean symbolic value, corresponding to a specific condition true when the value corresponding to this case of the union; and we `assume` this boolean condition when we perform a case-based reasoning. In a sense this boolean variable is assumed on demand, not by default, which allows multiple incompatible assumptions to co-exist in a single abstract state.

Compound memory blocks

A generalization of the finite array, that we have shortly presented in Example 6.1.5 maps a fixed range of indices to a single bitvector symbolic value. This is useful to represent memory blocks of fixed-sized length in C or machine code, where values may span several contiguous addresses. The division in intervals correspond to the succession of `store` operations in the block. Because `load` and `store` operations span several addresses, they may not be aligned with the current layout of the block, which means that we have to concatenate and extract the bitvector symbolic values that were stored. Moreover, there are cases where a value spanning several bytes is accessed by small parts which are then concatenated again (for instance, when a structure is `memcpy`d). In this case, we use symbolic rewrites to retrieve the original values without losing information. This domain is the value-based equivalent of the cell-based memory abstraction of Miné (2006a), and they are interesting to compare because their implementation is quite different.

Regarding blocks of unbounded size, we have already mentionned a version of Cousot et al. (2011), which is parameterized to store AADTs instead of non-relational values. This improves over the original implementation in being automatically relational (in addition, we have the ability to handle arrays of any type instead of only integers, which is not mentionned in the original paper). We also have a simpler array-smashing (Blanchet et al., 2002) implementation. Interestingly, both rely on a notion of a symbolic value representing a set, closely corresponding to summarized dimensions of Gopan et al. (2004) (especially as formalized by Liu and Rival (2015b)).

A last memory-related AADT being implemented by Paul Robert, is a representation of strings abstracted using two integer variables, representing the capacity and position of the last zero. Interestingly, the implementation of some of these domain operations is easier to understood when viewed as a symbolic translation to SSA.

A common characteristic of memory blocks of unbounded size is that their SSA translation to the symbolic numerical domain of Chapter 4 is sound, but not exact. This means that the result of this translation can be used to prove the validity of an assertion, but the counter-examples that it finds may be spurious. A solution to restore exactness is for the SSA language to also include array operators. There is an interesting line of research to explore there, especially by combining over-approximating AADTs with under-approximating ones (e.g., Milanese and Miné (2024); Le et al. (2022)).

6.3.4 Flow-sensitive vs flow-insensitive information and limitations

The design of AADT operations (Section 6.1.3), where regular operations are forward transfer functions creating new AADTs whereas `assume` does not change AADTs but constrains the abstract state, dictates how we store information. In short, flow-sensitive information, that may be improved using `assume`, should be stored in the abstract state, while flow-insensitive information can be stored in AADTs. For instance, numerical information is stored in abstract states as it is commonly refined by predicates. On the other hand, the fact that a pair is a pair is invariant in a typed language, so this can be stored using an AADT. The `Switch_Abstraction` is an interesting example of mixture, where the list of possible values in different cases is stored in flow-insensitive way, but this can be refined by the set of possible cases using a flow-sensitive symbolic value enumerating the possible choices (the choice variable). Thinking in terms of flow-sensitive vs flow-insensitive information often

provides interesting perspectives. For instance, we can directly put abstract values (like intervals) inside AADTs, instead of integer symbolic values, but the interval cannot be refined by an `assume` statement. As another example, the forward operations in AADTs (like `read`) produce flow-insensitive AADTs using flow-sensitive information. For instance it is possible in Chapter 5 to rewrite a term like $e_1 * e_2$ to e_1 when $e_2 = 1$ in the current abstract state; the fact that the symbolic value is represented by e_1 is flow-insensitive, but the choice of e_1 instead of $e_1 * e_2$ is flow-sensitive.

The benefit of this design is simplicity, as we only need to implement forward operations on AADTs, and we do not need to update AADT values on calls to `assume`. But this can also be a limitation as we cannot refine AADTs after they have been created, notably by calls to `assume`⁵. For instance, the result of a call to `read` in `Finite_Array_Abstraction` is more precise when we know more information about the indices, so we would like to reevaluate this function when we get more information about indices, and we do not do that currently.

A workaround to the absence of refinement by `assume` is the use of local disjunctions such as the `Switch_Abstraction`. For instance, instead of merging the values using `join_value`, `read` could return a `Switch_Abstraction` where each case correspond to a possible concrete value of the index. Then, future propagation of information on the array indices would propagate to conditions on the choice variable, indirectly allowing to refine the loaded value. An interesting research direction is thus to understand whether the lack of backward propagation on AADT operations is a limitation in practice (for instance to implement shape analysis based on separation logic (Chang and Rival, 2013, 2008)), and how to lift it.

6.3.5 Symbolic view of AADT, Exactness, and Compilation.

AADT as a symbolic representation The use of `Switch_Abstraction` to make `read` more precise is actually an instance of a more general technique, which is the use of a symbolic representation to delay operations that may lose precision. One can view the replacement of `join_value` by a `Switch_Abstraction` to be delaying the merge of variables by doing it symbolically; this symbolic representation can then be evaluated on demand, in particular when more information is known about the state.

In general, like Abstract Data Types are strongly related to their algebraic specification and to the universal algebra over their signature, AADTs can be viewed as the implementation of a set of (over-approximating) rewriting rules. For instance, the pair abstract data type can be seen as an implementation of the rewriting rules $\text{first}(\text{pair}(\alpha_1, \alpha_2)) = \alpha_1$, $\text{second}(\text{pair}(\alpha_1, \alpha_2)) = \alpha_2$. AADTs extend over simple rewriting rules by incorporating the notion of constraints contained in the abstract state, the notion of anti-unification, and the possibility to perform over-approximating rewrites instead of exact rewrites. But this is an interesting connection that would be worth exploring in the future (in general, I think the combination between symbolic and semantic analyses is an interesting research direction).

Exact compilation We already saw in Chapter 4 that using symbolic abstract domains allowed for exact analyses. When we view our AADTs as performing symbolic rewrites, it is clear that most of the operations on AADT are exact (for instance,

⁵but also by other operations. For instance, we may want to refine the knowledge of the heap when we `load` values. In CODEX, there is a work around for this operation which is that the `load` signature also returns a new heap. In this case it makes sense, as the concrete `load` operation has memory side-effects such as changing cache lines, but it illustrates what could be a limitation.

pairs). In fact, all the operations on all the AADTs presented in this chapter are exact, when operations on the base domain are exact. If we view AADT-based analysis as a compilation, the analysis is thus compiling a program in a C-like memory model to an SSA program using only numerical operations. It can thus be seen as a sophisticated form of scalar replacement of aggregates (Muchnick, 1997, §12.2) transformation.

An interesting application of this is that the target SSA program is easily translated into Horn clauses and given to a software model-checking tool like SPACER (Komuravelli et al., 2016), with the following property: the assertions of the source program are valid (resp. satisfiable) if, and only if, the Horn clauses are valid (resp. satisfiable). Thus, using the same translation, we can either prove that the program is correct or provide a counter-example. When the operations on the AADT are sound but not exact (e.g., when the memory is not bounded), then the validity of the Horn assertions imply the validity of the source program, but the counter-examples found from the Horn clauses may be spurious. Thus, an interesting direction would be to combine over-approximating memory operations with under-approximating operations (Milanese and Miné, 2024), to check if properties that cannot be proved have a counter example. As part of this, it would be interesting to record when the operations risk introducing spurious counter-examples (which implies programs with unbounded loops accessing an unbounded number of memory cells) ; this is related to the logics for local completeness (Bruni et al., 2021).

Reasons for exact analysis The reason why the analysis is exact is due first to the fact that the base numeric domain of Chapter 4 is exact (for one of the concretizations), but also to the fact that the programs only manipulate a finite number of memory cells. If the source program had unbounded arrays or could perform dynamic memory allocations in a loop, we would have to introduce over-approximations (as we do in Chapter 7). Alternatively, we could introduce unbounded arrays in the target language (or even the full memory heap, as in the memory SSA form (Novillo, 2007)).

Note that small changes in the concrete semantics can make the analysis inexact. For instance, OCAML allows comparing pairs by physical equality, and the above abstraction cannot represent this precisely (we can prove that some pairs are not physically different if the values they contain are different, but we cannot say anything about physical equality when the values are equal). To restore equality, we would require adding to abstract pairs a symbolic variable that represent its addresses, that need to be comparable by equality.

Finally, while the ability to have an exact analysis is useful, the ability to perform inexact transformations is also useful, allowing to adjust the precision vs performance benefits, in particular for rarely-useful properties. For instance, verifying memory-safety of many string-manipulating functions does not require the precise content of the string, but only to know the index of the first zero and capacity. We can thus represent strings very efficiently using only a pair of integers, which suffices in the majority of C programs.

Chapter 7

Memory analyses

MEMORY ANALYSES do not need to be motivated: whoever has programmed in the C language knows that proper handling of pointers is one of the main sources of bugs. It is also the source of 70% of cybersecurity vulnerabilities (Rebert and Kern, 2024). In addition, most of the proof effort when verifying low-level programs like OS kernels amounts to preservation of datastructure invariants (Walker et al., 1980; Klein et al., 2014).

The goal of this chapter is to illustrate how symbolic value-based analyses in general, and AADT in particular, can be used for precise memory analysis in low-level languages like C and machine code. We first detail an analysis in the simpler setting of *whole-program analysis* of programs allocating a bounded amount of memory. In this setting, the finite memory can be represented exactly by representing the content of each memory cell; our analysis is interesting in that it uses AADT to perform disjunctive reasoning on the possible targets of pointers in a precise and efficient way (which is difficult to achieve in existing variable-based analysers (Chevalier, 2020)). In the second part, we explain how AADT can also be used for modular analyses of programs performing unbounded memory allocation. We discuss shape analyses based on separation logic (Chang and Rival, 2008) (developed in collaboration with Xavier Rival, where our PhD student Hugo Illous (2019) worked on shape analyses that compute heap transformations (NFM'17; FMSD'21) usable as interprocedural summaries (SAS'20)), and type-based analyses, a work that started during Olivier Nicole's PhD (Nicole, 2022) (also co-supervised with Xavier), that I am still actively researching (with the help of Julien Simonnet and Paul Robert, whom I co-supervise with Mihaela Sighireanu). This latter work shows that, in addition to program translation and analysis, abstract interpretation is also a suitable technique for program type-checking.

whole-program analysis

7.1 Whole-program analysis for a finite, C-like memory model

In this section, we will use AADTs to implement an analyser for a language with a C-like memory model (in the style of Leroy and Blazy (2008)). Its construction is built as layers of AADTs, representing (integers, addresses, values, blocks and memories) that follow the layers of definition in the concrete. The abstraction is inspired

by the design of the `VALUE` (kept in `EVA`) abstract interpreters, that build layers of non-relational abstractions by application of a succession of lattice transformers on a base interval lattice. The difference is that we build layers of AADTs over a base integer symbolic variable; as a consequence, the abstraction that we describe here are relational.

The concrete semantics (Chapter I) and definition of the analysis (Chapter J) are quite short (respectively ≈ 200 and ≈ 450 lines, not counting comments), but are moved to Appendices due to lack of space; we present here only a formalization of the analysis. Despite its small size, the analysis has several interesting features: it is *relational* (there are relations between the memory cells) and quite precise; it is built *modularly*; and it is *sound and exact* (when using the exact concretization as $\gamma_{\S\#}$, which is itself a sound and exact abstraction of SSA programs manipulating integer expressions. In terms of translation, it means that the analysis that we will be defining allows exact, semantics-preserving translation from programs with a finite C heap to an SSA program manipulating only integers. In practical terms, it means that we can send unproved assertions to an automatic solver that only understands integer expressions, and the answer to that solver will be valid iff the original assertion is valid, and satisfiable iff the original assertion is satisfiable. In other words, value-based abstract interpretation with AADT is a pragmatic tool to implement a verification-condition generator for C.

7.1.1 Memory model

The main feature of this language is a memory heap similar to C's (see e.g. the memory model of Compcert (Leroy and Blazy, 2008)), where the heap is composed of set of *block* (contiguous sequence of cells, where each cell can contain a based value), each identified with a *block name* $b \in \mathcal{BN}$. This model differs from C mainly in that the base type are infinite integers instead of finite bitvectors. In addition, we assume here that the set of block names in a given program is finite.

block

block name

```

module Block_Name = struct
  type t = {name:string; id:int; size: int}

  let id_to_block: (int,t) Hashtbl.t = Hashtbl.create 17
  let count = ref 0
  let create name size =
    let id = !count in
    let res = {name;id;size} in
    incr count; Hashtbl.replace id_to_block id res; res
  let total() = !count
  let id_to_block id = Hashtbl.find id_to_block id
end

```

We will thus use a finite set of block names (`Block_Name.t` in the code, and $b \in \mathcal{BN}$ in the formalization). To each block name corresponds a unique (integer) identifier `id`, a positive size, and a name. In C, a block name would correspond to the definition of a global variable; e.g., a block name whose name is "g" and whose size is 12 would correspond to the C global declaration `char g[12];`. There is a unique correspondence between block names and integers, and the above interface help manipulate this correspondence.

```

module type HEAP_OPERATIONS = sig
  type address      type integer      type value      type heap      type boolean
  val base_address: Block_Name.t → address
  val pointer_add: address → integer → address
  val pointer_eq: address → address → boolean
  val load: heap → address → value
  val store: heap → address → value → heap
  val initial_heap: Block_Name.t list → value → heap
end

```

In addition to the standard `integer` and `boolean`, our C memory model operates on 3 type of values: `heap`, `address`, and `value`. Basically a `heap` is a map from `address` to `value`.

The main operations in the model are as follow: `base_address` returns the first address in a block; this corresponds to `&g` in C (where `g` is the name of a global variable). `pointer_add` performs pointer arithmetics, `pointer_eq` compares two pointers for equality, `load` retrieves a values from the heap and `store` updates the heap. Finally, `initial_heap l` creates a heap composed of a set of blocks `l`. Note that this describes a finite, fixed-size heap as found, for instance, in embedded systems; we could add a `malloc` function to make it infinite (but this makes the abstraction more complex).

7.1.2 Syntax

When can now integrate `HEAP_OPERATIONS` in our `WHILE` expressions.

```

module Typ = struct
  include Typ
  let heap: ['heap] Typ.t = Typ.new_tag()
  module Heap = (val heap)
end
module type C_LIKE_EXPRESSIONS = sig
  include INTEGER_AND_BOOLEAN_EXPRESSIONS
  include HEAP_OPERATIONS
  with type address := ['integer] expr
  and type value := ['integer] expr
  and type integer := ['integer] expr
  and type heap := ['heap] expr
  and type boolean := ['boolean] expr
end

```

We integrate the heap a way that does not distinguish address and integer values using different types. Instead, we consider both `integer` and `address` to be part of generic `value` (that we type as "integer". Such a low-level description is suitable to a low-level language like machine code, where addresses will eventually be compiled to integer values (or bitvectors) and pointer arithmetic to simple addition. However our language, like C, does not allow arbitrary operations on values, and ill-defined operations are undefined behaviour. Here, we use a particularly strict semantics where all arithmetics operations on address values (including addition with an integer value that is not done with `pointer_add`) are undefined. Adding more undefined behaviour makes the analysis simpler as we can make more assumptions in the analysis (that still need to be checked). When analysing machine code, we

use a less strict analysis where for instance we reinterpret arithmetic additions as `pointer_add`.

Also note that the heap will just be stored in a variable of type `heap` (this allows minimal changes in our implementation).

7.1.3 Concrete semantics

The full concrete semantics of our language is defined in Chapter I (as an OCaml interpreter that extends our `WHILE` interpreter). Here we describe the formalization of the state.

Definition 7.1.1: Concrete state for `WHILE` with `C` memory

$\mathbb{A} \triangleq \mathcal{BN} \times \mathbb{Z}$	Concrete addresses
$\mathbb{CV} \triangleq \mathbb{Z} \cup \mathbb{A}$	Concrete "C" value
$\mathbb{B} \triangleq \{\text{true}, \text{false}\}$	Concrete booleans
$\Sigma = (\mathbb{X}_{\text{int}} \rightarrow \mathbb{CV}) \cup (\mathbb{X}_{\text{bool}} \rightarrow \mathbb{B})$	Concrete store
$\mathbb{H} \triangleq (b : \mathcal{BN}) \rightarrow (\mathbb{CV}^{b.\text{size}})$	Concrete heap
$\mathbb{M} \triangleq \Sigma \times \mathbb{H}$	Concrete memory
$\mathbb{S} = \mathbb{M} \times \mathbb{I}$	Concrete state

The program in our language manipulates "C" values $v \in \mathbb{CV}$ that are either integers $z \in \mathbb{Z}$ or addresses $(b, o) \in \mathbb{A} \triangleq \mathcal{BN} \times \mathbb{Z}$, where o is an offset inside a region b . Like in C, the intent of these addresses is that they can be compiled to integers, but the language restricts the operations that are allowed on them (notably using provenance (Memarian et al., 2019)). Then, block consists in a finite sequence of values $\mathbb{CV}^n$ for some length n . Finally, heaps $h \in \mathbb{H}$ are mappings from block names $b \in \mathcal{BN}$ to blocks whose length depends on the block name (i.e. the length of a block mapped to b is $\text{size}(b)$).

7.1.4 Abstract semantics

Using AADT, we can build an abstract state (Definition 7.1.3) that closely follows the construction of the concrete state (Definition 7.1.1).

Definition 7.1.2: The Heap AADT

$$\begin{aligned}
\text{Heap}(\mathbb{V}) &\triangleq (b : \mathcal{BN}) \rightarrow (\mathbb{V})^{b.\text{size}} \\
\gamma_{\mathbb{V}}(\hat{h}, \hat{s}^\#) &\triangleq \{(h, v) \in ((b : \mathcal{BN}) \rightarrow \mathbb{V}) \times \mathbb{V} \mid \\
&\quad \forall b \in \mathcal{BN} : \forall i \in [0..b.\text{size} - 1] : (h[b][i], v) \in \gamma_{\mathbb{V}}(\hat{h}[b][i], \hat{s}^\#)\}
\end{aligned}$$

First, we define `Finite_Heap_Abstraction` in Section H.1, which is a variation of `Finite_Array_Abstraction`, to represent finite heaps. Basically, a heap is an array of blocks (indexed by a finite set of block names, which can be encoded as integers), which is an array of values. We could be using `Finite_Array_Abstraction` twice and really view heaps as blocks where blocks are arrays of values, but doing so would be

less precise (e.g., loading a pointer would perform an intermediary merge of blocks), so we propose a direct implementation instead¹.

Definition 7.1.3: Abstract state for WHILE with C memory

$\mathcal{Q}_{\mathbb{B}}$	Abstract booleans
$\mathcal{Q}_{\mathbb{Z}}$	Abstract integers
$\mathcal{Q}_{\mathcal{BN}} \triangleq \mathcal{Q}_{\mathbb{Z}}$	Abstract block names
$\mathcal{Q}_{\mathbb{A}} \triangleq \text{Switch}(\mathcal{Q}_{\mathbb{Z}}) \approx \mathcal{Q}_{\mathcal{BN}} \times (\mathcal{BN} \rightarrow \mathcal{Q}_{\mathbb{Z}})$	Abstract addresses
$\mathcal{Q}_{\text{CV}} \triangleq \text{Ite}(\mathcal{Q}_{\mathbb{B}}, \mathcal{Q}_{\mathbb{Z}}, \mathcal{Q}_{\mathbb{A}})$	Abstract "C" values
$\mathcal{Q}_{\mathbb{H}} \triangleq \text{Heap}(\mathcal{Q}_{\text{CV}})$	Abstract heaps
$\mathcal{Q}_{\Sigma} \triangleq (\mathbb{X}_{\text{int}} \rightarrow \mathcal{Q}_{\text{CV}}) \cup (\mathbb{X}_{\text{bool}} \rightarrow \mathcal{Q}_{\mathbb{B}})$	Abstract stores
$\mathcal{Q}_{\mathbb{M}} = \mathcal{Q}_{\Sigma} \times \mathcal{Q}_{\mathbb{H}}$	Abstract memories
$\mathcal{S}^{\#} \triangleq \mathcal{Q}_{\mathbb{M}} \times \hat{\mathcal{S}}^{\#}$	Abstract states

The abstract state is built as a succession of layers. First, we have abstract (or symbolic) booleans and integers which correspond to the symbolic values of the [VALUE_BASED_DOMAIN](#) (for instance, the [Numerical_SSA_Domain](#) of Chapter 5). Then, we need an abstract representation $\mathcal{Q}_{\mathcal{BN}}$ for an unknown block name in a set. Here we simply reuse integer symbolic values $\mathcal{Q}_{\mathbb{Z}}$, taking advantage of the block name/unique id correspondence of module [Block_Name](#). In practice this choice is not very good as integers are abstracted using intervals in [Numerical_SSA_Domain](#), and thus a set of two block names would include all the block names whose id is in-between. In CODEX we have dedicated "enumeration types" that precisely represent a set of finite choices, for this purpose.

Abstract addresses are maps from the possible block names to which the pointer may point, to the possible offsets corresponding to this block name; and abstract block names $\mathcal{Q}_{\mathcal{BN}}$ are used as a choice variable to make this representation of addresses more precise using the [Switch_Abstraction](#). For instance, after a pointer equality test $p == q$, we learn that both pointers will point to the same set of block names using choice variables. Furthermore, pointers derived from another pointers using a call to [pointer_add](#) will also share their choice variable (see [pointer_add](#) in Chapter J).

"C"-like values are either an abstract address or an abstract integer. Again we use a choice variable to improve this representation (for instance, if two values are equal, they are either both an address or both an integer). Here the choice variable is a boolean telling whether the value is an address or a simple integer, so we use the [Ite_Abstraction](#) to represent this disjunction.

Then, blocks are finite arrays of "C" values represented using [Finite_Array_Abstraction](#), and heaps are finite arrays of [Block_Name](#) to blocks whose operations are implemented in [Finite_Heap_Abstraction](#). The [Abstract_Store](#) is the same than in Chapter 4. Note that in the formalization we have separated the heap from the store, but to simplify the implementation we have really put the heap as a variable in the store. Finally, the full state is described as just an abstract memory and an abstract SSA state (as provided by the [Numerical_SSA_Domain](#)).

¹The implementation is actually parametric on the implementation of blocks, but we will forget about this for the sake of simplicity.

The construction, based on opaque abstract datatypes, is modular in that it is easy to swap implementations to increase or decrease precision. For instance, Chapter J proposes a simpler implementation of abstract addresses not relying on `Switch_Abstraction`, and where the same offset is used for every possible target ($\mathcal{Q}_{\mathbb{A}} \triangleq \mathcal{Q}_{\mathcal{BN}} \times \mathcal{Q}_{\mathbb{Z}}$, as found in Astrée (Miné, 2006a) or Verasco (Jourdan et al., 2015)). Swapping the implementation requires no change to the others AADTs. Also note that despite this modularity, the construction is fully relational.

We do not mention the concretization here (but we already provided them for all the AADTs functors), nor the implementation (which is in Chapter J) and their proof, but we can prove that all the transfer functions are sound and exact when the base domain is.

7.2 Modular memory analysis of dynamically-allocating code

The goal of this section is to present how symbolic value-based analysis and abstract abstract data types can be used to perform modular analysis of programs with dynamic memory allocations. Specifically, I have studied two kinds of memory abstractions: a relational extension (NFM’17; FMSD’21; SAS’20) to separation logic-based shape analysis by Chang and Rival (2008, 2013); and type-based analysis (VMCAI’22; OOPSLA’24) that perform type checking via abstract interpretation, thereby guaranteeing (spatial) memory safety. For a more comprehensive introduction to this subject, including comparisons between the approaches, please refer to *Frama-C Book* (2024). Here, we focus on the implementation of these analysers.

Suppose we are given this C program, which we wish to verify as memory-safe.

```
struct data { int value; /* Smaller than 100. */ };
struct node { struct node *next; struct data *data; };

void decrement_list(struct node* head){
    struct node *ptr = head; struct data *dptr;
    while(ptr != 0){
        dptr = ptr->data;
        dptr->value = dptr->value - 1;
        ptr = ptr->next;
    }
}
```

A first observation is that we are not given the entire program, but only the `decrement_list` C function which is a part of a larger program. This is an example of a *modular analysis*: we want to analyse `decrement_list` without knowing precisely the context in which it is called. In particular, we do not know the length of `head` argument, which means that we need to summarize the memory. By memory, in C, we mean the *heap graph*, the graph whose nodes is the set of all memory blocks allocated by the program, and edges represent pointers between these blocks.

modular analysis

heap graph

Figure 7.2.1: Heap graph

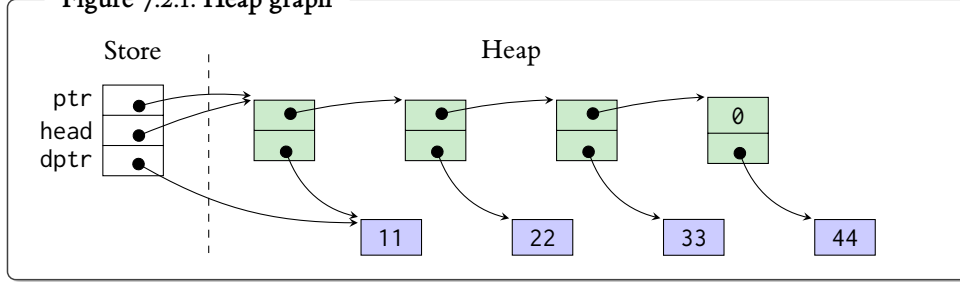


Figure 7.2.1 represents a possible concrete state in the execution of the `decrement_list` function on a linked list with 4 elements, when execution reaches the last statement of the `while` loop in the first iteration. The heap graph is the set of all list nodes (in green), and the data to which they point (in blue).

It is difficult to analyse the program without making some assumptions about its behaviour. For instance, if $dptr \rightarrow \text{value}$ could point to the end of the list, then decrementing it would change a null pointer, and we would not detect the end of the list. In machine code, `ptr` or `dptr` could point to the return address in the stack, or inside the code, and nothing much can then be inferred about the program. Therefore, we want to analyse the program under some assumptions about the shape of the memory (and warn, or analyse with weaker assumptions, if these assumptions do not hold), and in particular some assumption on the initial state (the precondition). Note that when analysing a whole program or a larger part of the code, these preconditions may be inferred automatically (SAS'20).

7.2.1 Shape analysis

Two strategies for automated verification this program are now possible. The first strategy, *shape analysis*, is to devise a very precise abstraction of the part of the heap graph that is accessible by the `decrement_list` function, and to track precisely the position of all the pointers (`head`, `dptr`, and `ptr`) within this graph. Note that the set of all the possible heap graphs in the initial state can be arbitrarily large, each heap graph in this set can be arbitrarily large, and so too is the distance between two pointers in the graph: consider for instance the distance between `head` and `ptr` at the end of the execution. For these reasons, the defining characteristic of shape analyses (Chang et al., 2020) is the use of *summaries* that can finitely represent unbounded parts of the heap graph, as well as unfolding/materializing/focusing operations that extract heap graph nodes from the summary (and their converse folding/dematerializing/unfocusing operations).

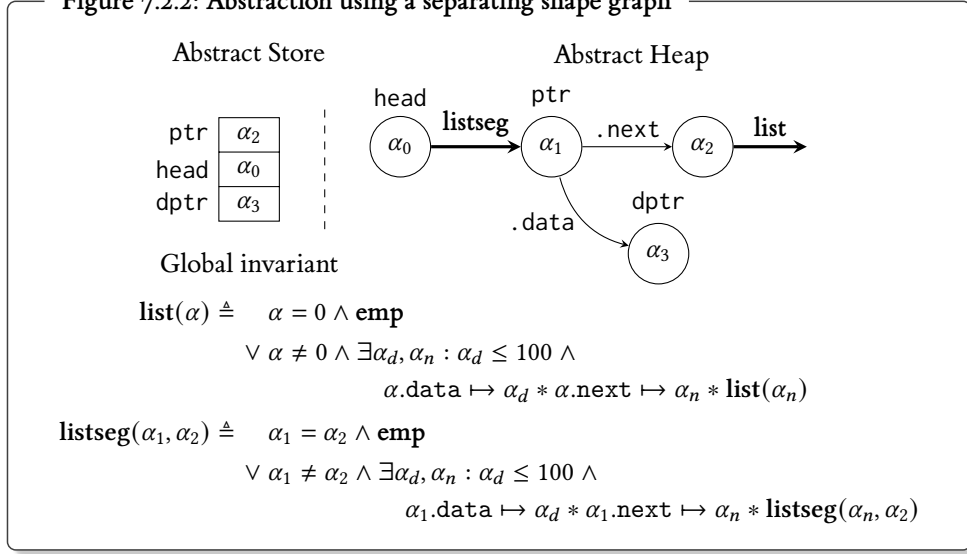
shape analysis

summaries

Figure 7.2.2 presents, in the framework of Chang and Rival (2008) based on separation logic, the abstract state in the `decrement_list` function before the execution of the last statement of the `while` loop. We already saw the abstract store in Chapter 4. In the abstract heap, nodes are symbolic values and edges represent disjoint memory locations. For instance, the $\alpha_1 \xrightarrow{\text{next}} \alpha_2$ edge represent the memory location accessed by $\text{ptr} \rightarrow \text{next}$ to get α_2 , the new value of `ptr`. `list` and `listseg` are inductive predicates, i.e. user-provided inductive definitions that are used to summarize memory regions. The use of `listseg` summarizes the start of the list, while `list` summarizes the end.

This abstraction should be implementable using AADTs; it is a map $\alpha \times \mathbb{F} \rightarrow \alpha$ from symbolic values (addresses) and field names to symbolic values (the content). The implementation detailed in Chang and Rival (2013) is not formalized as an

Figure 7.2.2: Abstraction using a separating shape graph



AADT, but as a full domain whose join operations uses and compute renaming functions ϕ , which I am confident could be formalized using weakening and anti-unification. It would be interesting to see if some uses of disjunctive completion could be replaced by local disjunctions, and whether this would make the analysis more efficient or if we would lose precision. This is an interesting perspective for future work.

The benefit of shape analysis is that we can obtain a precise invariant about the program heap. The analysis can accommodate changes in the heap graph structure to prove complex properties, such as absence of use-after-free or proper reuse of memory blocks to store different data structures of different types (sometimes referred to as a strong update). However, there are drawbacks of shape analysis in the context of automated verification:

- First, finding the right structure to represent a heap summary can be difficult. Analyses based on separation logic (Chang and Rival, 2008) can be parameterized with user-supplied inductive predicates that provides possible summaries to the analysis. However, providing this information requires precise knowledge of the aliasing relations in the heap graph. For instance, consider the `decrement_list` function: it can be called in situations where the list pointed by `head` is normal or circular, or when the tail of the list is shared with another list, or when some `struct` data nodes are shared between different list elements. All these correspond to different user-supplied predicates and initial states.
- The analysis may be costly; for example, when multiple pointers traverse a linked list, we may need to use case-by-case analysis and create a set of different summarized heap graphs based on the positions of each pointer in the graph. In the worst scenarios, the number of cases to consider grows exponentially (Li et al., 2017).
- Implementing the folding code is difficult, and the analysis can encounter situations where the heap graph cannot be summarized using the provided pred-

icates, and will fail (instead of just raising a false alarm). It is not possible to just forget information about some of the pointers or part of the heap graph (or the analysis will be unsound).

For these reasons, I believe that while shape analysis is a good solution for verifying the most complex parts of a program, it needs to be complemented by analyses that infers weaker invariants, but more automatically; so that we can verify the program incrementally.

7.2.2 Type-based analysis using a structural type system

The second strategy involves using a less precise representation of the heap, where we do not track the position of pointers in the heap graph. We also can safely forget about some pointers : this will prevent analysis of code using that pointer but not affect the rest of the program analysis). This approach is based on type-based analysis [VMCAI'22](#); [OOPSLA'24](#). More precisely, we perform a type-based analysis using a *structural type system* ([Walker, 2005](#)), whose rules do not require tracking all references to a memory block, which allows for a simpler and more automatable analysis.

The idea is to classify all memory blocks into a fixed set of *regions*, such that blocks within the same region share similar properties. A natural approach is to define one region per C type, as shown in Figure 7.2.3. There we define two region names (or type name), *data* and *node*, and attach constraints to these names (to the right of $:=$), that we describe later. Note that it is possible (and more precise) to define different regions with the same content : for this reason, our type system is *nominal*. For instance, in our experience with `TYPEDC`, we sometimes have to change the correspondance between region names and C types: sometimes we have several regions for the same original C type (because these regions have different invariants). Sometimes we also have a single region for different types (as a single memory block may be reused to contain values of different types), but in this case there is generally an existing `union` type in C.

We can now define an analysis based on these region names. The idea is that we represent a pointer value solely by its target region name, and offset within that region. For instance, Figure 7.2.3 also represent the abstract state before execution of the last statement of the `while` loop in the `decrement_list` function. We can see that the `src_c{` variable contain the address of a *node* region at offset 0, that the `head` variable contain the address of a *node* region at offset 0 or the `NULL` pointer of value 0, and that `dptr` contain the address of a *data* region at offset 0. The expression `ptr → next` computes an address within a *node* region at the offset corresponding to the `next` field (typically 4 or 8, depending on the architecture; in our idealized semantics, where the basic unit of storage is an integer of arbitrary size, it is 1).

structural type system

regions

nominal type system

Figure 7.2.3: Abstraction using type-based analysis

Abstract Store		Global invariant
ptr	node ★ +0	$\text{data} := \text{struct}\{ (\text{int with self} \leq 100) \text{ value}; \}$ $\text{node} := \text{struct}\{ ((\text{int with self} = 0) \cup \text{node} \star) \text{ next};$ $\text{data} \star \text{ data}; \}$
head	if($\alpha_2, 0, \text{node} \star +0$)	
dptr	data ★ +0	

This abstraction alone would not suffice to verify spatial memory safety in most

program. What we need to prove can be informally described as "safe navigation": programs "navigate" the heap graph by either pointer arithmetic (within a memory block), or by following pointers between blocks. We want to ensure that accessing memory at addresses obtained by any combination of these operations is safe. For this, we have to rely on some properties of the heap graph. A nice feature of type-based analysis is that the heap graph can be represented using only a global, flow-insensitive invariant over the contents of the regions. Observe that we do not have a represent of the heap graph in Figure 7.2.3. We only have a global invariant, encoded using global annotations on the region content (i.e., *type invariants*). What the TYPEDC annotation of Figure 7.2.3 expresses is that:

type invariants

- A **data** region contains an integer whose value is in the interval $[-\infty..100]$ range;
- A **node** region is the concatenation of two integer values, the first containing the address of a **data** region at offset 0, the second is either the address of a **node** region at offset 0, or the null pointer (of value 0).

When reading values from the heap, we make use of the global heap-graph invariant to improve our knowledge over the content of the value being read. For instance, we remember that the symbolic value returned by $\text{data} \rightarrow \text{value}$ is in the $[-\infty..100]$ range, that the symbolic value returned after evaluation of $\text{ptr} \rightarrow \text{data}$ is the address of a **data** region at offset 0, etc. Doing this suffices to prove safe navigation, and would be all that we have to do in a purely-functional program with no assignment.

However, we are analysing imperative programs, that also make changes to the heap. Thus, we also have to make sure that our global invariant is also preserved when writing to the heap. One necessary condition is that the block being modified still fulfils the invariant of the region of that block. This is the case for the write $\text{data} \rightarrow \text{value} = \text{data} \rightarrow \text{value} - 1$: as we are decrementing a value smaller than 100, invariant that the value being written is smaller than 100 will be fulfilled. But, it could be the case that data was pointing to a block that was both a **data** and a **node**, in which case this write could be messing up with the invariant that this block should also satisfy the invariants on **node** blocks. For this reason, our global invariant also features *separation*: there are no overlap in the address of blocks of different types. This implies, in particular, type-based alias analysis (Diwan et al., 1998): i.e., that the dptr pointer does not alias with the ptr ptr, as they point to blocks in different regions. With this last property, our memory model is complete.

separation

Implementation as an AADT

Describing a simple type-based analysis using AADT is easy (Definition 7.2.4). A full implementation, that can verify an encoding of our above C example in the WHILE language, is provided in Chapter K.

The structure is simpler, but similar, to the one of the whole program analysis of Section 7.1. Some things are the same: the definition of abstract booleans $\mathbb{Q}_{\mathbb{B}}$ and integers $\mathbb{Q}_{\mathbb{Z}}$, the definition of "C" values $\mathbb{Q}_{\text{CV}}$ as a disjunction between numerical values and addresses, or the abstract store $\mathbb{Q}_{\Sigma}$. Some things are similar: we represent abstract addresses $\mathbb{Q}_{\mathbb{A}}$ as maps from type names to their respective offset, and this structure is very similar to the representation of addresses as maps from block names to offsets in the whole-program analysis. There are differences in transfer functions

Definition 7.2.4: Abstract state for modular type-based analysis WHILE

$\mathcal{Q}_{\mathbb{B}}$	Abstract booleans
$\mathcal{Q}_{\mathbb{Z}}$	Abstract integers
$\mathcal{Q}_{\mathcal{RN}} \triangleq \mathcal{Q}_{\mathbb{Z}}$	Abstract region names
$\mathcal{Q}_{\mathbb{A}} \triangleq \text{Switch}(\mathcal{Q}_{\mathbb{Z}}) \approx \mathcal{Q}_{\mathcal{RN}} \times (\mathcal{RN} \rightarrow \mathcal{Q}_{\mathbb{Z}})$	Abstract addresses
$\mathcal{Q}_{\text{CV}} \triangleq \text{Ite}(\mathcal{Q}_{\mathbb{B}}, \mathcal{Q}_{\mathbb{Z}}, \mathcal{Q}_{\mathbb{A}})$	Abstract "C" values
$\mathcal{Q}_{\Sigma} \triangleq (\mathbb{X}_{\text{int}} \rightarrow \mathcal{Q}_{\text{CV}}) \cup (\mathbb{X}_{\text{bool}} \rightarrow \mathcal{Q}_{\mathbb{B}})$	Abstract stores
$\mathcal{Q}_{\mathbb{M}} = \mathcal{Q}_{\Sigma}$	Abstract memories
$\mathcal{S}^{\#} \triangleq \mathcal{Q}_{\mathbb{M}} \times \hat{\mathcal{S}}^{\#}$	Abstract states

and, in particular, when two values point to the same type name and offset, they may still be different; this is because type-names summarize a set of blocks, instead of representing individual blocks. Finally, some things are different: we do not have any abstraction for abstract blocks or abstract heaps in this analysis, as this information is entirely contained in the flow-insensitive type definitions of the regions `data` and `node`.

Extensions and concluding remarks

The type-based analysis that I have just presented is simple. Most of this example could be type-checked without abstract interpretation, using a simpler syntactic method (which could also be formalized using abstract interpretation). However, abstract interpretation is already useful to propagate the fact that `ptr` is non-null inside the `while` loop, or to check the refinement type constraints on the contents of `data` regions.

An important benefit of abstract interpretation is that it make it easy to extend, and to combine different reasoning methods to prove a result. Syntactic type systems work well in languages like Haskell, OCaml or Rust as programmers are taught to accept restrictions on the programs that they can write in exchange for guarantees that their programs behave correctly (in particular, are type-safe and memory-safe); even if limitations of the type system expressivity is a driving motivation for extending the type systems. In a language like C, the programming culture is absence of safety guards by the compiler, reliance on the programmer to satisfy subtle semantic invariants, and design and implementation of data structures that are optimized for the program being developped. In this context, the ability to easily combine different reasoning methods for proving type-safety and memory-safety is important. In the following, I will present some existing and future extensions of the type-based analysis presented here.

Cache domain The first important extension is to have a flow-sensitive representation of a part of the heap. Often programs load from the heap a previously-stored value on which we have some information; or load a value in the heap, make an assumption on it, and load it gain. A flow-insensitive invariant cannot represent these crucial additional informations. The solution is to add *retained points-to predicates*

retained points-to predicates

(VMCAI'22), that we write $\alpha_1 \mapsto \alpha_2$, which means that dereferencing the (symbolic) address α_1 yields the symbolic value α_2 . As we now have a name α_2 for the content of the heap, we can attach information to it (e.g., using a numerical domain as in Chapter 5, or if α_2 is an AADT as in Chapter 6). In CODEX these information is stored in something we call the *cache domain* (where the "cache" is the points-to predicates that we have retained), that implements focusing (when we load the address) or unfocusing (when we forget about the content of an address) operations, and relies on aliasing information about the addresses to update or drop values from the cache. With this cache domain, we have an analysis similar to the low-level liquid types of Rondon et al. (2010) (but with more precise decisions on where to insert focus and unfocusing operations). This cache domain can also be seen as an extension of LPAR'18, which can be described as an implementation of the cache domain restricted to single execution traces for use in symbolic execution.

cache domain

This cache domain can be extended in various ways. In VMCAI'22, we included staged points-to predicates in addition retained one. The idea is that after some memory writes, the program may not be satisfying some type invariants. Thus, we hold the writes and group them with further ones, that restore the invariant. During his PhD thesis, Julien Simonnet further extended the cache domain. In particular, we no longer cache individual memory locations, but whole memory blocks. This is important as different fields in a data structure may be related, and we want to remember the whole block to remember this relation. We can imagine many further extensions of this cache domain; for instance, it could include inductive predicates in separation logic, as seen in Section 7.2.1; this would also require stronger type invariants.

Nested regions In the analysis we have presented, all regions are separated: pointers to different regions do not alias. This simple model is too strict for languages like C, where, for instance, an `int*` pointer could point inside a block whose type is structure with an `int` field, like `data`. In object-oriented languages like C++, inheritance corresponds to defining a new record B as an extension of an existing record A with new fields, and to allowing conversion from pointers to B to pointers to A . Both of these examples call for equipping regions with a hierarchical structure, where some regions can include others. This defines an order relation between region names that is a tree, and two region names are separated if they are in different branches of this tree (VMCAI'22). In OOPSLA'24 we considerably extended this type system in several directions. First, we clarified the role of names in our system: regions are defined using a type system, some of these types are named, and pointers can only point to named types. Second, we added disjunctive types: e.g., a union type $\tau_1 \cup \tau_2$ is defined disjunctively as containing either a value of type τ_1 or a value of type τ_2 . We formalized the interaction between named types, disjunctive types (union and existential types), and type concatenations (record and array types). Among our findings, we found subtle (but still structural!) typing rules like mild update, where some blocks can change of type if no pointer existed that expected the block to be in the previous type. Finally, we allowed region names to have parameters, which allows sharing invariants across different blocks; for instance, to relate a block containing an array of size ℓ to a block containing a pointer to this array, and an integer ℓ . The goal of this extension was to push the limits of what can be expressed using a dependent, but structural, type system with automatic type checking.

Limitations of structural type systems There are limitations in what can be achieved using only a structural type system. The most important is the absence of strong update (Toman et al., 2020), i.e. we cannot change the type of a block if there are existing pointers to this block; because we do not track all the possible pointers to a block, we cannot guarantee the safety of some strong update operations. Strong updates are rare in practice, except in two cases: data structure initialization (but this case is easily handled using mechanisms that amount to recency abstraction (Balakrishnan and Reps, 2006)), and data-structure deallocation. For this reason, a structural type system cannot be used to guarantee absence of use-after-free errors. There are many situations where these do not matter, like embedded systems (which often allocate all data statically); systems using garbage collectors, like the runtime of GNU Emacs; or code accessing the heap without changing the shape of the heap graph. In the other systems, we can only guarantee spatial memory safety. An important future line of work is thus to extend our type-based analysis to support temporal safety properties; by integration with shape analysis like that of Section 7.2.1, using substructural type systems like ownership types (Clarke et al., 2013). Note that these extensions are quite orthogonal to our base structural type system: in short, our base type systems provides invariants on individual blocks of the heap graph (by assigning a region to each block, and properties about the region), while extensions would enforce topological restrictions on the heap graph (e.g., ensuring that some blocks can be pointed to by at most one reference).

Chapter 8

Conclusion

 MY RESEARCH CAREER has known two phases. In the first, I was doing systems research, in particular in the area of secure, safety-critical, and real-time operating systems. In 2013, I switched to automated methods for formal verification. The connecting thread of this work is that my current goal is to find and develop pragmatic tools for guaranteeing the correctness of the systems programs that I used to write: the knowledge of how systems programs work help me develop static analysis methods that focus on the most important invariants. For instance, as in OS verification, “invariant reasoning [on pointer-manipulating data structures] dominates the proof effort” (Klein et al., 2009; Walker et al., 1980), this lead me to work on automated verification using type-based annotations (OOPSLA’24; VMCAI’22), which successfully worked when applied to the verification of full embedded OS kernels (RTAS’21).

The answer that I have found to the problem of efficiently guaranteeing the correctness of systems programs is type-based modular sound static analysis (Section 7.2.2). An important difficulty when writing low-level system code is to make sure that your code preserves intricate low-level data structure invariants; calls functions with the right preconditions; and does not exhibit undefined behaviour (including memory-safety bugs). Using types, when interpreted in the style of Church, prevents one to write incorrect programs, and is thus a solution to the problem of writing incorrect code. But when interpreted in the style of Curry, or under the lens of abstract interpretation (Rival and Yi, 2020, §10.3.1), they are an incomplete static analysis that can also prevent you from writing semantically-correct program, where a more powerful analysis could have succeeded in proving that the code is correct. In systems code, I think what matters is proving properties about the final machine code program that is run by the hardware, whose semantics is untyped. We should use static analysis to try to prove these properties as automatically as possible, and types, when integrated inside a static analysis, are part of the toolbox to do this. Decidable syntactic type checking alone is insufficient for systems programs whose invariants are often best modelled using dependent types, and can have very uncommon design (e.g. Massalin, 1992). But type systems can play an important role in systems program verification. In particular, type systems promote modular verification, where types can be applied separately to different parts of the program, and are used to communicate the expected invariants on data types and across function boundaries. This methodology could also be applied to static analysers—even if the field generally focuses on unannotated verification, and it is true that writing specifications can be

cumbersome, which is why we should infer a part of those annotations, like we infer some types in languages like OCaml.

It remains to see how to design and implement such a static analyser, which is the subject of this manuscript. It describes symbolic value-based abstract interpretation, as I am convinced that it is a convenient way of implementing static analysers for low-level (and others) languages, in particular when we need to do complex reasoning about memory. To summarize, symbolic value-based abstract interpretation uses static analysis to translate the program to a language of formulas which is closely linked to the Static Single Assignment (SSA) form (Rastello and Bouchez Tichadou, 2022) found in compilers. This replaces complex reasoning about side effects to a combination of semantic reasoning about immutable formulas, and simple translation passes integrated in the static analyser, that include the manipulation of abstract abstract data types. I will not repeat here the benefits of this approach and its description; instead, I would like to explain how I got interested in the problem of checking correctness of low-level code, and how I came to the conclusion that value-based symbolic abstract interpretation was the answer.

8.1 From operating systems code to formal methods

Low-level system programming I have always been interested in low-level code. As a teenager, I learned x86 DOS programming, wrote assembly games for Texas Instrument calculators, or learned how to debug, disassemble and patch programs to bypass copy protections. I then became interested in OS design and implementation, reading great books like the second edition of Operating System Design and Implementation (Tanenbaum and Woodhull, 1996) (with the MINIX source code) or the memory management part of Knuth’s Art of Computer Programming (with all examples in a fictive assembly language), and as a student became invested in the short-lived GNU Hurd-on-L4 project. This is why I became greatly interested in working as an intern, then PhD student, then full-time researcher in CEA, in the lab headed by Vincent David (that I thank for advising and hiring me), who was leading the OASIS (David et al., 1998; HASE’11) and PHAROS (ISORC’11) projects. There, I practiced many aspects of operating system design and implementation. My first task as an intern was to do the first port the OASIS kernel to multicore processors. This got me interested in lock-free and wait-free (Herlihy, 1991) concurrent algorithms, and weak (non-sequentially consistent (Lamport, 1979)) memory models. This was long before those were standardized in C, which got me interested in the difference between C and machine behaviour (e.g. using memory and compiler fences, to ensure that both the CPU and compilers ensured that instructions would be executed in the correct order).

The initial goal of my PhD was to explore OS designs that would allow the co-existence of hard real-time and general-purpose tasks on a single system. I reframed the problem as ensuring resource security: how to guarantee that certain tasks meet their deadlines, even when other tasks actively attempt to delay them? The main issue was sharing resources, in particular in the trusted servers that are protecting the shared resources (like the X server shares the screen between different tasks). I came up with a design where all the tasks were properly attributed all the resources that they used, including when they were executing code in the trusted server, because they lent the resources that they owned to the server (IIES’09) (I think this has interesting connections with resource management in substructural type systems

(Walker, 2005)). This led to the design of data structures that were efficient but very complicated because we needed to bound the time spent on every operation (because of the hard real-time constraint), and that some of the resources (CPU time and memory) used by the shared server could disappear (because the task that lent them was killed). A nice example of this is the algorithm that controlled the use and recycling of pages as page tables (IIDS'10).

Pen-and-paper proofs Because it was very difficult to get this code right, I was naturally drawn to formalizing those data structures, first using pen-and-paper proofs, and then by formal verification (e.g., applying formal methods on the page table algorithm presented in IIDS'10 in join work with Nikolai Kosmatov, Allan Blanchard, Frédéric Loulergue, Céline Alec (TAP'14; FMICS'15)). I often found that formalization and trying to prove the correctness of an algorithm allowed to discover stronger invariants, which led to improving the efficiency of the operations on the data structure. This is also observed, I believe, in some Rust programs that are more efficient than C++ ones: this is because you can depend on more complex invariants with less fear of having incorrect code, a concept branded in Rust as "fearless concurrency".

I began formalizing and proving properties about many of the things that I was doing, and in particular schedulability analysis (RTSS'06; RTAS'08) (trying to prove, in a real-time system, that all the tasks would meet their deadlines). The OASIS/PHAROS system could express scheduling constraints that were much more expressive than the classical periodic tasks; if Christophe Aussaguès had already worked on finding methods to perform this analysis (Aussagues and David, 1998), there were interesting areas of extension, such as multiprocessor scheduling or more expressive timing constraints (RTAS'08; ICE'10). In retrospective, the concepts introduced there could be described as abstractions of the semantics of the real-time tasks that are executed, and operations like schedulability analysis or computing the maximum size for communication buffers compute a sound over-approximation of the real execution. This is an interesting perspective that would be worth exploring in the future.

Using formal methods This interest on proving the correctness of different algorithms got me interested in formal methods. Several times the solutions of concurrency problems that I had were found in the work of Leslie Lamport, and I learned about TLA+ (among other many interesting topics) by looking at his bibliography. His book on TLA+ (Lamport, 2002) was really easy to read, even for a non-specialist, and the tool suited well my needs for the verification of concurrent algorithms. I was quickly able to formalize some of the systems that I was working on, and to use the TLC explicit-state model-checker as a way to test its correctness. When writing concurrent code, you cannot test it without tooling due to the possibility of rare race conditions, so I could immediately see the advantages of the use of formal methods. I got funding to advise a PhD student, Amira Methni (Methni, 2016) (co-advised with colleague Belgacem Ben Hedia, and Serge Haddad, Kamel Barcaoui ¹) to keep working on this subject : she worked hard to develop the C2TLA+ tool, that performs an optimized translation from C to TLA+ specification that can be model-checked to check that they comply with an abstract TLA+ specification (FTSCS'14; ISCEA'15; VECoS'15; Methni, 2016; JRWRTC'14). Related to this manuscript, we used in par-

¹That I thank for their guidance, as my understanding of model checking and formal methods at the time was very naive.

ticular abstract interpretation to optimize the translation from C to TLA+, so that model-checking the specification required exploration of fewer states.

I had learned that there were nearby teams that were developing a theorem prover for TLA+ in the INRIA-Microsoft Joint Center, so I reached out to them, and I thank Stephan Merz and Damien Doligez for starting a collaboration where I was using the TLA+ Proof System that they developed (I think I was one of the first users external to the team). I really enjoyed the assertive hierarchical proof style (Lamport, 1995) of the system, which makes the formal proof quite close to a pen-and-paper one. Getting funding to work together took time, and we were funded only after I had left the lab; nevertheless, I continued working with Stephan, Damien and colleagues (ABZ'16) on a formalization of the pen-and-paper proof I had previously written in RTSS'12, which was an enriching and formative experience.

8.2 Static analysis of low-level code

As I wanted to better understand how formal methods worked, I got a growing interest in programming language semantics, and compilers in particular; I remember in particular learning a lot from Kennedy (2007); Appel (1998a). This influence can be seen in papers like RTSS'12, which I would now describe as a study of the properties of the concrete semantic of the real-time language ΨC used in OASIS and PHAROS (the results of this paper would later be proved in TLA+ in ABZ'16). I also designed a template language to help the generation of C code from tree-structured data like JSON; later, I would use abstract interpretation to perform the reverse operations, i.e. recovering the tree-structured data from a template and output (SAS'23).

In 2013, I had the opportunity to develop formal methods at the LSL laboratory within CEA. I am grateful to Benjamin Monate for trusting my potential despite my limited background in formal methods, and I thank Florent Kirchner for having welcomed me.

Value and mission An important skill that I had was the ability to have a good understanding and quickly navigate inside the C programs that were analysed by FRAMA-C. This is why I was tasked to help use and develop VALUE, the abstract interpretation plugin of FRAMA-C, originally developed by Pascal Cuoq, but maintained at the time by Boris Yakobowski. I knew very little about abstract interpretation, and Boris kindly taught me the basics of the tool, and in particular the structuring of the C analysis in layers that are similar to the ones in Section 7.1 (but for a non-relational abstract interpreter). At the time there were no good introductory books about the topic like Miné (2017) or Rival and Yi (2020); I remember learning abstract interpretation using David Pichardie's PhD (Pichardie, 2005), then classic papers like Cousot (1977); Cousot and Cousot (1979), as well as different papers related to Astrée Blanchet et al. (2002, 2003); Cousot et al. (2009); Mauborgne and Rival (2005); Cousot et al. (2006); Miné (2006a). I quickly started developing extensions, in particular for the Trust-In-Soft startup that was selling commercial solutions around a fork of the tool. But I also did a lot of case studies (LPAR'15), using the tool to check absence of undefined behaviour on various industrial embedded C code.

I have learned a lot from this experience. As a previous C developer, I immediately grasped the potential benefits of abstract interpretation to verify C code : the possibility to verify absence of undefined behaviour, and in particular of memory

safety errors, automatically with a low effort, seemed to be an ideal tool to be protected against the worst issues that can happen when writing C. VALUE had a good design that worked quite well for the verification of absence of undefined behaviour in embedded C code: its non-relational nature gave it a nice and simple structure (that influenced the design of Codex : the main concern of Chapter 6 is to obtain an analyser structure similar to that of VALUE, but that also allows relational reasoning), it could handle low-level C code with casts and unions quite well, and it compensated the lack of relations by aggressive use of case-based reasoning (behaving a bit like an explicit-state model checker), using memoization to maintain a good performance despite all the redundant computations.

There were however, important limitations. The most important was due to the inability for the tool to represent relational information between variables². This meant that simple transformations, like using temporary variables to store the result of an expression or using a copy of a variable, would make the analysis imprecise. In addition, learning from conditionals and assertions relied on a fragile pattern matching over program expressions. The second most important limitation was bad support for dynamic memory location or arrays of unbounded size; basically, any situation where you could not enumerate the memory cells in the program. A third limitation was the monolithic nature of the tool: all the parts were intertwined, which made it difficult to extend. Finally, VALUE could only perform whole-program analyses, i.e., you need to analyse all of the code starting from main, because the tool can not reason about unknown code (possibly manipulating an unbounded amount of memory cells). This meant that analysis times can be quite long, which was unfortunate because significative efforts are needed to find a suitable parameterization of the tool, mainly related to configuring disjunctions for case analysis. For these reasons, it was difficult to use the tool outside of embedded systems (that generally use static memory allocation on a finite amount of memory, and where most loops are bounded, which allows case-based reasoning), or as a tool to help developers when writing code, the way I would have loved to use it as a C programmer. From then on, *my main goal would be to advance the state of the art in abstract interpretation so that it can be used to assist systems programmers in their day-to-day work*, helping them to increase the confidence in the low-level code they write using an automated tool. This took a lot of time, but I think this goal is closed to be reached.

Symbolic value-based abstract interpretation inside CODEX I had identified two main directions where to improve the analyser. The first was to update the design of VALUE to improve numerical precision, but also to make the tool more easily extensible. The second was to change the memory abstraction to better handle dynamic memory allocation, and support modular analysis. As the first direction was a prerequisite to the second, I started working on this. This resulted in the first version of CODEX, which was at the time an analyser similar to the one in Section 7.1.

I am not sure how I came across the symbolic and value-based design of abstract interpreters. I had understood that the design of VALUE was quite different from the one in other tools supporting relational abstractions like ASTRÉE, and finding a way to combine the advantages of both approaches was definitively a motivation. The idea of building symbolic expressions on which to attach non-relational abstract values, as in Chapter 5 and PLDI'24, came early; one of the main goal was that we should be

²The support of relational information about the variable has a strong impact on the design of a static analysis tool, as explained in Section 1.3.

able to propagate constraints even when the information copied or put in temporary variables (we prove that the CODEX analysis does not lose precision on this transformation in (PLDI'24, , Lemma 8.2)). From there, the idea of attaching the abstract values to a symbolic representation of values instead of variables came naturally. I also was familiar with compilation techniques (in particular, SSA and CPS forms), as well as related formal techniques such as symbolic execution and weakest-precondition calculus, which are very related to SSA translation (Leino, 2005; Flanagan and Saxe, 2001), and thus to symbolic value-based analysis (POPL'23).

Another key idea was doing the analysis on a simpler language than the C intermediate language CIL (Necula et al., 2002). I see two problems with CIL: because it performs some simplifications on the C AST, analysis reports cannot be done on the original source code (location information is preserved, but the source code has changed). On the other hand, doing the analysis directly on a C-like AST (even with simplifications) required handling many programming constructs, leading to redundant code which is harder to maintain. This was even more important as VALUE needed to support the ACSL specification language in addition to C. From there followed the idea of having both a high-level source code for report of the analysis results, and a low-level one with a smaller set of core constructs to simplify the analysis implementation. This idea exists elsewhere (Dillig et al., 2009); the originality is that we simultaneously handle both languages (PLDI'24).

The initial plan was to incorporate those changes to a new version of VALUE (hence, the initial name for this project was VALUE v2. However, the ideas behind CODEX were met with high skepticism. A lot of the ideas were new, some of the benefits could not yet be demonstrated on practical use cases, and the design was not yet proven on large-scale programs nor fully formalized; in particular, despite a lot of work on the subject there were many aspects that I did not understand and could not explain well (there were working versions of Codex in 2014/2015, and papers based on it in 2021 (RTAS'21), but it was not until 2023 that I could completely understand and describe some parts of the core design (POPL'23)). For these reasons, another major update of VALUE, the Evolved Value Analyser EVA, was developed by David Bühler and Boris Yakobowski. This addressed the lack of relation issue by allowing Value to be extended with more familiar and reassuring variable-based abstract domains Blazy et al. (2017). As a result CODEX was no longer considered as the future of VALUE.

Modular analysis of C code with dynamic memory allocation I had then a working design for precise and relational numerical analysis in abstract interpreters, which could handle low-level access to local and global variables. However, there were still two important topics that I wanted to investigate: modular/compositional analysis, and the verification of generic C code with dynamic memory allocation. I had come across the work of Xavier Rival on several important topics related to my research (sound static analysis of C code with Astrée (Blanchet et al., 2002, 2003), his work on trace partitioning (Mauborgne and Rival, 2005; Rival and Mauborgne, 2007) that was the theoretical basis for disjunctive analysis in VALUE), and I was very interested to better understand his work on shape analysis based on separation logic (Chang and Rival, 2008; Laviro et al., 2010; Chang and Rival, 2013)). One aspect that I had noticed is that the shape analyses that he had developed were also value-based, which was uncommon. I thought there was a good opportunity for collaboration, and I had a PhD grant, so I contacted him. I would like to thank him for accepting to work

with me, as I learned a lot working with him – not only about shape analysis, but also about the formalization of static analysis by abstract interpretation. In particular, I learned how to properly formalize such an analysis, with a method in which you first provide the concrete semantics, then the abstractions that your analysis manipulates, then the relation between the two (the concretization), then the soundness theorems for all the transfer functions, that correspond to their specification; after that you can provide an implementation and a proof of all these transfer functions.

We co-supervised Hugo Illous during his PhD [Illous \(2019\)](#), who worked tirelessly on extending the shape analysis based on separation logic so that it can reason about program transformations (i.e., input-output relation, or denotational semantics), in addition to states. This was challenging because when you reason about an unbounded number of memory cells, you cannot just use a numerical domain expressing how the cells in the input and output memory are related. He ended finding a fairly simple extension of separation logic capable of describing these program transformations ([NFM’17](#); [FMSD’21](#)), that has a good structure for automated reasoning on abstract transformations. One of the main applications of abstract transformations is that it can be used to summarize the behaviour of a function, and we can apply this summary to perform a modular analysis (following the functional approach of [Sharir and Pnueli \(1981\)](#)). Following this idea, Hugo implemented a modular shape analysis, and applied it in particular to a non-trivial subset of the real Emacs source code ([SAS’20](#)). Interestingly, the analysis was not purely bottom-up; rather, it computes the context in which functions are called, and uses this to compute a more precise summary of the function based on this context.

An interesting observation was that while analysis based on separation can compute very strong invariant over the memory, it also requires knowledge of the aliasing structure to parameterize the analysis; the analysis fails when this aliasing structure is not properly specified and tracked. Because Emacs uses a conservative garbage collector, there is a lot of arbitrary sharing between objects, which complicates analyses based on separation logic ([Li et al., 2015](#)). Actually because of the garbage collection, the shape of the memory for many parts of Emacs is not an important property (only spatial memory safety is), and simpler methods would suffice; in particular, methods that do not need to track aliasing (in terms of type theory, tracking aliasing requires the use of substructural type systems, while structural ones suffice for spatial memory safety). Retrospectively, a motivation for working on type-based analysis was the goal of having a method that is simpler to setup and automatize when weaker invariants are sufficient.

Binary and type-based analysis In 2016 Vincent David had left CEA and was a scientific director at Krono-safe, a startup company based on extensions of the OASIS/PHAROS technology. He was interested in the use of formal methods for safety and security certifications of the products developed by the company. One issue was the certification of the code generation toolchain, for which I proposed in particular using abstract interpretation to recover the data used to produce a text output by a template processing language ([SAS’23](#)). The other part was the use of machine code analysis to certify the kernel of ASTERIOS, their successor to OASIS and PHAROS. This was an excellent occasion to apply the methods that I was developing precisely for this kind of code! I then started a collaboration with Sébastien Bardin, who was leading the research around the BINSEC platform for binary code analysis, which provided in particular a parser and instruction decoder for binary executable that I

needed. I ported a first version of CODEX to BINSEC so that it could handle machine code in a few months, which I think demonstrated that CODEX makes it easy to create new analyses. I think there are two reasons why it works well: the first is the tagless-final style of writing compilation passes presented in this manuscript and PLDI'24; in particular for the SSA_Translation (POPL'23) to the "SSA intermediate language" (really just the Integer_and_Boolean_Value_Based_Domain signature). The second is the use of a library of abstract abstract datatypes (Chapter 6), that can be used to compose an abstraction representing the memory of the program; a large part could be reused between the C and binary code.

I contacted Xavier Rival again to work on the subject; he was also interested in the application of static analysis to the verification of OS components (Liu and Rival, 2015a; Giet, 2024), and accepted to co-advise Olivier Nicole (Nicole, 2022) with me. One of the main challenges was to verify (and certify) the ASTERIOS kernel executable independently of the application that ran on top of it. This required to use a memory abstraction able to reason about an unbounded number of memory cells. This abstraction was mainly used to verify the kernel runtime, and was therefore more concerned with the preservation of invariants than the establishment of new memory invariants. Furthermore, the memory in the kernel was statically allocated, so we were not interested in checking temporal memory safety. Finally, we had no access to the kernel source code, but we had access to the type definitions used to interface the kernel with the application. The first version of the type-based domain came as a way to combine all these ingredients. This, and a few other good ideas (expressing absence of privilege escalation without the need to write a specification, modelling the behaviour of user code to obtain a loop used to perform a fixpoint and compute invariants, using different techniques for analysing the initialization code than the runtime, etc.) and a tremendous engineering effort from Olivier working on all the difficulties of a real code, allowed to verify the absence of privilege escalation and of runtime errors in ASTERIOS (after having found a bug), which was awarded the best paper at RTAS'21. This was an excellent result, and since I have been working on the verification of other industrial kernels (not named for the sake of confidentiality) from their executable.

There were two reasons why the binary analysis worked very well. The first is the symbolic value-based analysis coupled with the numerical based on constraint propagation of Chapter 5: as proved in PLDI'24, the approach can recover from precision loss induced by compilation (in particular, the breaking of large expressions into sequence of small ones). The second is linked to the type domain. The danger in machine-code analysis is that if you lose too much precision, you end up writing to a pointer that can point anywhere in memory; after that, the analysis cannot continue. This happens in particular when writing in pointers coming from the heap. The type-based analysis provides enough structure to ensure that this problem no longer happens. We continued studying properties of this domain, and could indeed observe that machine code analysis was almost as precise as the source-code one (VMCAI'22).

I worked with Sébastien Bardin with a number of other subjects that are not directly related to this manuscript (but related to the topics of machine-code or memory analysis). After Richard Bonichon left, I replaced him as the BINSEC maintainer and as a co-advisor (with Sébastien and Roland Groz) to Manh-Dung Nguyen's PhD (Nguyen, 2021) on fuzzing for use-after-free vulnerabilities (RAID'20). I used abstract interpretation to detect infeasible test cases (IFM'20), or to improve symbolic execution (LPAR'18), or to help formalize a static analysis for compliance of inline-

assembly specifications (ICSE'21). Finally, we advised together (with J-Y. Marion) Charles Babu for his PhD that was supposed to combine techniques for malware analysis, but Charles persuaded us to focus this work on automated methods to improve trace partitioning, which ended up working quite well (SAS'24).

Goal met: Type-based modular analysis using symbolic and value-based abstract interpretation In 2022 all the ingredients were there: I knew how to develop a modular static analysis (SAS'20), we had a type-based abstraction that only used lightweight type annotations but was quite precise (VMCAI'22), and an abstract interpreter that could handle both C and machine code, and was demonstrated on a challenging industrial case study (RTAS'21). However, to make the technology more mature, I had to better understand how all of this worked (what was true for system programming remains true in static analysis: efficient implementation requires strong formalization).

One issue that I had when trying to publish on the design of Codex was that many aspects were entangled. The idea of symbolic and tagless-final abstract interpretation means analysis can produce code, that can be analysed. "Joining" memory abstractions requires renaming on the numeric domain, an uncommon operation that only exists in value-based analysers, which are uncommon and there was no name for this concept. An issue that I had was handling loops, and the analysis was based on a special kind of path expressions (Tarjan, 1981b; Cyphert et al., 2019), using a closure operator instead of the traditional widening. As a result, the symbolic language that we could generate by abstract interpretation was term-graph with μ -recursion (Ariola and Klop, 1996) instead of SSA, and the ability to generate formulas in this language was not a great motivation. This changed when I tried to change CODEX so that it could use a standard static analysis interface with widening, and realized that the resulting formula language was quite close to SSA (POPL'23). I learned while writing this paper that this corresponds in particular to sea-of-nodes SSA (Click and Paleczny, 1995; Demange et al., 2018), a lower-level description of the SSA graph of Chapter 4, and very recently that Click also used map merging to insert ϕ nodes (Click, 1995, Ch7: Parse-time optimizations), but in a pessimistic analysis. Following this, I got a PhD grant where I could hire Dorian Lesbre to work on SSA-based analysis, and he formalized other aspects of CODEX (the use of functor domains as compilation passes, and the numerical analysis presented in Chapter 5) and generalized the idea of symbolic abstract domains and semantic methods for compilation (PLDI'24). We also started working on SSA-based numerical abstractions (NSAD'24; PLDI'25).

I also wanted to better understand the type-based memory analysis. I asked Mihaela Sighireanu, that I knew from his work on interprocedural combined shape/numerical analysis (Bouajjani et al., 2012a,b; Dragoi et al., 2013; Bouajjani et al., 2011), to advise Julien Simonnet with me and work together on the subject. I am grateful that she accepted, and our collaboration proved to be both fruitful and truly enjoyable. One practical aspect that we wanted to explore was extending the type-based analysis so that it could describe arbitrary static memory layouts, and in particular support unions (especially when tags are used to discriminate between several cases) and relations between different memory blocs. Finding a suitable formalization of the combination of unions, record and named pointers was quite challenging, and we discovered unsuspected properties like the idea of mild update, explaining how we can mutably change the kind of variant in a union type without changing the

type (OOPSLA’24). We also did a lot of work on improving the idea and theory of abstract abstract datatypes (Chapter 6), notably for precise handling of disjunctions, with promising results (not yet submitted for publication); Julien also worked hard to implement a library of block abstractions based on AADTs, which was required to precisely analyse programs whose invariants are described with these more complex types.

8.3 Future directions

Symbolic value-based abstract interpretation is thus a suitable technique to implement typed-based modular analyses of system code. There are still a lot of improvements that needs to be done, and directions that I would like to explore.

Robust modular type-based static analysis of low-level code We have certainly used CODEX in interesting use cases, but it is not yet a standard tool used by industrial or individual users to prove that their program is memory-safe. One goal of my employer, CEA, is technological transfer for use in the industry, and I would very much like this to happen. I think we are quite close to that goal for memory safety. We now have a complete nice theory for TYPEDC (VMCAI’22; OOPSLA’24); we need to improve the implementation, and write a robust library of datatype abstractions. The AADT framework of is Chapter 6 is a good way of building this, but this need to be demonstrated in practice.

Temporal memory safety and concurrency We also want to statically verify temporal safety in addition to memory safety. We can build on our working type-based analysis for spatial memory-safety to add features for temporal safety. Possibilities includes combination with analyses based on separation logic (Chang and Rival, 2008; SAS’20), the use of substructural type system features like ownership transfer, or other methods. In 2022, I obtained a grant for the EMAS project (with Xavier Rival, Julia Lawall and Nikolai Kosmatov) where we want to explore this direction.

Analysis of more low-level languages Automatic verification of programs containing TYPEDC specifications can be useful beyond C and machine code; it can be useful for any language aimed at being compiled. It is useful for ensuring memory-safety in the languages that assume absence of memory corruption without checks (like C, C++, or unsafe Rust); it can also be used to make programs that insert dynamic checks more efficient (e.g. Webassembly, Java, etc.); it could also be used to generate memory-safety proof in proof-carrying code (Necula, 1997; Ahmed et al., 2010), starting from arbitrary programs compiled by arbitrary compilers (that do not have to be modified to generate proofs). Having a working solution to the problem of proof generation in proof-carrying code could make native machine code programs with proof a viable alternative to languages like EBPF or Webassembly, where more verification would be made statically rather than dynamically, resulting in more efficient execution (Geller et al., 2024).

Automated generation of specifications Annotating a program with TYPEDC specifications requires some work, which is not significative when TYPEDC is used during

development (as this corresponds to invariants known to the developer), but becomes important if we want to verify an existing program. Thus, an important goal is to decrease the effort as much as possible, in particular by generating automatically some parts of the specification [Machiry et al. \(2022\)](#). I have started advising Paul Robert as a PhD student (with Mihaela Sighireanu) who has begun working on this subject.

More efficient analyses I am very interested in algorithmic ideas to make numerical static analyses more precise and/or more efficient. One challenge in SSA-based analyses is that the number of variables in scope at a certain program point may increase, and this causes problems for relational abstract domains because the complexity of their operation is supra-linear in terms of the number of dimensions that they relate. There are techniques to mitigate this problem like packing ([Blanchet et al., 2003](#); [Venet and Brat, 2004](#)) or garbage collection ([Chang and Rival, 2013](#)); with the latter, value-based analysers have fewer dimensions to relate than an equivalent variable-based analyser, due to the fact that some memory locations contain the same symbolic value. But I would like to explore the topic of sparse analyses ([Rastello and Bouchez Tichadou, 2022](#), ch.8), i.e. having a single numerical domain covering the whole program, rather than duplicating the numerical state at each program point. I am interested in particular in sparse relational analyses (e.g. ([Miriaz and Pichardie, 2022](#); [Maalej et al., 2018](#); [Bodík et al., 2000](#))). I think that the SSA properties (the fact that variables are not modified, so have the same value between different program points) can allow more efficient static analyses; the results in [NSAD'24](#); [PLDI'25](#) are a first step in that direction. Another topic to investigate is how to reduce redundant work; for example, there are symbolic expressions that we produce several times, once for each loop iteration. Possibly ideas from the bottom-up logic inference community (like semi-naïve evaluation ([Ramakrishnan et al., 1992](#))) could help solve this problem.

Control-flow analyses and compilation We discovered the idea of compiling with abstract interpretation and semantic methods for compilation by serendipity, but I think there are a lot of applications to this idea. One is decompilation: using suitable abstraction of what a control state means, we could turn obfuscated code (e.g. using flattening ([Blazy and Trieu, 2016](#)) or virtual machines ([Salwan et al., 2018](#)), without suffering from the problems of path-oriented techniques like symbolic execution ([Ollivier et al., 2019](#))). Coupled with the advanced memory analyses that we perform, that can recover the control-flow of binary programs, we could be developing tools that could soundly transform parts of binary executables back to C ([Cifuentes and Gough, 1995](#)) or other IR ([Park et al., 2025](#)), with applications in reverse engineering. One is compilation: compiling with continuation seem to be a very good tool for implementing partial evaluation ([Futamura, 1999](#)), with its applications like generating compilers from interpreter, or JIT compilation by specializing a program in a specific context. I would also be interested in knowing how easy or useful it would to formalize compiling with abstract interpretation in interactive provers like Coq.

Chapter 9

Bibliography

9.1 Publications

- [PLDI’25] Dorian Lesbre, Matthieu Lemerre, Hichem Rami Ait-El-Hara and François Bobot. 2025. «Relational Abstractions Based on Labeled Union-Find». *Proc. ACM Program. Lang.* 9, PLDI <https://doi.org/10.1145/3729298>
- [NSAD’24] Matthieu Lemerre and Dorian Lesbre. 2024. «Labeled union-find for constraint factorization». In *10th International Workshop on Numerical and Symbolic Abstract Domains* <https://2024.splashcon.org/details/nsad-2024-papers/5/WiP-Labeled-Union-Find-for-Constraint-Factorization>
- [PLDI’24] Dorian Lesbre and Matthieu Lemerre. 2024. «Compiling with Abstract Interpretation». *Proc. ACM Program. Lang.* 8, PLDI, 368–393. <https://doi.org/10.1145/3656392>
- [OOPSLA’24] Julien Simonnet, Matthieu Lemerre, and Mihaela Sighireanu. 2024. «A Dependent Nominal Physical Type System for Static Analysis of Memory in Low Level Code». *Proc. ACM Program. Lang.* 8, OOPSLA2 (2024), 30–59. <https://doi.org/10.1145/3689712>
- [SAS’24] Charles Babu M., Matthieu Lemerre, Sebastien Bardin, and Jean-Yves Marion. 2024. «Trace Partitioning as an Optimization Problem». *Static Analysis Symposium (SAS)* (2024).
- [Frama-C Book (2024)] Matthieu Lemerre, Xavier Rival, Olivier Nicole, and Hugo Illous. 2024. «Advanced Memory and Shape Analyses». In *Guide to Software Verification with Frama-C: Core Components, Usages, and Applications*. Springer, 487–520.
- [SAS’23] Matthieu Lemerre. 2023. «Reverse Template Processing Using Abstract Interpretation». In *30th International Static Analysis Symposium (Lecture Notes in Computer Science, Vol. 14284)*. Springer, 403–433. https://doi.org/10.1007/978-3-031-44245-2_18
- [POPL’23] Matthieu Lemerre. 2023. SSA Translation is an Abstract Interpretation. *Proc. ACM Program. Lang.* 7, POPL, 1895–1924. <https://doi.org/10.1145/3571258> Distinguished paper award.

- [VMCAI'22] Olivier Nicole, Matthieu Lemerre, and Xavier Rival. 2022. «Lightweight Shape Analysis Based on Physical Types». In *23rd International Conference on Verification, Model Checking, and Abstract Interpretation – VMCAI 2022 (Lecture Notes in Computer Science, Vol. 13182)*. Springer, 219–241. https://doi.org/10.1007/978-3-030-94583-1_11
- [RTAS'21] Olivier Nicole, Matthieu Lemerre, Sébastien Bardin, and Xavier Rival. 2021. «No Crash, No Exploit: Automated Verification of Embedded Kernels». In *27th IEEE Real-Time and Embedded Technology and Applications Symposium*. 27–39. <https://doi.org/10.1109/RTAS52030.2021.00011> **Best paper award.**
- [ICSE'21] Frédéric Recoules, Sébastien Bardin, Richard Bonichon, Matthieu Lemerre, Laurent Mounier, and Marie-Laure Potet. 2021. «Interface Compliance of Inline Assembly: Automatically Check, Patch and Refine». In *43rd IEEE/ACM International Conference on Software Engineering, ICSE 2021, Madrid, Spain, 22-30 May 2021*. IEEE, 1236–1247. <https://doi.org/10.1109/ICSE43902.2021.00113> **Distinguished paper award.**
- [FMSD'21] Hugo Illous, Matthieu Lemerre, and Xavier Rival. 2021. «A relational shape abstract domain». *Formal Methods Syst. Des.* 57, 3 (2021), 343–400. <https://doi.org/10.1007/s10703-021-00366-4>
- [SAS'20] Hugo Illous, Matthieu Lemerre, and Xavier Rival. 2020. «Interprocedural Shape Analysis Using Separation Logic-Based Transformer Summaries». In *Static Analysis - 27th International Symposium, SAS 2020, Virtual Event, November 18-20, 2020, Proceedings (Lecture Notes in Computer Science, Vol. 12389)*, David Pichardie and Mihaela Sighireanu (Eds.). Springer, 248–273. https://doi.org/10.1007/978-3-030-65474-0_12
- [RAID'20] Manh-Dung Nguyen, Sébastien Bardin, Richard Bonichon, Roland Groz, and Matthieu Lemerre. 2020. «Binary-level Directed Fuzzing for Use-After-Free Vulnerabilities». In *23rd International Symposium on Research in Attacks, Intrusions and Defenses, RAID 2020, San Sebastian, Spain, October 14-15, 2020*, Manuel Egele and Leyla Bilge (Eds.). USENIX Association, 47–62.
- [IFM'20] Thibault Martin, Nikolai Kosmatov, Virgile Prevosto, and Matthieu Lemerre. 2020. «Detection of Polluting Test Objectives for Dataflow Criteria». In *Integrated Formal Methods - 16th International Conference, IFM 2020, Lugano, Switzerland, November 16-20, 2020, Proceedings (Lecture Notes in Computer Science, Vol. 12546)*, Brijesh Dongol and Elena Troubitsyna (Eds.). Springer, 337–345. https://doi.org/10.1007/978-3-030-63461-2_18
- [LPAR'18] Benjamin Farinier, Robin David, Sébastien Bardin, and Matthieu Lemerre. 2018. «Arrays Made Simpler: An Efficient, Scalable and Thorough Pre-processing». In *22nd International Conference on Logic for Programming, Artificial Intelligence and Reasoning (EPIC Series in Computing, Vol. 57)*, Gilles Barthe, Geoff Sutcliffe, and Margus Veanes (Eds.). EasyChair, 363–380. <https://doi.org/10.29007/dc9b>
- [NFM'17] Hugo Illous, Matthieu Lemerre, and Xavier Rival. 2017. «A Relational Shape Abstract Domain». In *NASA Formal Methods - 9th International Symposium, NFM 2017, Moffett Field, CA, USA, May 16-18, 2017, Proceedings (Lecture Notes in*

- Computer Science*, Vol. 10227), Clark W. Barrett, Misty D. Davies, and Temesghen Kahsai (Eds.). 212–229. https://doi.org/10.1007/978-3-319-57288-8_15
- [ABZ’16] Selma Azaiez, Damien Doligez, Matthieu Lemerre, Tomer Libal, and Stephan Merz. 2016. «Proving Determinacy of the PharOS Real-Time Operating System». In *Abstract State Machines, Alloy, B, TLA, VDM, and Z - 5th International Conference, ABZ 2016, Linz, Austria, May 23-27, 2016, Proceedings (Lecture Notes in Computer Science, Vol. 9675)*, Michael J. Butler, Klaus-Dieter Schewe, Atif Mashkoor, and Miklós Biró (Eds.). Springer, 70–85. https://doi.org/10.1007/978-3-319-33600-8_4
- [SCAM’16] Allan Blanchard, Nikolai Kosmatov, Matthieu Lemerre, and Frédéric Loulergue. 2016. «Conc2Seq: A Frama-C Plugin for Verification of Parallel Compositions of C Programs». In *16th IEEE International Working Conference on Source Code Analysis and Manipulation, SCAM 2016, Raleigh, NC, USA, October 2-3, 2016*. IEEE Computer Society, 67–72. <https://doi.org/10.1109/SCAM.2016.18>
- [FMICS’15] Allan Blanchard, Nikolai Kosmatov, Matthieu Lemerre, and Frédéric Loulergue. 2015. «A Case Study on Formal Verification of the Anaxagoras Hypervisor Paging System with Frama-C». In *20th International Workshop on Formal Methods for Industrial Critical Systems (Lecture Notes in Computer Science, Vol. 9128)*. https://doi.org/10.1007/978-3-319-19458-5_2
- [LPAR’15] Daniel Fava, Julien Signoles, Matthieu Lemerre, Martin Schäfer, and Ashish Tiwari. 2015. «Gamifying Program Analysis». In *Logic for Programming, Artificial Intelligence, and Reasoning - 20th International Conference, LPAR-20 2015, Suva, Fiji, November 24-28, 2015, Proceedings (Lecture Notes in Computer Science, Vol. 9450)*, Martin Davis, Ansgar Fehnker, Annabelle McIver, and Andrei Voronkov (Eds.). Springer, 591–605. https://doi.org/10.1007/978-3-662-48899-7_41
- [VECoS’15] Amira Methni, B Ben Hedia, Matthieu Lemerre, Serge Haddad, and Kamel Barkaoui. 2015a. «State space reduction strategies for model checking concurrent C programs». In *9th Workshop on Verification and Evaluation of Computer and Communication Systems, VECoS 2015*, Vol. 1431. CEUR-WS, 65–75.
- [ISCEA’15] Amira Methni, Matthieu Lemerre, Belgacem Ben Hedia, Serge Haddad, and Kamel Barkaoui. 2015b. «Verifying and Constructing Abstract TLA Specifications: Application to the Verification of C programs». In *Tenth International Conference on Software Engineering Advances (ISCEA 2015)*.
- [TAP’14] Nikolai Kosmatov, Matthieu Lemerre, and Céline Alec. 2014. «A Case Study on Verification of a Cloud Hypervisor by Proof and Structural Testing». In *Tests and Proofs - 8th International Conference, TAP@STAF 2014, York, UK, July 24-25, 2014. Proceedings (Lecture Notes in Computer Science, Vol. 8570)*, Martina Seidl and Nikolai Tillmann (Eds.). Springer, 158–164. https://doi.org/10.1007/978-3-319-09099-3_12
- [JRWRTC’14] Amira Methni, Matthieu Lemerre, Belgacem Ben Hedia, Kamel Barkaoui, and Serge Haddad. 2014a. «An approach for verifying concurrent C programs». In *8th Junior Researcher Workshop on Real-Time Computing*. 33–36.
- [FTSCS’14] Amira Methni, Matthieu Lemerre, Belgacem Ben Hedia, Serge Haddad, and Kamel Barkaoui. 2014b. «Specifying and Verifying Concurrent C Programs

- with TLA+». In *Formal Techniques for Safety-Critical Systems - Third International Workshop, FTSCS 2014, Luxembourg, November 6-7, 2014. Revised Selected Papers (Communications in Computer and Information Science, Vol. 476)*, Cyrille Artho and Peter Csaba Ölveczky (Eds.). Springer, 206–222. https://doi.org/10.1007/978-3-319-17581-2_14
- [RTSS'12] Matthieu Lemerre and Emmanuel Ohayon. 2012. «A Model of Parallel Deterministic Real-Time Computation». In *Proceedings of RTSS 2012*.
- [Ewili'12] Vincent Legout and Matthieu Lemerre. 2012. «Paravirtualizing Linux in a real-time hypervisor». *SIGBED Rev.* 9, 2 (2012), 33–37. <https://doi.org/10.1145/2318836.2318842>
- [HPCS'12] Frédéric Loulergue, Frédéric Gava, Nikolai Kosmatov, and Matthieu Lemerre. 2012. «Towards verified cloud computing environments». In *2012 International Conference on High Performance Computing & Simulation (HPCS)*. 91–97. <https://doi.org/10.1109/HPCSim.2012.6266896>
- [HASE'11] Stephane Louise, Matthieu Lemerre, Christophe Aussagues, and Vincent David. 2011. «The OASIS Kernel: A Framework for High Dependability Real-Time Systems». In *2011 IEEE 13th International Symposium on High-Assurance Systems Engineering*. 95–103. <https://doi.org/10.1109/HASE.2011.38>
- [ISORC'11] Matthieu Lemerre, Emmanuel Ohayon, Damien Chabrol, Mathieu Jan, and Marie-Benedicte Jacques. 2011. «Method and Tools for Mixed-Criticality Real-Time Applications within PharOS». In *2011 14th IEEE International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing Workshops*. 41–48. <https://doi.org/10.1109/ISORCW.2011.15>
- [ICE'10] Matthieu Lemerre, Vincent David, Christophe Assaguès, and Guy Vidal Naquet. 2010. «An Introduction to Time-Constrained Automata». In *ICE 2010 3rd Interaction and Concurrency Experience*. 83–98.
- [Lococo'10] Emmanuel Ohayon, Matthieu Lemerre, and Vincent David. 2010. «CONFIGEN: A tool for managing configuration options». In *Proceedings First International Workshop on Logics for Component Configuration, LoCoCo 2010, Edinburgh, UK, 10th July 2010 (EPTCS, Vol. 29)*, Inês Lynce and Ralf Treinen (Eds.). 32–43. <https://doi.org/10.4204/EPTCS.29.4>
- [IIDS'10] Matthieu Lemerre, Vincent David, and Guy Vidal-Naquet. 2010. «A dependable kernel design for resource isolation and protection». In *The First Workshop on Isolation and Integration in Dependable Systems (IIDS'2010)*. ACM.
- [IIES'09] Matthieu Lemerre, Vincent David, and Guy Vidal-Naquet. 2009. «A Communication Mechanism for Resource Isolation». In *Second Workshop on Isolation and Integration in Embedded Systems (IIES '09)*. ACM, 16. <https://doi.org/10.1145/1519130.1519131>
- [RTAS'08] Matthieu Lemerre, Vincent David, Christophe Aussaguès, and Guy Vidal-Naquet. 2008. «Equivalence between Schedule Representations: Theory and Applications». In *Proceedings of the 14th IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS 2008, April 22-24, 2008, St. Louis, Missouri, USA*. IEEE Computer Society, 237–247. <https://doi.org/10.1109/RTAS.2008.17>

[RTSS'06] Matthieu Lemerre, Vincent David, Christophe Aussagues, and Guy Vidal-Naquet. 2006. «A new representation for the scheduling problem and its applications». *Proceedings WiP IEEE RTSS 6* (2006), 89–92.

9.2 Other references

Carmine Abate, Roberto Blanco, Ștefan Ciobăcă, Adrien Durier, Deepak Garg, Catalin Hritcu, Marco Patrignani, Éric Tanter, and Jérémy Thibault. 2021. «An Extended Account of Trace-relating Compiler Correctness and Secure Compilation». *ACM Trans. Program. Lang. Syst.* 43, 4 (2021), 14:1–14:48. doi:10.1145/3460860

Amal Ahmed, Andrew W. Appel, Christopher D. Richards, Kedar N. Swadi, Gang Tan, and Daniel C. Wang. 2010. «Semantic foundations for typed assembly languages». *ACM Trans. Program. Lang. Syst.* 32, 3 (2010), 7:1–7:67. doi:10.1145/1709093.1709094

Frances E. Allen. 1970. «Control flow analysis». In *Proceedings of a Symposium on Compiler Optimization, Urbana-Champaign, Illinois, USA, July 27-28, 1970*, Robert S. Northcote (Ed.). ACM, 1–19. doi:10.1145/800028.808479

Bowen Alpern and Fred B. Schneider. 1987. «Recognizing Safety and Liveness». *Distributed Comput.* 2, 3 (1987), 117–126. doi:10.1007/BF01782772

Bowen Alpern, Mark N. Wegman, and F. Kenneth Zadeck. 1988. «Detecting Equality of Variables in Programs». In *15th ACM Symposium on Principles of Programming Languages (POPL 1988)*. 1–11. doi:10.1145/73560.73561

Stephen Alstrup and Mikkel Thorup. 2000. «Optimal Pointer Algorithms for Finding Nearest Common Ancestors in Dynamic Trees». *J. Algorithms* 35, 2 (2000), 169–188. doi:10.1006/JAGM.2000.1079

C Scott Ananian. 2001. *The static single information form*. Master's thesis. Massachusetts Institute of Technology.

Andrew W Appel. 1998a. *Modern compiler implementation in ML*. Cambridge University Press. doi:10.1017/CB09780511811449

Andrew W. Appel. 1998b. «SSA is Functional Programming». *SIGPLAN Notices* 33, 4 (4 1998), 17–20. doi:10.1145/278283.278285

Zena M Ariola and Jan Willem Klop. 1996. «Equational term graph rewriting». *Fundamenta Informaticae* 26, 3, 4 (1996), 207–240. doi:10.3233/FI-1996-263401

Christophe Aussagues and Vincent David. 1998. «A method and a technique to model and ensure timeliness in safety critical real-time systems». In *Proceedings. Fourth IEEE International Conference on Engineering of Complex Computer Systems (Cat. No. 98EX193)*. IEEE, 2–12.

Gogul Balakrishnan and Thomas Reps. 2004. «Analyzing memory accesses in x86 executables». In *International conference on compiler construction*. Springer, 5–23.

- Gogul Balakrishnan and Thomas Reps. 2006. «Recency-abstraction for heap-allocated storage». In *Static Analysis*. Springer, 221–239.
- Robert A. Ballance, Arthur B. Maccabe, and Karl J. Ottenstein. 1990. «The Program Dependence Web: A Representation Supporting Control, Data, and Demand-Driven Interpretation of Imperative Languages». In *Proceedings of the ACM SIG-PLAN'90 Conference on Programming Language Design and Implementation (PLDI), White Plains, New York, USA, June 20-22, 1990*, Bernard N. Fischer (Ed.). ACM, 257–271. doi:10.1145/93542.93578
- Sébastien Bardin, Robin David, and Jean-Yves Marion. 2017. «Backward-Bounded DSE: Targeting Infeasibility Questions on Obfuscated Codes». In *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*. IEEE Computer Society, 633–651. doi:10.1109/SP.2017.36
- Sébastien Bardin, Philippe Herrmann, and Franck Védrine. 2011. «Refinement-Based CFG Reconstruction from Unstructured Programs». In *12th International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI 2011, Vol. 6538)*. Springer, 54–69. doi:10.1007/978-3-642-18275-4_6
- Gilles Barthe, Delphine Demange, and David Pichardie. 2014. «Formal Verification of an SSA-Based Middle-End for CompCert». *ACM Trans. Program. Lang. Syst.* 36, 1 (2014), 4:1–4:35. doi:10.1145/2579080
- Santiago Bautista. 2023. *Static Analysis of Algebraic Data Types and Arrays*. Ph.D. Dissertation. ENS Rennes.
- Santiago Bautista, Thomas Jensen, and Benoît Montagu. 2024. «An input–output relational domain for algebraic data types and functional arrays». *Formal Methods in System Design* (6 2024). doi:10.1007/s10703-024-00456-z
- Santiago Bautista, Thomas P. Jensen, and Benoît Montagu. 2022. «Lifting Numeric Relational Domains to Algebraic Data Types». In *Static Analysis - 29th International Symposium, SAS 2022, Auckland, New Zealand, December 5-7, 2022, Proceedings (Lecture Notes in Computer Science, Vol. 13790)*, Gagandeep Singh and Caterina Urban (Eds.). Springer, 104–134. doi:10.1007/978-3-031-22308-2_6
- Frédéric Benhamou, Frédéric Goualard, Laurent Granvilliers, and Jean-Francois Puget. 1999. «Revising Hull and Box Consistency». In *Logic Programming: The 1999 International Conference, Las Cruces, New Mexico, USA, November 29 - December 4, 1999*, Danny De Schreye (Ed.). MIT Press, 230–244.
- Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. 1999. «Symbolic Model Checking without BDDs». In *Tools and Algorithms for Construction and Analysis of Systems, 5th International Conference, TACAS '99, Held as Part of the European Joint Conferences on the Theory and Practice of Software, ETAPS'99, Amsterdam, The Netherlands, March 22-28, 1999, Proceedings (Lecture Notes in Computer Science, Vol. 1579)*, Rance Cleaveland (Ed.). Springer, 193–207. doi:10.1007/3-540-49059-0_14
- B. Blanchet, P. Cousot, R. Cousot, J. Feret, L. Mauborgne, A. Miné, D. Monniaux, and X. Rival. 2002. «Design and implementation of a special-purpose static program analyzer for safety-critical real-time embedded software». *The Essence of*

- Computation: Complexity, Analysis, Transformation. Essays Dedicated to Neil D. Jones* 2566 (10 2002), 85–108. doi:10.1007/3-540-36377-7_5
- B. Blanchet, P. Cousot, R. Cousot, J. Feret, L. Mauborgne, A. Miné, D. Monniaux, and X. Rival. 2003. «A Static Analyzer for Large Safety-Critical Software». In *Proceedings of the ACM SIGPLAN 2003 Conference on Programming Language Design and Implementation (PLDI'03)*. ACM Press, San Diego, California, USA, 196–207.
- Sandrine Blazy, David Bühler, and Boris Yakobowski. 2017. «Structuring Abstract Interpreters Through State and Value Abstractions». In *18th International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI 2017, Vol. 10145)*, Ahmed Bouajjani and David Monniaux (Eds.). Springer, 112–130. doi:10.1007/978-3-319-52234-0_7
- Sandrine Blazy and Alix Trieu. 2016. «Formal verification of control-flow graph flattening». In *Proceedings of the 5th ACM SIGPLAN Conference on Certified Programs and Proofs, Saint Petersburg, FL, USA, January 20-22, 2016*, Jeremy Avigad and Adam Chlipala (Eds.). ACM, 176–187. doi:10.1145/2854065.2854082
- Rastislav Bodík, Rajiv Gupta, and Vivek Sarkar. 2000. «ABCD: eliminating array bounds checks on demand». In *Proceedings of the 2000 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), Vancouver, British Columbia, Canada, June 18-21, 2000*, Monica S. Lam (Ed.). ACM, 321–333. doi:10.1145/349299.349342
- Benoit Boissinot, Philip Brisk, Alain Darte, and Fabrice Rastello. 2012. «SSI Properties Revisited». *ACM Trans. Embed. Comput. Syst.* 11S, 1 (6 2012), 21:1–21:23. doi:10.1145/2180887.2180898
- Marcello M. Bonsangue, Jurriaan Rot, Davide Ancona, Frank S. de Boer, and Jan J. M. M. Rutten. 2014. «A Coalgebraic Foundation for Coinductive Union Types». In *Automata, Languages, and Programming - 41st International Colloquium, ICALP 2014, Copenhagen, Denmark, July 8-11, 2014, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 8573)*, Javier Esparza, Pierre Fraigniaud, Thore Husfeldt, and Elias Koutsoupias (Eds.). Springer, 62–73. doi:10.1007/978-3-662-43951-7_6
- Lucas Bordeaux, Youssef Hamadi, and Moshe Y. Vardi. 2007. «An Analysis of Slow Convergence in Interval Propagation». In *Principles and Practice of Constraint Programming - CP 2007, 13th International Conference, CP 2007, Providence, RI, USA, September 23-27, 2007, Proceedings (Lecture Notes in Computer Science, Vol. 4741)*, Christian Bessiere (Ed.). Springer, 790–797. doi:10.1007/978-3-540-74970-7_56
- Ahmed Bouajjani, Cezara Dragoi, Constantin Enea, and Mihaela Sighireanu. 2011. «On inter-procedural analysis of programs with lists and data». In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2011, San Jose, CA, USA, June 4-8, 2011*, Mary W. Hall and David A. Padua (Eds.). ACM, 578–589. doi:10.1145/1993498.1993566
- Ahmed Bouajjani, Cezara Dragoi, Constantin Enea, and Mihaela Sighireanu. 2012a. «Abstract Domains for Automated Reasoning about List-Manipulating Programs with Infinite Data». In *Verification, Model Checking, and Abstract Interpretation - 13th International Conference, VMCAI 2012, Philadelphia, PA, USA, January 22-24, 2012*.

- Proceedings (Lecture Notes in Computer Science, Vol. 7148)*, Viktor Kuncak and Andrey Rybalchenko (Eds.). Springer, 1–22. doi:10.1007/978-3-642-27940-9_1
- Ahmed Bouajjani, Cezara Dragoi, Constantin Enea, and Mihaela Sighireanu. 2012b. «Accurate Invariant Checking for Programs Manipulating Lists and Arrays with Infinite Data». In *Automated Technology for Verification and Analysis - 10th International Symposium, ATVA 2012, Thiruvananthapuram, India, October 3-6, 2012. Proceedings (Lecture Notes in Computer Science, Vol. 7561)*, Supratik Chakraborty and Madhavan Mukund (Eds.). Springer, 167–182. doi:10.1007/978-3-642-33386-6_14
- François Bourdoncle. 1993. «Efficient chaotic iteration strategies with widenings». In *Formal Methods in Programming and their Applications*. Springer, 128–141. doi:10.1007/BFb0039704
- William J. Bowman. 2025. «The Ethical Compiler: Addressing the Is-Ought Gap in Compilation (Invited Talk)». In *Proceedings of the 2025 ACM SIGPLAN International Workshop on Partial Evaluation and Program Manipulation, PEPM 2025, Denver, CO, USA, 21 January 2025*, Guillaume Allais and Yanhong Annie Liu (Eds.). ACM, 1–9. doi:10.1145/3704253.3706135
- David Brumley, JongHyup Lee, Edward J. Schwartz, and Maverick Woo. 2013. «Native x86 Decompilation Using Semantics-Preserving Structural Analysis and Iterative Control-Flow Structuring». In *22th USENIX Security Symposium*. USENIX Association, 353–368. <https://www.usenix.org/conference/usenixsecurity13/technical-sessions/presentation/schwartz>.
- Roberto Bruni, Roberto Giacobazzi, Roberta Gori, Isabel Garcia-Contreras, and Dusko Pavlovic. 2020. «Abstract extensionality: on the properties of incomplete abstract interpretations». *Proc. ACM Program. Lang.* 4, POPL (2020), 28:1–28:28. doi:10.1145/3371096
- Roberto Bruni, Roberto Giacobazzi, Roberta Gori, and Francesco Ranzato. 2021. «A Logic for Locally Complete Abstract Interpretations». In *36th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2021, Rome, Italy, June 29 - July 2, 2021*. IEEE, 1–13. doi:10.1109/LICS52264.2021.9470608
- Cristiano Calcagno, Dino Distefano, Peter W. O’Hearn, and Hongseok Yang. 2006. «Beyond Reachability: Shape Abstraction in the Presence of Pointer Arithmetic». In *Static Analysis, 13th International Symposium, SAS 2006, Seoul, Korea, August 29-31, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 4134)*, Kwangkeun Yi (Ed.). Springer, 182–203. doi:10.1007/11823230_13
- Jacques Carette, Oleg Kiselyov, and Chung-chieh Shan. 2009. «Finally tagless, partially evaluated: Tagless staged interpreters for simpler typed languages». *Journal of Functional Programming* 19, 05 (2009), 509–543.
- David M. Cerna and Temur Kutsia. 2023. «Anti-unification and Generalization: A Survey». In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI 2023, 19th-25th August 2023, Macao, SAR, China*. ijcai.org, 6563–6573. doi:10.24963/IJCAI.2023/736
- Bor-Yuh Evan Chang, Cezara Dragoi, Roman Manevich, Noam Rinetzkky, and Xavier Rival. 2020. «Shape Analysis». *Found. Trends Program. Lang.* 6, 1–2 (2020), 1–158. doi:10.1561/25000000037

- Bor-Yuh Evan Chang and K. Rustan M. Leino. 2005. *Abstract Interpretation with Alien Expressions and Heap Structures*. Springer, 147–163. doi:10.1007/978-3-540-30579-8_11
- Bor-Yuh Evan Chang and Xavier Rival. 2008. «Relational Inductive Shape Analysis». In *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '08)*. ACM, New York, NY, USA, 247–260. doi:10.1145/1328438.1328469
- Bor-Yuh Evan Chang and Xavier Rival. 2013. «Modular Construction of Shape-Numeric Analyzers». In *Festschrift for Dave Schmidt (EPTCS, Vol. 129)*, Anindya Banerjee, Olivier Danvy, Kyung-Goo Doh, and John Hatcliff (Eds.). doi:10.48550/arXiv.1309.5138
- David R. Chase, Mark N. Wegman, and F. Kenneth Zadeck. 1990. «Analysis of pointers and structures (with retrospective)». In *20 Years of the ACM SIGPLAN Conference on Programming Language Design and Implementation 1979-1999, A Selection*, Kathryn S. McKinley (Ed.). ACM, 343–359. doi:10.1145/989393.989429
- Marc Chevalier. 2020. *Proving the security of software-intensive embedded systems by abstract interpretation. (Analyse de la sécurité de systèmes critiques embarqués à forte composante logicielle par interprétation abstraite)*. Ph. D. Dissertation. École Normale Supérieure, Paris, France. <https://tel.archives-ouvertes.fr/tel-03127921>
- Marc Chevalier and Jérôme Feret. 2020. «Sharing Ghost Variables in a Collection of Abstract Domains». In *Verification, Model Checking, and Abstract Interpretation - 21st International Conference, VMCAI 2020, New Orleans, LA, USA, January 16-21, 2020, Proceedings (Lecture Notes in Computer Science, Vol. 11990)*, Dirk Beyer and Damien Zufferey (Eds.). Springer, 158–179. doi:10.1007/978-3-030-39322-9_8
- Cristina Cifuentes and K. John Gough. 1995. «Decompilation of Binary Programs». *Softw. Pract. Exp.* 25, 7 (1995), 811–829. doi:10.1002/SPE.4380250706
- Dave Clarke, Johan Östlund, Ilya Sergey, and Tobias Wrigstad. 2013. «Ownership Types: A Survey». In *Aliasing in Object-Oriented Programming. Types, Analysis and Verification*, Dave Clarke, James Noble, and Tobias Wrigstad (Eds.). Lecture Notes in Computer Science, Vol. 7850. Springer, 15–58. doi:10.1007/978-3-642-36946-9_3
- Cliff Click. 1995. *Combining Analyses, Combining Optimizations*. Ph. D. Dissertation. Rice University.
- Cliff Click and Michael Paleczny. 1995. «A simple graph-based intermediate representation». *ACM Sigplan Notices* 30, 3 (1995), 35–49. doi:10.1145/202530.202534
- Richard Cole and Ramesh Hariharan. 2005. «Dynamic LCA Queries on Trees». *SIAM J. Comput.* 34, 4 (2005), 894–923. doi:10.1137/S0097539700370539
- Keith D Cooper, Timothy J Harvey, and Ken Kennedy. 2001. «A simple, fast dominance algorithm». *Software Practice & Experience* 4 (2001), 1–10.
- Patrick Cousot. 1977. *Asynchronous iterative methods for solving a fixed point system of monotone equations in a complete lattice*. Technical Report. Laboratoire IMAG, Université scientifique et médicale de Grenoble, Grenoble, France. <https://www.di.ens.fr/~cousot/publications.www/Cousot-IMAG-RR88-Sep-1977.pdf>

- Patrick Cousot. 1978. *Méthodes itératives de construction et d'approximation de points fixes d'opérateurs monotones sur un treillis, analyse sémantique de programmes (in French)*. Ph. D. Dissertation. Université Joseph Fourier, Grenoble, France. <https://www.di.ens.fr/~cousot/publications.www/CousotTheseEsSciences1978.pdf>
- Patrick Cousot. 1999. «The Calculational Design of a Generic Abstract Interpreter». In *Calculational System Design*, M. Broy and R. Steinbrüggen (Eds.). NATO ASI Series F. IOS Press, Amsterdam.
- Patrick Cousot. 2002. «Constructive design of a hierarchy of semantics of a transition system by abstract interpretation». *Theor. Comput. Sci.* 277, 1-2 (2002), 47–103. doi:10.1016/S0304-3975(00)00313-3
- Patrick Cousot. 2021. *Principles of abstract interpretation*. MIT Press.
- Patrick Cousot and Radhia Cousot. 1977. «Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints». In *4th ACM Symposium on Principles of Programming Languages (POPL 1977)*. 238–252. doi:10.1145/512950.512973
- Patrick Cousot and Radhia Cousot. 1979. «Systematic design of program analysis frameworks». In *6th ACM Symposium on Principles of Programming Languages (POPL 1979)*. 269–282. doi:10.1145/567752.567778
- Patrick Cousot and Radhia Cousot. 1992. «Abstract Interpretation Frameworks». *Journal of Logic and Computation* 2, 4 (1992), 511–547.
- Patrick Cousot and Radhia Cousot. 2002a. «Modular Static Program Analysis». In *Proceedings of the Eleventh International Conference on Compiler Construction (CC 2002)*, R.N. Horspool (Ed.). LNCS 2304, Springer, Berlin, Grenoble, France, 159–178. doi:10.1007/3-540-45937-5_13
- Patrick Cousot and Radhia Cousot. 2002b. «Systematic design of program transformation frameworks by abstract interpretation». In *29th Symposium on Principles of Programming Languages (POPL 2002)*, John Launchbury and John C. Mitchell (Eds.). ACM, 178–190. doi:10.1145/503272.503290
- Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. 2006. «Combination of Abstractions in the ASTRÉE Static Analyzer». In *Revised Selected Papers from the 11th Asian Computing Science Conference on Advances in Computer Science - Secure Software and Related Issues – ASIAN 2006 (Lecture Notes in Computer Science, Vol. 4435)*. Springer, 272–300. doi:10.1007/978-3-540-77505-8_23
- Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, and Xavier Rival. 2009. «Why does Astrée scale up?» *Formal Methods Syst. Des.* 35, 3 (2009), 229–264. doi:10.1007/S10703-009-0089-6
- P. Cousot, R. Cousot, and F. Logozzo. 2011. «A Parametric Segmentation Functor for Fully Automatic and Scalable Array Content Analysis». In *Conference Record of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM Press, New York, Austin, Texas, 105–118. doi:10.1145/1925844.1926399

- Patrick Cousot and Nicolas Halbwachs. 1978. «Automatic discovery of linear restraints among variables of a program». In *Conference Record of the Fifth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM Press, New York, NY, Tucson, Arizona, 84–97. doi:10.1145/512760.512770
- John Cyphert, Jason Breck, Zachary Kincaid, and Thomas W. Reps. 2019. «Refinement of path expressions for static analysis». *Proc. ACM Program. Lang.* 3, POPL (2019), 45:1–45:29. doi:10.1145/3290358
- Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. 1991. «Efficiently Computing Static Single Assignment Form and the Control Dependence Graph». *ACM Trans. Program. Lang. Syst.* 13, 4 (10 1991), 451–490. doi:10.1145/115372.115320
- Vincent David, Jean Delcoigne, Evelyne Leret, Alain Ourghanlian, Philippe Hilsenkopf, and Philippe Paris. 1998. «Safety Properties Ensured by the OASIS Model for Safety Critical Real-Time Systems». In *Computer Safety, Reliability and Security, 17th International Conference, SAFECOMP'98, Heidelberg, Germany, October 5-7, 1998, Proceedings (Lecture Notes in Computer Science, Vol. 1516)*, Wolfgang D. Ehrenberger (Ed.). Springer, 45–59. doi:10.1007/3-540-49646-7_4
- Jacobus Willem de Bakker and Lambert Meertens. 1975. «On the Completeness of the Inductive Assertion Method». *J. Comput. Syst. Sci.* 11, 3 (1975), 323–357. doi:10.1016/S0022-0000(75)80056-0
- Delphine Demange, Yon Fernández de Retana, and David Pichardie. 2018. «Semantic reasoning about the sea of nodes». In *27th International Conference on Compiler Construction (CC 2018)*, Christophe Dubach and Jingling Xue (Eds.). ACM, 163–173. doi:10.1145/3178372.3179503
- Frama-C development community. [n. d.]. Frama-C value analysis plug-in. <http://frama-c.com/value.html>.
- Paul F. Dietz. 1982. «Maintaining Order in a Linked List». In *Proceedings of the 14th Annual ACM Symposium on Theory of Computing, May 5-7, 1982, San Francisco, California, USA*, Harry R. Lewis, Barbara B. Simons, Walter A. Burkhard, and Lawrence H. Landweber (Eds.). ACM, 122–127. doi:10.1145/800070.802184
- Edsger W. Dijkstra. 1975. «Guarded commands, nondeterminacy and formal derivation of programs». *Commun. ACM* 18, 8 (8 1975), 453–457. doi:10.1145/360933.360975
- Isil Dillig, Thomas Dillig, and Alex Aiken. 2009. «SAIL: Static analysis intermediate language with a two-level representation». *Technical Report, Tech. Rep.* (2009).
- Isil Dillig, Thomas Dillig, and Alex Aiken. 2010. «Fluid updates: Beyond strong vs. weak updates». In *Programming Languages and Systems*. Springer, 246–266.
- Dino Distefano, Peter W Ohearn, and Hongseok Yang. 2006. «A local shape analysis based on separation logic». In *Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 287–302.

- Amer Diwan, Kathryn S. McKinley, and J. Eliot B. Moss. 1998. «Type-based Alias Analysis». In *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation (PLDI '98)*. ACM, New York, NY, USA, 106–117. doi:10.1145/277650.277670
- Adel Djoudi, Sébastien Bardin, and Éric Goubault. 2016. «Recovering High-Level Conditions from Binary Programs». In : *21st International Symposium on Formal Methods (FM 2016)*. 235–253. doi:10.1007/978-3-319-48989-6_15
- Robert W Dockins. 2012. *Operational refinement for compiler correctness*. Ph. D. Dissertation. Princeton University.
- Cezara Dragoi, Constantin Enea, and Mihaela Sighireanu. 2013. «Local Shape Analysis for Overlaid Data Structures». In *Static Analysis - 20th International Symposium, SAS 2013, Seattle, WA, USA, June 20-22, 2013. Proceedings (Lecture Notes in Computer Science, Vol. 7935)*, Francesco Logozzo and Manuel Fähndrich (Eds.). Springer, 150–171. doi:10.1007/978-3-642-38856-9_10
- Archibald Samuel Elliott, Andrew Ruef, Michael Hicks, and David Tarditi. 2018. «Checked C: Making C Safe by Extension». In *2018 IEEE Cybersecurity Development, SecDev 2018, Cambridge, MA, USA, September 30 - October 2, 2018*. IEEE Computer Society, 53–60. doi:10.1109/SecDev.2018.00015
- Cormac Flanagan and James B. Saxe. 2001. «Avoiding exponential explosion: generating compact verification conditions». In *Proceedings of the 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '01)*. Association for Computing Machinery, New York, NY, USA, 193205. doi:10.1145/360204.360220
- Yoshihiko Futamura. 1999. «Partial Evaluation of Computation Process – An Approach to a Compiler-Compiler». *High. Order Symb. Comput.* 12, 4 (1999), 381–391. doi:10.1023/A:1010095604496
- Harold N. Gabow. 1990. «Data Structures for Weighted Matching and Nearest Common Ancestors with Linking». In *Proceedings of the First Annual ACM-SIAM Symposium on Discrete Algorithms, 22-24 January 1990, San Francisco, California, USA*, David S. Johnson (Ed.). SIAM, 434–443.
- Bernard A. Galler and Michael J. Fischer. 1964. «An improved equivalence algorithm». *Commun. ACM* 7, 5 (1964), 301–303. doi:10.1145/364099.364331
- Graeme Gange, Jorge A. Navas, Peter Schachte, Harald Søndergaard, and Peter J. Stuckey. 2015. «Horn clauses as an intermediate representation for program analysis and transformation». *Theory Pract. Log. Program.* 15, 4-5 (2015), 526–542. doi:10.1017/S1471068415000204
- Graeme Gange, Jorge A. Navas, Peter Schachte, Harald Søndergaard, and Peter J. Stuckey. 2016. «An Abstract Domain of Uninterpreted Functions». In *Verification, Model Checking, and Abstract Interpretation (VMCAI 2016)*, Barbara Jobstmann and K. Rustan M. Leino (Eds.). Springer, 85–103. doi:10.1007/978-3-662-49122-5_4

- Adam T. Geller, Justin Frank, and William J. Bowman. 2024. «Indexed Types for a Statically Safe WebAssembly». *Proc. ACM Program. Lang.* 8, POPL (2024), 2395–2424. doi:10.1145/3632922
- Roberto Giacobazzi and Isabella Mastroeni. 2016. «Making abstract models complete». *Math. Struct. Comput. Sci.* 26, 4 (2016), 658–701. doi:10.1017/S0960129514000358
- Roberto Giacobazzi and Elisa Quintarelli. 2001. «Incompleteness, Counterexamples, and Refinements in Abstract Model-Checking». In *Static Analysis, 8th International Symposium, SAS 2001, Paris, France, July 16-18, 2001, Proceedings (Lecture Notes in Computer Science, Vol. 2126)*, Patrick Cousot (Ed.). Springer, 356–373. doi:10.1007/3-540-47764-0_20
- Josselin Giet. 2024. *Vérification automatique d'ordonnancements de tâches*. Ph. D. Dissertation. ENS Paris.
- Laure Gonnord and Peter Schrammel. 2014. «Abstract Acceleration in Linear Relation Analysis». *Science of Computer Programming* 93 (2014), 125–153. doi:10.1016/j.scico.2013.09.016 Author version : http://hal.inria.fr/hal-00787212/en.
- Denis Gopan, Frank DiMaio, Nurit Dor, Thomas Reps, and Mooly Sagiv. 2004. «Numeric domains with summarized dimensions». In *Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 512–529.
- Susanne Graf and Hassen Saïdi. 1997. «Construction of Abstract State Graphs with PVS». In *Computer Aided Verification, 9th International Conference, CAV '97, Haifa, Israel, June 22-25, 1997, Proceedings (Lecture Notes in Computer Science, Vol. 1254)*, Orna Grumberg (Ed.). Springer, 72–83. doi:10.1007/3-540-63166-6_10
- Philippe Granger. 1989. «Static analysis of arithmetical congruences». *International Journal of Computer Mathematics* 30, 3-4 (1989), 165–190. doi:10.1080/00207168908803778
- Sumit Gulwani and George C. Necula. 2004. «A Polynomial-Time Algorithm for Global Value Numbering». In *Static Analysis Symposium (SAS 2004)*, Roberto Giacobazzi (Ed.). Springer, 212–227. doi:10.1007/978-3-540-27864-1_17
- Sumit Gulwani and Ashish Tiwari. 2006. *Static Analysis of Heap-Manipulating Low-Level Software*. Technical Report. Microsoft.
- Sumit Gulwani and Ashish Tiwari. 2007. «An Abstract Domain for Analyzing Heap-Manipulating Low-Level Software». In *Computer Aided Verification, 19th International Conference, CAV 2007, Berlin, Germany, July 3-7, 2007, Proceedings (Lecture Notes in Computer Science, Vol. 4590)*, Werner Damm and Holger Hermanns (Eds.). Springer, 379–392. doi:10.1007/978-3-540-73368-3_42
- Arie Gurfinkel. 2022a. «Program Verification with Constrained Horn Clauses (Invited Paper)». In *Computer Aided Verification - 34th International Conference, CAV 2022, Haifa, Israel, August 7-10, 2022, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 13371)*, Sharon Shoham and Yakir Vizel (Eds.). Springer, 19–29. doi:10.1007/978-3-031-13185-1_2

- Arie Gurfinkel. 2022b. «Program Verification with Constrained Horn Clauses (Invited Paper)». In *Computer Aided Verification - 34th International Conference, CAV 2022, Haifa, Israel, August 7-10, 2022, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 13371)*, Sharon Shoham and Yakir Vizel (Eds.). Springer, 19–29. doi:10.1007/978-3-031-13185-1_2
- Yann Herklotz, Delphine Demange, and Sandrine Blazy. 2023. «Mechanised Semantics for Gated Static Single Assignment». In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic (Eds.). ACM, 182–196. doi:10.1145/3573105.3575681
- Maurice Herlihy. 1991. «Wait-Free Synchronization». *ACM Trans. Program. Lang. Syst.* 13, 1 (1991), 124–149. doi:10.1145/114005.102808
- Hugo Illous. 2019. *Abstract Heap Relations for a Compositional Shape Analysis*. Ph.D. Dissertation. Université Paris sciences et lettres. <https://tel.archives-ouvertes.fr/tel-02399767>
- Bertrand Jeannet and Antoine Miné. 2009. «Apron: A Library of Numerical Abstract Domains for Static Analysis». In *Computer Aided Verification, 21st International Conference, CAV 2009, Grenoble, France, June 26 - July 2, 2009. Proceedings (Lecture Notes in Computer Science, Vol. 5643)*, Ahmed Bouajjani and Oded Maler (Eds.). Springer, 661–667. doi:10.1007/978-3-642-02658-4_52
- Bertrand Jeannet, Peter Schrammel, and Sriram Sankaranarayanan. 2014. «Abstract acceleration of general linear loops». In *The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '14, San Diego, CA, USA, January 20-21, 2014*, Suresh Jagannathan and Peter Sewell (Eds.). ACM, 529–540. doi:10.1145/2535838.2535843
- Jacques-Henri Jourdan. 2016. *Verasco: a Formally Verified C Static Analyzer*. Ph.D. Dissertation. Université Paris Diderot (Paris 7) Sorbonne Paris Cité.
- Jacques-Henri Jourdan, Vincent Laporte, Sandrine Blazy, Xavier Leroy, and David Pichardie. 2015. «A formally-verified C static analyzer». *ACM SIGPLAN Notices* 50, 1 (2015), 247–259.
- Matthieu Journault, Antoine Miné, Raphaël Monat, and Abdelraouf Ouadjaout. 2019. «Combinations of Reusable Abstract Domains for a Multilingual Static Analyzer». In *11th International Conference on Verified Software Theories, Tools, and Experiments - Revised Selected Papers (Lecture Notes in Computer Science, Vol. 12031)*, Supratik Chakraborty and Jorge A. Navas (Eds.). Springer, 1–18. doi:10.1007/978-3-030-41600-3_1
- Andrew W. Keep and R. Kent Dybvig. 2013. «A nanopass framework for commercial compiler development». In *ACM SIGPLAN International Conference on Functional Programming, ICFP'13, Boston, MA, USA - September 25 - 27, 2013*, Greg Morrisett and Tarmo Uustalu (Eds.). ACM, 343–350. doi:10.1145/2500365.2500618
- Sven Keidel, Sebastian Erdweg, and Tobias Hombücher. 2023. «Combinator-Based Fixpoint Algorithms for Big-Step Abstract Interpreters». *Proc. ACM Program. Lang.* 7, ICFP (2023), 955–981. doi:10.1145/3607863

- Richard Kelsey. 1995. «A Correspondence between Continuation Passing Style and Static Single Assignment Form». In *Proceedings ACM SIGPLAN Workshop on Intermediate Representations (IR'95)*, Michael D. Ernst (Ed.). ACM, 13–23. doi:10.1145/202529.202532
- Andrew Kennedy. 2007. «Compiling with continuations, continued». In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming, ICFP 2007, Freiburg, Germany, October 1-3, 2007*, Ralf Hinze and Norman Ramsey (Eds.). ACM, 177–190. doi:10.1145/1291151.1291179
- Ken Kennedy. 1981. «A Survey of Data Flow Analysis Techniques». In *Program Flow Analysis: Theory and Applications*, Neil D. Jones and Steven S. Muchnick (Eds.). Prentice-Hall.
- Gary A Kildall. 1973. «A unified approach to global program optimization». In *1st annual ACM SIGACT-SIGPLAN Symposium on Principles of programming languages (POPL 1973)*. doi:10.1145/512927.512945
- Sung Kook Kim, Arnaud J. Venet, and Aditya V. Thakur. 2020. «Memory-efficient Fixpoint Computation». In *27th Static Analysis Symposium (SAS)*. To appear.
- Zachary Kincaid, Thomas W. Reps, and John Cyphert. 2021. «Algebraic Program Analysis». In *Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 12759)*, Alexandra Silva and K. Rustan M. Leino (Eds.). Springer, 46–83. doi:10.1007/978-3-030-81685-8_3
- Johannes Kinder, Florian Zuleger, and Helmut Veith. 2009. «An Abstract Interpretation-Based Framework for Control Flow Reconstruction from Binaries». In *Verification, Model Checking, and Abstract Interpretation (VMCAI 2009)*, Neil D. Jones and Markus Müller-Olm (Eds.). Springer, 214–228. doi:10.1007/978-3-540-93900-9_19
- James C. King. 1976. «Symbolic Execution and Program Testing». *Commun. ACM* 19, 7 (1976), 385–394. doi:10.1145/360248.360252
- Oleg Kiselyov. 2010. «Typed Tagless Final Interpreters». In *Generic and Indexed Programming - International Spring School, SSGIP 2010, Oxford, UK, March 22-26, 2010, Revised Lectures (Lecture Notes in Computer Science, Vol. 7470)*, Jeremy Gibbons (Ed.). Springer, 130–174. doi:10.1007/978-3-642-32202-0_3
- Gerwin Klein, June Andronick, Kevin Elphinstone, Toby Murray, Thomas Sewell, Rafal Kolanski, and Gernot Heiser. 2014. «Comprehensive Formal Verification of an OS Microkernel». *ACM Trans. Comput. Syst.* 32, 1 (2 2014), 2:1–2:70. doi:10.1145/2560537
- Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. «seL4: Formal Verification of an OS Kernel». In *Proceedings of the ACM SIGOPS 22Nd Symposium on Operating Systems Principles (SOSP '09)*. ACM, New York, NY, USA, 207–220. doi:10.1145/1629575.1629596

- Anvesh Komuravelli, Arie Gurfinkel, and Sagar Chaki. 2016. «SMT-based model checking for recursive programs». *Formal Methods in System Design* 48, 3 (2016), 175–205.
- Lamport. 1979. «How to make a multiprocessor computer that correctly executes multiprocess programs». *IEEE transactions on computers* 100, 9 (1979), 690–691.
- Leslie Lamport. 1995. «How to write a proof». *The American mathematical monthly* 102, 7 (1995), 600–608.
- Leslie Lamport. 2002. *Specifying Systems, The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.
- Vincent Laviron, Bor-Yuh Evan Chang, and Xavier Rival. 2010. «Separating Shape Graphs». In *Proceedings of the 19th European Conference on Programming Languages and Systems (ESOP’10)*. Springer-Verlag, Berlin, Heidelberg, 387–406. doi:10.1007/978-3-642-11957-6_21
- Quang Loc Le, Azalea Raad, Jules Villard, Josh Berdine, Derek Dreyer, and Peter W. O’Hearn. 2022. «Finding real bugs in big programs with incorrectness logic». *Proc. ACM Program. Lang.* 6, OOPSLA1 (2022), 1–27. doi:10.1145/3527325
- Juneyoung Lee, Yoonseung Kim, Youngju Song, Chung-Kil Hur, Sanjoy Das, David Majnemer, John Regehr, and Nuno P. Lopes. 2017. «Taming undefined behavior in LLVM». In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18-23, 2017*, Albert Cohen and Martin T. Vechev (Eds.). ACM, 633–647. doi:10.1145/3062341.3062343
- K.R.M. Leino. 2005. «Efficient weakest preconditions». *Inform. Process. Lett.* 93, 6 (2005), 281–288. doi:10.1016/j.ipl.2004.10.015
- Matthieu Lemerre and Sébastien Bardin. 2017. «Abstract Interpretation using a Language of Symbolic Approximation». *CoRR* abs/1712.10058 (2017).
- Thomas Lengauer and Robert Endre Tarjan. 1979. «A Fast Algorithm for Finding Dominators in a Flowgraph». *ACM Trans. Program. Lang. Syst.* 1, 1 (1979), 121–141. doi:10.1145/357062.357071
- Sorin Lerner, David Grove, and Craig Chambers. 2002. «Composing dataflow analyses and transformations». In *29th SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2002)*, John Launchbury and John C. Mitchell (Eds.). ACM, 270–282. doi:10.1145/503272.503298
- Xavier Leroy. 2006. «Formal certification of a compiler back-end or: programming a compiler with a proof assistant». In *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2006, Charleston, South Carolina, USA, January 11-13, 2006*, J. Gregory Morrisett and Simon L. Peyton Jones (Eds.). ACM, 42–54. doi:10.1145/1111037.1111042
- Xavier Leroy. 2009. «A Formally Verified Compiler Back-end». *J. Autom. Reason.* 43, 4 (2009), 363–446. doi:10.1007/S10817-009-9155-4

- Xavier Leroy and Sandrine Blazy. 2008. «Formal Verification of a C-like Memory Model and Its Uses for Verifying Program Transformations». *J. Autom. Reason.* 41, 1 (2008), 1–31. doi:10.1007/S10817-008-9099-0
- Huisong Li, Francois Berenger, Bor-Yuh Evan Chang, and Xavier Rival. 2017. «Semantic-directed clumping of disjunctive abstract states». In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL 2017, Paris, France, January 18-20, 2017*, Giuseppe Castagna and Andrew D. Gordon (Eds.). ACM, 32–45. doi:10.1145/3093333.3009881
- Huisong Li, Xavier Rival, and Bor-Yuh Evan Chang. 2015. «Shape Analysis for Unstructured Sharing». In *Static Analysis - 22nd International Symposium, SAS 2015, Saint-Malo, France, September 9-11, 2015, Proceedings (Lecture Notes in Computer Science, Vol. 9291)*, Sandrine Blazy and Thomas P. Jensen (Eds.). Springer, 90–108. doi:10.1007/978-3-662-48288-9_6
- Jiangchao Liu and Xavier Rival. 2015a. «Abstraction of Arrays Based on Non Contiguous Partitions». In *Verification, Model Checking, and Abstract Interpretation - 16th International Conference, VMCAI 2015, Mumbai, India, January 12-14, 2015. Proceedings (Lecture Notes in Computer Science, Vol. 8931)*, Deepak D’Souza, Akash Lal, and Kim Guldstrand Larsen (Eds.). Springer, 282–299. doi:10.1007/978-3-662-46081-8_16
- Jiangchao Liu and Xavier Rival. 2015b. «Abstraction of Optional Numerical Values». In *Programming Languages and Systems - 13th Asian Symposium, APLAS 2015, Pohang, South Korea, November 30 - December 2, 2015, Proceedings (Lecture Notes in Computer Science, Vol. 9458)*, Xinyu Feng and Sungwoo Park (Eds.). Springer, 146–166. doi:10.1007/978-3-319-26529-2_9
- Francesco Logozzo and Manuel Fähndrich. 2008. *On the Relative Completeness of Bytecode Analysis Versus Source Code Analysis*. Springer Berlin Heidelberg, Berlin, Heidelberg, 197–212. doi:10.1007/978-3-540-78791-4_14
- Maroua Maalej, Vitor Paisante, Fernando Magno Quintão Pereira, and Laure Gonnord. 2018. «Combining range and inequality information for pointer disambiguation». *Science of Computer Programming* 152 (2018), 161 – 184. doi:10.1016/j.scico.2017.10.014
- Aravind Machiry, John H. Kastner, Matt McCutchen, Aaron Eline, Kyle Headley, and Michael Hicks. 2022. «C to Checked C by 3C (extended version)». *CoRR abs/2203.13445* (2022). doi:10.48550/arXiv.2203.13445
- Alan K. Mackworth. 1977. «Consistency in networks of relations». *Artificial Intelligence* 8, 1 (1977), 99 – 118. doi:10.1016/0004-3702(77)90007-8
- Henry Massalin. 1992. *Synthesis: An Efficient Implementation of Fundamental Operating System Services*. Ph. D. Dissertation. Department of Computer Science, Columbia University.
- Laurent Mauborgne and Xavier Rival. 2005. «Trace Partitioning in Abstract Interpretation Based Static Analyzers». In *European Symposium on Programming (ESOP’05) (Lecture Notes in Computer Science, Vol. 3444)*, M. Sagiv (Ed.). Springer-Verlag, 5–20.

- Kayvan Memarian, Victor B. F. Gomes, Brooks Davis, Stephen Kell, Alexander Richardson, Robert N. M. Watson, and Peter Sewell. 2019. «Exploring C semantics and pointer provenance». *Proc. ACM Program. Lang.* 3, POPL (2019), 67:1–67:32. doi:10.1145/3290380
- Amira Methni. 2016. *Méthode de conception de logiciel système critique couplée à une démarche de vérification formelle. (A method for designing critical software system coupled with a formal verification approach)*. Ph. D. Dissertation. Conservatoire national des arts et métiers, France. <https://tel.archives-ouvertes.fr/tel-01445983>
- Donald Michie. 1968. «Memo functions and machine learning». *Nature* 218, 5136 (1968), 19–22.
- Marco Milanese and Antoine Miné. 2024. «Under-approximating Memory Abstractions». In *31th International Symposium on Static Analysis, SAS 2024*.
- Antoine Miné. 2001. «A New Numerical Abstract Domain Based on Difference-Bound Matrices». In *Programs as Data Objects, Second Symposium, PADO 2001, Aarhus, Denmark, May 21-23, 2001, Proceedings (Lecture Notes in Computer Science, Vol. 2053)*, Olivier Danvy and Andrzej Filinski (Eds.). Springer, 155–172. doi:10.1007/3-540-44978-7_10
- Antoine Miné. 2006a. «Field-sensitive value analysis of embedded C programs with union types and pointer arithmetics». In *Proc. of the ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES'06)*. ACM, 54–63. <http://www.di.ens.fr/~mine/publi/article-mine-lctes06.pdf>.
- Antoine Miné. 2006b. «The octagon abstract domain». *Higher-order and symbolic computation* 19, 1 (2006), 31–100. doi:10.1109/WCRE.2001.957836
- Antoine Miné. 2006c. «Symbolic methods to enhance the precision of numerical abstract domains». In *International Workshop on Verification, Model Checking, and Abstract Interpretation (VMCAI 2006)*. Springer, 348–363. doi:10.1007/11609773_23
- Antoine Miné. 2012. «Abstract domains for bit-level machine integer and floating-point operations». In *WING'12 - 4th International Workshop on invariant Generation*. Manchester, United Kingdom, 16.
- Antoine Miné. 2017. «Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation». *Found. Trends Program. Lang.* 4, 3-4 (2017), 120–372. doi:10.1561/25000000034
- Solène Miriaz and David Pichardie. 2022. «A Flow-Insensitive-Complete Program Representation». In *23th International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI 2022)*. doi:10.1007/978-3-030-94583-1_10
- Anders Møller and Michael I. Schwartzbach. 2018. *Static Program Analysis*. Department of Computer Science, Aarhus University, <http://cs.au.dk/~amoeller/spa/>.
- Raphael Monat. 2021. *Static Type and Value Analysis by Abstract Interpretation of Python Programs with Native C Libraries*. Ph. D. Dissertation. Sorbonne Université.

- David Monniaux. 2007. «Verification of device drivers and intelligent controllers: a case study». In *Proceedings of the 7th ACM & IEEE International conference on Embedded software, EMSOFT 2007, September 30 - October 3, 2007, Salzburg, Austria*, Christoph M. Kirsch and Reinhard Wilhelm (Eds.). ACM, 30–36. doi:10.1145/1289927.1289937
- Steven S. Muchnick. 1997. *Advanced Compiler Design and Implementation*. Morgan Kaufmann.
- Manasij Mukherjee and John Regehr. 2024. «Hydra: Generalizing Peephole Optimizations with Program Synthesis». *Proc. ACM Program. Lang.* 8, OOPSLA1 (2024), 725–753. doi:10.1145/3649837
- Kedar S. Namjoshi and Zvonimir Pavlinovic. 2018. «The Impact of Program Transformations on Static Program Analysis». In *Static Analysis - 25th International Symposium, SAS 2018, Freiburg, Germany, August 29-31, 2018, Proceedings (Lecture Notes in Computer Science, Vol. 11002)*, Andreas Podelski (Ed.). Springer, 306–325. doi:10.1007/978-3-319-99725-4_19
- George C. Necula. 1997. «Proof-Carrying Code». In *Conference Record of POPL'97: The 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Papers Presented at the Symposium, Paris, France, 15-17 January 1997*, Peter Lee, Fritz Henglein, and Neil D. Jones (Eds.). ACM Press, 106–119. doi:10.1145/263699.263712
- George C. Necula, Scott McPeak, Shree Prakash Rahul, and Westley Weimer. 2002. «CIL: Intermediate Language and Tools for Analysis and Transformation of C Programs». In *Compiler Construction, 11th International Conference, CC 2002, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2002, Grenoble, France, April 8-12, 2002, Proceedings (Lecture Notes in Computer Science, Vol. 2304)*, R. Nigel Horspool (Ed.). Springer, 213–228. doi:10.1007/3-540-45937-5_16
- Greg Nelson. 1980. *Techniques for program verification*. Ph.D. Dissertation. Stanford University, CA, USA.
- Greg Nelson and Derek C. Oppen. 1980. «Fast Decision Procedures Based on Congruence Closure». *J. ACM* 27, 2 (1980), 356–364. doi:10.1145/322186.322198
- Manh-Dung Nguyen. 2021. *Binary-level directed fuzzing for complex vulnerabilities. (Guidage du test en frelatage de codes binaires pour la détection de vulnérabilités logicielles complexes)*. Ph. D. Dissertation. Grenoble Alpes University, France. <https://tel.archives-ouvertes.fr/tel-03238343>
- Olivier Nicole. 2022. *Automated Verification of Systems Code using Type-Based Memory Abstractions*. Ph.D. Dissertation. Université PSL/ENS/CEA LIST. <https://theses.hal.science/tel-03962643v2>
- Diego Novillo. 2007. «Memory SSA—a unified approach for sparsely representing memory operations». In *Proceedings of the GCC Developers Summit*. Citeseer, 97–110.

- Hakjoo Oh, Kihong Heo, Wonchan Lee, Woosuk Lee, and Kwangkeun Yi. 2012. «Design and Implementation of Sparse Global Analyses for C-like Languages». In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '12)*. ACM, New York, NY, USA, 229–238. doi:10.1145/2254064.2254092
- Chris Okasaki and Andrew Gill. 1998. «Fast Mergeable Integer Maps». In *Workshop on ML*. 77–86.
- Mathilde Ollivier, Sébastien Bardin, Richard Bonichon, and Jean-Yves Marion. 2019. «How to kill symbolic deobfuscation for free (or: unleashing the potential of path-oriented protections)». In *Proceedings of the 35th Annual Computer Security Applications Conference, ACSAC 2019, San Juan, PR, USA, December 09-13, 2019*, David M. Balenson (Ed.). ACM, 177–189. doi:10.1145/3359789.3359812
- Vitor Paisante, Maroua Maalej, Leonardo Barbosa e Oliveira, Laure Gonnord, and Fernando Magno Quintão Pereira. 2016. «Symbolic range analysis of pointers». In *Proceedings of the 2016 International Symposium on Code Generation and Optimization, CGO 2016, Barcelona, Spain, March 12-18, 2016*, Björn Franke, Youfeng Wu, and Fabrice Rastello (Eds.). ACM, 171–181. doi:10.1145/2854038.2854050
- Jihee Park, Insu Yun, and Sukyoung Ryu. 2025. «Bridging the Gap between Real-World and Formal Binary Lifting through Filtered-Simulation». *Proc. ACM Program. Lang.* 9, OOPSLA1 (4 2025). doi:10.1145/3720524
- David Pichardie. 2005. *Interprétation abstraite en logique intuitionniste: extraction d'analyseurs Java certifiés*. Ph.D. Dissertation. Université de Rennes I.
- Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. «Translation Validation». In *Tools and Algorithms for Construction and Analysis of Systems, 4th International Conference, TACAS '98, Held as Part of the European Joint Conferences on the Theory and Practice of Software, ETAPS'98, Lisbon, Portugal, March 28 - April 4, 1998, Proceedings (Lecture Notes in Computer Science, Vol. 1384)*, Bernhard Steffen (Ed.). Springer, 151–166. doi:10.1007/BFb0054170
- François Pottier and Didier Rémy. 2005. *The Essence of ML Type Inference*. MIT Press, Chapter 10, 389–489.
- Raghu Ramakrishnan, Divesh Srivastava, and S Sudarshan. 1992. «Efficient bottom-up evaluation of logic programs». *Computer Systems and Software Engineering: State-of-the-Art (1992)*, 287–324.
- Fabrice Rastello and Florent Bouchez Tichadou (Eds.). 2022. *SSA-based Compiler Design*. Springer.
- Alex Rebert and Christoph Kern. 2024. *Secure by Design: Google's Perspective on Memory Safety*. Technical Report. Google Security Engineering.
- John Regehr and Usit Duongsaa. 2006. «Deriving abstract transfer functions for analyzing embedded software». In *Proceedings of the 2006 ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES'06)*, Ottawa, Ontario, Canada, June 14-16, 2006, Mary Jane Irwin and Koen De Bosschere (Eds.). ACM, 34–43. doi:10.1145/1134650.1134657

- John H. Reif and Harry R. Lewis. 1986. «Efficient Symbolic Analysis of Programs». *J. Comput. Syst. Sci.* 32, 3 (1986), 280–314. doi:10.1016/0022-0000(86)90031-0
- John H. Reif and Robert Endre Tarjan. 1982. «Symbolic Program Analysis in Almost-Linear Time». *SIAM J. Comput.* 11, 1 (1982), 81–93. doi:10.1137/0211007
- Thomas W. Reps, Gogul Balakrishnan, and Junghee Lim. 2006. «Intermediate-representation recovery from low-level code». In *Proceedings of the 2006 ACM SIGPLAN Workshop on Partial Evaluation and Semantics-based Program Manipulation, 2006, Charleston, South Carolina, USA, January 9-10, 2006*, John Hatcliff and Frank Tip (Eds.). ACM, 100–111. doi:10.1145/1111542.1111560
- Thomas W. Reps, Shmuel Sagiv, and Greta Yorsh. 2004. «Symbolic Implementation of the Best Transformer». In *5th International Conference on Verification, Model Checking, and Abstract Interpretation – VMCAI 2004 (Lecture Notes in Computer Science, Vol. 2937)*, Bernhard Steffen and Giorgio Levi (Eds.). Springer, 252–266. doi:10.1007/978-3-540-24622-0_21
- Xavier Rival. 2011. *Abstract Domains for the Static Analysis of Programs Manipulating Complex Data Structures*. Ph. D. Dissertation.
- Xavier Rival and Laurent Mauborgne. 2007. «The Trace Partitioning Abstract Domain». *ACM Transactions on Programming Languages and Systems (TOPLAS)* 29, 5 (2007).
- Xavier Rival and Kwangkeun Yi. 2020. *Introduction to static analysis: an abstract interpretation perspective*. Mit Press.
- Patrick Maxim Rondon, Ming Kawaguchi, and Ranjit Jhala. 2010. «Low-level liquid types». In *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2010, Madrid, Spain, January 17-23, 2010*, Manuel V. Hermenegildo and Jens Palsberg (Eds.). ACM, 131–144. doi:10.1145/1706299.1706316
- Barry K. Rosen. 1977. «High-Level Data Flow Analysis». *Commun. ACM* 20, 10 (1977), 712–724. doi:10.1145/359842.359849
- Barry K Rosen, Mark N Wegman, and F Kenneth Zadeck. 1988. «Global value numbers and redundant computations». In *15th ACM SIGPLAN-SIGACT symposium on Principles of programming languages (POPL 1988)*. 12–27. doi:10.1145/73560.73562
- Juriaan Rot. 2018. «Finite and infinite traces, inductively and coinductively». *WAIT* (2018). <https://matryoshka-project.github.io/wait2018/slides/WAIT-2018-Rot-Finite-and-Infinite-Traces,-Inductively-and-Coinductively.pdf>.
- Barbara G. Ryder and Marvin C. Paull. 1986. «Elimination Algorithms for Data Flow Analysis». *ACM Comput. Surv.* 18, 3 (1986), 277–316. doi:10.1145/27632.27649
- Shmuel Sagiv, Thomas W. Reps, and Reinhard Wilhelm. 1998. «Solving Shape-Analysis Problems in Languages with Destructive Updating». *ACM Trans. Program. Lang. Syst.* 20, 1 (1998), 1–50. doi:10.1145/271510.271517

- Jonathan Salwan, Sébastien Bardin, and Marie-Laure Potet. 2018. «Symbolic Deobfuscation: From Virtualized Code Back to the Original». In *Detection of Intrusions and Malware, and Vulnerability Assessment - 15th International Conference, DIMVA 2018, Saclay, France, June 28-29, 2018, Proceedings (Lecture Notes in Computer Science, Vol. 10885)*, Cristiano Giuffrida, Sébastien Bardin, and Gregory Blanc (Eds.). Springer, 372–392. doi:10.1007/978-3-319-93411-2_17
- Sriram Sankaranarayanan, Franjo Ivancic, Ilya Shlyakhter, and Aarti Gupta. 2006. «Static Analysis in Disjunctive Numerical Domains». In *Static Analysis, 13th International Symposium, SAS 2006, Seoul, Korea, August 29-31, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 4134)*, Kwangkeun Yi (Ed.). Springer, 3–17. doi:10.1007/11823230_2
- Sigurd Schneider. 2013. *Semantics of an Intermediate Language for Program Transformation*. Master’s thesis. Saarland University. <https://www.ps.uni-saarland.de/~sdschn/msc.pdf>
- Dana S Scott and Christopher Strachey. 1971. *Toward a mathematical semantics for computer languages*. Technical Report.
- Micha Sharir. 1980. «Structural Analysis: A New Approach to Flow Analysis in Optimizing Compilers». *Comput. Lang.* 5, 3 (1980), 141–153. doi:10.1016/0096-0551(80)90007-7
- Micha Sharir and Amir Pnueli. 1981. «Two approaches to interprocedural data flow analysis». *Program flow analysis: theory and applications* (1981).
- Holger Siegel, Bogdan Mihaila, and Axel Simon. 2013. «The Undefined Domain: Precise Relational Information for Entities That Do Not Exist». In *Programming Languages and Systems - 11th Asian Symposium, APLAS 2013, Melbourne, VIC, Australia, December 9-11, 2013. Proceedings (Lecture Notes in Computer Science, Vol. 8301)*, Chung-chieh Shan (Ed.). Springer, 74–89. doi:10.1007/978-3-319-03542-0_6
- Axel Simon and Andy King. 2010. «The two variable per inequality abstract domain». *High. Order Symb. Comput.* 23, 1 (2010), 87–143. doi:10.1007/s10990-010-9062-8
- Vugranam C Sreedhar. 1995. *Efficient program analysis using DJ graphs*. Ph. D. Dissertation.
- Yannick Stade, Sarah Tilscher, and Helmut Seidl. 2024. «The Top-Down Solver Verified: Building Confidence in Static Analyzers». In *Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24-27, 2024, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 14681)*, Arie Gurfinkel and Vijay Ganesh (Eds.). Springer, 303–324. doi:10.1007/978-3-031-65627-9_15
- Benno Stein, Bor-Yuh Evan Chang, and Manu Sridharan. 2024. «Interactive Abstract Interpretation with Demanded Summarization». *ACM Trans. Program. Lang. Syst.* 46, 1 (2024), 4:1–4:40. doi:10.1145/3648441
- Andrew S. Tanenbaum and Albert S. Woodhull. 1996. *Operating systems - design and implementation, 2nd Edition*. Pearson Education.
- Robert Endre Tarjan. 1981a. «A Unified Approach to Path Problems». *J. ACM* 28, 3 (7 1981), 577–593. doi:10.1145/322261.322272

- Robert Endre Tarjan. 1981b. «Fast algorithms for solving path problems». *Journal of the ACM (JACM)* 28, 3 (1981), 594–614.
- Robert Endre Tarjan and Jan van Leeuwen. 1984. «Worst-case Analysis of Set Union Algorithms». *J. ACM* 31, 2 (1984), 245–281. doi:10.1145/62.2160
- André Luiz Camargos Tavares, Benoit Boissinot, Fernando Magno Quintão Pereira, and Fabrice Rastello. 2014. «Parameterized Construction of Program Representations for Sparse Dataflow Analyses». In *23rd International Conference on Compiler Construction – CC 2014 (Lecture Notes in Computer Science, Vol. 8409)*, Albert Cohen (Ed.). Springer, 18–39. doi:10.1007/978-3-642-54807-9_2
- John Toman, Ren Siqi, Kohei Suenaga, Atsushi Igarashi, and Naoki Kobayashi. 2020. «ConSORT: Context- and Flow-Sensitive Ownership Refinement Types for Imperative Programs». In *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings (Lecture Notes in Computer Science, Vol. 12075)*, Peter Müller (Ed.). Springer, 684–714. doi:10.1007/978-3-030-44914-8_25
- Athanasios K. Tsakalidis. 1988. «The Nearest Common Ancestor in a Dynamic Tree». *Acta Informatica* 25, 1 (1988), 37–54. doi:10.1007/BF00268844
- Arnaud Venet and Guillaume P. Brat. 2004. «Precise and efficient static array bound checking for large embedded C programs». In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation 2004, Washington, DC, USA, June 9-11, 2004*. 231–242. doi:10.1145/996841.996869
- Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2022. «Sound, Precise, and Fast Abstract Interpretation with Tristate Numbers». In *IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2022, Seoul, Korea, Republic of, April 2-6, 2022*, Jae W. Lee, Sebastian Hack, and Tatiana Shpeisman (Eds.). IEEE, 254–265. doi:10.1109/CGO53902.2022.9741267
- Vesal Vojdani, Kalmer Apinis, Vootele Rõtov, Helmut Seidl, Varmo Vene, and Ralf Vogler. 2016. «Static race detection for device drivers: the Goblint approach». In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, ASE 2016, Singapore, September 3-7, 2016*, David Lo, Sven Apel, and Sarfraz Khurshid (Eds.). ACM, 391–402. doi:10.1145/2970276.2970337
- Bruce J. Walker, Richard A. Kemmerer, and Gerald J. Popek. 1980. «Specification and Verification of the UCLA Unix Security Kernel». *Commun. ACM* 23, 2 (2 1980), 118–131. doi:10.1145/358818.358825
- David Walker. 2005. *Advanced topics in types and programming languages*. The MIT Press Cambridge, Chapter Substructural type systems, 3–44.
- Ben Wegbreit. 1976. «Faster Retrieval from Context Trees». *Commun. ACM* 19, 9 (1976), 526–529. doi:10.1145/360336.360347
- Mark D. Weiser. 1984. «Program Slicing». *IEEE Trans. Software Eng.* 10, 4 (1984), 352–357. doi:10.1109/TSE.1984.5010248

- Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. «egg: Fast and extensible equality saturation». *Proc. ACM Program. Lang.* 5, POPL (2021), 1–29. doi:[10.1145/3434304](https://doi.org/10.1145/3434304)
- Li-yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic. 2020. «Interaction trees: representing recursive and impure programs in Coq». *Proc. ACM Program. Lang.* 4, POPL (2020), 51:1–51:32. doi:[10.1145/3371119](https://doi.org/10.1145/3371119)

Chapter 10

Appendices

Appendix A

Module PatriciaTree

This is a portion of the interface for `PatriciaTree` an implementation of Patricia trees [?, also known as Okasaki’s fast mergeable integer maps \(Okasaki and Gill, 1998\)](#). This code, that I have developped with the help of my PhD student Dorian Lesbre, has been separated from CODEX as it can be useful outside of the contexte of abstract interpretation. Here we provide only the part of the interface which is needed for the code in this manuscript.

The main reason to use these maps instead of the standard maps in OCaml are that they have a fixed tree layout that allow faster merge (union, intersection) operations, as in many cases you do not have to traverse the tree to perform the merge [Okasaki and Gill \(1998\)](#). In particular, when merging trees that have shared subtrees, we can take advantage of physical equality to avoid visiting these shared subtrees, and reuse them as is. This is useful in abstract interpretation as we often have to merge large maps that derive from a common map [\(Blanchet et al., 2002\)](#) (this is related to the notion of confluent persistency [\(???\)](#)). As a consequence, the design of this library tries to take advantage and preserve physical sharing as much as possible (unlike other OCaml libraries that implement Patricia trees).

There are other elements that differentiate this library from standard maps and sets: it has heterogeneous versions, that allow maps with keys of different types of keys mapping to different types of values. It relies on keys to provide a unique integer identifier (that we generate on creation time). There are other useful features that we do not use here, such as functions that can merge a map with a set, etc.

A.1 Homogeneous maps

```
module type KEY = sig
  type t
  val to_int: t → int
end
```

We need keys to be uniquely identified by an integer value (which is the real key used by the Patricia tree algorithms).

```
module MakeMap(Key:KEY):sig
  type τ t
```

```

val empty:  $\tau$  t
val is_empty:  $\tau$  t  $\rightarrow$  bool
val add : Key.t  $\rightarrow$   $\tau$   $\rightarrow$   $\tau$  t  $\rightarrow$   $\tau$  t
val singleton : Key.t  $\rightarrow$   $\tau$   $\rightarrow$   $\tau$  t
val remove : Key.t  $\rightarrow$   $\tau$  t  $\rightarrow$   $\tau$  t
val find : Key.t  $\rightarrow$   $\tau$  t  $\rightarrow$   $\tau$ 
val filter : (Key.t  $\rightarrow$   $\tau$   $\rightarrow$  bool)  $\rightarrow$   $\tau$  t  $\rightarrow$   $\tau$  t
val to_list :  $\tau$  t  $\rightarrow$  (Key.t *  $\tau$ ) list
val iter : (Key.t  $\rightarrow$   $\tau$   $\rightarrow$  unit)  $\rightarrow$   $\tau$  t  $\rightarrow$  unit
val fold : (Key.t  $\rightarrow$   $\tau$   $\rightarrow$   $\tau_2$   $\rightarrow$   $\tau_2$ )  $\rightarrow$   $\tau$  t  $\rightarrow$   $\tau_2$   $\rightarrow$   $\tau_2$ 

```

These functions are compatible with those of the Map module provided by the standard library of OCaml. `empty` returns an empty map, and `is_empty` checks for emptiness. `add` adds a binding to an existing map and `remove` removes it, returning a new map; `singleton` creates a map with a single binding in it. `find` retrieves the value associated with a key in a map. `filter` keeps only the bindings for which the predicate returns `true`. `to_list` returns the list of bindings in a map (in the order of the keys). `iter` and `fold` iterate on the bindings of the map, in the order of the keys.

```

val unsigned_max_binding:  $\tau$  t  $\rightarrow$  Key.t *  $\tau$ 

```

We use a non-standard ordering on OCaml `int`, which are seen like unsigned values in C (i.e. -1 is interpreted as $2^w - 1$, where w is 32 or 64 depending on the architecture, and is the highest key value). `unsigned_max_binding` returns the binding whose key is maximal according to this ordering.

```

val nonidempotent_inter_no_share:
  (Key.t  $\rightarrow$   $\tau$   $\rightarrow$   $\tau_2$   $\rightarrow$   $\tau_3$ )  $\rightarrow$   $\tau$  t  $\rightarrow$   $\tau_2$  t  $\rightarrow$   $\tau_3$  t

```

`nonidempotent_inter_no_share` computes an intersection of two maps (i.e., returns a map whose keys are present in the two arguments, using the supplied function to merge the value). This version with this type signature does not preserve physical sharing, and we use longer names (than the `inter` name of the OCaml `stdlib`) to make that explicit.

```

val fold_on_nonequal_inter:
  (Key.t  $\rightarrow$   $\tau$   $\rightarrow$   $\tau$   $\rightarrow$   $\tau_2$   $\rightarrow$   $\tau_2$ )  $\rightarrow$   $\tau$  t  $\rightarrow$   $\tau$  t  $\rightarrow$   $\tau_2$   $\rightarrow$   $\tau_2$ 
val fold_on_nonequal_union:
  (Key.t  $\rightarrow$   $\tau$  option  $\rightarrow$   $\tau$  option  $\rightarrow$   $\tau_2$   $\rightarrow$   $\tau_2$ )  $\rightarrow$   $\tau$  t  $\rightarrow$   $\tau$  t  $\rightarrow$   $\tau_2$   $\rightarrow$   $\tau_2$ 
end

```

`fold_on_nonequal_inter` $f m_1 m_2$ calls f with arguments $k, m_1[k], m_2[k]$ whenever both $m_1[k]$ and $m_2[k]$ exist, and $m_1[k] \neq m_2[k]$. `fold_on_nonequal_union` does the same but when either $m_1[k]$ or $m_2[k]$ exist, or both exist and $m_1[k] \neq m_2[k]$.

A.2 Heterogeneous keys

Heterogeneous maps and sets have keys (or elements) of different types, on which we mandate the following signature:

```

type (_, _) cmp = Eq : ( $\kappa$ ,  $\kappa$ ) cmp | Diff : ( $\kappa_1$ ,  $\kappa_2$ ) cmp

module type HETEROGENEOUS_KEY = sig
  type  $\kappa$  t
  val to_int:  $\kappa$  t  $\rightarrow$  int
  val polyeq:  $\kappa_1$  t  $\rightarrow$   $\kappa_2$  t  $\rightarrow$  ( $\kappa_1$ ,  $\kappa_2$ ) cmp
end

```

The module type `HETEROGENEOUS_KEY` specifies keys equipped with a polymorphic equality test. Compared to the homogeneous case, it adds a `polyeq` function comparing keys and returning a type equality witness (of type `cmp`) certifying that equal keys have the same type.

A.3 Heterogeneous sets

```

module MakeHeterogeneousSet(Key:HETEROGENEOUS_KEY):sig
  type t
  val empty: t
  val mem:  $\kappa$  Key.t  $\rightarrow$  t  $\rightarrow$  bool
  val add:  $\kappa$  Key.t  $\rightarrow$  t  $\rightarrow$  t

  type  $\delta$  polyfold = { f:  $\kappa$ .  $\kappa$  Key.t  $\rightarrow$   $\delta \rightarrow \delta$  } [@@unboxed]
  val fold:  $\delta$  polyfold  $\rightarrow$  t  $\rightarrow$   $\delta \rightarrow \delta$ 
end

```

The module `MakeHeterogeneousSet` defines sets with heterogeneous elements (or keys). The operations `empty`, `mem`, and `add` respectively create an empty set, test membership, and add an element a set. Because of heterogeneity, `folding` over the elements requires a function of higher-rank polymorphic type, which needs in OCaml to be encapsulated inside a record (defined in `polyfold`).

A.4 Heterogeneous maps

```

module type VALUE = sig type ( $\kappa$ ,  $\mu$ ) t end

module MakeHeterogeneousMap(Key:HETEROGENEOUS_KEY)(Value:VALUE):sig

  type  $\mu$  t
  val empty:  $\mu$  t
  val find:  $\kappa$  Key.t  $\rightarrow$   $\mu$  t  $\rightarrow$  ( $\kappa$ ,  $\mu$ ) Value.t
  val mem:  $\kappa$  Key.t  $\rightarrow$   $\mu$  t  $\rightarrow$  bool
  val add:  $\kappa$  Key.t  $\rightarrow$  ( $\kappa$ ,  $\mu$ ) Value.t  $\rightarrow$   $\mu$  t  $\rightarrow$   $\mu$  t
  val singleton:  $\kappa$  Key.t  $\rightarrow$  ( $\kappa$ ,  $\mu$ ) Value.t  $\rightarrow$   $\mu$  t

```

The module `MakeHeterogeneousMap` defines maps with heterogeneous keys and values. Note that the type of the value depends on both the type of the key and the type of the map; this allows operations combining different heterogeneous maps mapping the same keys to different types of vales.

The operations `empty`, `find`, `mem`, `add`, and `singleton` provide basic map operations: creating an empty map, retrieving a value by key, checking membership, adding a binding, and creating a map with a single binding. These functions are similar to their homogeneous counterparts, but handle heterogeneous key-value pairs.

```
type  $\mu$  polyiter = { f :  $\kappa$ .  $\kappa$  Key.t  $\rightarrow$  ( $\kappa$ ,  $\mu$ ) Value.t  $\rightarrow$  unit; } [@@unboxed]
val iter :  $\mu$  polyiter  $\rightarrow$   $\mu$  t  $\rightarrow$  unit

type ( $\delta$ ,  $\mu$ ) polyfold =
  { f :  $\kappa$ .  $\kappa$  Key.t  $\rightarrow$  ( $\kappa$ ,  $\mu$ ) Value.t  $\rightarrow$   $\delta \rightarrow \delta$  } [@@unboxed]
val fold : ( $\delta$ ,  $\mu$ ) polyfold  $\rightarrow$   $\mu$  t  $\rightarrow$   $\delta \rightarrow \delta$ 
```

Again, due to the heterogeneity of keys, operations like `fold` and `iter` are higher-rank, and need the function argument to be encapsulated in a record.

```
type ( $\delta$ ,  $\mu$ ) polyfold2_union =
  { f :  $\kappa$ .  $\kappa$  Key.t  $\rightarrow$ 
    ( $\kappa$ ,  $\mu$ ) Value.t option  $\rightarrow$  ( $\kappa$ ,  $\mu$ ) Value.t option  $\rightarrow$   $\delta \rightarrow \delta$  } [@@unboxed]
val fold_on_nonequal_union : ( $\delta$ ,  $\mu$ ) polyfold2_union  $\rightarrow$   $\mu$  t  $\rightarrow$   $\mu$  t  $\rightarrow$   $\delta \rightarrow \delta$ 
```

The function `fold_on_nonequal_union` is similar to the homogeneous function `fold_on_nonequal_union` (calling the argument on all the keys for which the value differ), but is also higher-rank.

```
type ( $\mu_1$ ,  $\mu_2$ ,  $\mu_3$ ) polyunion =
  { f :  $\kappa$ .  $\kappa$  Key.t  $\rightarrow$ 
    ( $\kappa$ ,  $\mu_1$ ) Value.t  $\rightarrow$  ( $\kappa$ ,  $\mu_2$ ) Value.t  $\rightarrow$  ( $\kappa$ ,  $\mu_3$ ) Value.t } [@@unboxed]
val idempotent_union : ( $\kappa$ ,  $\kappa$ ,  $\kappa$ ) polyunion  $\rightarrow$   $\kappa$  t  $\rightarrow$   $\kappa$  t  $\rightarrow$   $\kappa$  t

type ( $\mu_1$ ,  $\mu_2$ ,  $\mu_3$ ) polyinter =
  { f :  $\kappa$ .  $\kappa$  Key.t  $\rightarrow$ 
    ( $\kappa$ ,  $\mu_1$ ) Value.t  $\rightarrow$  ( $\kappa$ ,  $\mu_2$ ) Value.t  $\rightarrow$  ( $\kappa$ ,  $\mu_3$ ) Value.t } [@@unboxed]
val idempotent_inter : ( $\kappa$ ,  $\kappa$ ,  $\kappa$ ) polyinter  $\rightarrow$   $\kappa$  t  $\rightarrow$   $\kappa$  t  $\rightarrow$   $\kappa$  t
end
```

The function `idempotent_union` performs the union of two maps. It assumes that the argument f is idempotent, i.e., that $f\ k\ v\ v = v$. Thus, when the algorithm needs to merge two identical subtrees (where identity is detected using physical equality), it avoids traversing this subtree and returns it as is. This use and preservation of physical sharing allows handling large maps efficiently.

The function `idempotent_inter` is similar, but performs an intersection of the two maps instead of a union. Like the union, this operation is higher-ranked to support heterogeneity.

Appendix B

Module Term

B.1 Interface

```
| module Term: sig
```

This module builds terms (i.e., initial representation) corresponding to the (final) signature of `INTEGER_AND_BOOLEAN_EXPRESSIONS`, and some helper functions.

The terms that we build is actually a term graph, i.e. a directed acyclic structure where each term points to its subterms, and subterms can be shared (in Chapter 4, it is used to build cyclic term graphs (Ariola and Klop, 1996)). In order to have maximal sharing, we use hash-consing to identify terms that are structurally equal.

```
type  $\tau$  function_symbol = ..
type  $\tau$  function_symbol +=
  | True : ['boolean] function_symbol
  | Const : Z.t  $\rightarrow$  ['integer] function_symbol
  | Not : (['boolean]  $\rightarrow$  ['boolean]) function_symbol
  | And : (['boolean]  $\rightarrow$  ['boolean]  $\rightarrow$  ['boolean]) function_symbol
  | Or : (['boolean]  $\rightarrow$  ['boolean]  $\rightarrow$  ['boolean]) function_symbol
  | Add : (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['integer]) function_symbol
  | Sub : (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['integer]) function_symbol
  | Mul : (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['integer]) function_symbol
  | Div : (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['integer]) function_symbol
  | Eq : (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['boolean]) function_symbol
  | Leq : (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['boolean]) function_symbol
  | Unknown:  $\tau$  Typ.t  $\rightarrow$   $\tau$  function_symbol
type  $\tau$  t =
  | Ar0:{ id:int; symb: $\tau$  function_symbol; }  $\rightarrow$   $\tau$  t
  | Ar1:{ id:int; symb:( $\tau \rightarrow \tau_2$ ) function_symbol; arg: $\tau$  t; }  $\rightarrow$   $\tau_2$  t
  | Ar2:{ id:int; symb:( $\tau \rightarrow \tau_2 \rightarrow \tau_3$ ) function_symbol;
        arg1:  $\tau$  t; arg2:  $\tau_2$  t; }  $\rightarrow$   $\tau_3$  t
```

We group terms according to the arity of the function symbol: e.g., terms whose symbol has arity 2 point to 2 subterms. The function symbol is typed using a GADT, and used as a tag in the term.

```

val to_int:  $\tau$  t  $\rightarrow$  int
val polyeq:  $\tau$  t  $\rightarrow$   $\tau_2$  t  $\rightarrow$  ( $\tau, \tau_2$ ) PatriciaTree.cmp
val typeof:  $\tau$  t  $\rightarrow$   $\tau$  Typ.t
type ext_pp = { pp :  $\tau$ . Format.formatter  $\rightarrow$   $\tau$  t  $\rightarrow$  unit; }
val pp: ext_pp  $\rightarrow$  Format.formatter  $\rightarrow$   $\tau$  t  $\rightarrow$  unit
type any = Any:  $\tau$  t  $\rightarrow$  any [@@unboxed];;
val phys_equal:  $\tau$  t  $\rightarrow$   $\tau_2$  t  $\rightarrow$  bool
end

```

We need some utility functions for our terms. `to_int` and `polyeq` return unique identifier for the term (i.e., a value number (Rosen et al., 1988)), and compare terms returning a type equality witness; they allow our terms to be used as keys of a PatriciaTree. `typeof` returns a representation of the return type of term. `pp` pretty-prints the term given a function that can pretty-print the subterms (this open recursion allows the term to be extensible). The `any` type is an existential wrapper around the term type, which simplifies code that does not need type information of the term. `phys_equal` is syntactic equality between terms (which amount to `==`, as the terms are hash-consed; but unlike `==`, it allows comparison between terms of different types).

```

val term_id_count: int ref
val reset: unit  $\rightarrow$  unit

```

We use and print the terms in several chapters, especially Chapter 3 and Chapter 4. Between analyses, we `reset` the counter giving ids to terms so that term ids remain small, and the two analyses of Chapter 4 have the same ids for terms. We also make `term_id_count` available for when we extend the terms to include variables.

```

module Build_Integer_And_Boolean_Expressions:
  INTEGER_AND_BOOLEAN_EXPRESSIONS with type  $\tau$  expr =  $\tau$  Term.t

```

`Build_Integer_And_Boolean_Expressions` is a set of smart constructors building the terms with new ids if needed, and taking care of the hash-consing. For instance, `add t1 t2` builds a term representing the addition of subterms t_1 and t_2 .

B.2 Implementation

The implementation is unsurprising. We begin by the definition of term and definition of helper functions:

```

open Ch1_language

module Term = struct
  type  $\tau$  function_symbol = ..
  type  $\tau$  function_symbol +=
    | True: ['boolean] function_symbol
    | Const: Z.t  $\rightarrow$  ['integer] function_symbol
    | Not: (['boolean]  $\rightarrow$  ['boolean]) function_symbol
    | And: (['boolean]  $\rightarrow$  ['boolean]  $\rightarrow$  ['boolean]) function_symbol
    | Or: (['boolean]  $\rightarrow$  ['boolean]  $\rightarrow$  ['boolean]) function_symbol
    | Add: (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['integer]) function_symbol
    | Sub: (['integer]  $\rightarrow$  ['integer]  $\rightarrow$  ['integer]) function_symbol

```

```

| Mul: ([‘integer] → [‘integer] → [‘integer]) function_symbol
| Div: ([‘integer] → [‘integer] → [‘integer]) function_symbol
| Eq: ([‘integer] → [‘integer] → [‘boolean]) function_symbol
| Leq: ([‘integer] → [‘integer] → [‘boolean]) function_symbol
| Unknown:  $\tau$  Typ.t →  $\tau$  function_symbol

type  $\tau$  t =
| Ar0: {id:int; symb: $\tau$  function_symbol} →  $\tau$  t
| Ar1: {id:int; symb:( $\tau$  →  $\tau_2$ ) function_symbol; arg: $\tau$  t} →  $\tau_2$  t
| Ar2: {id:int; symb:( $\tau$  →  $\tau_2$  →  $\tau_3$ ) function_symbol; arg1: $\tau$  t; arg2: $\tau_2$ 
t} →  $\tau_3$  t

let get_id = function
| Ar0{id;_} → id
| Ar1{id;_} → id
| Ar2{id;_} → id
let to_int = get_id

type any = Any:  $\tau$  t → any [@@unboxed];;
let phys_equal:  $\tau$  t →  $\tau_2$  t → bool = fun a b → (Any a) == (Any b);;

let typeof (type a) (t:a t) : a Typ.t = match t with
| Ar0{symb=True;_} → Typ.boolean
| Ar0{symb=Const _;_} → Typ.integer
| Ar1{symb=Not;_} → Typ.boolean
| Ar2{symb=And;_} → Typ.boolean
| Ar2{symb=Or;_} → Typ.boolean
| Ar2{symb=Add;_} → Typ.integer
| Ar2{symb=Sub;_} → Typ.integer
| Ar2{symb=Mul;_} → Typ.integer
| Ar2{symb=Div;_} → Typ.integer
| Ar2{symb=Eq;_} → Typ.boolean
| Ar2{symb=Leq;_} → Typ.boolean
| Ar0{symb=Unknown typ;_} → typ
| _ → assert false

let polyeq (type a) (a:a t) (type b) (b:b t) : (a,b) PatriciaTree.cmp =
if phys_equal a b then Typ.polyeq (typeof a) (typeof b) else Diff

type ext_pp = {pp:  $\tau$ . Format.formatter →  $\tau$  t → unit}

let pp: type a. ext_pp → Format.formatter → a t → unit =
let fprintf = Format.fprintf in
fun {pp} fmt → function
| Ar0{symb=True;_} → fprintf fmt "True"
| Ar0{symb=Const k;_} → fprintf fmt "%s" (Z.to_string k)
| Ar0{symb=Unknown typ;id} when Typ.equal typ boolean → fprintf fmt
"unknown_boolean_%d" id
| Ar0{symb=Unknown typ;id} when Typ.equal typ integer → fprintf fmt
"unknown_integer_%d" id
| Ar1{symb=Not;arg;_} → fprintf fmt "not(%a)" pp arg
| Ar2{symb=And;arg1;arg2;_} → fprintf fmt "(%a && %a)" pp arg1 pp arg2
| Ar2{symb=Or;arg1;arg2;_} → fprintf fmt "(%a || %a)" pp arg1 pp arg2
| Ar2{symb=Add;arg1;arg2;_} → fprintf fmt "%a + %a" pp arg1 pp arg2

```

```

| Ar2{symb=Sub;arg1;arg2;_} → fprintf fmt "%a - %a" pp arg1 pp arg2
| Ar2{symb=Mul;arg1;arg2;_} → fprintf fmt "%a * %a" pp arg1 pp arg2
| Ar2{symb=Div;arg1;arg2;_} → fprintf fmt "%a / %a" pp arg1 pp arg2
| Ar2{symb=Eq;arg1;arg2;_} → fprintf fmt "%a = %a" pp arg1 pp arg2
| Ar2{symb=Leq;arg1;arg2;_} → fprintf fmt "%a ≤ %a" pp arg1 pp arg2
| _ → assert false
end

```

Now, we define some global variables and code to perform the hash-consing: in particular, a global counter for ids, and a generic memoization function. This function creates a hash table from equal and hash functions, and provides a hashconsing function to its argument. It also registers a resetting function inside the reset_hook.

Note that this creates several different hash-consing functions, one for each kind of term (this is especially useful in the case of concurrent hash-consing functions to diminish contention).

```

let term_id_count = ref 0

let reset_hook = ref []

let memo (type a) ~equal ~hash f =
  let module WeakHash = Weak.Make(struct
    type t = a
    let equal = equal
    let hash = hash
  end) in
  let cache = WeakHash.create 8 in
  reset_hook := (fun () → WeakHash.clear cache)::!reset_hook;
  let hashcons = fun tentative →
    incr term_id_count;
    (* We could increment only when res == tentative, but then the ids
       do not match between the different analyses. *)
    WeakHash.merge cache tentative
  in f ~hashcons

```

We use this hash-consing function to create our smart constructors building the term graph:

```

module Build_Integer_And_Boolean_Expressions:
  INTEGER_AND_BOOLEAN_EXPRESSIONS
  with type τ expr = τ Term.t
= struct
  include Term
  type τ expr = τ Term.t
  type boolean = ['boolean] expr
  type integer = ['integer] expr

  (* True is a constant, so we build it once and do not need hash-
     consing. *)
  let true = Ar0{id= !term_id_count;symb=True};;
  incr term_id_count;;

  (* Each call to unknown_* returns a different term: no hash-consing.
     *)

```

```

let unknown typ =
  let id = !term_id_count in incr term_id_count;
  Ar0{id;symb=Unknown typ}
let unknown_boolean() = unknown Typ.boolean
let unknown_integer() = unknown Typ.integer

let const =
  let equal a b = match a,b with
    | Ar0{symb=Const a;_},Ar0{symb=Const b;_} → Z.equal a b
    | _ → assert false in
  let hash = function
    | Ar0{symb=Const a;_} → Z.hash a
    | _ → assert false in
  memo ~equal ~hash (fun ~hashcons z → hashcons @@ Ar0{id= !term_id_
count;symb=Const z})

let ¬ =
  let equal a b = match a,b with
    | Ar1{arg=a_arg;_},Ar1{arg=b_arg;_} → phys_equal a_arg b_arg
    | _ → assert false in
  let hash = function
    | Ar1{arg;_} → (get_id arg)
    | _ → assert false in
  memo ~equal ~hash (fun ~hashcons arg → hashcons @@ Ar1{id= !term_id_
count;symb=Not;arg})

let binop =
  let equal a b = match a,b with
    | Ar2{arg1=a_arg1;arg2=a_arg2;_},Ar2{arg1=b_arg1;arg2=b_arg2;_} →
      phys_equal a_arg1 b_arg1 ∧ phys_equal a_arg2 b_arg2
    | _ → assert false in
  let sdbm x y = y + (x lsl 16) + (x lsl 6) - x in
  let hash = function
    | Ar2{arg1;arg2;_} → sdbm (get_id arg1) (get_id arg2)
    | _ → assert false in
  fun symb →
    memo ~equal ~hash (fun ~hashcons arg1 arg2 → hashcons @@ Ar2{id= !
term_id_count;symb;arg1;arg2})
;;

let ∧ = binop And
let ∨ = binop Or
let add = binop Add
let sub = binop Sub
let mul = binop Mul
let div = binop Div
let eq = binop Eq
let leq = binop Leq
end

```

Finally, the reset function iterates on all functions on the reset hook, and reset `term_id_count`, but not to zero: because some constant terms are already created, like `true`, we need to reset to the last id used to create the constants.

```
let reset =  
  (* Last id used to create the constants. *)  
  let reset_to = !term_id_count in  
  fun () →  
    term_id_count := reset_to;  
    !reset_hook |> List.iter (fun f → f())
```

Appendix C

Module Cfg2dot

```
module CFG2Dot = struct
  open Cfg
```

This code has been used to create Figure 3.4.1. It just traverses the graph using a depth-first search algorithm, and prints a description of each node and edge to be processed by the graphviz tool. We need a collection to mark already visited nodes in the DFS; for this we just use a hash on the label of each node, as each label is unique to each node.

```
let rec show = function
| Entry → "Entry"
| Loop_head { loop_id; _ } → "Loop_head " ^ string_of_int loop_id
| Join (a, b) → "Join((" ^ show a ^ "), (" ^ show b ^ ")")
| Assume (cond, node) →
  Format.asprintf "%s; assume(%a)" (show node) Term_With_ProgVar.pp
  cond
| Assign (var, exp, node) →
  Format.asprintf "%s; %a := %a" (show node) ProgVar.pp var
  Term_With_ProgVar.pp exp;;

let dfs f node =
  let cache = Hashtbl.create 17 in
  let rec loop elt =
    let id = show elt in
    if not @@ Hashtbl.mem cache id then (
      Hashtbl.replace cache id ();
      f elt;
      predecessors elt |> List.iter loop)
  in
  loop node;;

let dump_graph filename final_node =
  let output = open_out filename in
  let f node =
    Printf.fprintf output "\"%s\" [label=\"%s\"]\n"
      (show node) (show node);
```

```

match node with
| Assume (cond, _) →
  List.iter
    (fun pred →
      Printf.fprintf output "\"%s\" -> \"%s\" [label=\"%s\"]\n"
        (show pred) (show node)
        (Format.asprintf "%a" Term_With_ProgVar.pp cond))
    (predecessors node)
| Assign (var, exp, _) →
  List.iter
    (fun pred →
      Printf.fprintf output "\"%s\" -> \"%s\" [label=\"%s\"]\n"
        (show pred) (show node)
        (Format.asprintf "%a := %a"
          ProgVar.pp var Term_With_ProgVar.pp exp))
    (predecessors node)
| _ →
  List.iter
    (fun pred →
      Printf.fprintf output "\"%s\" -> \"%s\"\n"
        (show pred) (show node))
    (predecessors node)
in
match final_node with
| None → assert false
| Some s →
  Printf.fprintf output "digraph {\n";
  ignore (dfs f s);
  Printf.fprintf output "}\n%!";
  close_out output
end;;

```

Appendix D

Module Symbolic_Store_2_Dot

```
module Symbolic_Store_2_Dot = struct
```

This section describes the code used to generate the value graph Figure 4.1 and SSA graph of Figure 4.2, using GRAPHVIZ. It is included mainly for completeness, but also characterizes the graphical representation of our terms and sea-of-node/SSA language.

```
open Term
open Term_With_Symvar

let preds t = match t with
| Ar0{symb=Var{definition=Phi(a,b);_};_} → List.map (fun x → Any x) (
  Option.to_list a @ Option.to_list b)
| Ar0{symb=Var{definition=Head(d);_};_} → List.map (fun x → Any x) (
  Option.to_list d)
| Ar0 _ → []
| Ar1{arg;_} → [Any arg]
| Ar2{arg1;arg2;_} → [Any arg1;Any arg2]
```

Each of our node contains a link to the predecessor node (generally its subterms, but for recursion variables (Ariola and Klop, 1996), it is the definition.

```
module Term_Set = PatriciaTree.MakeHeterogeneousSet(Term_With_Symvar)

let rec dfs f cache (Any node) =
  if Term_Set.mem node cache
  then cache
  else
    let cache = Term_Set.add node cache in
    f (Any node);
    List.fold_left (dfs f) cache (preds node)
```

To iterate on every node, we need to perform a depth-first search (we do it in reverse order, starting from a "last" node. We use a heterogeneous set to remember the nodes that we already have traversed.

```

let show x = Format.asprintf "%a" Term_With_Symvar.pp x
let pp_sep fmt () = Format.fprintf fmt ", "
let show_definition from_loop definition = match definition with
| Phi(a,b) →
    Format.asprintf "φ(%a)"
      (Format.pp_print_list ~pp_sep Term_With_Symvar.pp)
      (Option.to_list a @ Option.to_list b)
| Head(a) →
    Format.asprintf "head%s(%t)"
      (show_from_loop from_loop)
      (fun fmt → Format.pp_print_option Term_With_Symvar.pp fmt a)

```

Some pretty printer for terms, and for recursion variables, of their definition.

```

let dump_symbolic_store file symbolic_store =
  let outc = open_out file in
  let print fmt = Printf.fprintf outc fmt in
  let control_state_list = ref [] in

```

This is the main printing function. It does two things simultaneously: print the node and edges for every term, but also collect all the control states to be displayed later as clusters.

```

let rec add_control_state control_state =
  if List.memq control_state !control_state_list then ()
  else begin
    control_state_list := control_state::!control_state_list;
    match control_state with
    | Join {predecessors=(pred1,_),(pred2,_)} →
        add_control_state pred1;
        add_control_state pred2
    | Loop_head {definition=(pred,_)} → add_control_state pred
    | _ → ()
  end in

```

Whenever we see a control state, we add it and all its predecessors.

```

(* On each node. *)
let do_node (Any node) =
  begin match node with
  (* Recursion variable *)
  | Ar0{symb=Var{definition;control_state;from_loop;_};_} →
      print "%d [penwidth=3,style=filled,fillcolor=white,color=black,\n"
        label="%s = %s"]\n"
        (Term.to_int node) (show node)
        (show_definition from_loop definition);
      begin match control_state with
      | No_control_state → ()
      | _ → add_control_state control_state
      end
  (* Normal term. *)
  | _ → print "%d [label=\"%s\"]\n" (Term.to_int node) (show node)
  end

```

```

end;
(preds node) |> List.iter (fun (Any pred) →
  print "\"%d\" -> \"%d\"\\n" (Term.to_int pred) (Term.to_int node)) in

```

We print each node, collect the control state for recursion variables, and display the link between each node and its subterm or definition.

```

begin
  print "digraph {\n%!";
  print "ranksep=1;\n%!";
  print "compound=true;\n%!";
  print "newrank=true;\n%!";

```

We now start the main printing algorithm. First, we print a preamble.

```

let f _key value visited_nodes =
  print "%d [xlabel=\"%s\"]\n%!\" (Term.to_int value) (Format.asprintf \"%a\"
    ProgVar.pp _key);
  dfs do_node visited_nodes (Any value) in
let _ = Symbolic_Store.fold {f} symbolic_store (Term_Set.empty) in

```

Then, we handle all the entries in the symbolic store; for each we do a depth-first search. Note that we accumulate the set of visited nodes between each depth-first search, as some subterms between may be shared.

```

!control_state_list |> List.iteri (fun i cluster →
  print "subgraph cluster_%d {\n\" i;
  print "style=filled; color=lightgrey; rank=same;\n%!";
  (* Invisible node to ensure cluster are there and we can connect
     them *)
  print "invisible_%d [shape=point,style=invis]\n%!\" i;
  let  $\phi$ , label = match cluster with
  | Loop_head {id; $\phi$ } →  $\phi$ , "loop_head_" ^ (string_of_int id)
  | Join { $\phi$ } →  $\phi$ , "join"
  | Entry → Phi.empty, "entry"
  | _ → assert false in
  Phi.fold {f=(fun _key1 _key2 value () →
    let () = print "%d \" (Term.to_int value) in
    ()
  )}  $\phi$  ();
  print "\n label = \"%s\"; }\n\" label
);

```

As we have visited every term and subterm, we have accumulated a set of control states, that we can now print.

```

!control_state_list |> List.iteri (fun i cluster →
  let print_edge_from_predecessor (pred,cond) =
    let Some idx =
      List.find_index ((=) pred) !control_state_list in
    print "invisible_%d -> invisible_%d \
      [penwidth=5,color=black,ltail=cluster_%d,\
        lhead=cluster_%d,label=\"%s\"]\n%!\"

```

```

        idx i idx i (Format.asprintf "%a" pp_condition cond) in
    match cluster with
    | Join{predecessors = (pred1,pred2)} →
        print_edge_from_predecessor pred1;
        print_edge_from_predecessor pred2
    | Loop_head { $\phi$ ;definition} →
        print_edge_from_predecessor definition
    | _ → ();
    print "}\n!";
    close_out outc
end
end;;

```

Finally, we can print the edges between the different clusters, and the closing text.

Appendix E

Module Term_Option_Pair_Map

Here we describe the `Term_Option_Pair_Map` module that maps two terms to one term, build upon the `PatriciaTree` and `Term` modules. It is used to represent a renaming, the result of an anti-unification operation.

```
module Term_Option_Pair_Map: sig
  type t
  val empty: t
  val mem:  $\tau$  Term.t option  $\rightarrow$   $\tau$  Term.t option  $\rightarrow$  t  $\rightarrow$  bool
  val find:  $\tau$  Term.t option  $\rightarrow$   $\tau$  Term.t option  $\rightarrow$  t  $\rightarrow$   $\tau$  Term.t
  val add:  $\tau$  Term.t option  $\rightarrow$   $\tau$  Term.t option  $\rightarrow$   $\tau$  Term.t  $\rightarrow$  t  $\rightarrow$  t
  type  $\delta$  polyfold =
    { f:  $\tau$ .  $\tau$  Term.t option  $\rightarrow$   $\tau$  Term.t option  $\rightarrow$   $\tau$  Term.t  $\rightarrow$   $\delta$   $\rightarrow$   $\delta$  }
  [@@unboxed]
  val fold:  $\tau$  polyfold  $\rightarrow$  t  $\rightarrow$   $\tau$   $\rightarrow$   $\tau$ 
  val mem1:  $\tau$  Term.t option  $\rightarrow$  t  $\rightarrow$  bool
end =
```

The `Term_Option_Pair_Map` module can be seen as an heterogeneous map from pairs of terms to a term. Mostly we follow the same signature than in OCaml's stdlib of the `PatriciaTree` module, but we pass two keys argument instead of one when needed. In the signature, `empty` returns an empty map; `find` retrieves the value associated with two keys in a map, and `mem` just tests if this value exists; `add` adds a binding to an existing map; `fold` iterates on the bindings if the map (encapsulating the argument in a record because the heterogeneity). We also have a function `mem1`, where `mem1 t1 m` tests if there is any binding $(t_1, t_2) \mapsto t$ in m .

```
struct
  module Key = struct
    type  $\tau$  t =  $\tau$  Term.t option
    let to_int = function None  $\rightarrow$  -1 | Some a  $\rightarrow$  Term.to_int a
    let polyeq a b : ( $\tau, \tau_2$ ) PatriciaTree.cmp = match a, b with
      | None, None  $\rightarrow$  (Obj.magic PatriciaTree.Eq)
      | None, _ | _, None  $\rightarrow$  Diff
      | Some a, Some b  $\rightarrow$  Term_With_Symvar.polyeq a b
  end
end
```

We extend the comparison functions on keys to option types. Note that the `polyeq` function is not type-safe: from two `None` values, we cannot return a proof that they correspond to the same type. For this we should use a special option type where `None` also includes a proof that it is of the correct type.

Thus, we could for instance create a map `m` with a binding $(\text{None}, \text{None}) \mapsto \text{true}$, and use `find` on `m` with the keys $(\text{None}, \text{None})$ in a context where the term is expected to be an `[integer] Term.t`, resulting in an unsafe cast. To prevent that, we assert that we never create a binding where both keys are `None`. This guarantees type safety, using the invariant that the type of every value correspond to the type of at least one of the keys: all the uses of this module are type-safe.

```
module Value2 = struct
  type ( $\kappa, \mu$ ) t =  $\kappa$  Term.t
end
module Map2 = PatriciaTree.MakeHeterogeneousMap(Key)(Value2)
module Value1 = struct
  type ( $\kappa, \text{'value}$ ) t = unit Map2.t
end
module Map1 = PatriciaTree.MakeHeterogeneousMap(Key)(Value1)
type t = unit Map1.t
```

We implement the map of pairs of term to term, as a map from terms to a map of terms to terms.

```
type  $\delta$  polyfold =
  { f:  $\tau$ .  $\tau$  Term.t option  $\rightarrow$   $\tau$  Term.t option  $\rightarrow$   $\tau$  Term.t  $\rightarrow$   $\delta \rightarrow \delta$  }
  [@@unboxed]

let fold (type acc) ({f}:acc polyfold) map (acc:acc) :acc =
  let f: type a. a Term.t option  $\rightarrow$  unit Map2.t  $\rightarrow$  acc  $\rightarrow$  acc =
    fun (key1:a Term.t option) (map2:unit Map2.t) acc  $\rightarrow$ 
      let f: type b. b Term.t option  $\rightarrow$  b Term.t  $\rightarrow$  acc  $\rightarrow$  acc =
        fun (key2:b Term.t option) value acc  $\rightarrow$ 
          let key1: b Term.t option = Obj.magic key1 in
            f key1 key2 value acc in
    Map2.fold {f} map2 acc in
  Map1.fold {f} map acc
```

The untyped semantics of what the `fold` function does is not difficult: it just iterates on all the bindings in both maps, recursively. What is difficult is to make this function type-check. The problem is that both `Map1.fold` and `Map2.fold` require an explicitly-polymorphic function on their keys, but they have to call the `f` function argument, that will unify the type of both keys (and of the value).

Actually, `Map2` is not an heterogeneous map – it is a polymorphic homogeneous map, i.e. a “ τ map” mapping “ τ keys” to “ τ values”. Folding on such a map does not require an explicitly polymorphic function. We should thus extend `PatriciaTree` to provide this type of map. Here we use `Obj.magic` instead: this is safe as types `a` and `b` are indeed the same type. Alternatively, we could also use a dynamic check returning an equality witness asserting that the type of both keys are the same.

```
let pp_term_option fmt = function
  | None  $\rightarrow$  Format.fprintf fmt "None"
```

```

| Some x → Term_With_Symvar.pp fmt x

let pp_map2 fmt map2 =
  let f key value = Format.fprintf fmt "%a -> %a\n%" pp_term_option key
    Term_With_Symvar.pp value in
  Map2.iter {f} map2

let pp fmt map =
  let f key map2 = Format.fprintf fmt "%a -> (%a)\n%" pp_term_option key
    pp_map2 map2 in
  Map1.iter {f} map

```

The pretty-printing functions are not surprising.

```

let find (key1:τ Term.t option) (key2:τ Term.t option) map =
  match Map1.find key1 map with
  | exception Not_found → raise Not_found
  | _ → Map2.find key2 (Map1.find key1 map);;
let mem key1 key2 map =
  match find key1 key2 map with
  | exception Not_found → false | _ → true
let mem1 key1 map = Map1.mem key1 map
let empty = Map1.empty

let add: τ Term.t option → τ Term.t option → τ Term.t → t → t =
  fun key1 key2 value map1 →
    assert(key1 != None ∨ key2 != None);
    let map2 = match Map1.find key1 map1 with
      | exception Not_found → Map2.singleton key2 value
      | map2 → Map2.add key2 value map2
    in Map1.add key1 map2 map1
end

```

The implementation of functions on the map are also not surprising. Just note that, as mentioned earlier, we ensure that there are no maps for which keys are both None, to ensure type safety.

Appendix F

Standard non-relational abstract interpretation

In Section 1.3 we introduced the concepts of variable-based and non-relational analyses, and in Chapter 3 we introduced the module type `ABSTRACT_DOMAIN` used to implement variable-based static analyzers. This chapter implements an example of such analyses. For this, we first implement the `Interval_Single_Value_Abstraction`, which is reused in the value-based static analysis of Chapter 5.

F.1 Module `Single_Value_Abstraction`

As seen in Chapter 5, a single-value abstraction is a combination of lattices, each describing a set of values of some type; and forward and backward transfer functions, that combine lattice elements to form new lattice elements. In addition, we have query, which allows the analysis to make decision based on the current state of the analysis.

The Quadrivalent abstraction

```
module Quadrivalent = struct

  module Lattice = struct
    type t = Bottom | True | False | Top
    let pp fmt x = Format.fprintf fmt "%s" (match x with
      | Bottom → "Bottom" | True → "True" | False → "False" | Top → "Top")

    let join a b = match (a,b) with
      | Bottom, x | x, Bottom → x
      | False, False → False
      | True, True → True
      | _ → Top

    let fixpoint_step older newer =
      let grow = join older newer in grow, grow = older
```

```

let inter a b = match a,b with
| Bottom, _ | _, Bottom → Bottom
| Top, x | x, Top → x
| True, False | False, True → Bottom
| True, True → True | False, False → False
end
include Lattice

```

A constant boolean expression either returns true (e.g. for “ $10 < 12$ ”), false (for “ $12 < 10$ ”), or does not return (for “ $3/0 < 10$ ”). Given a boolean expression, the set of all its possible return values in any state is $\{\}$ (for “ $x/0 < 10$ ”), $\{\text{true}\}$ (for “ $x * 0 = 0$ ”), $\{\text{false}\}$ (for “ $x * 2 = 1$ ”), or $\{\text{true}, \text{false}\}$ (for “ $x \leq 3$ ”). Thus, there are only 4 possible set of return values for boolean expressions; the set of set of return values for boolean expressions is finite. This set of set of return values is thus directly representable without the need for an abstraction; the following defines a computable representation isomorphic to $\mathcal{P}(\{\text{true}, \text{false}\})$, that we call a quadri-valent value, (by analogy with the different trivalent values, and the fact that there are 4 elements in this set).

We implement standard operations on this lattice: join ($\sqcup$), inter ($\sqcap$), combined widening and inclusion testing (where we do not need to widen as the lattice is finite).

```

module Forward = struct
  open Lattice
  let true = True
  let ¬ = function
    | Bottom → Bottom | True → False | False → True | Top → Top
  let ∧ a b = match a,b with
    | Bottom, _ | _, Bottom → Bottom
    | False, _ | _, False → False
    | True, True → True
    | Top, True | True, Top | Top, Top → Top
  let ∨ a b = match a,b with
    | Bottom, _ | _, Bottom → Bottom
    | True, _ | _, True → True
    | False, False → False
    | Top, False | False, Top | Top, Top → Top
end

module Backward = struct
  open Lattice

  let ¬ arg res =
    let notres = Forward.¬ res in
    let newarg = Lattice.inter arg notres in
    if newarg = arg then None else Some(newarg)

  let ∧ a b res =
    let a',b' = match a,b,res with
      | _,_,(Bottom|Top) → a,b
      | _,_,True → True,True
      | True,_,False → True,False
    in

```

```

    | _, True, False → False, True
    | _ → a, b in
  let a' = Lattice.inter a a' and b' = Lattice.inter b b' in
  (if a = a' then None else Some a'),
  (if b = b' then None else Some b')

let v a b res =
  let a', b' = match a, b, res with
    | _, _, (Bottom|Top) → a, b
    | _, _, False → False, False
    | False, _, True → False, True
    | _, False, True → True, False
    | _ → a, b in
  let a' = Lattice.inter a a' and b' = Lattice.inter b b' in
  (if a = a' then None else Some a'),
  (if b = b' then None else Some b')
end
end

```

As the quadrivalent lattice exactly represents $\mathcal{P}(\{\text{true}, \text{false}\})$, we can easily implement the best Forward and Backward transfer functions, which are sound and exact.

The Interval abstraction

We can now build our interval abstraction, that builds upon `Quadrivalent`. First, to represent possibly-unbounded intervals, we need to have a representation of infinite bounds:

```

module Integer_Or_Inf = struct
  type t =
    | Ninf (* Negative infinity *)
    | Pinf (* Positive infinity. *)
    | Finite of Z.t (* Finite value. *)
  let pp fmt = function
    | Ninf → Format.fprintf fmt "-oo"
    | Finite x → Format.fprintf fmt "%s" (Z.to_string x)
    | Pinf → Format.fprintf fmt "+oo"
  let min a b = match a, b with
    | Ninf, _ | _, Ninf → Ninf
    | x, Pinf | Pinf, x → x
    | Finite a, Finite b → Finite(min a b)
  let max a b = match a, b with
    | Pinf, _ | _, Pinf → Pinf
    | x, Ninf | Ninf, x → x
    | Finite a, Finite b → Finite(max a b)
  let add a b = match a, b with
    | Ninf, Pinf | Pinf, Ninf → assert false
    | Ninf, _ | _, Ninf → Ninf
    | Pinf, _ | _, Pinf → Pinf
    | Finite a, Finite b → Finite(Z.add a b)
  let neg = function
    | Ninf → Pinf
    | Pinf → Ninf

```

```

| Finite a → Finite(Z.neg a)
let mul x y =
  match x, y with
  | Ninf, Ninf | Pinf, Pinf → Pinf
  | Ninf, Pinf | Pinf, Ninf → Ninf
  | Finite v1, Finite v2 → Finite (Z.mul v1 v2)
  | (Ninf | Pinf as x), Finite v | Finite v, (Ninf | Pinf as x) →
    if Z.gt v Z.zero then x
    else if Z.lt v Z.zero then neg x
    else Finite Z.zero
let equal a b = match a,b with
| Ninf, Ninf | Pinf, Pinf → true
| Finite a, Finite b → Z.equal a b
| _ → false
let pred = function Pinf → Pinf | Ninf → Ninf | Finite x → Finite (Z.
pred x)
let succ = function Pinf → Pinf | Ninf → Ninf | Finite x → Finite (Z.
succ x)
end

```

The above code defines basic mathematical functions over extended numbers in $\mathbb{Z} \cup \{+\infty, -\infty\}$. We use these to define intervals as pairs of such numbers:

```

module Interval = struct
  (* Constraint: the min cannot be Pinf, the max cannot be Ninf. *)
  type t = Integer_Or_Inf.t * Integer_Or_Inf.t
  open Integer_Or_Inf
  let pp fmt (min,max) = Format.fprintf fmt "[%a,%a]"
    Integer_Or_Inf.pp min Integer_Or_Inf.pp max
  let top = (Ninf,Pinf)
  let const a = let ext = Finite a in (ext,ext)
  let join (mina,maxa) (minb,maxb) = (min mina minb, max maxa maxb)
  let inter (mina,maxa) (minb,maxb) = (max mina minb, min maxa maxb)
  let fixpoint_step (prev_min,prev_max) (next_min,next_max) =
    (* Compare the bounds: if smaller, directly go to infinite. *)
    let newmax,included1 = match prev_max,next_max with
    | Pinf, _ → next_max, true
    | Finite _, Pinf → Pinf, false
    | Finite prev, Finite next →
      if Z.lt prev next then Pinf,false
      else next_max, true
    | Ninf, _ | _, Ninf → assert false in
    let newmin,included2 = match prev_min,next_min with
    | Ninf, _ → next_min, true
    | Finite _, Ninf → Ninf, false
    | Finite prev, Finite next →
      if Z.lt next prev then Ninf,false
      else next_min, true
    | Pinf, _ | _, Pinf → assert false
    in (newmin,newmax),included1 ^& included2

  let equal (mina,maxa) (minb,maxb) =
    Integer_Or_Inf.equal mina minb ^& Integer_Or_Inf.equal maxa maxb
end

```

The above code defines the classical lattice of interval (Cousot and Cousot, 1977).

```

module Interval_Forward = struct
  open Integer_Or_Inf
  let add (mina,maxa) (minb,maxb) = (add mina minb, add maxa maxb)
  let neg (min,max) = (neg max, neg min)
  let sub itva itvb = add itva (neg itvb)
  let mul (mina,maxa) (minb,maxb) =
    let x = mul mina minb in
    let y = mul mina maxb in
    let z = mul maxa minb in
    let t = mul maxa maxb in
    let res_min = min x (min y (min z t)) in
    let res_max = max x (max y (max z t)) in
    (res_min,res_max)
  let div _ _ = Interval.top

  let leq (mina,maxa) (minb,maxb) =
    let open Quadrivalent.Lattice in
    match maxa,minb with
    | Finite maxa, Finite minb when Z.leq maxa minb → True
    | _ → begin
        match mina,maxb with
        | Finite mina, Finite maxb when Z.gt mina maxb → False
        | _ → Top
      end
    end

  let eq (mina,maxa) (minb,maxb) =
    let open Quadrivalent.Lattice in
    match mina,maxa,minb,maxb with
    | _, Finite maxa, Finite minb, _ when Z.lt maxa minb → False
    | Finite mina, _, _, Finite maxb when Z.gt mina maxb → False
    | Finite mina, Finite maxa, Finite minb, Finite maxb when
        Z.equal mina maxa ∧ Z.equal mina minb ∧ Z.equal mina maxb → True
    | _ → Top
  end
end

```

The forward transfer functions directly derive from their soundness theorems (Definition 5.1.3). In the case of div, we are sound but not computing the best approximation.

```

module Interval_Backward = struct
  open Quadrivalent

  let inter_and_compare arg refined =
    let newarg = Interval.inter arg refined in
    if Interval.equal arg newarg then None else Some (newarg)

  let eq arg1 arg2 res =
    match res with
    | Top → (None,None)
    | True → (inter_and_compare arg1 arg2, inter_and_compare arg2 arg1)
    | False → None, None
    | Bottom → None, None
end

```

```

let leq ((min1,max1) as arg1) ((min2,max2) as arg2) res =
  match res with
  | Top → (None,None)
  | True → (* arg1 <= arg2. *)
    (inter_and_compare arg1 (Ninf, max2),
     inter_and_compare arg2 (min1,Pinf))
  | False → (* arg1 > arg2. *)
    (inter_and_compare arg1 (Integer_Or_Inf.succ min2,Pinf),
     inter_and_compare arg2 (Ninf,Integer_Or_Inf.pred max1))
  | Bottom → None,None

let default_ar2 _ _ _ = (None,None)
let mul = default_ar2
let div = default_ar2
let add arg1 arg2 res =
  (inter_and_compare arg1 (Interval_Forward.sub res arg2),
   inter_and_compare arg2 (Interval_Forward.sub res arg1))
let sub arg1 arg2 res =
  (inter_and_compare arg1 (Interval_Forward.add res arg2),
   inter_and_compare arg2 (Interval_Forward.sub arg1 res))
end

```

Similarly, our backward transfer functions derive from the definitions in Definition 5.1.4, although we grossly over-approximate some of them.

The Interval_Single_Value_Abstraction

```

module type INTEGER_SINGLE_VALUE_ABSTRACTION = sig
  include INTEGER_AND_BOOLEAN_EXPRESSIONS
  val to_quadrivalent: ['boolean] expr → Quadrivalent.t
  val to_interval: ['integer] expr → Interval.t
  val pp: τ Typ.t → Format.formatter → τ expr → unit
  val join: τ Typ.t → τ expr → τ expr → τ expr
  val inter: τ Typ.t → τ expr → τ expr → τ expr
  val fixpoint_step: τ Typ.t → τ expr → τ expr → τ expr * bool
  module Backward:sig
    val ¬: ['boolean] expr → ['boolean] expr →
      ['boolean] expr option
    val ∧: ['boolean] expr → ['boolean] expr → ['boolean] expr →
      ['boolean] expr option * ['boolean] expr option
    val ∨: ['boolean] expr → ['boolean] expr → ['boolean] expr →
      ['boolean] expr option * ['boolean] expr option
    val eq: ['integer] expr → ['integer] expr → ['boolean] expr →
      ['integer] expr option * ['integer] expr option
    val leq: ['integer] expr → ['integer] expr → ['boolean] expr →
      ['integer] expr option * ['integer] expr option
    val add: ['integer] expr → ['integer] expr → ['integer] expr →
      ['integer] expr option * ['integer] expr option
    val sub: ['integer] expr → ['integer] expr → ['integer] expr →
      ['integer] expr option * ['integer] expr option
    val mul: ['integer] expr → ['integer] expr → ['integer] expr →
      ['integer] expr option * ['integer] expr option
  end
end

```

```

    val div: ['integer] expr → ['integer] expr → ['integer] expr →
      ['integer] expr option * ['integer] expr option
  end
end

```

An `INTEGER_SINGLE_VALUE_ABSTRACTION` is the combination of a boolean lattice, an integer lattice, and the forward and backward transfer functions listed in `INTEGER_AND_BOOLEAN_EXPRESSIONS`. We implement one example of `INTEGER_SINGLE_VALUE_ABSTRACTION`, the `Interval_Single_Value_Abstraction`, where the boolean lattice is `Quadrivalent` and the integer lattice is `Interval`.

```

module Interval_Single_Value_Abstraction: INTEGER_SINGLE_VALUE_ABSTRACTION =
struct
  type  $\tau$  expr;;
  type boolean = ['boolean] expr
  external of_quadrivalent: Quadrivalent.t → ['boolean] expr = "%identity"
  external to_quadrivalent: ['boolean] expr → Quadrivalent.t = "%identity"
  external of_interval: Interval.t → ['integer] expr = "%identity"
  external to_interval: ['integer] expr → Interval.t = "%identity"

```

We need to use a single τ expr type so that `['boolean] expr` corresponds to `Quadrivalent.t` and `['integer] expr` corresponds to `Interval.t`. The usual way to do this would be:

```

type  $\tau$  expr = Boolean: Quadrivalent.t → ['boolean] expr
           | Integer: Interval.t → ['integer] expr

```

However, at runtime this requires an extra boxing and tag that we do not need, as we do not need to discriminate based on the runtime values of a τ expr (we always already know the type of the value on which we operate). So, it is best to use a tagless type family, i.e. to view the expr type constructor as a function that directly maps `['boolean]` to `Quadrivalent.t` and `['integer]` to `Interval.t`. Unfortunately, type families like this are not directly expressible by the OCaml compiler; however, we can emulate it using abstract type and conversion functions, which is what we have done above.

```

let boolean = (of_quadrivalent,to_quadrivalent)
let integer = (of_interval,to_interval)

module Forward = struct
  let ar0 (ofres,_) v# = ofres v#
  let true = ar0 boolean Quadrivalent.Forward.true
  let unknown_boolean() = ar0 boolean Quadrivalent.Top
  let const z = ar0 integer (Interval.const z)
  let unknown_integer() = ar0 integer Interval.top

  let ar1 (ofa,toa) (ofres,tores) f a = ofres (f (toa a))
  let  $\neg$  = ar1 boolean boolean Quadrivalent.Forward. $\neg$ 

  let ar2 (_,toa) (_,tob) (ofres,_) f a b = ofres (f (toa a) (tob b))
  let  $\wedge$  = ar2 boolean boolean boolean Quadrivalent.Forward. $\wedge$ 
  let  $\vee$  = ar2 boolean boolean boolean Quadrivalent.Forward. $\vee$ 
  let eq = ar2 integer integer boolean Interval_Forward.eq

```

```

let leq = ar2 integer integer boolean Interval_Forward.leq
let add = ar2 integer integer integer Interval_Forward.add
let sub = ar2 integer integer integer Interval_Forward.sub
let mul = ar2 integer integer integer Interval_Forward.mul
let div = ar2 integer integer integer Interval_Forward.div
end
include Forward

module Backward = struct
  let ar1 (ofa,toa) (ofres,tores) f a res =
    Option.map ofa (f (toa a) (tores res));;
  let  $\neg$  = ar1 boolean boolean Quadrivalent.Backward. $\neg$ 

  let ar2 (ofa,toa) (ofb,tob) (ofres,tores) f a b res =
    let a',b' = (f (toa a) (tob b) (tores res)) in
    Option.map ofa a', Option.map ofb b'

  let  $\wedge$  = ar2 boolean boolean boolean Quadrivalent.Backward. $\wedge$ 
  let  $\vee$  = ar2 boolean boolean boolean Quadrivalent.Backward. $\vee$ 
  let eq = ar2 integer integer boolean Interval_Backward.eq
  let leq = ar2 integer integer boolean Interval_Backward.leq
  let add = ar2 integer integer integer Interval_Backward.add
  let sub = ar2 integer integer integer Interval_Backward.sub
  let mul = ar2 integer integer integer Interval_Backward.mul
  let div = ar2 integer integer integer Interval_Backward.div
end

```

This code lift all the transfer functions from [Quadrivalent](#) and [Interval](#) so that they take and return values of type τ expr. Because the conversion is identity, the untyped semantics of this code is equivalent to a direct call to the underlying function (e.g., we could do `let add = Obj.magic Interval_Forward.add`. However, the compiler can realise this, and we can implement this cleaner (but more verbose) code.

```

let pp (type a) ((module T):a Typ.t) fmt (v#:a expr) = match T.Tag with
| Typ.Boolean.Tag → Quadrivalent.pp fmt (to_quadrivalent v#)
| Typ.Integer.Tag → Interval.pp fmt (to_interval v#)

let join (type a) ((module T):a Typ.t) (v1#:a expr) (v2#:a expr) : a expr =
  match T.Tag with
  | Typ.Boolean.Tag →
    of_quadrivalent
    (Quadrivalent.join (to_quadrivalent v1#) (to_quadrivalent v2#))
  | Typ.Integer.Tag →
    of_interval (Interval.join (to_interval v1#) (to_interval v2#))

let inter (type a) ((module T):a Typ.t) (v1#:a expr) (v2#:a expr) : a expr =
  match T.Tag with
  | Typ.Boolean.Tag →
    of_quadrivalent
    (Quadrivalent.inter (to_quadrivalent v1#) (to_quadrivalent v2#))
  | Typ.Integer.Tag →
    of_interval (Interval.inter (to_interval v1#) (to_interval v2#))

let fixpoint_step

```

```

    (type a) ((module T):a Typ.t) (prev:a expr) (next:a expr) : (a expr *
    bool) =
    let doit fpstep of_type to_type =
      let (v,b) = fpstep (to_type prev) (to_type next) in (of_type v,b) in
    match T.Tag with
    | Typ.Boolean.Tag →
      doit Quadrivalent.fixpoint_step of_quadrivalent to_quadrivalent
    | Typ.Integer.Tag → doit Interval.fixpoint_step of_interval to_
      interval
    end

```

In the above code, we use the typ argument as a tag to discriminate between the different operations. The type arguments here are necessary to help the OCaml type checker, due to the use of GADTs.

F.1.1 A Standard_Non_Relational_Abstract_Domain

```

module type INTEGER_ABSTRACT_DOMAIN = sig
  type state
  type  $\tau$  expr
  include ABSTRACT_DOMAIN
  with type state := state
  and type  $\tau$  Expr.expr =  $\tau$  expr
  and module type Expr.EXPR_TYPE =
    (INTEGER\_AND\_BOOLEAN\_EXPRESSIONS with type  $\tau$  expr =  $\tau$  expr)
  val pp_state: Format.formatter → state → unit
end

```

An [INTEGER_ABSTRACT_DOMAIN](#) is just an [ABSTRACT_DOMAIN](#) whose transfer functions are those described in [INTEGER_AND_BOOLEAN_EXPRESSIONS](#); i.e. it is suitable to analyze [While_Lang_Integer_and_Boolean](#) programs.

```

module type NON_RELATIONAL_INTEGER_ABSTRACT_DOMAIN = sig
  module SVA: INTEGER_SINGLE_VALUE_ABSTRACTION
  module Abstract_Store:STORE with type  $\tau$  value =  $\tau$  SVA.expr
  type state = Abstract_Store.t
  type  $\tau$  expr = state →  $\tau$  SVA.expr * ( $\tau$  SVA.expr → state → state)
  include INTEGER_ABSTRACT_DOMAIN
  with type state := state
  and type  $\tau$  expr :=  $\tau$  expr
end

```

A [NON_RELATIONAL_INTEGER_ABSTRACT_DOMAIN](#) is an [INTEGER_ABSTRACT_DOMAIN](#) whose abstract state is an abstract store mapping program variables to abstract values. These abstract values are provided by a single-value abstraction.

It is easy to implement non-relational abstract domains in a way such that the implementation of every expression mimicks its concrete counterpart (i.e., [While_Lang_Integer_and_Boolean_Interpreter](#)). In that case, we would have `type  $\tau$  expr = state →  $\tau$  SVA.expr`; it is like a concrete interpreter where would have just replaced the concrete values with abstract values.

However, such a design does not use backward transfer functions at all, and thus cannot learn anything from conditions; i.e., we can learn that $x \in [-5..5]$ after the statement $x := 5/y$, but not that $x \in [3..+\infty]$ after the condition `if`($x > 2$).

To handle backward transfer functions, the evaluation of expression returns not only an abstract value, but also a function that updates the state if we learn something about the possible values for this expression, used for backward propagation.

Note that this shows that our analysis is purely variable-based. As the state only mentions the value of variables, improving the state implies updating these values. We cannot attach any information outside of memory locations. This works relatively well in this simple language, but we quickly encounter problems in more complex languages like C where lvalues can point to several memory locations (e.g. array references). Even with this simple example, propagating conditions such as $x*x \leq 1 \wedge x*x > 0$ fail to discover that this is not possible, because we do not keep any information on the value of the expression $x*x$, only on the value of the variable x (on which we learn nothing). All these problems disappear in a value-based analysis like in Chapter 5, where the numerical analysis only has to refine its knowledge about the values.

```

module Make_Standard_Non_Relational_Abstract_Domain
  (SVA:INTEGER_SINGLE_VALUE_ABSTRACTION)
  : NON_RELATIONAL_INTEGER_ABSTRACT_DOMAIN with module SVA = SVA =
struct
  module SVA = SVA
  module Abstract_Store = Make_Store(struct type  $\tau$  t =  $\tau$  SVA.expr end)
  type state = Abstract_Store.t
  module State = struct type t = state end
  let entry_state = Abstract_Store.empty

```

We can build our `NON_RELATIONAL_INTEGER_ABSTRACT_DOMAIN` in a fully generic way, that only depends on the definition of the `INTEGER_SINGLE_VALUE_ABSTRACTION`.

```

type  $\tau$  expr = state  $\rightarrow$  ( $\tau$  SVA.expr * ( $\tau$  SVA.expr  $\rightarrow$  state  $\rightarrow$  state))

let var x  $s^\#$  = Abstract_Store.find x  $s^\#$ ,
    (fun  $v^\#$   $s^\# \rightarrow$  Abstract_Store.add x  $v^\#$   $s^\#$ )

let ar0:  $\tau$  SVA.expr  $\rightarrow$   $\tau$  expr = fun  $v^\#$   $s^\# \rightarrow$  ( $v^\#$ , fun  $_v^\#$   $s^{\#'} \rightarrow s^{\#'}\)$ )

let ar1 forward backward (e1: $\tau_1$  expr)  $s^\#$  =
  let ( $v_1^\#, r_1$ ) = e1  $s^\#$  in
  let  $v^\#$  = forward  $v_1^\#$  in
   $v^\#, \text{fun } v^\# s^{\#'} \rightarrow \text{let } v_1^{\#'} = \text{backward } v_1^\# v^\# \text{ in}$ 
    match  $v_1^{\#'}$  with None  $\rightarrow s^{\#'} \mid$  Some  $v_1^{\#'} \rightarrow r_1 v_1^{\#'} s^{\#'}$ 

let ar2 forward backward (e1: $\tau_1$  expr) (e2: $\tau_2$  expr)  $s^\#$  =
  let ( $v_1^\#, r_1$ ) = e1  $s^\#$  in let ( $v_2^\#, r_2$ ) = e2  $s^\#$  in
  let  $v^\#$  = forward  $v_1^\# v_2^\#$  in
   $v^\#, \text{fun } v^\# s^{\#'} \rightarrow \text{let } v_1^{\#'}, v_2^{\#'} = \text{backward } v_1^\# v_2^\# v^\# \text{ in}$ 
    let  $s^{\#'} = \text{match } v_1^{\#'} \text{ with None } \rightarrow s^{\#'} \mid \text{Some } v_1^{\#'} \rightarrow r_1 v_1^{\#'} s^{\#'} \text{ in}$ 
    let  $s^{\#'} = \text{match } v_2^{\#'} \text{ with None } \rightarrow s^{\#'} \mid \text{Some } v_2^{\#'} \rightarrow r_2 v_2^{\#'} s^{\#'} \text{ in}$ 
     $s^{\#'}$ 

```

These functions help building expression from sub-expressions, by composition. The forward evaluation of a variable just returns its value in the abstract store, while the backward evaluation updates the abstract store with the new value. For other expressions of arity 0, the backward evaluation does nothing.

For operators of arity 1, the forward evaluation is just like in a concrete interpreter. The backward evaluation checks if we learn something new, and if so, pass it to the backward evaluation of the subexpression $e1$. The case of operators of arity 2 is similar.

```

module Expr = struct
  module Expr = struct
    type  $\tau$  expr = state  $\rightarrow$  ( $\tau$  SVA.expr * ( $\tau$  SVA.expr  $\rightarrow$  state  $\rightarrow$  state))
    let true = ar0 SVA.true
    let  $\neg$  = ar1 SVA. $\neg$  SVA.Backward. $\neg$ 
    let  $\wedge$  = ar2 SVA. $\wedge$  SVA.Backward. $\wedge$ 
    let  $\vee$  = ar2 SVA. $\vee$  SVA.Backward. $\vee$ 
    let unknown_boolean() = ar0 (SVA.unknown_boolean())

    let const k = ar0 (SVA.const k)
    let unknown_integer() = ar0 (SVA.unknown_integer())
    let add = ar2 SVA.add SVA.Backward.add
    let sub = ar2 SVA.sub SVA.Backward.sub
    let mul = ar2 SVA.mul SVA.Backward.mul
    let div = ar2 SVA.div SVA.Backward.div
    let eq = ar2 SVA.eq SVA.Backward.eq
    let leq = ar2 SVA.leq SVA.Backward.leq
  end
  module type EXPR_TYPE = module type of Expr
  include Expr
end

```

Using the arity functions, we can lift SVA transfer functions to the handling of expressions in our abstract domain.

```

let assign x e state = let (e,_) = e state in
  Some(Abstract_Store.add x e state)
let assume cond state = let (cond,r_cond) = cond state in
  let open Quadivalent in
  match (SVA.to_quadivalent cond) with
  | Bottom | False  $\rightarrow$  None
  | True  $\rightarrow$  Some state
  | Top  $\rightarrow$  Some (r_cond SVA.true state)

```

All the backward evaluation is not used for assignment, where we only need forward evaluation. However, in assume backward evaluation is used to perform constraint propagation. We start by assuming that the boolean condition is true, and we update the abstract state based on this fact.

Note that this constraint propagation is much more limited than the one in Chapter 5: in particular, we only propagate constraints inside a single statement, and not accross statements. This is the reason why splitting a statement using temporary variables make the analysis less precise (Logozzo and Fähndrich, 2008), a problem that we solve in PLDI'24 and Chapter 5.

```

let join s1# s2# =
  let f key v1# v2# = SVA.join key.ProgVar.typ v1# v2# in
  Abstract_Store.idempotent_union {f} s1# s2#

let fixpoint_step _loop_id prev next =
  let includes = ref true in
  let f key v1# v2# =
    let (v#, b) = SVA.fixpoint_step key.ProgVar.typ v1# v2# in
    includes := b ∧ !includes; v#
  in
  let s# = Abstract_Store.idempotent_union {f} prev next in
  (s#, !includes)
end

```

Join and widening/inclusion testing are implemented simply, by joining/widening/testing inclusion between the abstract value associated with each variable.

```

module Box_Abstract_Domain =
  Make_Standard_Non_Relational_Abstract_Domain
  (Interval_Single_Value_Abstraction)
module Example_Abstract_Interpretation =
  Example(Fixpoint(Box_Abstract_Domain));;
assert (let initial_state = Box_Abstract_Domain.entry_state in
  let final_state = Option.get @@
    Example_Abstract_Interpretation.factorial5 initial_state in
  Format.asprintf "%a" Box_Abstract_Domain.pp_state final_state =
    "[x -> [0,0] y -> [1,+oo] ]");;

```

We can finally build and test our domain. Note that the `ABSTRACT_DOMAIN` whose abstract values are `Interval` is called the box abstract domain (as its graphical representation looks like rectangular boxes).

Appendix G

Module Ite_Abstraction

G.1 The Ite Abstract Abstract Datatype

The `Ite_Abstraction` is a variant of the `Switch_Abstraction` with only two choices (but each of these choices may have different types). To summarize, we have

Definition G.1.1: The Ite AADT

$$\begin{aligned} \text{Ite}(\mathbb{Q}_B, \mathbb{Q}_l, \mathbb{Q}_r) &\triangleq \mathbb{Q}_B \times (\mathbb{Q}_l \cup \{\text{none}\}) \times (\mathbb{Q}_r \cup \{\text{none}\}) \\ \gamma_{\mathbb{Q}_l}(\text{none}, \hat{s}^\#) &= \gamma_{\mathbb{Q}_r}(\text{none}, \hat{s}^\#) \triangleq \{\} \\ \gamma_{\mathbb{Q}}((\alpha_b, \alpha_l, \alpha_r), \hat{s}^\#) &\triangleq \{(v_l, v) \in \gamma_{\mathbb{Q}_l}(\alpha_l, \hat{s}^\#) \mid v[\alpha_b] = \text{true}\} \\ &\quad \cup \{(v_r, v) \in \gamma_{\mathbb{Q}_r}(\alpha_r, \hat{s}^\#) \mid v[\alpha_b] = \text{false}\} \end{aligned}$$

```
module Ite_Abstraction(Arg:sig
  module Sub:VALUE_BASED_DOMAIN;; open Sub
  type Q_left
  type Q_right
  val serialize_left:
    Q_left option → S# → Q_left option → S# → τ Sub.φ_arguments
    → (Q_left,τ) Sub.serialization * S# * S#
  val serialize_right:
    Q_right option → S# → Q_right option → S# → τ Sub.φ_arguments
    → (Q_right,τ) Sub.serialization * S# * S#
end) = struct
```

We can build the `Ite_Abstraction` as a functor, passing the left and right AADT as arguments, and returning an AADT. Each AADT only consists in a type definition, a pretty-printing, and a serialization function.

```
open Arg;; open Polymorphic_State_Option_Monad;;
type Q = { is_left : ['boolean] Sub.Q;
  if_left : Arg.Q_left option;
  if_right : Arg.Q_right option }
```

The above is the definition of a symbolic ite: if the boolean has value left, return the left value, otherwise the right one. The left and right value may be None in the case where the condition is constant.

```
let make_left symval_left =
  let* true = Sub.Expr.true in
  return { is_left = true; if_left=Some symval_left; if_right=None }

let make_right symval_right =
  let* false = Sub.Expr.¬ Sub.Expr.true in
  return { is_left = false; if_left=None; if_right = Some symval_right }
```

These constructors represent the case where the condition of the ite is constant, i.e. the value is always a left value, or always a right value.

```
let reduce symval state =
  let reduce_left symval =
    match symval.if_left with
    | None → symval (* Already reduced. *)
    | Some _ → begin
        match Sub.assume symval.is_left state with
        | None → {symval with if_left = None}
        | Some _ → symval
      end in
  let reduce_right symval =
    match symval.if_right with
    | None → symval (* Already reduced. *)
    | Some _ → begin
        match Sub.Expr.¬ (return symval.is_left) state with
        | Some(is_right,state)
        | _ → {symval with if_right = None}
      when Sub.assume is_right state <> None → symval
    end in
  reduce_right (reduce_left symval)
```

The reduce function is a fundamental operation in this abstraction: we check if, in the current context, the ite condition has to be true or has to be false; in this case, we update the ite by replacing its possible value by None. This corresponds to a reduction, i.e. an operation that improves the precision in the abstract, without changing the concretization.

```
let ar1_lift (op_left,op_right) symval state =
  let {is_left;if_left;if_right} = reduce symval state in
  let if_left,state = match if_left with
  | None → None, state
  | Some if_left → op_left symval state in
  let if_right,state = match if_right with
  | None → None, state
  | Some if_right → op_right symval state in
  {is_left;if_left;if_right}
```

ar1_lift transforms a pair of unary transfer function (respectively operating on the left and right value of the ite) into a unary transfer function on the ite. An

interesting property of this operation is that it preserves the condition: the new and old value share the same condition, i.e. they are both simultaneously left values or right values.

```

let serialize_option serialize v1 s1 v2 s2  $\phi$ _args =
  match v1,v2 with
  | None, None  $\rightarrow$ 
    Sub.Serialization( $\phi$ _args, fun  $\phi$ _res s  $\rightarrow$  None,  $\phi$ _res, s), s1, s2
  | _  $\rightarrow$ 
    let Sub.Serialization( $\phi$ _args, d_value), s1, s2 =
      serialize v1 s1 v2 s2  $\phi$ _args in
    Sub.Serialization( $\phi$ _args, fun  $\phi$ _res s  $\rightarrow$ 
      let v,  $\phi$ _res, s = d_value  $\phi$ _res s in
      Some(v),  $\phi$ _res, s), s1, s2

let serialize v1 s1 v2 s2  $\phi$ _args =
  let v1 = reduce v1 s1 and v2 = reduce v2 s2 in
  let Serialization( $\phi$ _args, d_is_left), s1, s2 =
    Sub.serialize Typ.boolean
    (Some v1.is_left) s1 (Some v2.is_left) s2  $\phi$ _args in
  let Serialization( $\phi$ _args, d_if_left), s1, s2 =
    serialize_option serialize_left
    v1.if_left s1 v2.if_left s2  $\phi$ _args in
  let Serialization( $\phi$ _args, d_if_right), s1, s2 =
    serialize_option serialize_right
    v1.if_right s1 v2.if_right s2  $\phi$ _args in
  Sub.Serialization( $\phi$ _args, fun  $\phi$ _res s  $\rightarrow$ 
    let if_right,  $\phi$ _res, s = d_if_right  $\phi$ _res s in
    let if_left,  $\phi$ _res, s = d_if_left  $\phi$ _res s in
    let is_left,  $\phi$ _res, s = d_is_left  $\phi$ _res s in
    {is_left; if_left; if_right},  $\phi$ _res, s), s1, s2
end

```

Finally, the anti-unification operation on ite abstractions can be formalized simply as:

$$\Phi(\text{None}, \text{None}) = \text{None} \quad \Phi(\text{None}, \alpha) = \Phi(\epsilon, \alpha) \quad \Phi(\alpha, \text{None}) = \Phi(\alpha, \epsilon)$$

$$\Phi(\text{Some}(\alpha_1), \text{Some}(\alpha_2)) = \text{Some}(\Phi(\alpha_1, \alpha_2))$$

$$\Phi(\text{Ite}(\alpha_1, \alpha_2, \alpha_3), \text{Ite}(\alpha_4, \alpha_5, \alpha_6)) = \text{Ite}(\Phi(\alpha_1, \alpha_4), \Phi(\alpha_2, \alpha_5), \Phi(\alpha_3, \alpha_6))$$

where ϵ denotes the absence of a symbolic value.